

Datasets and Benchmarking in Quantum Computing

Table of Contents

Optimising Clifford Circuits with Quantomatic	2
MQT Bench: Benchmarking Software and Design Automation Tools for Quantum Computing	22
Publicly Available Quantum Circuit Datasets for ML Training	32
QASMBench: A Low-Level Quantum Benchmark Suite for NISQ Evaluation and Simulation	38
Equivalence Checking of Sequential Quantum Circuits	59
QCircuitNet: A Large-Scale Hierarchical Dataset for Quantum Algorithm Design	89
Clifford Circuit Optimization with Templates and Symbolic Pauli Gates	118
Optimality Study of Existing Quantum Computing Layout Synthesis Tools	132
END	146

Optimising Clifford Circuits with Quantomatic

Andrew Fagan¹

Ross Duncan^{1,2}

`andrew.fagan.2014@uni.strath.ac.uk` `ross.duncan@strath.ac.uk`

¹Department of Computer and Information Sciences
University of Strathclyde
26 Richmond Street, Glasgow, United Kingdom

²Cambridge Quantum Computing Ltd
9a Bridge Street, Cambridge, United Kingdom

We present a system of equations between Clifford circuits, all derivable in the ZX-calculus, and formalised as rewrite rules in the Quantomatic proof assistant. By combining these rules with some non-trivial simplification procedures defined in the Quantomatic tactic language, we demonstrate the use of Quantomatic as a circuit optimisation tool. We prove that the system always reduces Clifford circuits of one or two qubits to their minimal form, and give numerical results demonstrating its performance on larger Clifford circuits.

1 Introduction

Remarkable advances in the past two years have seen quantum computing hardware reach the point where the deployment of quantum devices for non-trivial tasks is now a near-term prospect. However, these machines still suffer from severe limitations, both in terms of memory size and the coherence time of their qubits. It is therefore of paramount importance to extract the most useful work from the fewest operations: a poorly optimised quantum program may not be able to finish before it is undone by noise.

In this paper we study the automated optimisation of *Clifford circuits*. Clifford circuits are not universal for quantum computation – they are well known to be efficiently simulable by a classical computer [2] – however adding any non-Clifford gate to the Cliffords yields a set of approximately universal operations hence it is likely that the vast majority of operations in any quantum program will be Clifford operations, and hence reducing the Clifford depth and gate count of a circuit will have substantial benefit.

A secondary reason to focus on Cliffords is that the corresponding subtheory of quantum mechanics is well-understood in terms of the ZX-calculus [7]. Backens [3] has shown that the ZX-calculus is sound and complete for *stabilizer quantum theory* – that is the fragment of quantum mechanics containing only the Clifford operations and states which can be produced from them. While recent extensions to the ZX-calculus have been proposed which are complete for the Clifford+T fragment [16] and for the full qubit theory [22], both these extensions are significantly larger and more complex than the stabilizer subtheory, and both axiomatisations are undergoing rapid development. Clifford circuits therefore provide a stable platform to develop techniques which could later be extended to a universal language.

In this work we present a *circuit optimiser* which can transform a Clifford circuit into an equivalent circuit with reduced depth and total number of operations. The system always reduces Clifford circuits of one or two qubits to their minimal form, and yields significant size reductions for larger circuits.

The optimiser has been developed as a *simplification procedure* in the graphical proof-assistant Quantomatic [17, 18]. By working inside the proof-assistant we obtain an important benefit: alongside the optimised circuit our optimiser produces a formal proof certifying the correctness of its circuit transformations.

Quantomatic is a flexible graph rewrite engine which is especially well-adapted to working with the ZX-calculus. It is designed for interactive use, and a typical session involves interleaving automated tactics¹, backtracking, and manually choosing rewrites. However, our circuit optimiser is designed to operate without user intervention, and we have necessarily made extensive use of the Quantomatic’s powerful tactic combinators and its built-in Python interpreter. In doing so, we have constructed the largest and most sophisticated Quantomatic development to date.

As a graph rewriting engine, Quantomatic normally inspects only the local subgraph structure of a term while searching for a rewrite to apply. However, quantum circuits have a global causal structure which must be maintained to preserve the property of “being a circuit”. A key contribution of this work is the development of techniques to infer the global structure and use it to guide the rewrite process.

Selinger [25] presented generators and relations for all the n -qubit Clifford groups as a rewrite system over string diagrams. This work has some similarities to that: most notably, we also treat Clifford circuits as a free PROP subject to an equivalence relation defined on small subgraphs. However Selinger’s approach is based on transforming circuits to a standard form which is often larger than the smallest equivalent circuit, whereas we aim for minimal forms. Selinger’s rewrite system has a much larger number of rules than the axioms of the ZX-calculus. Maslov and Roetteler [21] demonstrate that all Clifford unitaries can be implemented in depth no more than 7 stages deep, using the $\wedge Z$ as the only two-qubit gate. The $\wedge Z$ has formal disadvantages in ZX-calculus, so in our development we have used the CNOT and swap gates instead. In the examples we have tested, our optimiser usually attains similar depth over our gate set; however on some examples it halts while some simplifications are still possible. Quantomatic has previously been used to produce mechanised correctness proofs for quantum error correcting codes [14, 6, 11], and many of the techniques used in these works are applied here.

We assume that the reader is at least somewhat familiar with both the ZX-calculus and the Quantomatic system; an accessible introduction to both is found in the earlier formalisation effort [14].

Acknowledgements The authors wish to thank Aleks Kissinger and Hector Miller-Bakewell for their technical assistance throughout this project.

The Quantomatic project files All the proofs which appear in this paper and its appendix are publicly available as a downloadable Quantomatic project at <https://gitlab.cis.strath.ac.uk/kwb13215/Clifford-Quanto/>.

2 The Clifford Group

Let $\omega = e^{i\pi/4}$. The n th Clifford group is the group of unitary matrices acting on $\mathbb{C}^{2^n}$, finitely generated by the matrices

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix} \quad V = \frac{1}{\sqrt{2}} \begin{pmatrix} \omega & \bar{\omega} \\ \bar{\omega} & \omega \end{pmatrix} \quad \text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \quad \sigma = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

under tensor product and matrix composition. Observe that S and V are both order 4, the Pauli matrices are given by $Z = S^2$ and $X = V^2$, and the usual Hadamard matrix is obtained as $H = \bar{\omega}SVS$. Throughout

¹A *tactic* is a short program which automates a particular proof strategy.

this paper we will quotient the group by global scalar factors, so that $A = zA$ for all non-zero $z \in \mathbb{C}$. The resulting quotient yields the n th *reduced* Clifford group, which we denote $\mathcal{C}_n$.

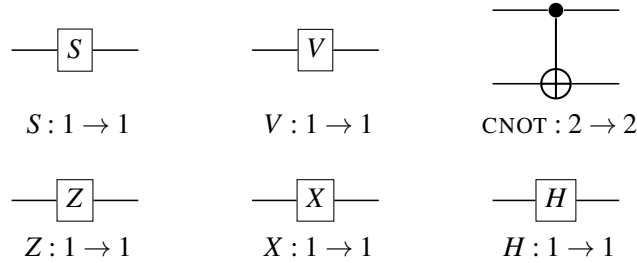
$\mathcal{C}_n$ is finite for all n and the order of the group is given by the expression²

$$|\mathcal{C}_n| = \prod_{j=1}^n 2(4^j - 1)4^j = 2^{2n+1}(2^{2n} - 1)|\mathcal{C}_{n-1}|.$$

The order of $\mathcal{C}_n$ grows very quickly: $|\mathcal{C}_1| = 24$, $|\mathcal{C}_2| = 11520$, $|\mathcal{C}_3| = 92827280$, and so on.

The Clifford group [15] is equivalently defined as the *normalizer* of the Pauli group within the unitaries. Let $\mathcal{P}_n$ be the subgroup of $\mathcal{C}_n$ generated by the Z and X operators³. Then for all $C \in \mathcal{C}_n$ and all $P \in \mathcal{P}_n$ we have $CP = P'C$ for some $P' \in \mathcal{P}_n$. This crucial fact will heavily exploited in our circuit optimisation procedure.

A more holistic view of the Cliffords is as a class of *circuit diagrams* whose gates are the generators of the group, rather than the group itself. This can be formalised as a free $\dagger$ -PROP [20, 19] generated by morphisms



We refer to this PROP as **Cliff**. Note that the swap σ is automatically present due to the PROP structure. The morphisms Z , X , and H are not essential, but it will be convenient later to include them among the generators. The matrix valuations above suffice to define a standard interpretation functor $\llbracket \cdot \rrbracket_C : \mathbf{Cliff} \rightarrow \mathbf{fdHilb}$, such that $\llbracket \mathbf{Cliff}(n, n) \rrbracket_C = \mathcal{C}_n$. We will use this PROP only to define the translation from **Cliff** to the ZX-calculus as a functor; we refer the interested reader to Selinger [25] for more details of this perspective.

Remark 2.1. Note that **Cliff**, as a free PROP, does not include any non-trivial equations between Clifford circuits and is not therefore a presentation of the Clifford *group* itself: **Cliff** distinguishes between different implementations of the same unitary map, which are identified in the image of $\llbracket \cdot \rrbracket_C$. In particular $\mathbf{Cliff}(n, n)$ is infinite for all $n > 0$.

Remark 2.2. Although it will play only a minor role in this paper, we stress that **fdHilb** denotes the category of finite dimensional Hilbert spaces and linear maps under the quotient described above: $A = zA$ for all non-zero $z \in \mathbb{C}$. It is common to impose this quotient only for $z = e^{i\alpha}$, however in this work the question of normalisation will not arise.

3 The ZX-calculus

The ZX-calculus [7] is a formal graphical notation for representing quantum states and processes, and an equational theory for reasoning about them. It is *universal*, meaning that every linear map has a

²The only scalars which arise in the quotient are ω^k for $k \in \mathbb{N}$ so the usual Clifford group is eight times larger.

³Under our quotient we have $Y = ZX = XZ$ so these generators suffice.

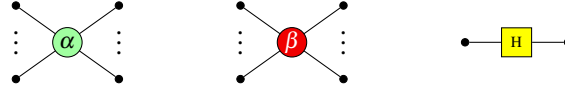


Figure 1: Allowed interior vertices

corresponding ZX-calculus term, and *sound*, meaning that any equation derivable in the calculus is true in its standard Hilbert space interpretation.

The ZX-calculus is formalised using *open graphs*, a generalisation of the usual notion of graph. Open graphs have three kinds of edges: *closed* edges have a vertex at each end, as in a conventional graph; *half-open* edges are incident upon a single vertex, the other end being open; and *open* edges are open at both ends, being incident upon no vertices. The set of open ends of the edges form the *boundary* of the open graph. An open graph is called *framed* if additionally the boundary is partitioned into two linearly ordered sets of *inputs* and *outputs*.

Definition 3.1. A *term*, or *diagram*, of the ZX-calculus is a finite undirected framed open graph whose interior vertices are of the following types:

- $Z(\alpha)$ vertices, labelled by an angle α where $0 \leq \alpha < 2\pi$. These are depicted as green or light grey circles; if $\alpha = 0$ then the label is omitted.
- $X(\beta)$ vertices, labelled by an angle β , where $0 \leq \beta < 2\pi$. These are depicted as red or dark grey circles; again, if $\beta = 0$ then the label is omitted.
- H vertices; unlike the other types H vertices are constrained to have degree exactly 2. They are depicted as yellow squares.

Two terms are considered equal if they are isomorphic as framed labelled graphs.

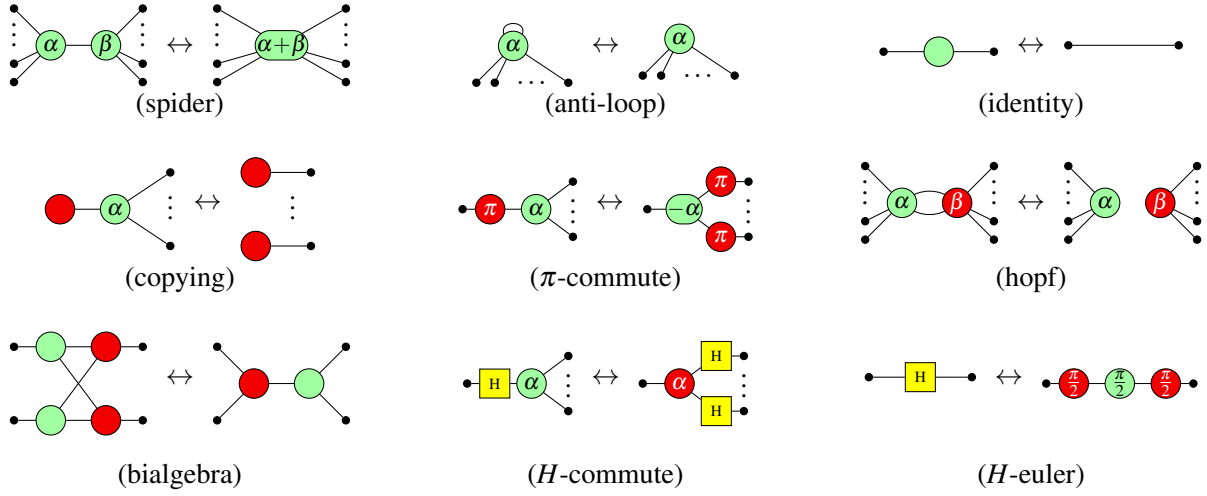
The allowed vertex types are shown in Figure 1. We adopt the convention that inputs are on the left, and outputs on the right.

Compound terms may be formed by joining some number (maybe zero) of the outputs of one term to the inputs of another. Given a diagram $D : n \rightarrow m$ we define its *adjoint* $D^\dagger : m \rightarrow n$ to be the diagram obtained by reflecting the diagram around the vertical axis and negating all the angles. Thus the terms of the ZX-calculus naturally form a $\dagger$ -PROP [8, 10], just like the Clifford circuits of the previous section. The three types of single vertex shown in Figure 1 can then be seen as the generators of this PROP, which we call **ZX**.

Remark 3.2. We consider the *scalar-free* fragment of the ZX-calculus; or equivalently, we quotient everything by non-zero scalar factors, just as we did in the previous section. This greatly reduces the complexity of the diagrams and poses no danger; Backens has demonstrated how to modify the calculus to preserve equality while respecting scalars [4].

Definition 3.3. Given a ZX-term $D : m \rightarrow n$, its *standard interpretation* is a linear map $\llbracket D \rrbracket : (\mathbb{C}^2)^{\otimes m} \rightarrow (\mathbb{C}^2)^{\otimes n}$ defined on single vertices as follows:

$$\begin{aligned} \left[\begin{array}{c} \vdots \\ \text{green circle } \alpha \\ \vdots \end{array} \right] &= \left\{ \begin{array}{ll} |0\rangle^{\otimes m} & \mapsto |0\rangle^{\otimes n} \\ |1\rangle^{\otimes m} & \mapsto e^{i\alpha} |1\rangle^{\otimes n} \end{array} \right. & \left[\begin{array}{c} \vdots \\ \text{red circle } \beta \\ \vdots \end{array} \right] &= \left\{ \begin{array}{ll} |+\rangle^{\otimes m} & \mapsto |+\rangle^{\otimes n} \\ |-\rangle^{\otimes m} & \mapsto e^{i\beta} |-\rangle^{\otimes n} \end{array} \right. \end{aligned}$$

Figure 2: The rules of the ZX-calculus. Note all arithmetic is modulo 2π .

$$\llbracket \bullet \text{---} \boxed{H} \text{---} \bullet \rrbracket = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

Since the above diagrams are the generators of $\mathbf{ZX}$, the above definition extends uniquely to a strict $\dagger$ -symmetric monoidal functor $\llbracket \cdot \rrbracket : \mathbf{ZX} \rightarrow \mathbf{fdHilb}$.

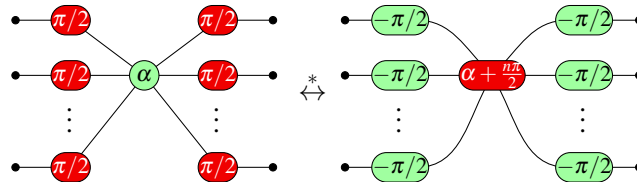
The ZX-calculus has a rich equational theory based on the theory of Frobenius-Hopf algebras [7, 10]. Various axiomatisations have been proposed ([12, 13, 5, 23, 16, 22]) with various advantages and drawbacks. Here we adopt the scheme of Backens [3] which is clean, concise, and adequate for the treatment of the Clifford group. These are shown in Figure 2. Note that, due to the H -commute rule, the colour swapped versions of all the rules are admissible, and we shall use these colour swapped versions without further comment.

Definition 3.4. Let $\leftrightarrow$ be the one-step rewrite relation on the terms of $\mathbf{ZX}$ generated by the pairs of terms shown in Figure 2; let $\overset{*}{\leftrightarrow}$ be the least equivalence relation containing $\leftrightarrow$. We say that $\mathbf{ZX}$ terms a and b are *equivalent* when $a \overset{*}{\leftrightarrow} b$.

We reserve the notation $a = b$ for the case where a and b are equal as graphs, in the sense of Definition 3.1.

Thanks to the H -euler rule, the H vertices are redundant. With this in mind, the following colour changing rule will be useful.

Lemma 3.5. *The following rule is admissible:*



where n is the number of red vertices in the left hand side diagram.

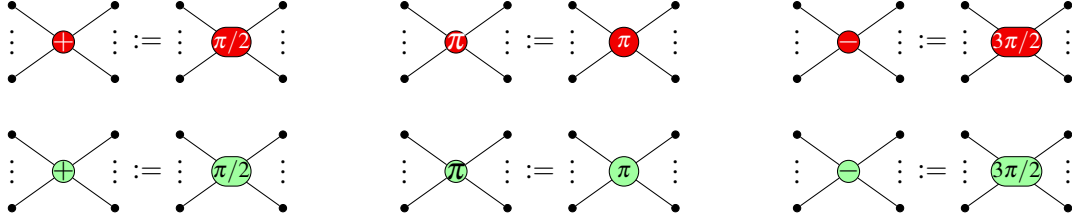
The proof is given in Appendix C.

4 Representing Clifford Circuits in the ZX-calculus

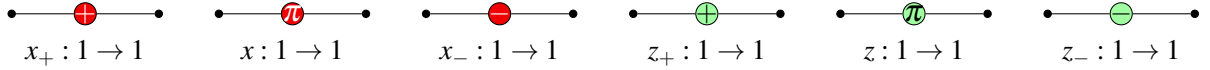
For the rest of the paper we will use only the *stabilizer* sub-language of the full ZX-calculus.

Definition 4.1. Let $\mathbf{ZX}_s$ denote the sub-PROP of $\mathbf{ZX}$ obtained by restricting to those terms whose Z and X vertices are labelled only by angles $\alpha \in \{0, \pi/2, \pi, 3\pi/2\}$.

Since only four vertex labels can occur in the terms of $\mathbf{ZX}_s$, we will adopt a more compact notation as shown in below.

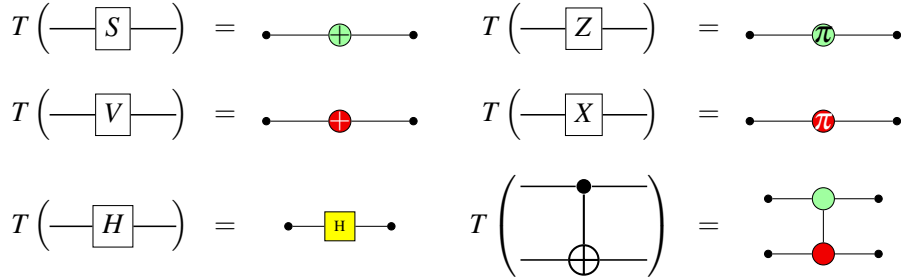


Since the one-qubit instances of these will occur very frequently in the rest of the paper, we introduce the names:



We refer to x and z as the *Paulis*.

We define the *translation* functor $T : \mathbf{Cliff} \rightarrow \mathbf{ZX}_s$ on the generators as shown below.



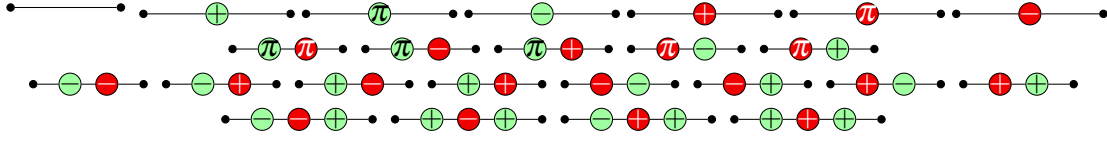
This extends to the whole PROP in the usual way. Further we have the following:

Proposition 4.2. For all $c : n \rightarrow m$ in $\mathbf{Cliff}$ we have $\llbracket c \rrbracket_C = \llbracket T(c) \rrbracket$.

Subject to the proviso that the variables α and β occurring in the rules of Figure 2 must take values in the set $\{0, \pi/2, \pi, 3\pi/2\}$, it is immediate that if $a \leftrightarrow^* b$ then a is in $\mathbf{ZX}_s$ if and only if b is. Further, the equational theory of $\mathbf{ZX}_s$ is sound and complete for its standard interpretation.

Theorem 4.3 (Backens [3]). Let a and b be terms of $\mathbf{ZX}_s$. Then $a \leftrightarrow^* b$ if and only if $\llbracket a \rrbracket = \llbracket b \rrbracket$.

This theorem guarantees that if a given Clifford circuit has a smaller equivalent circuit then there is a proof of their equivalence in the ZX-calculus. The challenge is to find it. We start by considering minimal forms for $\mathcal{C}_1$ and $\mathcal{C}_2$.

Figure 3: $\mathcal{C}_1$: standard minimal forms for single-qubit Cliffords

4.1 1-Qubit Cliffords

Let $\mathcal{C}_1$ denote the 24 diagrams shown Figure 3. Note that all of them are in the image of the translation functor T , so they correspond to Clifford circuits. Further, each of the diagrams has a distinct interpretation under $\llbracket \cdot \rrbracket$, so they cover $\mathcal{C}_1$ and, by soundness, they are not equivalent.

Proposition 4.4. *Given $t \in \mathbf{Cliff}(1, 1)$ then $T(t) \xrightarrow{*} c$ for some $c \in \mathcal{C}_1$.*

Proof. From its type, we have $\llbracket t \rrbracket_C \in \mathcal{C}_1$ and since $|\mathcal{C}_1| = 24$, necessarily $\llbracket t \rrbracket_C = \llbracket c \rrbracket$ for some $c \in \mathcal{C}_1$, and thus by Proposition 4.2 $\llbracket T(t) \rrbracket = \llbracket c \rrbracket$. From this point the result follows by Theorem 4.3. $\square$

Since we will require an effective procedure later, we offer an alternative, constructive, proof which produces the required c . The ideas of this proof form an important part of our optimisation procedure.

Recall that a *line graph* is a framed open graph with one input and one output, which is connected and whose vertices all have degree two. In other words, the graph has the shape of a line. We can treat a line graph as a sequence of vertices starting from the input; let $\text{pos}(v)$ denote the sequence position of vertex v , i.e. if v is the first vertex then $\text{pos}(v) = 0$.

Lemma 4.5. *Let $t \in \mathbf{ZX}_s(1, 1)$ be a line graph; then $t \xrightarrow{*} s$ for some $s \in \mathbf{ZX}_s(1, 1)$ which has the following properties:*

- its vertices form an alternating sequence of $Z(\alpha)$ and $X(\beta)$ vertices, where none of the labels are zero;
- it contains at most two Pauli vertices, and if present these are first vertices after the input.

We say that s is in Pauli-standard form.

Proof. Observe that by applying the H -euler rule all the H vertices can be removed. Then, by repeated application of the spider and identity rules, we obtain a graph of alternating coloured vertices, none of which are labelled by zero. Now we proceed by induction; define

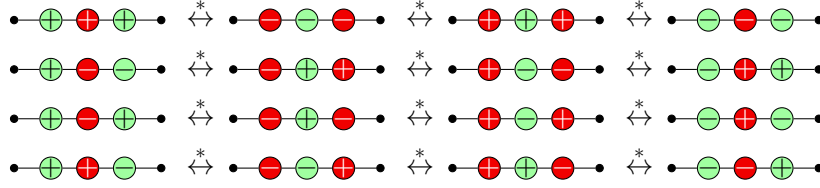
$$m(t) = \sum_{v \text{ is a Pauli}} \text{pos}(v)$$

Observe that if $m(t) < 2$ then t has the required form. Otherwise there is at least one Pauli vertex v with $\text{pos}(v) \geq 2$. Suppose that v is a Pauli Z vertex; then we may rewrite $t \xrightarrow{*} t'$ by the following rewrites:

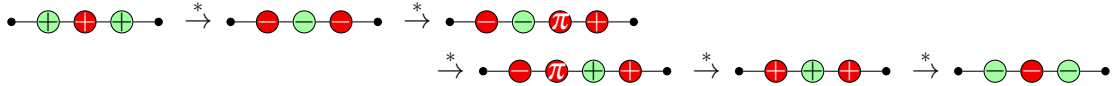
$$\bullet \xrightarrow{*} \bullet \xrightarrow{*} \bullet$$

and applying the identity rule if possible. Note that $Z(\alpha + \pi)$ is never a Pauli, so the number of Paulis is not increased, and the number vertices overall is reduced. If $X(\gamma - \beta)$ is a Pauli then its position is $\text{pos}(v) - 1$. Hence $m(t') < m(t)$, and result is achieved by induction. If v is a Pauli X vertex, then the same argument goes through with the colours reversed. $\square$

Lemma 4.6. *We have the following equations:*



Proof. For the first sequence of equations, consider the following rewrites



where the first and last steps use Lemma 3.5. The other three cases are established using very similar reasoning. $\square$

Alternative proof of Prop 4.4 First observe that for any $t \in \mathbf{Cliff}(1, 1)$ its image $T(t)$ has the form of a line graph. Now, by Lemma 4.5, $T(t) \xleftrightarrow{*} t_1$ where t_1 is in Pauli-standard form. Now we proceed by induction on n , the number of vertices of t_1 .

If $n = 0$ or 1 then t_1 is already in $\mathcal{C}_1$. If $n = 2$ and t_1 has no Pauli nodes, then t_1 is already in $\mathcal{C}_1$; otherwise $t_1 \xleftrightarrow{*} c$ for some c in $\mathcal{C}_1$ by the π -commute rule.

If $n = 3$ and t_1 contains at least one Pauli vertex, then by applying the π commute and spider rules, t_1 can rewrite to a term with only two vertices or fewer. For example

$$\bullet \text{---} \pi \text{---} \alpha \text{---} \beta \text{---} \bullet \xrightarrow{*} \bullet \text{---} \alpha \text{---} \beta + \pi \text{---} \bullet \quad (1)$$

Otherwise, none of the vertices are Paulis; there are 16 line graphs of this form; by Lemma 4.6 these are partitioned in four equivalence classes. Each of the four equivalence classes contains an element of $\mathcal{C}_1$, hence $t_1 \xleftrightarrow{*} c \in \mathcal{C}_1$.

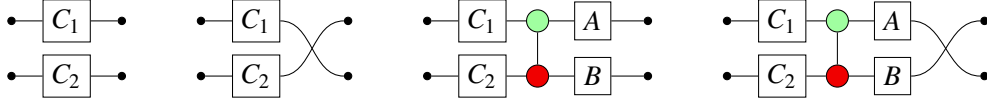
Finally, suppose $n \geq 4$. If t_1 has any Pauli vertices, then $t_1 \xleftrightarrow{*} t_2$ by (1) where t_2 has strictly fewer Pauli vertices, and strictly fewer vertices overall. Otherwise, t_1 has no Pauli vertices; therefore any sequence of vertices of length three can be rewritten, using Lemma 4.6, to a sequence with the colours exchanged. Then by applying the spider and identity rules, and Lemma 4.5 $t_1 \xrightarrow{*} t_2$ where t_2 is in Pauli standard form and has strictly fewer vertices than t_1 . Hence we obtain the result by induction. $\square$

By enumerating all diagrams with fewer than four vertices, it's easy to check that no $c \in \mathcal{C}_1$ is equivalent to a smaller diagram, so these diagrams are the minimal forms for $\mathcal{C}_1$.

4.2 2-Qubit Cliffords

Let $\mathcal{C}_2$ be the set of diagrams defined in Figure 4. Notice that $\mathcal{C}_2$ has $24^2(1 + 1 + 9 + 9) = 11520$ elements. As in the case of $\mathcal{C}_1$, all the elements of $\mathcal{C}_2$ are evidently circuits, and it can be mechanically checked that they are all distinct under the standard interpretation. Hence we adopt $\mathcal{C}_2$ as the normal forms for $\mathcal{C}_2$. Again, by the completeness of the axioms (Thm 4.3) we can immediately conclude that every 2-qubit Clifford circuit is equivalent to some $k \in \mathcal{C}_2$. Zooming in a little closer we have the following:

Proposition 4.7. *Let $k \in \mathcal{C}_2$ and let g be a generator of $\mathcal{C}_2$ as a ZX-calculus term; then $g \circ k \xleftrightarrow{*} k'$ for some $k' \in \mathcal{C}_2$.*



where $C_1, C_2 \in \mathfrak{C}_1$, $A \in \mathfrak{A}$ and $B \in \mathfrak{B}$.

$$\mathfrak{A} = \{ \text{---}, \text{---} \oplus \text{---}, \text{---} \oplus \text{---} \oplus \text{---} \} \quad \mathfrak{B} = \{ \text{---}, \text{---} \oplus \text{---}, \text{---} \oplus \text{---} \oplus \text{---} \}$$

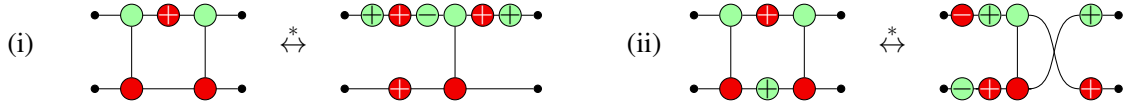
Figure 4: $\mathfrak{C}_2$: standard minimal forms for two-qubit Cliffords

Proof. Notice that the 1-qubit Cliffords C_1 and C_2 which occur in k play no part in the computation, hence we ignore them.

There are effectively four choices for $g : (u \otimes \text{id}), (\text{id} \otimes u), \text{CNOT}$, or σ , where u ranges over the generators of $\mathfrak{C}_1$. If $g = \sigma$, the result is trivial.

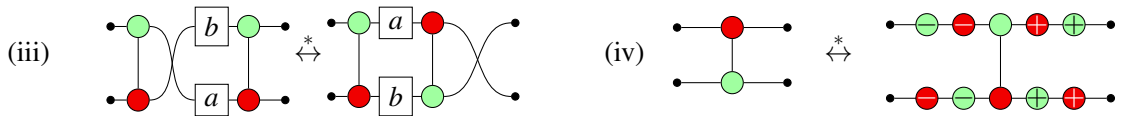
Further, for any $u \in \mathfrak{C}_1$, we have $(u \otimes \text{id}) \circ \text{CNOT} = (a \otimes \text{id}) \circ \text{CNOT} \circ (u_1 \otimes u_2)$ for some $u_1, u_2 \in \mathfrak{C}_1$ and $a \in \mathfrak{A}$, either by commuting vertices of the same colour, or taking out factors of Paulis. Similarly $(\text{id} \otimes u) \circ \text{CNOT} = (\text{id} \otimes b) \circ \text{CNOT} \circ (u_1 \otimes u_2)$ for some $b \in \mathfrak{B}$. Hence the only non-trivial cases to consider are:

1. $g = \text{CNOT}$ and $k = (a \otimes b) \circ \text{CNOT}$. There are nine possibilities for this case, of which we consider two here. If $a = x_+$ and $b = \text{id}$, then by using the bialgebra rule we reduce as shown in (i) below. If $a = x_+$ and $b = z_+$, then a similar argument yields (ii).



They remaining cases can all be derived from these two, except the case where $a = b = \text{id}$, which is trivial.

2. $g = \text{CNOT}$ and $k = (b \otimes a) \circ \sigma \circ \text{CNOT}$. In this case we have (iii) below. However, by exploiting the H rules we have the equivalence (iv) so this case can be reduced to the previous one.



3. $g = \text{CNOT}$ and $k = \sigma$. Again we can apply (iv).

□

As in the single qubit case these reductions will provide the basis for the rewrite rules of our optimiser.

5 Quantomatic

We briefly sketch the Quantomatic system; for a full description see [18]; to obtain it, see [17].

Quantomatic is an interactive theorem prover which can prove equations between terms of the ZX-calculus. The user draws the desired term in the graphical editor, and builds the proof by applying *rewrite*

rules to the current graph. A rewrite rule is a *directed* equation, i.e. a pair of graphs which have the same boundary. Rules are applied in two-step process. First the system searches for subgraphs of the term which are isomorphic to the LHS of the rule; each such subgraph is called a *match*. Quantomatic displays all the possible matches of the chosen rule in the current term, and allows the user to select where the rule is to be applied. The matched subgraph is then replaced by the RHS of the rule to produce a new term. A proof therefore consists of a sequence of terms linked by the application of a particular rule at a particular location in the term.

Quantomatic allows the user to define automated proof tactics, known as simplification procedures or *simprocs*. The name is slightly misleading since there is no need for a simproc to actually *simplify* the graph, in the sense of reducing the number of vertices or edges; however failure to do so will typically result in non-termination. Simprocs are written in the Python programming language, interpreted by the built-in Jython interpreter [11], and augmented with various tactic combinators. The most useful of these are:

- $\text{REWRITE}(r)$: given a rule r , apply the rewrite to the first match found.
- $\text{REWRITE_METRIC}(r, m)$: given a rule r and a *metric* function $m : \mathbf{ZX} \rightarrow \mathbb{N}$ apply the rewrite to the first match found which *reduces* the metric.
- $\text{REWRITE_TARGETED}(r, v, t)$: given a rule r , a vertex v in the LHS of the rule, and a *targeting* function $t : \mathbf{ZX} \rightarrow \text{Vert}$ apply the rewrite r to the first match where vertex v of the rule is matched to the vertex $t(G)$ in the term.

All of these combinators can also accept a list of rules, in which case the rewrites are attempted in the order of the list. Simprocs can be combined in sequence, and also using the combinator $\text{REDUCE}(s)$ which repeats the simproc s until no new rewrites can be performed.

An important point is that when multiple matches are obtained, the system will select one without user intervention. Which rewrite is selected depends on the internal representation of the term, and is effectively non-deterministic.

The axioms of the ZX-calculus, considered as a rewrite system, are neither terminating nor confluent: for example, if $G \rightarrow G'$ via the π -commute rule then the rule can be applied again immediately to rewrite $G' \rightarrow G$. This and similar cases can easily lead to a non-terminating rewrite strategy. Further although many of the axioms can be oriented in a direction which reduces the graph complexity, the resulting rewrite system is no longer complete with respect to the standard interpretation: in some cases it is necessary to increase the graph size to derive a desired equation.

6 Optimising Clifford Circuits with Quantomatic

In this section we describe our optimisation procedure and summarise the results. The reasons for our choices are discussed in the following section.

By *circuit* we mean a ZX-calculus term which is the image of the translation functor $T : \mathbf{Cliff} \rightarrow \mathbf{ZX}$ defined in Section 4. Note that since $\mathbf{Cliff}$ is a $\dagger$ -PROP and $\llbracket \cdot \rrbracket$ is a strict monoidal functor, the circuits form a sub-PROP of $\mathbf{ZX}$: they are closed under composition and tensor product; however they are not closed with respect to the equivalence relation $\xrightarrow{*}$.

Most ZX-calculus terms are *not* circuits: some of these don't correspond to Clifford unitaries, and so do not concern us, however there are many non-circuits which are equivalent to circuits via rewriting. We don't attempt to find circuit forms for such terms. Rather, knowing that the input is a circuit, we use rewrites which replace circuits with circuits, and thus stay at all times within the class of circuits.

The axioms of the ZX-calculus are mostly equations between graphs which are evidently not circuits. Therefore our first step is to derive new equations between circuits which will serve as rewrites for use in the main simproc. These rules are listed in Appendix D.

While the derived rules are “obviously” equations between circuits, the circuit structure of a larger graph may not be apparent by looking at a local subgraph. It’s necessary to look at the entire graph in order to guide the rewrite engine.

Definition 6.1. A ZX-calculus diagram is called *simple* if it satisfies the following criteria:

- It is a simple graph.
- No vertex is adjacent to a vertex of the same colour.
- Every degree 2 vertex has a non-zero label.

Lemma 6.2. Every ZX-calculus diagram is equal to a simple one.

Proof. Starting with an arbitrary diagram d , first replace all H boxes using the H-euler rule. Then apply the spider rule to fuse spiders, the hopf rule to remove parallel edges, the anti-loop rule to remove any self loops, and the identity to remove trivial spiders everywhere. $\square$

Definition 6.3. [9] Given a diagram d , with vertices V , inputs I , and outputs O , a *causal flow* $(f, \prec)$ on d consists of a function $f : I + V \rightarrow V + O$ and a partial order $\prec$ on the set $I + V + O$ satisfying the following properties:

$$(F1) \ f(v) \sim v$$

$$(F2) \ v \prec f(v)$$

$$(F3) \ \text{If } u \sim f(v) \text{ then } v \prec u$$

where $v \sim u$ means that u and v are adjacent in the graph.

In the one-way model [24], the existence of a causal flow on a graph implies that measurement-based quantum computation on the corresponding graph state is deterministic [9]. It plays a similar role for ZX-diagrams.

Definition 6.4. A simple ZX-diagram is called *circuit-like* if has the same number of inputs and outputs and its underlying graph admits a *causal flow*.

Proposition 6.5. Every circuit-like diagram corresponds to a quantum circuit.

Proof. Using the given flow $(f, \prec)$ we will translate a circuit-like diagram d into a circuit c such that c is obvious in the image to of T .

We must first determine which edges in the diagram correspond to qubits and which correspond to interactions between qubits. Note that for each input i , the flow function f defines a path through the diagram $i \rightarrow f(i) \rightarrow f(f(i)) \rightarrow \dots$. By (F2) and the fact that $\prec$ is a partial order, such a path must terminate at an output; and by (F3) each such input-to-output path is disjoint. Further, since the numbers of inputs and outputs are the same then every vertex of the diagram must occur on one such path.

Now we translate the vertices into gates tracing along the qubit paths. If the vertex has degree 2 it immediately yields a Z_α or an X_β gate. If vertex v has higher degree, let its neighbours, excluding its immediate predecessor and successor, be $u_1, \dots, u_n$. We decompose v into $v_0, v_1, \dots, v_n$ as shown below:



Note that the new vertices must be compatible with the original ordering, in the sense that if $u_i \sim v \sim u_j$ where $u_i \prec u_j$ then we require that $v_i \prec v_j$. By the flow conditions, none of the u_i belong to the same qubit as v , and by the simplicity of the diagram all the u_i are the opposite colour to v . Note that the updated diagram also admits a causal flow, obtained from the original one in the obvious manner. Hence all the higher degree vertices may be decomposed in the same way, to leave the diagram with no vertex of higher degree than three, and non-zero phases only in the degree two vertices. Then all the links between qubits maybe replaced by CNOT gates; since $\prec$ defines a partial order on the vertices, we are guaranteed this is well-defined. $\square$

Intuitively the flow path cover specifies which edges of the ZX-diagram are the “qubits” of the circuit; those edges not in the path cover define a CNOT acting on two qubits. Assuming that the graph is a circuit, the path cover is relatively easy to compute by starting at each input and finding a successor among its neighbours. If there is ever a choice of successor for a given vertex this choice can be resolved by looking at the other wires. This information is needed frequently by our optimisation procedure. If an unambiguous path cover cannot be constructed then the procedure halts with an error – in this case the input graph was not a circuit.

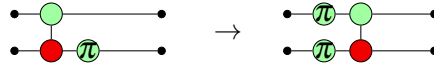
Our optimiser can be summarised as the repeated alternation of two phases:

- Simplification: apply everywhere possible a strictly reducing set of rules.
- Commutation: using targeting and metric rewrites, apply reversible rules to (i) move Pauli operators closer to the inputs. (ii) move CNOT and TONC gates which act on the same two qubits closed together.

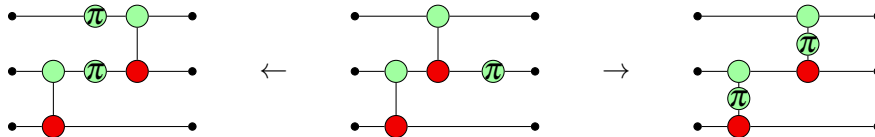
In order to reduce the number of rules, there is an initial phase which replaces all Hadamard, $Z(-\frac{\pi}{2})$ and $X(-\frac{\pi}{2})$ gates with S , V and Paulis as appropriate. The single qubit gates are then “tidied up” by a final pass.

The simplification phase is straightforward, exploiting the different cases of 4.7 as the key rules. The commutation phase is more delicate, and makes heavy use of the path cover computation.

For the single-qubit commutation rules we use targeting function which traverses each path starting from the input and applies the rule at the first possible position in the graph. This works very well and, in concert with the simplification rules, is guaranteed to reduce all single qubit Cliffords to one of the forms of $\mathcal{C}_1$. However this approach fails for rules where a Pauli must commute with a CNOT gate. The REWRITE_TARGETED method allows the user to identify a single vertex in the rewrite rule with a single vertex in the graph to be matched. However this is rarely sufficient to uniquely determine the overall match. For example, the following rule



can be applied in two different ways to the central diagram below:



Note that the righthand rewrite does not produce a circuit. The current implementation provides no direct way for a simproc to specify that one matching should be preferred to another, even when it has the

width	depth	input size	output size	size reduction	proof steps	time $\pm \sigma$
1	20	10.8	2.4	0.26	19.6	7 ± 3
2	20	30.2	9.0	0.30	78.9	1021 ± 680
3	20	44.2	14.7	0.32	148	3144 ± 2189
5	20	78.5	22.6	0.28	239	6679 ± 3091

Table 1: Summary of results on randomly generated circuits. Random circuits were generated with the program `randomCliffs.py` which is available from the project repository. Tests were performed on a basic desktop PC.

necessary information. For this reason, we have defined a metric which favours graphs where the Paulis are nearer to the inputs, and maximally penalises graphs with vertices which are not in any path.

The approach sketched here is very conservative: the optimiser only performs rewrites which result in diagrams which meet the (also very conservative) definition of circuit, even in cases where that diagram could be later rewritten to circuit.

7 Summary of results and discussion

Since we have formalised circuits in the ZX-calculus, our measure of size is the number of non-trivial vertices in the graph. This quantity is the gate count over the generating set S , Z , V , X , and CNOT , although each CNOT contributes 2 to the measure. A less obvious consequence of the ZX-calculus formalism is that the swap gate is not represented at all, and CNOT gates may act on any qubits without penalty. This may or may not be realistic for a given quantum architecture, but it is a fundamental feature of the ZX-calculus, and removing it would require a string diagrammatic formalism that takes into account the planarity of diagrams. This would take us far beyond the scope of this work.

Our optimiser reduces all one- and two-qubit circuits to elements of $\mathcal{C}_1$ and $\mathcal{C}_2$ respectively; however in larger circuits it does not always find a minimal form. Notably our ruleset does not include any reductions for circuits of three or more qubits, so there are relatively simple CNOT circuits which cannot be reduced. Further, due to the extreme conservativity of our metric and targeting procedures, the optimiser does not always find valid rewrites among the ruleset. However, on a test set of randomly generated Clifford circuits it typically produces significant reductions of circuit size; see Table [1](#).

There is striking increase in compute time per proof step as the width of the circuit increases. This can be attributed to the increased number of CNOT gates: as the average vertex degree increases, the number of candidate matches including a given vertex increases geometrically. Since many of these matches will be invalid as circuit rewrites, a lot of time is spent searching for matches which are then rejected. Further, wider circuits require two-qubit commutation rules, which are metric-driven rather than targeted, and hence impose a much greater computational load.

Indeed, the main difficulty we have faced is taking control of the matching engine. Targeted rewriting can select good rewrite positions with high efficiency but does not discriminate between good and bad rewrites at that position. Metric-driven rewrites can perform such discrimination but at the cost of losing control over the search. Combining these features in a single combinator would greatly ease this difficulty.

Further, the targeting and metric functors are required to be stateless; this imposes another avoidable performance cost, as the path cover of the graph must be recomputed on every invocation, regardless of whether a rewrite was performed. Given the poor performance of interpreted Python compared to the Scala core, significant speed-up could be achieved by providing a range of built-in metrics for use in

simprocs.

8 Conclusions and Future Work

Since Quantomatic was designed for interactive use, its simproc environment is currently quite limited. In repurposing the system to operate as a fully automated circuit optimisation tool, we have run up against its current limits, and our frequent resort to brute force and perversity is a reflection of these limitations, rather than the authors' mental states⁴. These limitations are not fundamental, and could be removed by minor extensions of the simproc API. Although there is much scope for improvement in our optimiser, we have demonstrated that the Quantomatic system can serve as the basis for an effective circuit optimisation tool.

References

- [1] *The Jython Project : Python for the Java Platform*. Available at <http://www.jython.org>.
- [2] S. Aaronson & D Gottesman (2004): *Improved Simulation of Stabilizer Circuits*. *Phys. Rev. A* 70(052328), doi:[10.1103/PhysRevA.70.052328](https://doi.org/10.1103/PhysRevA.70.052328).
- [3] Miriam Backens (2014): *The ZX-calculus is complete for stabilizer quantum mechanics*. *New Journal of Physics* 16(9), p. 093021, doi:[10.1088/1367-2630/16/9/093021](https://doi.org/10.1088/1367-2630/16/9/093021).
- [4] Miriam Backens (2015): *Making the stabilizer ZX-calculus complete for scalars*. In Chris Heunen, Peter Selinger & Jamie Vicary, editors: *Proceedings of the 12th International Workshop on Quantum Physics and Logic (QPL 2015)*, *Electronic Proceedings in Theoretical Computer Science* 195, pp. 17–32, doi:[10.4204/EPTCS.195.2](https://doi.org/10.4204/EPTCS.195.2).
- [5] Miriam Backens, Simon Perdrix & Quanlong Wang (2016): *A Simplified Stabilizer ZX-calculus*. In R. Duncan & C. Heunen, editors: *Proceedings 13th International Conference on Quantum Physics and Logic (QPL 2016)*, *Electronic Proceedings in Theoretical Computer Science* 236, pp. 1–20, doi:[10.4204/EPTCS.236.1](https://doi.org/10.4204/EPTCS.236.1).
- [6] Nicholas Chancellor, Aleks Kissinger, Stefan Zohren & Dominic Horsman (2016): *Coherent Parity Check Construction for Quantum Error Correction*. *arXiv.org preprint* (arXiv:[1611.08012](https://arxiv.org/abs/1611.08012)).
- [7] Bob Coecke & Ross Duncan (2011): *Interacting Quantum Observables: Categorical Algebra and Diagrammatics*. *New J. Phys* 13(043016), doi:[10.1088/1367-2630/13/4/043016](https://doi.org/10.1088/1367-2630/13/4/043016).
- [8] Bob Coecke, Ross Duncan, Aleks Kissinger & Quanlong Wang (2015): *Generalised Compositional Theories and Diagrammatic Reasoning*. In G. Chiribella & R. Spekkens, editors: *Quantum Theory: Informational Foundations and Foils*, Springer, doi:[10.1007/978-94-017-7303-4_10](https://doi.org/10.1007/978-94-017-7303-4_10).
- [9] V. Danos & E. Kashefi (2006): *Determinism in the one-way model*. *Phys. Rev. A* 74(052310), doi:[10.1103/PhysRevA.74.052310](https://doi.org/10.1103/PhysRevA.74.052310).
- [10] Ross Duncan & Kevin Dunne (2016): *Interacting Frobenius Algebras are Hopf*. In Martin Grohe, Eric Koskinen & Natarajan Shankar, editors: *Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science, LICS '16*, New York, NY, USA, July 5-8, 2016, *LICS '16*, ACM, pp. 535–544, doi:[10.1145/2933575.2934550](https://doi.org/10.1145/2933575.2934550).
- [11] Ross Duncan & Maxime Lucas (2014): *Verifying the Steane code with Quantomatic*. In Bob Coecke & Matty Hoban, editors: *Proceedings 10th International Workshop on Quantum Physics and Logic (QPL 2013)*, *Electronic Proceedings in Theoretical Computer Science* 171, pp. 33–49, doi:[10.4204/EPTCS.171.4](https://doi.org/10.4204/EPTCS.171.4).
- [12] Ross Duncan & Simon Perdrix (2009): *Graph States and the Necessity of Euler Decomposition*. In K. Ambos-Spies, B. Löwe & W. Merkle, editors: *Computability in Europe: Mathematical Theory and Computational*

⁴However the first author's mental state did take a beating from the lack of debugging facilities.

A Electronic Resources

All the work described in this paper is available as a downloadable quantomatic project from the following URL.

<https://gitlab.cis.strath.ac.uk/kwb13215/Clifford-Quanto/>

Please download it and play around!

B An aside on CNOT and SWAP

We have already seen the ZX-calculus representation of CNOT in Section 4. Note that the upper (z) vertex corresponds to the control qubit, while the lower (x) is the target qubit. We could also consider the CNOT with control and target reversed, which we denote TONC.

$$\text{TONC} := \begin{array}{c} \bullet \quad \bullet \\ \diagdown \quad \diagup \\ \bullet \quad \bullet \\ \diagup \quad \diagdown \\ \bullet \quad \bullet \end{array} = \begin{array}{c} \bullet \quad \bullet \\ \text{---} \quad \text{---} \\ \bullet \quad \bullet \end{array}$$

It is a well known fact that the swap itself can be represented as $\text{CNOT} \circ \text{TONC} \circ \text{CNOT}$, and indeed this is provable in the ZX-calculus.

$$\begin{array}{c} \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \\ \diagdown \quad \diagup \quad \diagdown \quad \diagup \quad \diagdown \quad \diagup \\ \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \end{array} \stackrel{*}{\longleftrightarrow} \begin{array}{c} \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \\ \diagdown \quad \diagup \quad \diagdown \quad \diagup \quad \diagdown \quad \diagup \\ \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \end{array} \stackrel{*}{\longleftrightarrow} \begin{array}{c} \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \\ \diagdown \quad \diagup \quad \diagdown \quad \diagup \quad \diagdown \quad \diagup \\ \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \end{array} \stackrel{*}{\longleftrightarrow} \begin{array}{c} \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \\ \diagdown \quad \diagup \quad \diagdown \quad \diagup \quad \diagdown \quad \diagup \\ \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \end{array} \stackrel{*}{\longleftrightarrow} \begin{array}{c} \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \\ \diagdown \quad \diagup \quad \diagdown \quad \diagup \quad \diagdown \quad \diagup \\ \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \end{array} \quad (2)$$

Note that the bialgebra rule gives the first equation, and the Hopf rule the second: we cannot rely purely on the group structure.

Alternatively, by the colour duality principle we can express TONC in terms of CNOT and H :

$$\begin{array}{c} \bullet \quad \bullet \\ \diagdown \quad \diagup \\ \bullet \quad \bullet \end{array} \stackrel{*}{\longleftrightarrow} \begin{array}{c} \text{H} \quad \bullet \quad \text{H} \\ \diagdown \quad \diagup \\ \text{H} \quad \bullet \quad \text{H} \end{array} \stackrel{*}{\longleftrightarrow} \begin{array}{c} \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \\ \diagdown \quad \diagup \quad \diagdown \quad \diagup \quad \diagdown \quad \diagup \\ \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \quad \bullet \end{array} \quad (3)$$

Combining (3) and (2) we obtain:

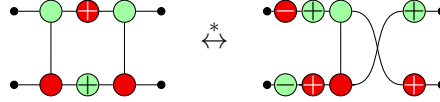
Lemma B.1.

$$\begin{array}{c} \bullet \quad \bullet \\ \diagdown \quad \diagup \\ \bullet \quad \bullet \end{array} \stackrel{*}{\longleftrightarrow} \begin{array}{c} \bullet \quad \oplus \quad \oplus \quad \oplus \quad \oplus \quad \oplus \quad \pi \\ \diagdown \quad \diagup \quad \diagdown \quad \diagup \quad \diagdown \quad \diagup \\ \bullet \quad \oplus \quad \oplus \quad \oplus \quad \oplus \quad \oplus \quad \pi \end{array}$$

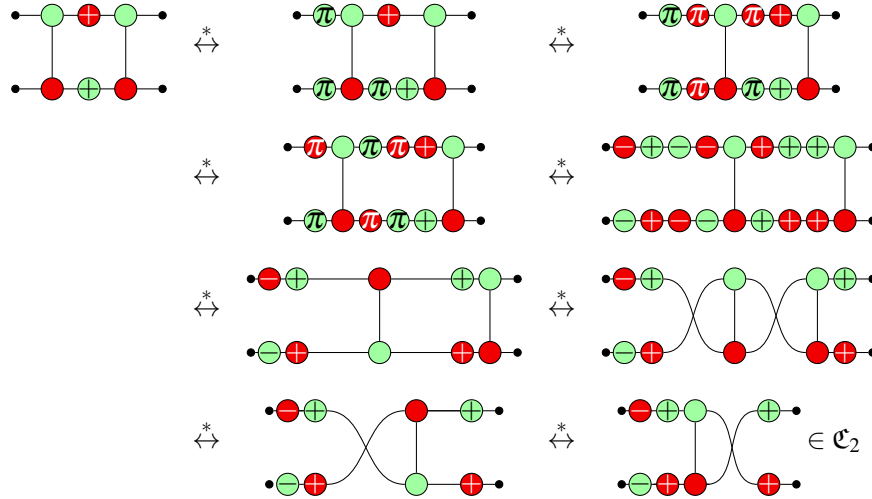
Lemma B.1 indicates that we can dispense with the swap as well as TONC and work purely with CNOT. Since we have formalised the system as a PROP it is more convenient to treat σ as one of the generators of $\mathcal{C}_2$.

□

Equation (ii):



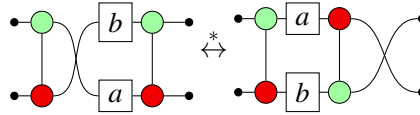
Proof.



Note that we have used both Equations (2) and (3) in the heart of this proof.

□

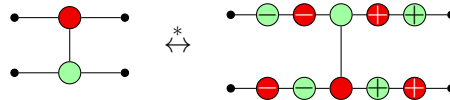
Equation (iii):



Proof. This is a straightforward application of the “topology” principle; no rules are required, they are equal in the sense of Definition 3.1

□

Equation (iv):



Proof. This is simply Equation (3) from the preceding section.

□

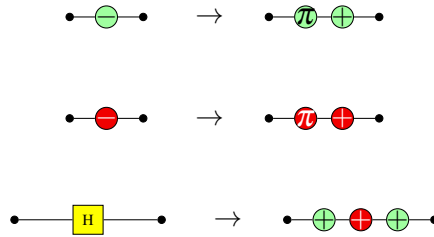
D Main Rules

The following derived rules are the core of the circuit transformations in the main proof development. The rules themselves are found in `derived-rules/` in the Quantomatic project, and their derivations are found in `derived-rules-proofs/`.

In all diagrams inputs are to the left, outputs to the right.

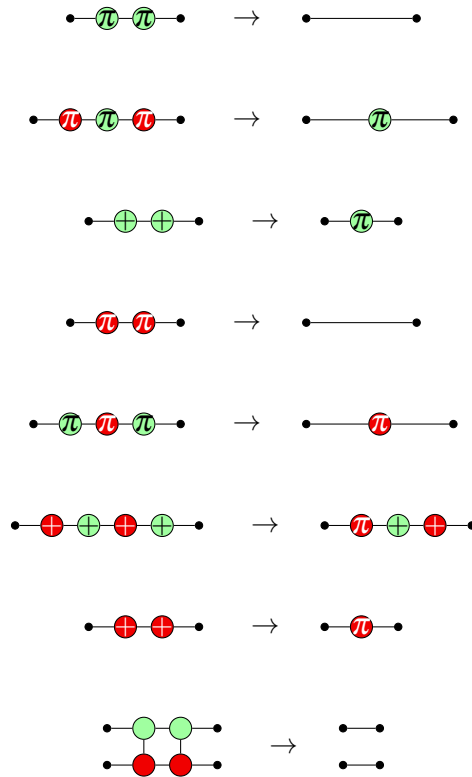
D.1 Init

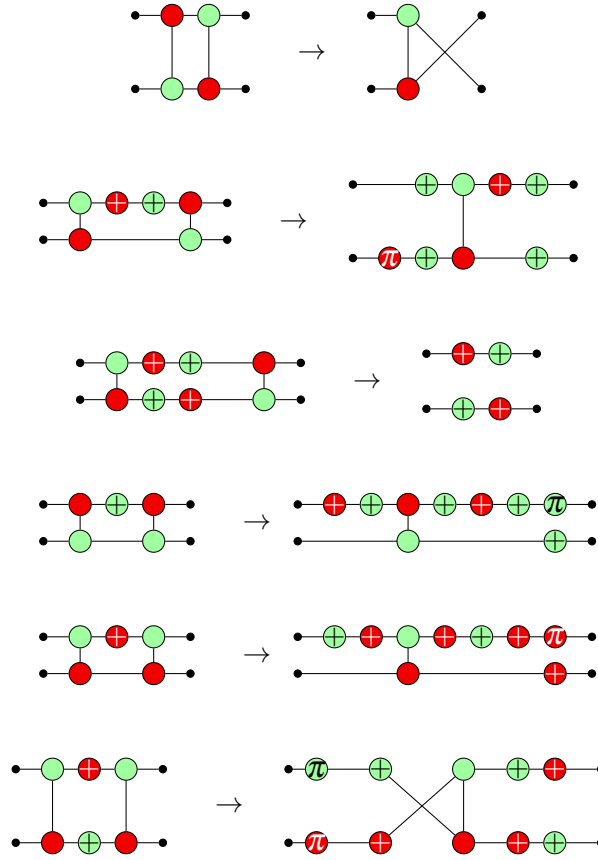
These rules remove all minus and H nodes from the graph.



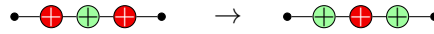
D.2 Always Rules

These are strictly reducing so can always be applied.





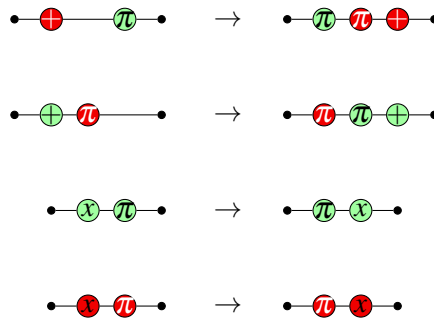
D.3 EulerProc



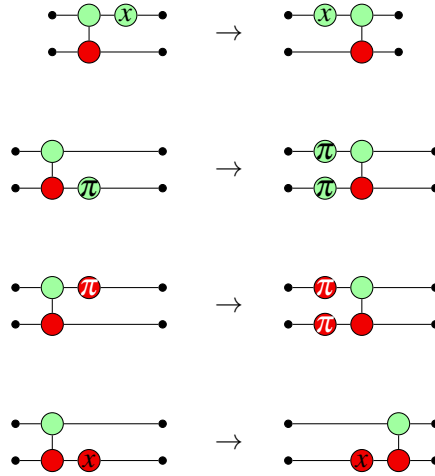
The inverse is also used.

D.4 Pauli Commutation rules

Each of these is managed by a custom simproc which searches for a good place to apply them.

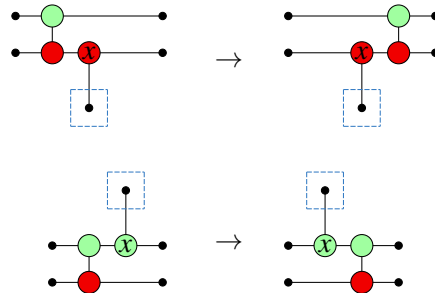


These rules are used by the Pauli Commutation proc, which is based on REDUCE_METRIC.



D.4.1 C2 Rules

These two alternate in the C2Proc simproc. Both have a custom targeting routine to select where they should be applied.



MQT Bench: Benchmarking Software and Design Automation Tools for Quantum Computing

Nils Quetschlich¹, Lukas Burgholzer¹, and Robert Wille^{1,2}

¹Chair for Design Automation, Technical University of Munich, Germany

²Software Competence Center Hagenberg GmbH (SCCH), Austria

Quantum software tools for a wide variety of design tasks on and across different levels of abstraction are crucial in order to eventually realize useful quantum applications. This requires practical and relevant benchmarks for new software tools to be empirically evaluated and compared to the current state of the art. Although benchmarks for specific design tasks are commonly available, the demand for an overarching cross-level benchmark suite has not yet been fully met and there is no mutual consolidation in how quantum software tools are evaluated thus far. In this work, we propose the *MQT Bench* benchmark suite (as part of the *Munich Quantum Toolkit*, MQT) based on four core traits: (1) cross-level support for different abstraction levels, (2) accessibility via an easy-to-use web interface (<https://www.cda.cit.tum.de/mqtbench>) and a Python package, (3) provision of a broad selection of benchmarks to facilitate generalizability, as well as (4) extendability to future algorithms, gate-sets, and hardware architectures. By comprising more than 70,000 benchmark circuits ranging from 2 to 130 qubits on four abstraction levels, MQT Bench presents a first step towards benchmarking different abstraction levels with a single benchmark suite to increase comparability, reproducibility, and transparency.

1 Introduction

Quantum computing has gained a lot of attention due to its promising applications and the advances in quantum computing hardware. But still, designing quantum algorithms often means to manually implement quantum circuits on the gate-level—similar to assembly language in classical computing. Hence, there is an urgent need to enhance the workflow of designing and testing quantum algorithms. Without software tools that aid in the design of quantum algorithms, the underlying hardware may not be efficiently utilized. Thus, researchers and developers have already proposed tools for various design tasks, such as quantum circuit simulation [1]–[13], compilation [14]–[34], or verification [35]–[50]. This already led to comprehensive quantum circuit design flows realized through toolkits such as IBM’s Qiskit [51], Google’s Cirq [52], or Rigetti’s Forest [53]—in addition to numerous tools and methods developed by other researchers and engineers in the field.

Generally, these software tools operate on and across different levels of abstraction and all tackle computationally hard problems. As a result, they have to compromise between resource demands and result quality. Usually, when a new software tool is proposed, its performance is empirically evaluated. The question how to evaluate the performance of a software tool is very challenging and not yet fully answered—especially for tools solving NP-complete problems. The most common approach is to run certain problem instances, so-called benchmarks, and compare the performance of the newly proposed method against state-of-the-art methods regarding a certain characteristic, e.g., run-time or solution quality.

Nils Quetschlich: nils.quetschlich@tum.de

Lukas Burgholzer: lukas.burgholzer@tum.de

Robert Wille: robert.wille@tum.de

Currently, this benchmarking is conducted using a wide variety of different benchmark suites—each with a specific focus. While some benchmark suites, e.g., [54], [55], provide benchmarks on higher abstraction levels, other benchmark suites focus on lower abstraction levels, e.g., [56], [57]. Although the intended target levels are properly covered, the demand for a cross-level benchmark suite is not fully met yet. As another consequence, there is no mutual consolidation which benchmarks to use for empirical evaluations yet—leading to lower comparability, reproducibility, and transparency.

In this paper, we propose *MQT Bench*—the benchmark suite from the *Munich Quantum Toolkit* (MQT) which explicitly aims to address those drawbacks. It aims at providing a first step towards benchmarking the whole quantum software stack with a single benchmark suite by offering the same benchmark algorithms on different levels of abstraction. To realize such a benchmark suite, several challenges have to be tackled:

- **Distinct requirements on different levels:** Benchmarks have to fulfill certain requirements depending on the abstraction level, e.g., only gates of a device’s native gate-set are allowed.
- **Accessibility:** To foster adoption of the benchmark suite, it needs to be as easy to use as possible.
- **Generalizability:** Providing a comprehensive set of benchmark algorithms to cover as many use cases as possible.
- **Extendability:** Future algorithms, gate-sets, and architectures should be easy to integrate.

MQT Bench has been developed with these challenges in mind and is based on four core traits:

1. **Cross-level benchmarking:** Provision of benchmarks on four abstraction levels.
2. **Accessibility:** To simplify the usage of MQT Bench, we provide a web interface (<https://www.cda.cit.tum.de/mqtbench>) and a Python package in addition to the open-source repository on GitHub (<https://github.com/cda-tum/mqt-bench>).
3. **Algorithm selection:** Broad selection of different algorithms ranging from building blocks, i.e., Quantum Fourier Transform (QFT), to applications, i.e., Grover’s algorithm.
4. **Extendability:** Algorithms, gate-sets, and hardware architectures are easily extendable and integrable into MQT Bench.

By this, MQT Bench aims to improve the comparability, reproducibility, and transparency of empirical evaluations for the whole quantum software stack.

The rest of this work is structured as follows: In [Section 2](#), we review the necessary basics of the quantum computing compilation flow and the respective software stack to keep this work self-contained. Afterwards, [Section 3](#) reviews the state of the art on how this quantum software stack is currently evaluated and benchmarked—motivating the benchmark suite proposed in this paper. Based on that, [Section 4](#) introduces the resulting benchmark suite and its core traits, before the generated benchmarks are evaluated in [Section 5](#). [Section 6](#) concludes this work.

2 Background

To keep this paper self-contained, this section gives a brief overview of the quantum circuit compilation flow with its different abstraction levels and the respective quantum software stack. The levels mentioned in this section are inspired by the structure proposed by the openQASM 3.0 specification [58] and can be found similarly in many other compilation flows as well.

2.1 Quantum Circuit Compilation Flow

Similar to the classical domain, executing a conceptual quantum algorithm on an actual device requires compiling it to a representation that adheres to all constraints imposed by the hardware. Usually, quantum algorithms are initially developed and tested on a hardware-agnostic level. Generally, this is described as a quantum circuit that consists of high-level building blocks without any restrictions to a certain gate-set or hardware architecture and is defined as the *algorithmic level*.

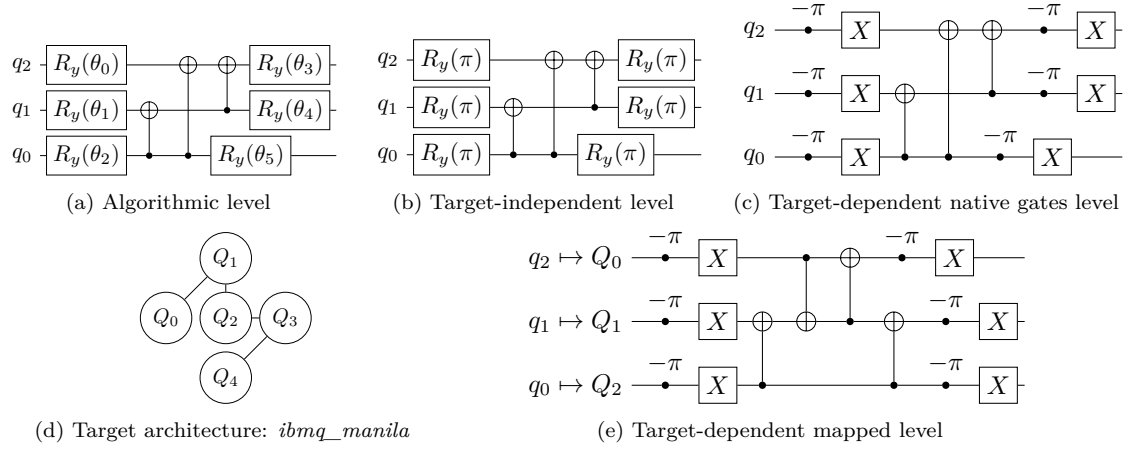


Figure 1: Compilation of a Variational Quantum Algorithm.

Example 1. Variational Quantum Algorithms (VQAs) are an emerging class of quantum algorithms with a wide range of applications. A respective circuit is depicted in Fig. 1a and shows an example of an ansatz function frequently used for Variational Quantum Eigensolvers (VQEs), a subclass of VQAs. On this abstraction level, the circuit is parameterized by the angles θ_i of the six single-qubit gates.

The first step in realizing a conceptual quantum algorithm on the algorithmic level for a particular problem is to synthesize the high-level building blocks and optimize the resulting representation independently of the actual target architecture. In analogy to a classical compiler, this involves tasks such as constant propagation and folding, gate modifier evaluation, synthesis, loop unrolling, and gate simplification. The resulting representation is defined as the *target-independent level*.

Example 2. VQAs are hybrid quantum-classical algorithms, where the parameters of the quantum ansatz are iteratively updated by a classical optimizer analogous to conventional gradient-based optimization. Consider again the circuit from Fig. 1a. Assuming that these parameters have been determined, e.g., $\theta_i = \pi$ for $i = 0, \dots, 5$, they are now propagated and the resulting quantum circuit is shown in Fig. 1b.

Today’s quantum devices impose several constraints on the circuits that may be executed on them. Thus, a target-dependent compilation phase is necessary. For one, devices only provide a particular set of native gates—typically consisting of an entangling gate (like the CX gate) and some family of single-qubit gates. Therefore, the gates of a circuit need to be translated to this native gate-set, which is typically followed by an optimization pass that aims to reduce the introduced overhead. This representation level is defined as the *target-dependent native gates level*.

Example 3. Different quantum computer realizations support different native gate-sets. In our example, we consider the `ibmq_manila` device as the target device that natively supports I , X , $\sqrt{X}$, R_z , and CX gates. Consequently, the R_y gates in Fig. 1b have to be converted using only these native gates. In this case, they are substituted by a sequence of R_z (denoted as $\bullet$ with a phase of $-\pi$) and X gates as shown in Fig. 1c.

In addition to the limited gate-set, today’s devices (at least those based on superconducting qubits) only feature limited connectivity between their qubits. Consequently, the circuit’s logical qubits need to be *mapped* to the targeted device’s physical qubits, so that multi-qubit gates are only applied to qubits directly connected on the device. Since such a mapping can rarely be determined in a static fashion, i.e., globally for the whole circuit, the mapping has to change dynamically throughout the circuit, which is frequently referred to as mapping or routing. Again, this is followed by a round of optimization in order to minimize the overhead caused by the compilation. This representation level is defined as the *target-dependent mapped level*.

Example 4. Consider again the scenario from [Example 3](#). The architecture of the `ibmq_manila` device is shown in [Fig. 1d](#) and defines between which qubits a two-qubit operation can be performed. Since the circuit shown in [Fig. 1c](#) contains CX gates operating between all combinations of qubits, there is no mapping directly matching the target architecture’s layout. As a consequence, a non-trivial mapping followed by a round of optimization leads to the resulting circuit shown in [Fig. 1e](#). This is also the reason for the different sequence of CX gates compared to the previous example.

At this point, the circuit is ready to be sent to the quantum computer’s backend for execution. From there, the individual gates are scheduled, linked to a particular calibration, and the resulting circuit is eventually passed to the target machine code generator. The resulting binaries are then forwarded to the execution engine to orchestrate the quantum computation on the actual device.

2.2 Quantum Software Stack

Quantum software is needed on all levels reviewed above in order to aid designers in eventually realizing useful quantum applications. In the following, we review three of the core design tasks for software tools.

Today, quantum computers are a scarce resource with limited availability and capability. Until more devices become available that are larger and less prone to errors, classical means to simulate quantum algorithm circuits (also known as *quantum circuit simulators*) are essential for fostering the development of quantum applications. Additionally, and in contrast to actual quantum computing devices, quantum circuit simulators allow for detailed insights into the quantum states throughout the circuit execution since the state’s amplitudes are explicitly tracked during the simulation. On actual hardware, information can only be extracted in the form of measurements from the final quantum state, which each yields a classical bit string according to the distribution described by the state’s amplitudes. Such simulators can be used on the highest possible level of abstraction, since they can be designed in a way that does not require restrictions on the circuit’s gate-set or the qubits’ connectivity, e.g., as proposed in [\[1\]–\[13\]](#). Furthermore, they might also be used on the lowest possible level in order to perform noise-aware simulations to estimate how a circuit is likely to perform on an actual device, e.g., as proposed in [\[59\]–\[62\]](#). Either way, on both abstraction levels, quantum circuit simulation is non-trivial since corresponding representations of states and operations, in general, require exponential space or runtime—requiring powerful software tools.

Due to the immense complexity of the tasks involved in quantum circuit compilation (as it has been illustrated in [Section 2.1](#)), e.g., mapping being NP-complete [\[63\]](#), manually conducting compilation often is not an option. Consequently, efficient software tools are needed across all levels of abstraction of the aforementioned flow in order to realize a conceptual algorithm on an actual device. Thus, various compilation tools have been proposed, e.g., [\[14\]–\[34\]](#).

During compilation, an algorithm’s description is considerably altered and transformed. Naturally, it is of utmost importance to ensure that the originally intended functionality is preserved through all levels of abstraction. This procedure is called verification. While the underlying principle of comparing the transformations represented by different quantum circuits is conceptually simple, the exponential size of the underlying matrices makes this problem challenging—it has been shown to be QMA-complete [\[64\]](#). Due to the increasing complexity of today’s compilation flows, tools for verifying their results become increasingly important. Examples of verification tools have been proposed in [\[35\]–\[50\]](#).

These design tasks demonstrate the wide variety of software tools operating on and across different levels of abstraction—leading to comprehensive quantum circuit design flows realized through toolkits such as IBM’s Qiskit [\[51\]](#), Google’s Cirq [\[52\]](#), or Rigetti’s Forest [\[53\]](#). At the same time, all these tasks have in common that they have an immensely large complexity. Therefore, a multitude of techniques have been proposed for each task—each with its own trade-off between resource demand and quality of the result. It is key in the development of scientific methods to empirically evaluate and compare their performances on practical, relevant benchmarks.

3 Benchmarking

The benchmark suite proposed in this paper is not the first and certainly not the last collection of quantum circuit benchmarks. In this section, we review some of the existing suites and their respective foci. Afterwards, we discuss how MQT Bench further complements this.

3.1 Current State of the Art

Providing a comprehensive set of benchmarks to satisfy the needs across all levels of the quantum circuit compilation flow is a challenging task. Several approaches to provide benchmark suites for some of the levels within the quantum circuit compilation flow have already been proposed and a non-exhaustive overview is given in the following:

Application-Oriented Performance Benchmarks for Quantum Computing [54]: Starting with an example located on the algorithmic level, this benchmark suite rather focuses on high-level descriptions. At the moment of writing, it provides 13 benchmark algorithms which can be generated with a variable number of qubits via jupyter notebooks using multiple software compilation flows and are classified into four categories of complexity. The resulting circuits are available as high-level Python objects without any further compilation steps.

SupermarQ [55]: Similarly, SupermarQ also focuses on the algorithmic level by providing benchmarks with an adjustable qubit range for eight algorithms as high-level Python objects via a Python package. Additionally, six feature vectors are proposed to describe the characteristics of the benchmarks.

QASMBench [56]: This benchmark suite focuses on the target-independent level. It offers numerous quantum circuits with a wide but fixed range of both the number of qubits and depth. It classifies the benchmarks into three categories of sizes and three categories of algorithm classes. All circuits are available in an intermediate representation according to the openQASM 2.0 specification [65].

RevLib [57]: This benchmark suite provides a wide variety of reversible circuits in an intermediate representation format specified by the authors. Classical functions are frequently embedded into reversible circuits in order to use them in quantum algorithms, e.g., oracle functions or Boolean building blocks such as the modular exponentiation in Shor’s algorithm [66]. In addition, decomposed versions of this benchmark suite have found widespread use in evaluating the performance of mapping tools, e.g., in [32]–[34]. Since reversible circuits merely form a subclass of quantum circuits and do not employ quantum mechanical effects such as superposition and entanglement, they do not serve as adequate benchmarks for the whole stack.

3.2 Motivation

All of the mentioned benchmark suites have in common that they each target a specific abstraction level within the quantum circuit compilation flow. Although the respectively intended target level is well covered, the demand for a cross-level benchmark suite is not fully met yet. Additionally, so far, there is no mutual consolidation which benchmarks to use for empirical evaluations of software tools. This results in a lower comparability, reproduceability, and transparency of results and may even cause confusion because benchmarks believed to realize the same particular task are quite frequently realized in a completely different fashion, e.g., a benchmark called "grover" could realize Grover’s algorithm with any particular oracle. Consequently, the demand for a benchmark suite that spans the whole stack of abstraction levels constitutes the main motivation of this contribution. In order to realize such a benchmark suite, several challenges have to be tackled:

- Distinct requirements for benchmark circuits on different levels: The closer a level is to the actual computing hardware, the more requirements have to be fulfilled, e.g., only gates from the device’s native gate-set are allowed and the connectivity of the hardware architecture must be considered.
- Accessibility: In order to facilitate its adoption, the benchmark suite needs to be as easy to use as possible. While most benchmark suites already provide open-source access, running and adapting the code to the user’s needs, e.g., obtaining a selection of relevant benchmarks, frequently is tedious or impossible and a solution that keeps this hurdle low is desirable.

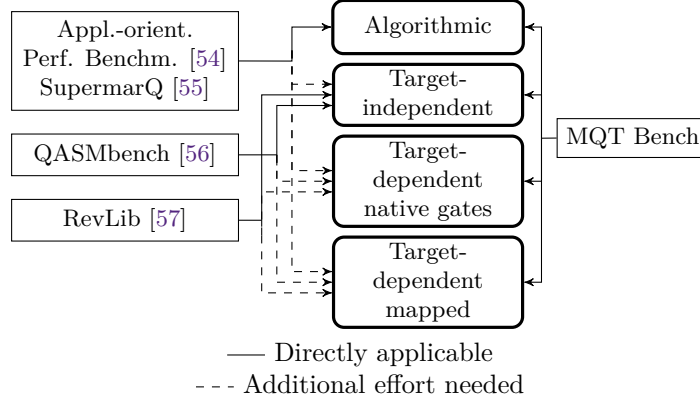


Figure 2: Benchmarking software tools with and without MQT Bench.

- Generalizability of a software tool: This can only be guaranteed if it is evaluated on a broad set of benchmarks, relating to several characteristics, e.g., the number of qubits, circuit depth, or the application domain.
- Extendability: In addition to that, quantum computing is rapidly advancing and the extendability of the benchmark suite to future algorithms, gate-sets, and architectures becomes even more important.

MQT Bench has been developed with these challenges in mind and aims to be a first step towards covering the whole quantum software stack with a benchmark suite based on the following four core traits (which are described in detail in Section 4):

1. Cross-level benchmarking: All benchmarks are provided on four abstraction levels reviewed in Section 2.1.
2. Accessibility: A website is provided (<https://www.cda.cit.tum.de/mqtbench>) to simplify the usage of MQT Bench as much as possible. Additionally, all benchmarks can also be generated on-demand using our Python package. While most users' needs should be covered by this, we also give access to the open-source repository on GitHub (<https://github.com/cda-tum/mqt-bench>).
3. Algorithm Selection: Broad selection of different benchmarks with parameterizable characteristics ranging from building blocks, i.e., QFT, to applications, i.e., Grover's algorithm.
4. Extendability: MQT Bench is easily extendable with respect to available benchmarks, native gate-sets, and hardware architectures.

Benchmarking quantum software tools and compilation flows with the same benchmarks aims to aid comparability, reproducibility, and transparency of empirical evaluations. How this leads to more consistent testing is shown in the following example.

Example 5. Consider a full quantum compilation flow across all four levels which shall be developed and tested as depicted in Fig. 2. On the left-hand side of the levels, the current state of the art of benchmarking is sketched. To comprehensively test each level, different benchmark suites have to be adapted and used. In contrast, MQT Bench replaces all of those and enables testing software tools with the same benchmarks throughout the whole quantum circuit compilation flow as shown on the right-hand side.

MQT Bench

[Benchmarks](#)
[Level and File Descriptions](#)
[Benchmark Description](#)
[More on our Work](#)
[Legal Information](#)

Algorithm Level Currently, only available using the Python package. Details can be found on our GitHub page .	Target-independent Level Select the used compiler: ☐ Qiskit ☐ TKET	Target-dependent Native Gates Level Select targeted native gate-set: ☐ IBM ☐ Rigetti ☐ OQC ☐ IonQ ☐ Quantinuum Select the used compiler: ☐ Qiskit ☐ TKET If Qiskit is selected, select its optimization level: ☐ Opt. 0 ☐ Opt. 1 ☐ Opt. 2 ☐ Opt. 3	Target-dependent Mapped Level Select a targeted device: ☐ IBM Washington (127 Qubits) ☐ IBM Montreal (27 Qubits) ☐ Rigetti Aspen-M2 (80 Qubits) ☐ OQC Lucy (8 Qubits) ☐ IonQ Harmony (11 Qubits) ☐ IonQ Aria 1 (25 Qubits) ☐ Quantinuum H2 (32 Qubits) Select the used compiler: ☐ Qiskit ☐ TKET If Qiskit is selected, select its optimization level: ☐ Opt. 0 ☐ Opt. 1 ☐ Opt. 2 ☐ Opt. 3 If TKET is selected, select its placement settings: ☐ Graph Placement ☐ Line Placement
---	---	--	---

Download
 Number of selected benchmarks: 0
 After the download button is clicked, all benchmarks provided as .qasm files are downloaded as a .zip archive. Details of the [file format](#) are provided. Alternatively, a pre-generated archive with all benchmarks can be [downloaded](#).

Download selected Benchmarks

Figure 3: User interface of the MQT Bench website.

4 MQT Bench

In this section, we describe in more detail how each of the four core traits mentioned above is implemented.

4.1 Cross-Level Benchmarking

As exemplarily illustrated in [Section 2](#), software development in quantum computing takes place on various levels. While there arguably are numerous possible levels, MQT Bench focuses on four as a balanced measure between specificity and generizability—inspired by the structure proposed in the openQASM 3.0 specification [58]. Providing these benchmarks on all of the four abstraction levels is the main contribution of this work. More precisely, the following levels are covered:

1. *Algorithmic level*: In this format, all kinds of quantum gates may be used and subsumed into high-level building blocks. Loops are possible and the circuit structure might be described by constants, variables, and mathematical expressions. This level provides benchmarks on the most generic level and specific values are assigned to its variables, e.g., the number of iterations of a loop. The number of qubits can be specified.
2. *Target-independent level*: Here, loop unrolling, constant folding and propagation are conducted. Also, naive gate simplification is enforced. Again, the number of qubits can be adapted through the website and the used compiler can be selected.
3. *Target-dependent native gates level*: On this level, a particular native gate-set is chosen and the circuit is transpiled and optimized accordingly. Here, the native gate-set, the compiler used and its settings can be specified.
4. *Target-dependent mapped level*: Finally, a dedicated architecture layout is chosen and the circuit is mapped to the targeted device such that it satisfies all its connectivity constraints and becomes executable on the device. Similar to the previous level, the used compiler and its settings are selectable—in addition to the target device.

All relevant information of a generated benchmark is denoted in short in the file name and in detail within the file itself.

4.2 Accessibility

MQT Bench strives to meet the needs of many. To accomplish this goal, it has to be as user-friendly as possible. On the one hand, this involves the benchmark library itself. All of the high-level, algorithmic benchmark scripts and routines to generate representations for the individual levels are publicly available on GitHub (<https://github.com/cda-tum/mqt-bench>). This allows for complete transparency on how the respective circuits have been generated. Additionally, a Python package is provided such that each benchmark can be generated on-demand.

On the other hand, and most importantly, this involves the way in which users interface with the benchmark suite. While most benchmark suites are provided as an accumulation of benchmark files—making it hard to extract the desired set of benchmarks, MQT Bench includes an easy-to-use, no-coding-required web interface (<https://www.cda.cit.tum.de/mqtbench>) that provides the means to filter the large number of pre-generated benchmarks according to the specific needs of the user. This web interface and its underlying server software is also part of our Python package, such that every user can utilize the same interface locally without the need to access our publicly available server. A screenshot of the website’s user interface and its configuration options is shown in Fig. 3.

4.3 Algorithm Selection

To provide a broad spectrum of different benchmarks, MQT Bench comprises most of today’s de-facto standard quantum algorithms. This includes building blocks, e.g., Quantum Fourier Transform (QFT) and the Greenberger-Horne-Zeilinger state preparation (GHZ), up to higher-level algorithms, e.g., Grover’s [67] and Shor’s [66] algorithm. At the time of writing, MQT Bench comprises the following benchmarks:

- Amplitude Estimation (AE)
- Deutsch-Jozsa (DJ) algorithm
- Graph state preparation
- GHZ state preparation
- Grover’s algorithm
- Quantum Approximation Optimization Algorithm (QAOA)
- Quantum Fourier Transformation (QFT)
- Quantum Phase Estimation (QPE)
- Quantum Walk
- Random circuit
- Shor’s algorithm
- Variational Quantum Eigensolver (VQE)
- Three VQA ansatz functions with randomly initialized parameters: Two Local, Real Amplitudes and Efficient SU2
- W-State preparation

Additionally, MQT Bench provides several application benchmarks specifically targeting variational quantum algorithms, since those algorithms are especially promising in the NISQ-era [68]. To this end, we use the classification provided by IBM Qiskit’s application levels: Optimization, Machine Learning, Finance, and Nature:

- *Optimization*: Travelling salesman problem and vehicle routing
- *Machine Learning*: Quantum Neural Network (QNN)
- *Finance*: Portfolio optimization and option pricing
- *Nature*: Ground state estimation

4.4 Extendability

Quantum computing is a rapidly evolving area of research and, especially in recent years, numerous promising algorithms and applications have emerged. Any benchmark suite has to be extendable and must continuously evolve in order to remain relevant. MQT Bench is designed to be easily extendable in a multitude of ways:

- *Algorithms*: New applications and algorithms can be easily integrated into MQT Bench by providing the corresponding algorithmic level description. The generation of all other levels and options (such as the native gate-sets, architectures, and compilers with their settings) is automatically handled by the proposed library.
- *Native gate-sets*: So far, native gate-sets of superconducting quantum computers from IBM, Rigetti, and Oxford Quantum Circuits are provided—in addition to the gate-sets of IonQ’s and Quantinuum’s ion trap-based quantum computers. In the future, additional gate-sets (such as those of Google’s and or AQT’s quantum computers) can be realized by adopting the necessary compilation routines in the proposed library.
- *Hardware architectures*: All major players currently rely on a modular platform for the architecture of their devices in order to further scale their quantum computers. It is straightforward to extend MQT Bench’s list of available architectures to accommodate future architectures.

Although the four levels MQT Bench is based on cover today’s main use cases, the number of abstraction levels most likely is going to increase in the future. While the mapped level has been considered the last one before a circuit is sent to the quantum computer for execution, pulse-level programming is envisioned to give even more control to software developers as proposed in [69]. On the other end of the spectrum, higher-level abstractions and programming languages will be required to foster the adoption of quantum technology. Currently, a programmatic description of the benchmarks is needed on the algorithmic level. An even higher level where this programmatic description will be automatically derived from is also envisioned. *QUARK* [70], a framework for quantum computing application benchmarking consisting of four levels where the lowest level considers the whole compilation flow discussed in Section 2.1, constitutes one step towards this direction.

5 Evaluation

At the time of writing, MQT Bench (*v1.0.0*) considers two compilers, five native gate-sets, and seven devices ranging from 8 to 127 qubits. In this section, we provide an overview of several characteristics and statistics of the resulting pre-generated benchmarks. To this end, the following metrics are illustrated in Fig. 4:

Number of qubits: A first broad overview is given in Fig. 4a, which shows the relative frequency of the generated benchmarks per number of qubits and per compiler. The relative frequency of benchmarks is decreasing with an increasing number of qubits due to fewer devices being available for higher qubit numbers and the maximal generation time for a benchmark file being exceeded more frequently. Furthermore, the benchmark generation using the *TKET* compiler generally took more time than *Qiskit*—leading to a smaller number of generated benchmarks, especially on the target-dependent mapped level.

Distribution of target-dependent mapped level benchmarks: The distribution of the target device for the mapped level is shown in Fig. 4b, with the number of qubits denoted in brackets. As expected, the larger the device, the more benchmarks are created for it. On the target-dependent native gates level, there is an equal distribution with respect to the number of benchmarks created for each of the five native gate-sets.

Distribution of benchmark characteristics: In Fig. 4c, six different characteristics with values between 0 and 1 are evaluated for all pre-generated benchmarks. Five of those characteristics (Program Communication, Critical Depth, Entanglement Ratio, Liveness, and Parallelism) have been proposed in [55]. Furthermore, the percentage of multi-qubit gates is evaluated.

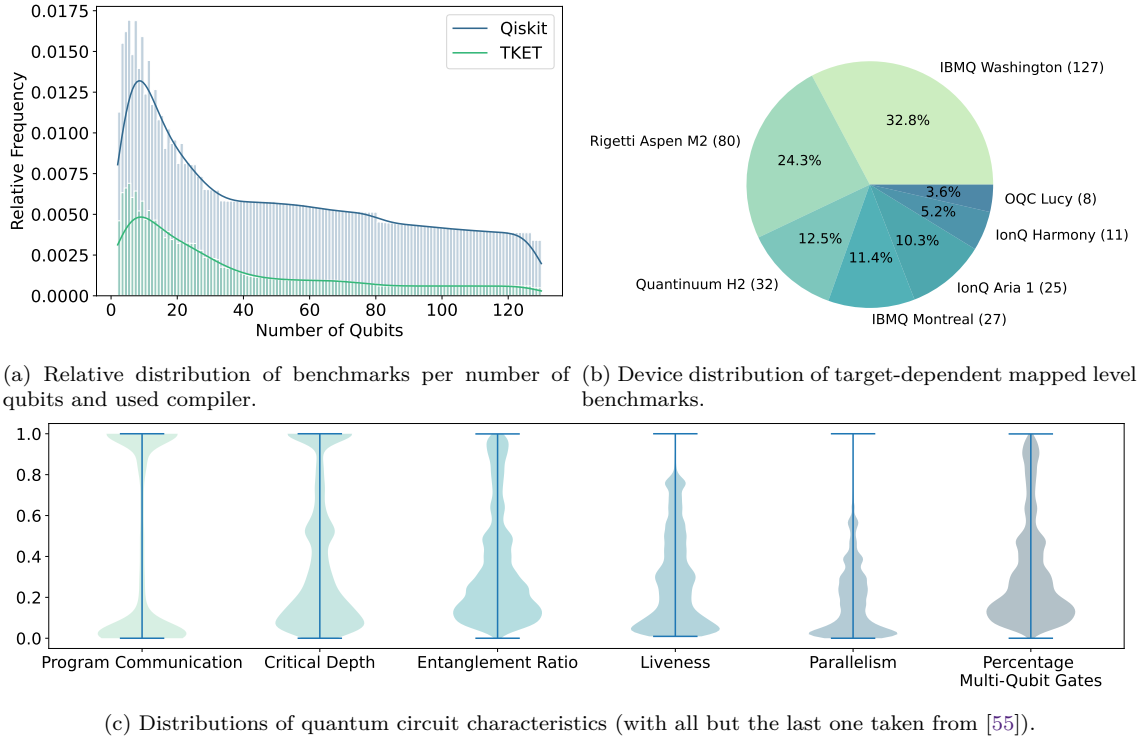


Figure 4: Insights into the provided benchmarks and their characteristics.

6 Conclusion

In this work, we proposed *MQT Bench*—a quantum circuit benchmark suite (as part of the *Munich Quantum Toolkit*, MQT) comprising different algorithms, compilers, native gate-sets, and target devices—resulting in more than 70,000 benchmark circuits ranging from 2 to 130 qubits on four abstraction levels. To keep this large number of benchmarks manageable, we provide an easy-to-use web interface (<https://www.cda.cit.tum.de/mqtbench>) allowing users to filter the benchmarks according to their needs. *MQT Bench* is also provided as a Python package (including the server software to start the web interface locally), such that each of the benchmarks can be easily generated on-demand. Furthermore, we give access to the open-source repository on GitHub (<https://github.com/cda-tum/mqt-bench>). By this, *MQT Bench* presents a first step towards serving a single benchmark suite for the whole quantum software stack—facilitating empirical evaluations of quantum software tools that are comparable, reproducible, and transparent.

Acknowledgments

This work received funding from the European Research Council (ERC) under the European Union’s Horizon 2020 research and innovation program (grant agreement No. 101001318), was part of the Munich Quantum Valley, which is supported by the Bavarian state government with funds from the Hightech Agenda Bayern Plus, and has been supported by the BMWK on the basis of a decision by the German Bundestag through project QuaST, as well as by the BMK, BMDW, and the State of Upper Austria in the frame of the COMET program (managed by the FFG).



Publicly Available Quantum Circuit Datasets for ML Training

Optimizing quantum circuits (minimizing gate counts and depth) benefits from rich datasets of circuit designs. Below is a comprehensive list of datasets and benchmark suites that provide quantum circuits (in OpenQASM 2.0/3.0 or other formats) for various real-world algorithms (Shor's, Grover's, VQE, etc.) and applications. Each entry describes the dataset's content, format, and how to access it.

QASMBench (OpenQASM Benchmark Suite)

QASMBench is a low-level **OpenQASM 2.0** benchmark suite consolidating many common quantum algorithm circuits. It includes routines from domains like quantum chemistry, simulation, linear algebra, search/optimization, arithmetic, machine learning, cryptography, and more ¹. The suite provides a diverse set of **real-use circuits** (e.g. Grover's algorithm, QAOA, QFT, quantum arithmetic circuits) with qubit counts ranging from very small (2 qubits) to quite large (over 100 qubits in some cases) ². All circuits are given as `.qasm` files that can be executed on quantum simulators or real devices (IBM Quantum Experience) without modification ³.

- **Content:** Dozens of quantum circuits covering **small-scale (2–10 qubits)**, **medium-scale (~10–27 qubits)**, and **large-scale (28–400+ qubits)** benchmarks. These include well-known algorithms (e.g. Bernstein–Vazirani, Grover's search, Quantum Fourier Transform), quantum chemistry ansatz circuits, arithmetic modules (adders, multipliers), error-correcting code circuits, etc. ⁴ ⁵. The circuits span many categories of algorithms, ensuring a broad training set for optimization tasks.
- **Format:** Provided in **OpenQASM 2** format (`.qasm` assembly files). This low-level representation lists quantum gates and measurements, making it suitable for gate count and depth analysis ⁴.
- **Access:** **Open-source on GitHub** in the *pnnl/QASMBench* repository ⁶. The repository contains the `.qasm` files organized by category and a research paper with details (Li *et al.*, TQC 2022). You can clone the repo or download the files directly to obtain the full benchmark suite.

MQTBench (Munich Quantum Toolkit Benchmark Library)

MQTBench is a **large-scale quantum circuit benchmark suite** that provides **around 70,000 circuits** ranging from 2 up to 130 qubits ⁷. Uniquely, it offers circuits at multiple abstraction levels – from high-level algorithm descriptions down to hardware-specific gate implementations – which can be useful for training ML models on different representations of quantum circuits. MQTBench includes a wide variety of **algorithmic circuits**: for example, it features Grover's search, Quantum Fourier Transform (QFT), Quantum Phase Estimation (QPE), Amplitude Estimation, quantum walks, and variational algorithm circuits (like QAOA and VQE ansatz) among others ⁸ ⁹. Notably, **Shor's algorithm** circuit is included as a fixed-size

benchmark in this suite ⁹, alongside other application-oriented examples (e.g. routing problems, portfolio optimization).

- **Content:** ~1,938 **base benchmark algorithms** (when considering unique algorithm types) instantiated across various sizes, totaling ~70k circuits ⁷. Scalable benchmarks (Grover, QFT, QAOA, VQE, etc.) can be generated for arbitrary qubit counts, and non-scalable ones like **Shor's algorithm** or quantum chemistry ground state circuits are provided at specific sizes ⁸ ¹⁰. This breadth covers most standard quantum computing use-cases, making the dataset highly representative.
- **Format:** Circuits are available as **OpenQASM 2.0** `.qasm` files. Users can select circuits via a web interface and download a ZIP file of the chosen benchmarks; the downloaded circuits are in QASM format (with optional metadata) ¹¹. The suite can also be accessed programmatically (e.g., via a PennyLane dataset integration ¹² ¹³).
- **Access:** MQTBench is accessible through its **official website** (TUM's Chair for Design Automation) which allows custom selection and bulk download of circuits ¹⁴. A pre-generated archive of *all* benchmarks is available for direct download ¹⁵. Additionally, the project's GitHub page provides tools and the Python package for generating and retrieving benchmarks ¹⁶ ¹⁷.

VeriQBench (Benchmark for Multiple Circuit Types)

VeriQBench is an open-source benchmark suite that covers **multiple types of quantum circuits**. It provides high-level circuit designs in four categories: **combinational** quantum circuits, **dynamic** circuits, **sequential** circuits, and **variational** (parameterized) circuits ¹⁸. For each type, VeriQBench includes common algorithms and tasks used in quantum computing, often with circuits at three scales – small (<20 qubits), medium (20–50 qubits), and large (>50 qubits) – to accommodate different use cases ¹⁸. All circuits are distributed as **OpenQASM** files, and scripts are provided to generate or scale circuits to arbitrary sizes if needed ¹⁹.

- **Content:** A broad set of **real-world quantum algorithms and applications**. Under *combinational* circuits, VeriQBench contains basic algorithm circuits like Bernstein–Vazirani, QFT, QPE, and reversible logic circuits (e.g. arithmetic modules, Clifford circuits) ²⁰ ²¹. The *dynamic* circuits include examples with mid-circuit measurements or classically controlled operations (useful for testing quantum control flow). *Sequential* circuits cover circuits that may incorporate loops or iterative patterns (e.g. iterative phase estimation, or algorithms split into stages). The *variational* circuits section provides templates for popular variational algorithms including **VQE (Variational Quantum Eigensolver) ansätze**, **QAOA (Quantum Approximate Optimization Algorithm)**, and other parametrized circuits like EfficientSU2 ²². This means the dataset directly features circuits used in quantum chemistry (via VQE for molecule Hamiltonians) and optimization problems.
- **Format:** All circuits are given in **OpenQASM 2.0** (`.qasm`) format ²³. The high-level abstractions (like oracle descriptions or algorithm structure) are preserved in the QASM files, which makes them readable and editable. Each circuit can be loaded into frameworks like Qiskit or run on quantum hardware supporting OpenQASM.
- **Access:** Available on **GitHub** under the *Veri-Q/Benchmark* repository ²⁴. The repository is organized by circuit type (combinational, dynamic, sequential, variational), each containing subfolders with the QASM files for various algorithms and scales. Users can clone the repository or download specific `.qasm` files. (VeriQBench was introduced by Chen *et al.* 2022, and the GitHub is openly accessible.)

NWQBench (NISQ Workload Benchmark Generators)

NWQBench is a collection of **parametric quantum circuit generators** for Near-Term Quantum (NISQ) algorithms, developed by PNNL. Unlike a static dataset, NWQBench provides Python scripts that can **generate circuits on demand** for a variety of algorithms, allowing the user to specify the number of qubits for each generated circuit ²⁵. It covers many common quantum routines—each in its own sub-directory (for example, `Adder`, `QFT`, `Wstate`, `Grover`, etc., each with a generator script) ²⁶. NWQBench can output circuits in **multiple representations**: by default it produces an OpenQASM description of the circuit, and it also includes tools to convert these into other frameworks such as Qiskit, Cirq, Q#, or PyQuil formats ²⁷ ²⁸.

- **Content:** A **large corpus of quantum benchmark routines** covering algorithms in domains like arithmetic (adders, multipliers), communication (W-state preparation), verification (quantum oracle circuits), and more ²⁷. Each generator can produce scalable circuits – e.g., a `QuantumAdder.py` script can create an n-qubit ripple-carry adder circuit of adjustable size, or a `Grover.py` can generate Grover’s algorithm circuit for a database of a given size. This allows creation of **randomly parameterized or scaled instances** of known algorithms. While not a fixed set of pre-made circuits, NWQBench is valuable for producing training data because you can generate arbitrarily many circuit instances (including random variations) for a given algorithm template.
- **Format:** **OpenQASM 2.0** is the primary output format. Running a generator (e.g. `python BenchmarkName.py N`) yields a `.qasm` file describing the circuit for N qubits ²⁹. A conversion utility is provided to translate the QASM into other language-specific circuit formats if needed ²⁸. This flexibility means the dataset can be tailored to the format your ML pipeline requires (QASM, or converted to Python objects, etc.).
- **Access:** The generators are available on **GitHub** in the *pnnl/NWQBench* repository ³⁰. After installing the required packages, users can run the provided scripts to produce circuit files. The repository’s README and example scripts show how to generate and convert circuits. (As NWQBench is code-based, there isn’t a direct “download all circuits” link – instead, one clones the repo and runs scripts to get the desired circuits.)

VQE-Generated Circuits Dataset (Qulacs-Osaka)

This is a **specialized dataset of quantum circuits produced by a Variational Quantum Eigensolver (VQE)** process. Created by researchers at Qulacs-Osaka (Nakayama *et al.*, 2023), the dataset contains thousands of **optimized ansatz circuits** for finding ground-state energies of various Hamiltonians ³¹ ³². The idea is to use these circuits for machine learning tasks like circuit classification or clustering, reflecting different problem instances solved by VQE. The circuits are grouped by qubit count (from 4 up to 20 qubits) and by the type of Hamiltonian problem (e.g., different models in quantum many-body physics).

- **Content:** Over **8,700 quantum circuits** resulting from VQE optimizations. Specifically, the dataset includes circuits for **5 problem Hamiltonians** (e.g., 1D transverse-field Ising model, 1D Heisenberg model, etc.) labeled 0–4 ³². For each Hamiltonian, circuits were generated and optimized on different system sizes: **4 qubits (1,500 circuits)**, and **8, 12, 16, 20 qubits (1,800 circuits each)** ³³. Each circuit is an ansatz (variational form) that was optimized to approximate the ground state of the given Hamiltonian. These are not random circuits; they are **purposeful circuits from quantum chemistry/physics applications** (finding ground energies) and thus serve as realistic test cases for optimization algorithms.

- **Format:** Provided in **OpenQASM 2.0** format under the `data/qasm` directory ³⁴. Each circuit is a `.qasm` file that can be loaded into quantum frameworks (the dataset also provides the corresponding Hamiltonian's ground state data for reference, though that may not be needed for circuit optimization tasks). The circuits typically consist of parameterized gates (with optimized parameters) arranged in a layered ansatz structure.
- **Access:** The dataset is openly available on **GitHub** at the repository *Qulacs-Osaka/VQE-generated-dataset* ³⁵ ³⁴. To obtain the data, you can clone the repository; the circuit files are located in subfolders named `04qubit`, `08qubit`, ..., `20qubit` inside the `data/qasm` folder ³⁶. No special request is needed. The repository also includes usage examples and visualization tools to explore the circuits.

RevLib (Reversible and Quantum Circuit Library)

RevLib is an online repository of **reversible and quantum circuits** that has long been used in quantum computing research (particularly for logic synthesis and optimization tasks). It hosts a large database of *reversible functions* (logic specifications) along with known **circuit realizations** of those functions ³⁷. Many circuits in RevLib are essentially building blocks for quantum algorithms – for example, arithmetic circuits (adders, multipliers, ALUs) which appear in Shor's algorithm, or oracles for Grover's algorithm can be derived from reversible logic functions. While some circuits are purely classical reversible networks (comprised of Toffoli, Fredkin, etc.), they can directly be treated as quantum circuits implemented with reversible gates.

- **Content: Hundreds of circuit designs** across various categories. RevLib's library includes over **150 reversible functions** and about **454 circuit implementations** in total ³⁸ ³⁹. Key categories relevant to quantum circuit optimization include: **Arithmetic circuits** (e.g. ripple-carry adders, multipliers – 44 examples ⁴⁰), **ALU and arithmetic logic units**, *encoding functions*, *symmetrical functions*, and **"quantum gates" category** which includes small-scale quantum gate networks. These circuits often serve as benchmarks for gate count optimization. For instance, the modular exponentiation component of Shor's algorithm (a large arithmetic circuit) can be constructed from the adder and multiplier circuits in RevLib. Likewise, Grover oracles for specific Boolean functions can be drawn from RevLib's reversible functions.
- **Format:** RevLib uses its own text-based formats for reversible circuits (such as *.real*, *.pla*, *.bench* files, depending on the circuit). The **RevLib format 2.0** is supported by tools like RevKit ⁴¹. In essence, these formats list gates (Toffoli, Fredkin, NOT, etc.) or truth tables for the functions. Many circuits are also given as **quantum gate level** descriptions that can be translated into equivalent OpenQASM or Circuit objects using conversion tools. (For example, a Toffoli gate is a standard quantum gate, so a sequence of Toffolis from a `.real` file directly corresponds to a quantum circuit.)
- **Access:** RevLib is publicly accessible through its **website** ³⁷. The site organizes circuits by category; users can browse and download individual circuit files. No signup is required – simply click on a function or circuit name to see its description and download links. (RevLib is a reference library, so it might not offer a bulk download of all circuits at once; you may download needed circuits one by one. Researchers sometimes also share a compiled archive upon request, but for most purposes the website is sufficient for access.)

QCircuitNet Dataset (Hierarchical Quantum Algorithm Dataset)

QCircuitNet is a newly introduced (2024) large-scale dataset specifically designed for AI model training in quantum circuit design ⁴². It provides a structured collection of quantum algorithm circuits, organized in a hierarchical manner by task type. The dataset covers a **wide range of quantum algorithms**, from basic building blocks (primitives) to advanced application-level circuits ⁴³ ⁴⁴. Unlike earlier benchmarks, QCircuitNet pairs each circuit with additional context like problem descriptions and verification data, making it especially suited for machine learning and reinforcement learning tasks that involve generating or optimizing circuits.

- **Content:** Thousands of circuits sorted into three main categories: **Oracle Construction**, **Algorithm Design**, and **Random Circuits** ⁴⁵. In the *Oracle Construction* set, you'll find circuits for Boolean oracle functions and problem encodings (including textbook oracles and more complex oracles for applications) – useful for Grover's algorithm and related tasks. The *Algorithm Design* category includes full implementations of many algorithms in quantum computing and quantum information (e.g., algorithms like Bernstein–Vazirani, Simon's algorithm, quantum key distribution protocols, teleportation, GHZ state preparation, etc., as well as advanced protocols) with circuits provided for various input sizes. For example, one subfolder contains **Bernstein–Vazirani circuits for n up to 133 qubits**, another has **GHZ state circuits up to 133 qubits** ⁴⁶, and so on (only a subset of sizes are kept in the public release for manageability, typically 10 sample circuits per algorithm size range). The *Random Circuits* section provides random Clifford and universal circuits at different scales for completeness. Overall, QCircuitNet spans everything from small 2–3 qubit puzzles to circuits over 100 qubits, giving a very comprehensive training set.
- **Format:** Circuits are given in **OpenQASM 3.0** format (each file starts with the `OPENQASM 3.0;` `include "stdgates.inc";` header) ⁴⁷. Alongside the QASM code, supplementary files (like `.txt` outputs or oracle truth tables) are provided for certain tasks to facilitate verification ⁴⁸ ⁴⁹. The use of OpenQASM 3 means the dataset can express advanced features (like timing or classical control) if needed, though most circuits are standard gate sequences. File naming conventions (e.g. `algorithmname_n<qubits>.qasm`) and directory structure are well-documented ⁵⁰ ⁵¹. This modern format ensures compatibility with current quantum software tools (many of which now support OpenQASM 3).
- **Access:** The dataset is **publicly available on GitHub** (see repository *EstelYang/QCircuitNet_Dataset*) ⁵². The repository contains the organized directories for each category and sub-task, with sample circuit files included as described. You can clone the repo to get all the data. (If a full release archive or additional data is needed, the accompanying research paper and Readme provide instructions; at present the GitHub is openly accessible without special requests.) QCircuitNet is a cutting-edge resource intended to stimulate AI research in quantum circuit optimization and generation.

Each of these datasets provides valuable **training data for ML or RL algorithms** focused on circuit optimization. By leveraging circuits from well-known quantum algorithms (Shor, Grover, QPE, VQE, etc.) and a variety of application domains, researchers can ensure their models learn to optimize not just random circuits but those with real structure and importance. All the above sources are publicly accessible (some via direct download, others via open repositories or web portals), making it convenient to gather a wide range of quantum circuits for your optimization tasks.

1 3 4 5 6 GitHub - pnnl/QASMBench: A low-level OpenQASM benchmark suite for NISQ evaluation and simulation. Please see our paper for details.

<https://github.com/pnnl/QASMBench>

2 QCCircuitNet: A Large-Scale Hierarchical Dataset for Quantum Algorithm Design

<https://arxiv.org/html/2410.07961v1>

7 8 9 10 11 12 13 14 15 16 17 MQT Bench

<https://www.cda.cit.tum.de/mqtbench/>

18 19 20 21 22 23 24 GitHub - Veri-Q/Benchmark

<https://github.com/Veri-Q/Benchmark>

25 26 27 28 29 30 GitHub - pnnl/nwqbench

<https://github.com/pnnl/nwqbench>

31 32 33 34 35 36 GitHub - Qulacs-Osaka/VQE-generated-dataset: Dataset for quantum machine learning, generated by the VQE algorithm.

<https://github.com/Qulacs-Osaka/VQE-generated-dataset>

37 38 39 40 41 RevLib - An Online resource for Reversible Benchmarks

<https://www.revlib.org/>

42 43 44 45 46 47 48 49 50 51 52 GitHub - EstelYang/QCircuitNet_Dataset

https://github.com/estelyang/qcircuitnet_dataset

QASMBench: A Low-Level Quantum Benchmark Suite for NISQ Evaluation and Simulation

ANG LI, SAMUEL STEIN, SRIRAM KRISHNAMOORTHY, and JAMES ANG, Pacific Northwest National Laboratory, USA

The rapid development of quantum computing (QC) in the NISQ era urgently demands a low-level benchmark suite and insightful evaluation metrics for characterizing the properties of prototype NISQ devices, the efficiency of QC programming compilers, schedulers and assemblers, and the capability of quantum system simulators in a classical computer. In this work, we fill this gap by proposing a low-level, easy-to-use benchmark suite called QASMBench based on the OpenQASM assembly representation. It consolidates commonly used quantum routines and kernels from a variety of domains including chemistry, simulation, linear algebra, searching, optimization, arithmetic, machine learning, fault tolerance, cryptography, etc., trading-off between generality and usability. To analyze these kernels in terms of NISQ device execution, in addition to circuit width and depth, we propose four circuit metrics including gate density, retention lifespan, measurement density, and entanglement variance, to extract more insights about the execution efficiency, the susceptibility to NISQ error, and the potential gain from machine-specific optimizations. Applications in QASMBench can be launched and verified on several NISQ platforms, including IBM-Q, Rigetti, IonQ and Quantinuum. For evaluation, we measure the execution fidelity of a subset of QASMBench applications on 12 IBM-Q machines through density matrix state tomography, which comprises 25K circuit evaluations. We also compare the fidelity of executions among the IBM-Q machines, the IonQ QPU and the Rigetti Aspen M-1 system. QASMBench is released at: <http://github.com/pnnl/QASMBench>.

CCS Concepts: • **Computer systems organization** → **Quantum computing**; • **Hardware** → **Quantum computation**; • **Software and its engineering** → **Software libraries and repositories**.

Additional Key Words and Phrases: Benchmark, OpenQASM, quantum metrics, NISQ

ACM Reference Format:

Ang Li, Samuel Stein, Sriram Krishnamoorthy, and James Ang. 2020. QASMBench: A Low-Level Quantum Benchmark Suite for NISQ Evaluation and Simulation. *ACM Trans. Quantum Comput.* 37, 4, Article 111 (August 2020), 26 pages. <https://doi.org/10.1145/1122445.1122456>

Quantum computing (QC) [9, 94] has been envisioned as one of the most promising computing paradigms beyond Moore’s Law for tackling difficult computing challenges that are classically intractable arising from various domains like chemistry [46, 66], machine learning [13, 109], cryptography [47, 112], linear algebra [22, 55], finance [105, 129], recommendation systems [68], physics simulation [32, 40], networking [69, 90], and so on. QC relies on fundamental quantum mechanisms such as superposition, entanglement, interference, tunneling, etc. for performing computation, in hopes of significant or even exponential speedups over existing algorithms in classical computers [24, 112].

Authors’ address: Ang Li, ang.li@pnnl.gov; Samuel Stein, samuel.stein@pnnl.gov; Sriram Krishnamoorthy, sriram@pnnl.gov; James Ang, ang@pnnl.gov, Pacific Northwest National Laboratory, 902 Battelle Blvd, Richland, WA, USA, 99354.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2020 Association for Computing Machinery.

2643-6817/2020/8-ART111 \$15.00

<https://doi.org/10.1145/1122445.1122456>

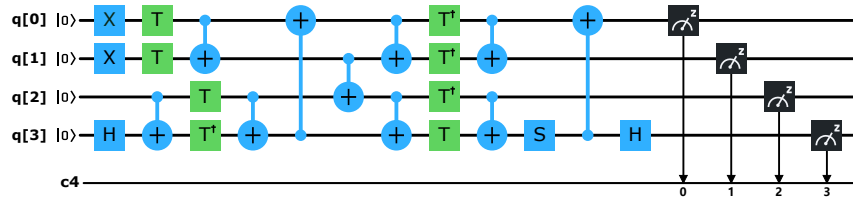


Fig. 1. A quantum adder circuit with width=4 (4 qubits) and depth=23 (23 gates) followed by measurement.

Despite holding great promise, QC in contemporary noisy-intermediate-scale-quantum (NISQ) [101] devices is still distant from outperforming classical computers regarding general problems. NISQ devices describe the near-term quantum platforms comprising 50 to hundreds of qubits, which is inadequate for full-scale error-correction but can already present promising results in a number of problems, and lay the foundations towards practical quantum demonstration [101]. The most recent public accessible NISQ device is IBM-Q Washington featuring 127 physical qubits.

The QC landscape is evolving rapidly nowadays. From the software perspective, large number of QC software are developed in classical programming languages (e.g., Python [49, 115], C/C++ [63], JavaScript [58], ML [3]). A list of opensource QC projects, quantum algorithms, and quantum simulators can be found in [41], [65], and [103], respectively. From the hardware perspective, the development of QC testbeds have already proceeded to a stage where small working prototypes are available through cloud service, such as IBM-Q [57], Microsoft Azure [88], Amazon Braket [48], and Rigetti [106], showing encouraging results [20, 35]. Finally, from the application perspective, Google has announced the demonstration of quantum supremacy on the task of sampling the output of a pseudo-random quantum circuit, using a 53-qubit NISQ system [5]. Several quantum machine learning frameworks (e.g., [16, 58]) and chemistry simulation packages have been released as well (e.g., [58, 83]).

To close the gap between practical quantum applications and real quantum machines, contributions from the computer system and architecture community are undoubtedly critical in tackling several key challenges [19], including software and hardware verification [120], defining and perforating abstraction boundaries [92], managing parallelism and communication [75], mapping and scheduling operations [110], elevating control complexity [76], hardware-specific optimizations [91, 114, 121], investigating and mitigating noise [93, 119], etc. These massive effort, however, are currently evaluated and verified using randomly selected QC benchmarks [30, 75, 76, 91, 93, 110, 119–121], lacking the common ground for judicious analysis and fair comparison among each other. The communities thus urgently desire a low-level, easy-to-use QC benchmark suite to foster the software-hardware codesign effort for the purpose of validation and verification. This is particularly vital in the NISQ era, given the noise and technology limitation continuously to be the major challenges in the near future.

In this work, we propose a low-level benchmark suite called QASMBench based on the OpenQASM assembly-level intermediate representation (IR) [27]. It collects commonly used quantum algorithms and routines (e.g., the adder circuit in Figure 1) from a variety of distinct domains, including quantum chemistry, simulation, linear algebra, searching, optimization, arithmetic, machine learning, fault tolerance, cryptography, etc. The design of the benchmark suite trades-off between generality and usability, covering a wide spectrum regarding circuit depth (i.e., number of gates) and width (i.e., number of qubits). Additionally, to analyze and compare the benchmark applications, we propose four circuit evaluation metrics including gate density, retention lifespan, measurement density, and entanglement variance, to offer deeper insights about the execution efficiency, the

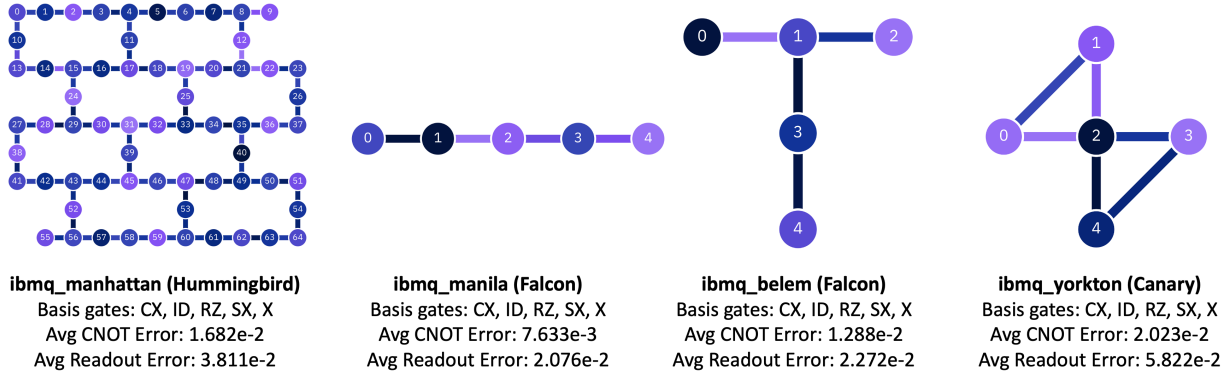


Fig. 2. IBM-Q device topology. The color of nodes implies frequency (GHz) of the qubit or how fast a 1-qubit gate can be executed. The connection color implies the gate time in nanoseconds for 2-qubit gates such as CX.

susceptibility to system/readout error, and the potential impact from device-specific optimizations, when mapping to a NISQ device. Circuits from QASMBench can be directly uploaded and evaluated in IBM-Q machines as well as other platforms (e.g., Rigetti through the Qiskit interface, IonQ and Quantinuum through Microsoft Azure Quantum [88]). Additionally, opensource tools such as *q-convert* and *QCOR* allow the translation of QASMBench to other QC programming languages or representations.

We evaluate QASMBench on 12 IBM-Q machines, and show the fidelity of circuit executions. The fidelity values are measured by calculating the Hellinger distance between the reconstructed density matrix of the resultant mixed state in a real quantum device through density matrix tomography, and the density matrix of the pure state by running the same circuit in a noiseless classical simulator. We observe that shallow circuits in general exhibit higher fidelity than deep circuits, likely due to less decay impact from a shorter execution time; whereas large quantum machines, such as the 127-qubit IBM-Q Washington, show reduced fidelity than small machines, despite using their best performing qubits. We also compare the fidelity of 4 QASMBench circuits among IBM-Q superconducting machines, IonQ’s QPU trapped-ion machine, and Rigetti’s Aspen M-1 superconducting machine, deriving interesting observations. Explanations are provided using the evaluation metrics proposed. Through such evaluation, we show how QASMBench and the 6 metrics can be used for evaluating contemporary and emerging NISQ platforms.

This paper is organized as follows: In Section 1, we briefly describe NISQ and the landscape of OpenQASM. In Section 2, we introduce the QASMBench applications. In Section 3, we propose our characterization metrics. We characterize the QASMBench applications and evaluate them on IBM-Q and other NISQ devices in Section 4. Finally, we summarize and draw the conclusion.

1 BACKGROUND: QUANTUM COMPUTING IN THE NISQ ERA

We describe NISQ devices and the OpenQASM programming environment in this section.

1.1 NISQ Devices

NISQ devices describe the near-term quantum platforms incorporating 50 to less than a thousand qubits [101]. The qubits are currently built using a variety of different technologies, including superconducting qubits [23, 107], trapped-ions qubits [21, 73], spin qubits [81, 100], photonic qubits [6, 95], etc. To correctly run a QC circuit, the physical qubits have to stay coherent long enough for allowing the accomplishment of all the gate operations, thus imposing rigid constraints on allowable circuit depth [25]. The **T1 coherence** time describes the time for a qubit to decay from

Table 1. Basis gates for IBM-Q, Rigetti, IonQ and Quantinuum NISQ devices.

Vendor	Technology	1-qubit basis gates	2-qubit basis gates	Reference
IBM-Q	Superconducting	ID, RZ, SX, X	CX	[57]
Rigetti	Superconducting	RX, RZ	CZ, XY	[106]
IonQ	Trapped-Ion	GPI, GPI2, GZ	MS	[59]
Quantinuum	Trapped-Ion	RX, RZ	ZZ	[87]

the excited state $|1\rangle$ to the ground state $|0\rangle$. The **T2 coherence** time describes the time for a qubit to transit its state due to environment interaction [121]. Consequently, gate operations should be quickly and precisely performed on the qubits during the coherent window for maintaining the quantum states. The probability of involving error during the gate operation is known as **gate error**. The measurement operation, which reads-out classical output from a qubit by collapsing its final quantum state, is also critical for the correctness and efficiency of QC [44]. The **readout error** describes the probability of inaccurate measurement of a qubit state [130]. To precisely sample the output distribution with the existence of gate and read-out error, a quantum circuit is often executed many times (known as **shots** [57] or repetitions [49]).

NISQ devices are mainly featured by two attributes at the system level: **basis gates** and **topology**. The basis gate set, also known as *instruction set architecture* (ISA) in classical computer architecture terminology, is hardware-defined. Upon execution, user quantum circuits are translated into an objective circuit only comprising basis gates. This process is called **quantum transpilation**. The basis gate sets for IBM-Q, Rigetti, IonQ and Quantinuum devices are listed in Table 1.

Additionally, due to technical limitations, it is possible that only certain qubits of a NISQ device are physically connected. In other words, the physical qubits in a NISQ device may follow a certain **topology** (also known as coupling). Figure 2 illustrates the topology of 4 IBM-Q devices. The topology limits the locations where two-qubit gates such as CX can be performed. If a CX is applied over two remote qubits, a series of SWAP gates are needed to relocate the two qubits to a connected tuple in the topology before the CX can be applied. For example, $CX(0, 4)$ in *ibmq_manila* in Figure 2 requires 4 extra SWAP gates. Additionally, more swaps may be needed to switch the qubit(s) back to their original position(s), adding extra overhead and error. **Qubit mapping** describes the process of mapping of the logical qubits of a quantum circuit to the physical qubits of a NISQ device, which significantly impacts QC accuracy and execution efficiency.

1.2 OpenQASM Ecosystem

OpenQASM (*Open Quantum Assembly Language*) [27] is a low-level intermediate representation (IR) of quantum instructions, which is similar to traditional *Hardware Description Language* (HDL) like *Verilog* and *VHDL*. OpenQASM is a unified low-level assembly language for IBM-Q quantum machines. These NISQ devices, being accessible through IBM-Q cloud or IBM-Q network [57], have been explored by many existing works [25, 56, 71, 91–93, 121]. An OpenQASM code can be directly uploaded and verified in an IBM-Q machine or launched through Qiskit.

Table 2 lists the types of gate that are defined in OpenQASM specification (i.e. the "qelib1.inc" header file) [27]. Within these gates, the first five, i.e., U3, U2, U1, CX, and ID, are *basic gates* that are expected to be supported by the quantum backend. From X to RZ are *standard gates* defined atomically in OpenQASM. These standard gates will be converted into basic gates during machine-specific assembling & mapping phase before actual execution. The remaining gates from CZ to C4X are *composition gates* that are constructed by standard gates. These gates are defined in qelib1.inc for the convenience of usage.

Table 2. OpenQASM gate definition (5 basic gates + 11 standard gates + 18 composition gates).

Gates	Meaning	Gates	Meaning	Gates	Meaning
U3	3 parameter 2 pulse 1-qubit	TDG	conjugate of sqrt(S)	CRZ	Controlled RZ rotation
U2	2 parameter 1 pulse 1-qubit	RX	X-axis rotation	CU1	Controlled phase rotation
U1	1 parameter 0 pulse 1-qubit	RY	Y-axis rotation	CU3	Controlled U3
CX	Controlled-NOT	RZ	Z-axis rotation	RXX	2-qubit XX rotation
ID	Idle gate or identity	CZ	Controlled phase	RZZ	2-qubit ZZ rotation
X	Pauli-X bit flip	CY	Controlled Y	RCCX	Relative-phase CXX
Y	Pauli-Y bit and phase flip	SWAP	Swap	RC3X	Relative-phase 3-controlled X
Z	Pauli-Z phase flip	CH	Controlled H	C3X	3-controlled X
H	Hadamard	CCX	Toffoli	C3XSQRTX	3-controlled sqrt(X)
S	sqrt(Z) phase	CSWAP	Fredkin	C4X	4-controlled X
SDG	conjugate of sqrt(Z)	CRX	Controlled RX rotation		
T	sqrt(S) phase	CRY	Controlled RY rotation		

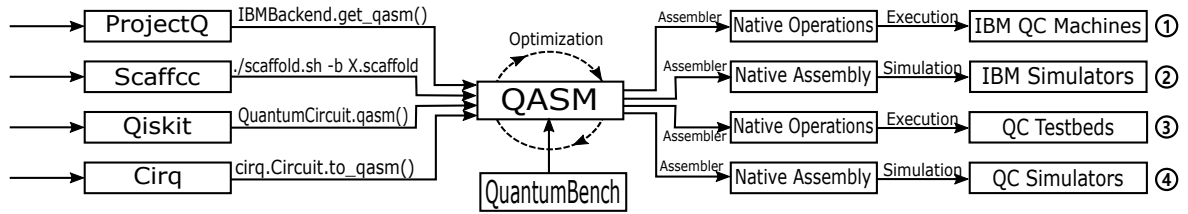


Fig. 3. The OpenQASM toolchain.

OpenQASM is a low-level assembly language. It is executed "sequentially" without any loops, branches or jumps, making it very convenient for static analysis and simulating in a classical simulator. OpenQASM is widely supported. Several popular QC software frameworks such as Qiskit [58], Cirq [49], Scaffold [63], ProjectQ [115] can dump the ready-to-execute circuit into an OpenQASM file, so it can be validated in the IBM-Q backends. Figure 3 shows the landscape of OpenQASM. As can be seen, OpenQASM stands at the conjunction between quantum software and hardware, being critical for both platform-agnostic or platform-specific optimization, mapping, scheduling, evaluation, profiling, and simulation. An OpenQASM based benchmark suite can be useful for all these activities. In the following, we briefly discuss each of the front-end frameworks.

Qiskit: The *Quantum Information Software Kit* (Qiskit) [58] is a quantum software platform developed by IBM. Qiskit is mainly based on Python but is also available in JavaScript and Swift [128]. The package comprises several tools, such as *qiskit-aer* for simulation, *qiskit-ignis* for hardware verification and noise addressing, and *qiskit-aqua* for exemplary applications. At present, Qiskit is probably the most widely used quantum programming language. Both Rigetti [106] and Microsoft Azure [88] (offering IonQ and Quantinuum system access) provide Qiskit interface. OpenQASM can be easily dumped from a Qiskit program using "*QuantumCircuit.qasm()*". Reversely, an OpenQASM circuit file can be loaded by Qiskit through "*QuantumCircuit.from_qasm_str()*".

Cirq: Cirq [49] is a QC software platform developed by Google. It is based on Python. Despite claimed to be the interface for connecting to their 72-qubit Bristlecone quantum computer, this is not yet available to the general users. Cirq incorporates a local simulator for generic gates, and a *Xmon-Simulator* for simulating the native gateset of Google's quantum computers. In addition, Cirq offers a function called "*to_qasm()*" in each circuit object so that a circuit can be dumped as an OpenQASM code that is runnable on IBM-Q backends. Note, based on our experience, not all Cirq code can be translated to OpenQASM.

Scaffold: Scaffold [63] is a quantum programming language embedded in the C/C++ programming language based on the LLVM compiler toolchain [72]. The major goal of Scaffold is to assist

Hidden Subgroup	Search and Optimization	Quantum Simulation	Machine Learning	Application Layer
Quantum Walk	Linear Equation	Quantum Arithmetic	Quantum Communication	
Logical Qubits	Logical Operations	Quantum Error Correction	Logic Simulation	Logical Layer
Physical Qubits	Physical Operations	Gate Error Rates	Physical Simulation	Physical Layer

Fig. 4. Quantum computing stack for the categorization of QASMBench. It is built based on [89] by augmenting the categories of quantum arithmetic, quantum machine learning and quantum communication.

in developing quantum algorithms and advanced optimization, leveraging the existing LLVM compiling flow. By supporting advanced language structures such as loops, Scaffold can generate very complex OpenQASM code, such as the VQE examples in QASMBench. A Scaffold program can be compiled by *Scaffcc* [63] to native OpenQASM code using the `"-b"` compiler option.

ProjectQ: ProjectQ is a quantum software platform developed by Steiger et al. from ETH Zürich [115]. Similar to Scaffold, ProjectQ does not have its own dedicated real quantum backend, but relies on classical simulation. However, it provides a way to generate OpenQASM code so that a ProjectQ program can be verified on an IBM testbed — through `"IBMBBackend.get_qasm()"`.

2 QASMBENCH

We introduce QASMBench in this section. QASMBench is an end-to-end package comprising a diverse variety of benchmark circuits, a system for evaluating the performance of a circuit, and characterizing circuit metrics for interpreting circuit characteristics both pre and post transpilation to real machines. Some of the benchmark routines are generated and reproduced from existing open-source QC software packages, including [27, 49, 58, 63, 83, 86, 108] through the approaches described in Figure 3 whereas others are locally developed.

Depending on the number of qubits used, QASMBench is partitioned into three categories:

- **Small-scale**, with qubits ranging from 2 to 5. The purpose is to allow intensive measures such as density matrix tomography, with limited cost. Also, the present IonQ QPU and most IBM-Q devices with public access feature 5 qubits. The benchmarks in this category are listed in Table 3.
- **Medium-scale**, with qubits ranging from 6 to 15, for general benchmarking usage. Another historical reason is that when QASMBench was originally developed in 2020, the publicly accessible IBM-Q machine with the maximum number of qubits is *ibmq_melbourne*, which has 15 qubits. However, *ibmq_melbourne* is already retired now.
- **Large-scale**, contains benchmarks with more than 15 qubits, as listed in Table 5. The biggest circuit in QASMBench is `adder_n127` that contains 3991 gates. This corresponds to the most recent IBM-Q machine – *ibmq_washington* which features 127 physical qubits.

For each item in Table 3, 4 and 5, we list its name, brief description, and the algorithm category it belongs to in the quantum stack (see Figure 4). We show the number of qubits and gates utilized in the routines. The gate number here refers to *standard* OpenQASM gates (not basic gates or composition gates) as discussed in Section 1.2. We also list the number of CX gates in the tables, which indicates a major source of delay and error. Each algorithm is briefly introduced under its respective category. However, certain routines appear in multiple categories, we only describe them in the first appearance. Last not the least, although QASMBench is developed based on OpenQASM, they can be converted to other representations such as Q#, PyQuil, Cirq, etc. through the Javascript

Table 3. QASMBench Small-scale Benchmark.

Benchmark	Description	Algorithms	Qubits	Gates	CX	Ref
adder	Quantum ripple-carry adder	Quantum Arithmetic	4	23	10	[63]
basis_change	Transform the single-particle bases of an linearly connected electronic structure	Quantum Simulation	3	53	10	[83]
basis_trotter	Implement Trotter steps for molecule LiH at equilibrium geometry	Quantum Simulation	4	1626	582	[83]
bell_state	Bell State	Logical Operation	2	3	1	[126]
cat_state	Cat State	Logical Operation	4	4	3	[74]
deutsch	Deutsch algorithm with 2 qubits for $f(x) = x$	Hidden Subgroup	2	5	1	[27]
dnn	Quantum Deep Neural Network	Quantum Machine Learning	2	268	84	[116]
fredkin_n3	Fredkin gate benchmark	Logical Operation	3	19	9	[98]
qec_dist3	Error correction with distance 3 and 5 qubits	Error Correction	5	114	49	[86]
grover	Grover's algorithm	Search and Optimization	2	16	2	[4]
hs4	Hidden subgroup problem	Hidden Subgroup	4	28	4	[63]
inverseqft	Performs an exact inversion of quantum Fourier transform	Hidden Subgroup	4	8	0	[27]
iSWAP	An entangling swapping gate	Logical Operation	2	9	2	[63]
linearsolver	Solver for a linear equation of one qubit	Linear Equation	3	19	4	[14]
lqn	Learning parity with noise	Machine Learning	5	11	2	[108]
pea	Phase estimation algorithm	Hidden Subgroup	5	98	42	[27]
qaoa	Quantum approximate optimization algorithm	Search and Optimization	3	15	6	[60]
qec_sm	Repetition code syndrome measurement	Error Correction	5	5	4	[27]
qec_en	Quantum repetition code encoder	Error Correction	5	25	10	[108]
qft	Quantum Fourier transform	Hidden Subgroup	4	36	12	[27]
qrng	Quantum Random Number Generator	Quantum Arithmetic	4	4	0	[27]
quantumwalks	Quantum walks on graphs with up to 4 nodes	Quantum Walk	2	11	3	[85]
shor	Shor's algorithm	Hidden Subgroup	5	64	30	[58]
toffoli	Toffoli gate	Logical Operation	3	18	6	[63]
teleportation	Quantum Teleportation	Quantum Communication	3	8	2	[39]
jellium	Variational ansatz for a Jellium Hamiltonian with a linear-swap network	Quantum Simulation	4	54	16	[83]
vqe_uccsd	Variational Quantum Eigensolver with UCCSD ansatz	Search and Optimization	4	220	88	[51]
wstate	W-state preparation and assessment	Logical Operation	3	30	9	[27]

Table 4. QASMBench Medium-scale Benchmarks.

Benchmark	Description	Domain	Qubits	Gates	CX	Ref
dnn	Quantum Deep Neural Network	Quantum Machine Learning	8	1200	384	[116]
adder	Quantum ripple-carry adder	Quantum Arithmetic	10	142	65	[27]
bb84	A quantum key distribution circuit	Quantum Communication	8	27	0	[49]
bv	Bernstein-Vazirani Algorithm	Hidden Subgroup	14	41	13	[27]
ising	Ising model simulation via QC	Quantum Simulation	10	480	90	[63]
multiplier	Quantum multiplier	Quantum Arithmetic	15	574	246	[49]
multiply	Performing 3×5 in a quantum circuit	Quantum Arithmetic	13	98	40	[4]
qaoa	Quantum approximate optimization algorithm	Search and Optimization	6	270	54	[49]
qf21	Using quantum phase estimation to factor the number 21	Hidden Subgroup	15	311	115	[4]
qft	Quantum Fourier transform	Hidden Subgroup	15	540	210	[27]
qpe	Quantum phase estimation algorithm	Hidden Subgroup	9	123	43	[4]
sat	Boolean satisfiability problem via QC	Searching and Optimization	11	679	252	[27]
seca	Shor's error correction algorithm for teleportation	Error Correction	11	216	84	[4]
simons	Simon's algorithm	Hidden Subgroup	6	44	14	[4]
vqe_uccsd	Variational quantum eigensolver with UCCSD	Linear Equation	8	10808	5488	[63]

based *q-convert* tool, which is available online: <http://quantum-circuit.com/qconvert>. We distribute QASMBench in other representations as well through a modified version of *q-convert*.

2.1 Small-scale Benchmarks

Adder: The quantum adder uses each qubit as a classical bit, and builds a traditional adder through quantum gates. Here, we build the full-adder circuit using a basic component — a 4-qubit ripple-carry adder [29, 124] with a carry-in and a carry-out qubit. Given the granularity of 4-qubits, a m -qubit adder would require $2m + m/4 + 1$ qubits in total where $m/4 + 1$ qubits are used for the carrying bits between the basic 4-qubit adder components. We included a 4-qubit adder (see Figure 1) in small-scale, a 10-qubit adder in medium-scale, and 18-/127-qubit adders in large-scale. This benchmark is motivated by its scalability, easy-to-verify and usefulness in building quantum numeric units in the post-NISQ era.

Basis_change: The OpenQASM routine of this benchmark is generated from Google's Cirq based OpenFermion library [83] for manipulating fermionic systems towards chemistry simulation on

Table 5. QASMBench Large-scale Benchmarks.

Benchmark	Description	Domain	Qubits	Gates	CX	Ref
adder	Quantum ripple-carry adder	Quantum Arithmetic	127	3991	910	[63]
bigadder	Quantum ripple-carry adder	Quantum Arithmetic	18	284	130	[27]
bv	Bernstein-Vazirani algorithm	Hidden Subgroup	19	56	18	[27]
cat_state	Cat State	Logical Operation	100	100	99	[74]
cc	Counterfeit coin finding problem via QC	Hidden Subgroup	18	34	17	[27]
ghz_state	GHZ State preparation and assessment	Logical Operation	23	23	22	[50]
ising	Ising model simulation via QC	Quantum Simulation	26	280	50	[63]
ising	Ising model simulation via QC	Quantum Simulation	60	654	118	[63]
multiplier	Quantum multiplier	Quantum Arithmetic	25	3723	750	[49]
qft	Quantum Fourier tranform	Hidden Subgroup	20	970	380	[27]
qft	Quantum Fourier tranform	Hidden Subgroup	85	17935	7140	[27]
square_root	Computing the square root of an number via amplitude amplification	Quantum Arithmetic	18	4640	898	[63]
swap_test	SWAP Test algorithm implementation	Logical Operation	25	446	96	[67]
wstate	W-state preparation and assessment	Logical Operation	27	446	96	[27]

quantum computers. It shows how to use a quantum circuit to transform the single-particle basis of an linearly connected electronic structure so as to realize exact evolution under a random Hermitian one-body fermionic operator. This is included as a quantum chemistry benchmark.

Basis_trotter: This routine is also generated from the Cirq based OpenFermion library [83] using the method described in Figure 3. It is designed to implement the Trotter [12, 77] steps for an actual molecule *LiH* at equilibrium geometry, so as to simulate basis molecular Hamiltonians taking the following form $H = \sum_{pq} T_{pq} a_p^\dagger a_q + \sum_{pqrs} V_{pqrs} a_p^\dagger a_q^\dagger a_r^\dagger a_s$. The circuit uses 4 qubits but is quite deep, challenging the T1 and T2 decay of a NISQ device. It is included as another chemistry benchmark.

Bell State: The Bell state describes the Bell Inequality Test. This algorithm implements the Bell experiment on a quantum computer. This is a fundamental quantum mechanics experiment, and consists primarily of local entanglement, and low SWAP cost no matter the topology.

Cat state: The Cat-state is an imperative quantum sub routine, and forms the basis of quantum networking's EPR pairs [54]. The circuit comprises entangling n qubits, where the readout bitstring of the algorithm is homogeneous in 0 or 1 (i.e. 00...0 or 11...1). The bell state challenges primarily the entangling capabilities of a system.

Deutsch: The Deutsch-Jozsa algorithm [31] is a generalization of Deutsch's algorithm. The problem is defined as – given an oracle function $f: \{0, 1\}^n \rightarrow \{0, 1\}$ which takes an n binary digit as input, and outputs either 0 or 1. The objective is to find whether the outputs are the same for all of the inputs (say constant), or are one value (i.e., 0 or 1) for half of the inputs and the other (i.e., 1 or 0) for the remaining half (say balanced). Here, if the 2^n possible inputs are encoded into an n -qubit register, and if the oracle function can be expressed as gate operations in a black-box, the quantum computer can get the answer with only one-time evaluation to the oracle function whereas a deterministic classical computer would require $2^{n-1} + 1$ times evaluations. This algorithm is among the first examples to demonstrate that a quantum algorithm can achieve exponentially speedup over any possible deterministic classical algorithms. It is included as a baseline circuit.

DNN: Quantum deep neural networks [116] is a quantum adaption of the classical neural network where variational quantum circuits with a layered-approach are parameterized by a bank of θ values. QNN consists of gates such as $RY(\theta)$, CX and $CRY(\theta)$ for encoding classical data and weight information as well as introducing entanglement. Tasks such as classification are commonly used in evaluating these models where an attainable objective function is described alongside quantum gate differentiation, where the circuit is optimized. The domain of Quantum Machine Learning (QML) and QNN have drawn substantial attention recently [16, 117] as the hope for a quantum advantage in machine learning, thereby motivating the inclusion of the quantum DNN benchmark.

Fredkin: The Fredkin operation in Quantum Computing forms the foundation for the SWAP test, an important machine learning operation [117]. Fredkin gates challenge topologies by interacting three qubits with each other.

QEC: Quantum error correction (QEC) is to introduce redundancy into the original circuit by using additional physical qubits so that the state of the original circuit is effectively protected against noise. The surface code is often considered to be among the most promising QEC approaches as it can cope with error rates around 10^{-2} [43]. The *qec_dist3* benchmark performing a 3-distance 5-qubit surface code which is developed based on [86]. It is the smallest code that can correct arbitrary single-qubit error. QASMBench includes other QEC benchmarks, such as **qec_en** and **qec_sm** for encoder and syndrome measurement of repetition code, and **seca** for Shor's QEC for teleportation. QEC is perhaps the most important algorithm for NISQ and the success of QC in the long run, hence we include four QEC benchmarks in QASMBench.

Grover: Grover's algorithm [52] is a quantum algorithm for searching in a database, offering quadratic speedup over classical searching methods. It takes a black-box oracle realizing the function: $f(x) = 1$ if $x = y$, $f(x) = 0$ if $x \neq y$, and find y within a randomly ordered sequence of N items using $O(\sqrt{N})$ operations and $O(N \log N)$ gates with probability $p \geq 2/3$. Grover's algorithm is appealing in that it determines with high probability the unique input given the output is pre-known. Grover's algorithm is one of the fundamental quantum algorithms considering the importance of searching in the big-data era.

HS4: Hidden subgroup problems [63] is a quantum problem capturing problems such as factoring and graph isomorphism. The algorithm is important in the theory of Shor's algorithm, and hence plays an important role in the evolution of benchmarking quantum computers.

QFT & InverseQFT: The Quantum Fourier Transform (QFT) [24] (and its inverse) applies (inverse) Fourier transformation to the wave function amplitudes. It is a linear transform over the states of qubits, which is the quantum analogue of the discrete (inverse) Fourier transform. QFT is a basic component of many well-known quantum algorithms, including Shor's algorithm, quantum phase estimation, hidden subgroup problem, etc. Besides, it has some interesting features such as zero entanglement variance, as well be discussed later.

iSWAP: The iSWAP operation is a challenging dual qubit operation comprising multiple CNOT operations as well as single qubit operations. It is well suited to challenge two local qubits on their gate performance.

LinearSolver: This benchmark realizes the HHL solver [55] for solving a linear equation of 1-qubit. The HHL algorithm estimates the outcome of a scalar measurement on the solution vector to a sparse linear equation system. We originally attempted to dump the multi-qubits HHL routine from Cirq [49], but encountered error in generating the OpenQASM code due to the mismatch between Google's native quantum IR and OpenQASM (e.g., missing the CCCry representation). HHL is a critical quantum algorithm given the importance of linear algebra in scientific computation and machine learning.

LPN: The LPN (*learning parity with noise*) benchmark is a machine learning problem of learning a hidden parity function defined by the unknown binary string in the presence of noise. This application is very promising for NISQ devices because (i) it is computationally intractable for classical approaches [97]; (ii) it actually leverages the noise presented in the NISQ devices [28], which is usually seen as a hurdle in other applications.

Pea: Quantum *phase-estimation* (QPE) [94] is an algorithm to compute the eigenvalue $e^{2\pi i\theta}$ of a unitary operator operating on m qubits with the eigenvector $|\psi\rangle$ such that $U|\psi\rangle = e^{2\pi i\theta}|\psi\rangle$, $0 \leq \theta < 1$. QPE is another fundamental quantum algorithm being the key component of Shor's

method. This *Pea* benchmark circuit describes a 2-qubit system including a read-out ancilla bit, and a physical system bit [33], showcasing the property of QPE.

QAOA: The quantum approximate optimization algorithm (QAOA) [37, 131] is another variational quantum algorithm that is particularly interesting to NISQ devices, given its ability to tolerate certain degree of noise through the unique quantum-classic hybrid algorithm design. QAOA is designed to solve combinatorial optimization problems [130], such as in graph analytics. In QAOA, a quantum subroutine is embedded in a classical search loop. The initial quantum state is prepared according to a set of variational parameters. These parameters are adjusted based on the measurement in the previous iteration. The QAOA benchmark in QASMBench represents the quantum workload for a sample iteration.

Quantum Walks: Quantum walks [2, 125] is the quantum analogue of the classical random-walk algorithm, which has been adopted for many problems including graph isomorphism [45] and ranking nodes in a network [96]. Both discrete and continuous-time quantum-walk algorithms have been formulated by existing works [2, 38]. The quantum-walk benchmark here represents the workload for a single step of the walks [85].

QEC SM & EN: Quantum Error correction codes are examples of current approaches to quantum error correction. These circuits form the foundation for Quantum Error Correction, and can be used to gain intuition as to the processors' performance for error correction.

QRNG: Quantum random number generator is an algorithm for generating completely random numbers. It is known that no absolutely random random number generator exists, and is a relatively simple quantum circuit with no entanglement. This routine, although simple, can be very useful for real world applications.

Shor: Shor's algorithm [111] is a famous QC algorithm for integer factorization in polynomial-time. It demonstrates theoretical exponential speedups over the classical algorithms, posing substantial threat and concern over the widely-used public-key cryptography systems like RSA. Given its importance in quantum cryptography, we include a 5-qubit sample here in QASMBench. We also include another example — **qf21** for showcasing the process of factoring the integer of 21.

Toffoli: Similar to the Fredkin gate, the Toffoli gate [122] or CCX gate is another universal gate. It refers to a circuit with three inputs and three outputs that inverts the third qubit if the first two qubits are both 1, otherwise all bits stay unchanged. We include the 3-qubit Toffoli gate as a baseline benchmark in QASMBench.

Teleportation: The teleportation routine is a quantum communication channel connecting two qubits, communicating through an ancilla third. Quantum teleportation challenges a processors' ability to communicate information between an ancilla qubit.

VQE: The variational quantum algorithm is to optimize a parameterized quantum circuit ansatz applied to some initial state for minimizing a cost function defined according to the output state [64, 84]. When applied in quantum simulation (i.e., simulating physical phenomena using QC rather than simulating QC in a classical computer), the goal of the algorithm is often to prepare ground states. The cost function is often the expectation value of a Hamiltonian. If the initial state is $|\psi\rangle$, the Hamiltonian is H , the ansatz is $U(\vec{\theta})$ where $\vec{\theta}$ is the variational parameter, then the cost function is $E(\vec{\theta}) = \langle\psi|U^\dagger(\vec{\theta})HU(\vec{\theta})|\psi\rangle$. The *Jellium* benchmark creates a variational ansatz for a Jellium Hamiltonian via a linear-swap network. We also include two *Unitary Coupled Cluster Single-Double* VQE ansatz [127] for estimating the ground energy of molecules due to their ultra deep circuits.

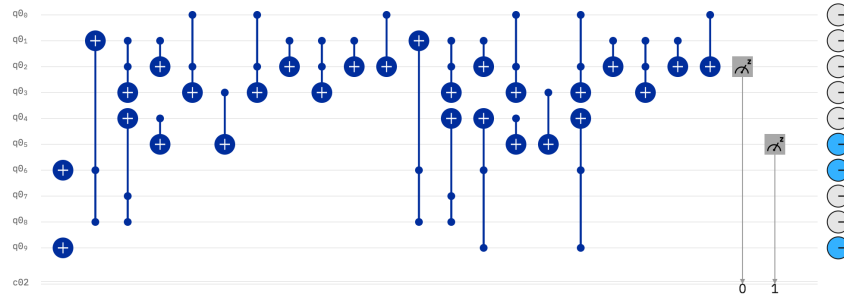


Fig. 5. A 2-qubit multiplier. The execution phases include: (1) *Preparation* (2) *Execution* (3) *Measurement*.

W_state: w_state is one of the basic states exhibiting properties not observed in a classical system. The w_state [36] is a special type of entangled quantum state generalized by:

$$|W\rangle = \frac{1}{\sqrt{n}}(|10\dots 0\rangle + |01\dots 0\rangle + \dots + |00\dots 1\rangle) \text{ for } n > 2$$

It describes the condition that for an n -qubit system, no matter which single qubit is measured or lost, the remaining $(n-1)$ qubits still remain entangled. This property makes it useful for improving the robustness of ensemble-based quantum memories [42]. This routine implements the 3-qubit condition where the three qubits follow $|W\rangle = \frac{1}{\sqrt{3}}(|001\rangle + |010\rangle + |100\rangle)$.

2.2 Medium-scale Benchmarks

BB84: BB84 [10] is a quantum key distribution (QKD) protocol which is the first quantum cryptographic protocol [15] based on the non-cloning quantum laws for showcasing provably secure key generation. It relies on the fact that it is not possible to obtain information distinguishing two non-orthogonal states without disturbing the signal. Since QKD is a critical component for quantum communication and quantum networks, we include BB84 as a sample benchmark in QASMBench.

BV: The Bernstein-Vazirani algorithm [11] is an extension to the Deutsch-Josza algorithm [31]. The problem is that, given a black-box function f , which takes in n bits as input, and outputs the bitwise product of a string s against the input. This is defined as $x, f(x) = s \times x \bmod(2)$ where the string s is hidden. Classically, to determine the hidden string, we need n evaluations (100..0, 0100..0, ..., 0000..1). This algorithm can decide s through a single query to the oracle on a quantum computer. Given it is another application that can showcase exponential quantum speedup, we include it in QASMBench.

Ising: The Quantum Ising Model [17, 70] consists of an array of quantum spins arranged in a certain lattice. The spins, which are expressed in quantum operators, can only interact with their neighbors. The phase transition is due to quantum fluctuation introduced by the transverse field. Given the quantum annealing machines (e.g., D-Wave) basically resolving optimization problems that can be expressed in an Ising model, we include it as a benchmark routine in QASMBench.

Multiplier: Similar to the adder, the quantum multiplier also uses a qubit as a classical bit to construct a multiplier unit. Our implementation is similar to the one from Cirq [49], which requires $5n$ qubits for an n -qubit multiplier and internally calls the n -qubit adder by n times for aggregating the partial results. The multiplier comprises the three phases: *preparing operand*, *multiply* and *measure*. Figure 5 shows an example of a 2-qubit multiplier using 10 qubits. Similarly to the adder, this is motivated by its simple scalability and easy verification.

SAT: The *Boolean Satisfiability* problem (SAT) determines whether there is an assignment of variables that satisfies a given Boolean function [7, 118] which is one of the first proven NP-complete problems. Since many optimization problem (e.g., in power-grid domain) can be attributed as a mixed-integer-linear-programming (MILP) problem, which can be formulated as a binary optimization problem [18] similar to SAT, we include SAT here as a benchmark in QASMBench.

Simon: Simon's algorithm [113] was among the first quantum algorithms to show exponential speedups over the best classical deterministic or probabilistic algorithm. It finds an unknown nonzero $s \in \{0, 1\}^n$ that satisfies $f(x) = f(x \oplus s)$. We include it here as another benchmark showing quantum advantage. Besides, Simon's algorithm is more sensitive to noise in NISQ devices.

2.3 Large-scale Benchmarks

CC: The *counterfeit coin* problem is a mathematical puzzle that attempts to find all false coins from a given bunch of coins using a balance scale [61]. Let's say you have a set of N coins, and up to k of them are counterfeit and hence lighter. Using a scale, which returns the information of "balanced" or "tilted", we can deduce if any, or how many of the coins are counterfeit. The classical algorithm requires a time complexity of $O(k \log(k/N))$. The quantum algorithm offers an exponential speed-up, with a time complexity of $O(k^{\frac{1}{4}})$ [62].

GHZ-state: A *Greenberger-Horne-Zeilinger* (GHZ) state [50] is a particular type of entangled quantum state that involves at least three qubits: $|GHZ\rangle = \frac{1}{\sqrt{2}}(|000\rangle + |111\rangle)$. Since it describes the superposition of all qubits being 0 with all of them being 1, it represents the maximally entangled quantum state. Consequently, characterizing GHZ state can provide very useful information about the fidelity of multi-qubit interactions, which is critical for constructing large-scale quantum computers. For an n -qubit system, GHZ state is generalized as:

$$|GHZ\rangle = \frac{1}{\sqrt{2}}(|0\rangle^{\otimes n} + |1\rangle^{\otimes n}) \text{ for } n > 2$$

To construct a GHZ-state for an n -qubit system, from an all-zero states $|\Psi\rangle = |0\rangle^{\otimes n}$, after an H gate is applied on the first qubit, $|\Psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle^{\otimes n} + |1\rangle \otimes |0\rangle^{\otimes n-1})$. We then apply a series of CX (or CNOT) gates from qubit 0 to qubit $(n-1)$. Running on a full-fidelity quantum machine with 5 logic qubits, the measurement result should be 50% $0^{\times n}$ and 50% $1^{\times n}$. This application represents another basic quantum state which exploits across all the available qubits. It can be used to prove CNOT gate stability of a NISQ device.

Squarer_root: This routine relies on amplitude amplification [53] to find the square root of an n -bit number using Grover's search technique. This is a complex application in QASMBench.

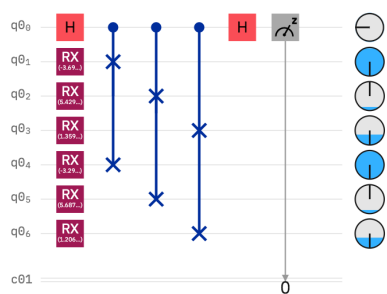


Fig. 6. 3-qubit swap_test.

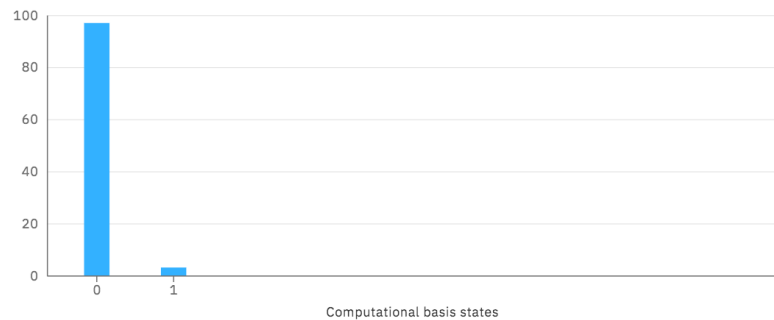


Fig. 7. Measurement result for a 3-qubit swap_test circuit using 7 qubits.

Swap-test: The swap-test is a very convenient and economic way to measure the difference between two quantum states, or how two quantum ($|\psi\rangle$ and $|\phi\rangle$) states are divergent from each other. The swap-test can be very useful in quantum machine learning for estimating the prediction accuracy (e.g., distance of prediction from the tag). It uses an ancilla qubit for example qubit 0. The initial state of the system is $|0, \psi, \phi\rangle$. The ancilla qubit is firstly put in superposition via an H gate, as shown in Figure 6, which converts the system state into $\frac{1}{\sqrt{2}}(|0, \psi, \phi\rangle + |1, \psi, \phi\rangle)$. By applying the series of controlled-swap gate (or CSWAP, which swaps the states of two qubits only if the control qubit is 1, see Table 2), the system state becomes $\frac{1}{\sqrt{2}}(|0, \psi, \phi\rangle + |1, \phi, \psi\rangle)$. Then, by applying another H gate on qubit-0, the system transits to $\frac{1}{2}(|0\rangle(|\psi, \phi\rangle + |\phi, \psi\rangle) + |1\rangle(|\psi, \phi\rangle - |\phi, \psi\rangle))$. When measuring qubit-0, the probability of obtaining 0 is $P(0) = \frac{1}{2} + \frac{1}{2}|\langle\psi|\phi\rangle|^2$. Therefore, if $|\psi\rangle$ and $|\phi\rangle$ are orthogonal, $|\langle\psi|\phi\rangle|^2 = 0$, the probability of obtaining 0 is 50%; if $|\psi\rangle$ and $|\phi\rangle$ are equal, $|\langle\psi|\phi\rangle|^2 = 1$, the probability of obtaining 0 is 100%. The more the two states are similar, the higher the probability of obtaining 0 approaches 1. Note that the swap-test needs sufficient number of repetitions to achieve an accurate sampling of the probability of the ancilla qubit. The measurement probability in Figure 7 shows that the two 3-qubit states in Figure 6 are similar to each other with $P(0) \approx 1$. The SWAP test is a crucial algorithm in many Quantum processes, and is easily scaled out to any application size with substantial entanglement, hence is an ideal NISQ evaluation benchmark.

3 QASMBENCH CHARACTERIZATION METRICS

We propose a set of quantum circuit based evaluation metrics representing various features of a quantum application. These metrics are designed such that through them certain estimation can be performed on executing a particular circuit over a particular NISQ device, practicing the software-hardware co-design paradigm. The metrics serve as useful indicators on how a quantum circuit can stress a NISQ hardware device. The scripts for profiling these metrics are provided along with the QASMBench code.

3.1 Circuit Width

Circuit width is defined as the number of qubits that enter the superposition state at least once within an application's lifespan. Qubits that are measured in the interim of a circuit and re-enter superposition are only counted as one qubit towards the circuit width. Circuit width dictates the spatial capacity required for a quantum device in order to run the quantum circuit. Circuit Width is mathematically defined in Equation 1 where $n_{q_{\text{active}}}$ is the number of qubits that are in-use or demanded by the circuit.

$$\text{Circuit Width} = n_{q_{\text{active}}} \quad (1)$$

3.2 Circuit Depth

Circuit depth is defined as the minimum time-evolution steps required to complete a quantum application. Time evolution is the process of completing all gates defined at time $t = t_j$, and once these are completed, the circuit moves onto time $t = t_{j+1}$, where the following gates are to be processed. To keep generality and avoid the impact from low-level optimization, circuit depth here is calculated solely using standard OpenQASM gates (see Section 1.2). That is to say, the OpenQASM code is decomposed to standard gates before counting the accurate time evolution steps required. As a concrete example, an X gate is a standard gate and hence requires 1 time-step, whereas a CRZ gate requires 4 standard gates to complete, thereby requiring 4 timesteps.

Circuit depth can be computed by decomposing OpenQASM code into a $n_q \times t$ matrix Q , where Q_{q_i, t_j} is the time-evolution steps to complete the gate on qubit i at time j . The sum of the maximum time in each column is then equal to the minimum time required for a quantum application. This is

described in Equation 2:

$$\text{Circuit Depth} = \sum_{j=1}^t \max_{0 \leq i < \text{width}} (Q_{q_i, t_j}) \quad (2)$$

3.3 Gate Density

Gate density, or operation density, describes the occupancy of gate slots along the time-evolution steps of a quantum circuit. As certain qubits might need to wait for other qubits in the time evolution (i.e., gate dependency), they remain idle by executing the ID gate (see Table 2). Consequently, if a gate slot is empty due to dependency, it implies a lower occupancy for the quantum hardware. This is similar to a classical processor, where data dependency introduces pipeline bubbles and reduced occupancy. We propose Gate Density to measure the likely occupancy of a circuit when mapping to a quantum hardware, as formulated in Equation 3. Note, the gate density is also calculated after decomposing down to the OpenQASM standard gates.

$$\text{Gate Density} = \frac{G_{1\text{-qubit}} + 2 \times G_{2\text{-qubit}}}{\text{Circuit Depth} \times \text{Circuit Width}} \quad (3)$$

where $G_{1\text{-qubit}}$ refers to the number of 1-qubit standard gates and $G_{2\text{-qubit}}$ refers to the number of 2-qubit standard gates (see Table 2).

3.4 Retention Lifespan

Retention Lifespan describes the maximum lifespan of a qubit within a system, and is motivated by the T1 and T2 coherence time of a quantum device (see Section 1.1). A longer lifespan of a quantum system implies more decay to the ground state (T1) and state-transition due to environment noise (T2), thus is more susceptible to information loss. Therefore, we propose taking the qubit with the longest lifespan to determine the system's retention lifespan. Using this metric, one can estimate if a particular circuit can be executed in a NISQ device with high fidelity, given its T1/T2 coherence time. Note, all IBM-Q machines offer T1/T2 coherence time as status indicators for the hardware. As circuit depth can grow substantially, we introduce the log operator to shrink the scale. If D_i is used to describe the lifespan or depth of the qubit i , Retention Lifespan, which measures the circuit size and sensitivity to quantum system error, is defined in Equation 4:

$$\text{Retention Lifespan} = \max_{0 \leq i < \text{width}} (\log(D_i)) \quad (4)$$

3.5 Measurement Density

Measurement density assesses the importance of measurements in a circuit. A higher measurement count implies the fact that each measurement might be of relatively less importance (e.g., periodic measurement in QEC, or measurement over ancilla qubits), whereas for application with less measurements, the measurement may be of utmost importance. The importance also increases when a measurement accounts for a wider and/or deeper circuit. A good example is the SWAP test (see Figure 7), where the circuit can be very large but only one measurement is taken to report the similarity. Consequently, this measurement is extremely important to the application. Measurement Density is defined in Equation 5, where $N_{\text{measurement}}$ is the number of measurements in an application. Since the circuit depth/width can be large and the importance of measurement decays when circuit depth/width keeps on increasing, we add a log to shrink the scale.

$$\text{Measurement Density} = \frac{\log(\text{Circuit Depth} \times \text{Circuit Width})}{N_{\text{measurement}}} \quad (5)$$

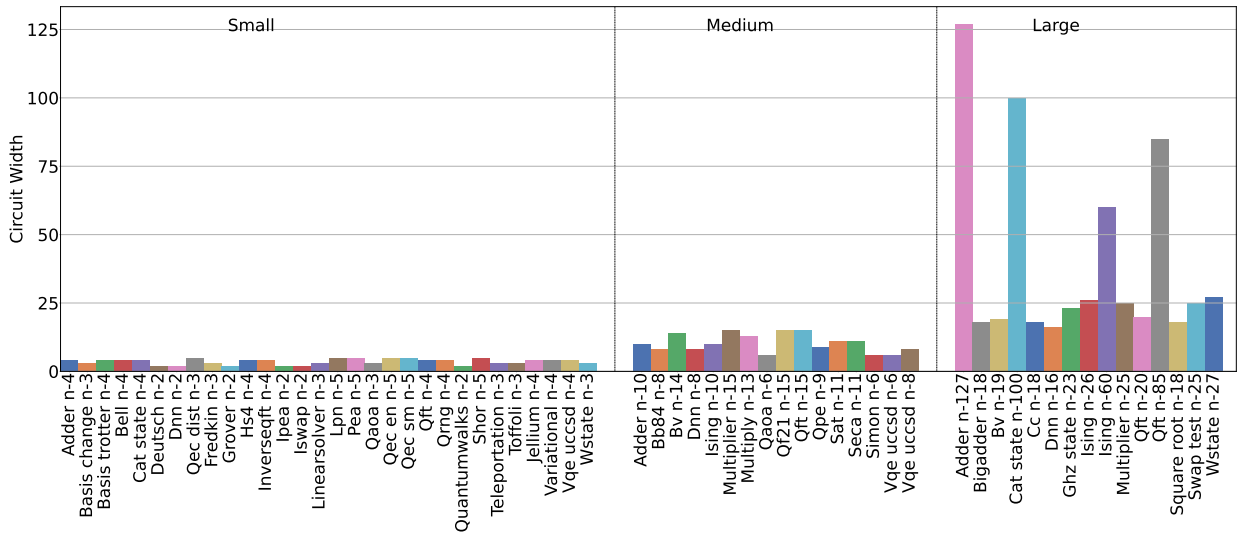


Fig. 8. Circuit Width of QASMBench circuits with respect to the small, medium and large categories.

3.6 Entanglement Variance

Entanglement Variance measures the balance of entanglement across the qubits of a circuit. Circuits with a higher Entanglement Variance indicate that certain qubits are more connected than other qubits (i.e., using more 2-qubit gates such as CX than others). This metric implies that when the circuit is mapped to a NISQ device: (i) less SWAP gates are needed if those hotspot qubits are mapped to the central vertices in the NISQ device topology, such as Qubit-1 in *ibmq_belem* and Qubit-2 in *ibmq_yorkton* in Figure 2). A higher entanglement variance implies a higher potential benefit from a good logic-physical qubit-mapping through quantum transpilation. If the entanglement variance is zero, little benefit should be expected from a better transpilation strategy; (ii) Given 2-qubit gate is one of the major sources introducing error, a higher Entanglement Variance implies uneven error introduction among qubits. We define Entanglement Variance in Equation 6, where G_{q_i} (2-qubits) is the number of 2-qubit gates operating on qubit- i , and $\overline{G_q}$ is the average number of 2-qubit gates operating over each qubit. The inclusion of plus 1 is to eliminate log errors with 0 variance.

$$\text{Entanglement Variance} = \frac{\log(\sum_{i=0}^{\text{width}} (G_{q_i}(\text{2-qubits}) - \overline{G_q}(\text{2-qubits}))^2 + 1)}{\text{Circuit Width}} \quad (6)$$

4 EVALUATION

We first evaluate QASMBench circuits using the proposed metrics and then benchmark a subset of the circuits on real NISQ devices.

4.1 Circuit Metrics

We evaluate QASMBench applications described in Section 2 using the circuit metrics proposed in Section 3.

4.1.1 Circuit Width. Figure 8 illustrates the width of the circuits in QASMBench. Since we partition QASMBench with respect to the number of qubits (1-5 for small-scale, 6-15 for medium-scale, and 15-127 for large-scale), this can be seen in Figure 8.

4.1.2 Circuit Depth. Figure 9 shows the depth of the benchmark circuits from QASMBench. Apparently, some circuits (e.g., *DNN*, *VQE*, *QFT*) are significantly deeper than other circuits. This figures shows that QASMBench covers a wide range of circuit depth from 11 in the *Linearsolver_n-3* to

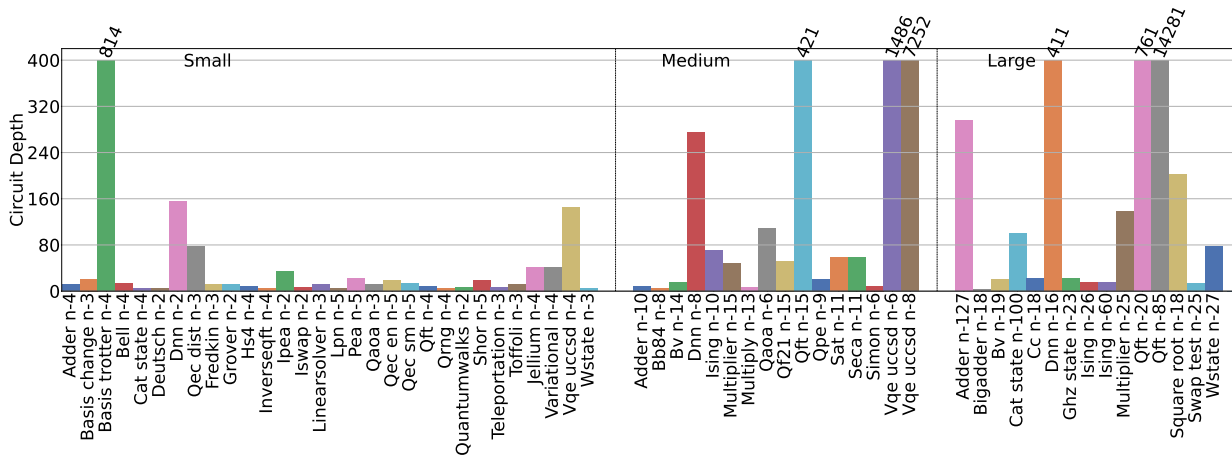


Fig. 9. Circuit Depth of QASMBench circuits.

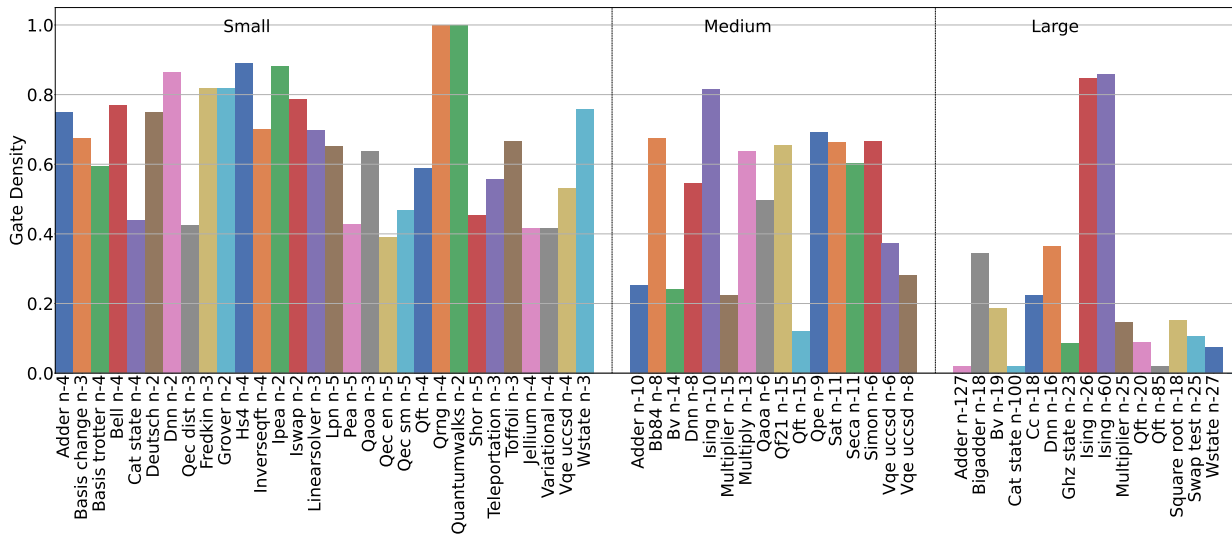


Fig. 10. Gate Density of QASMBench circuits.

14,281 in the *QFT_n-85*, offering a comprehensive circuit database for evaluating NISQ capacity and stability.

4.1.3 Gate Density. The plot of Gate Density for QASMBench is visualized in Figure 10. The general observation is that, with larger scale circuits, the gate density tends to be more sparse, implying reduced execution occupancy or efficiency. On the contrary, smaller circuits tend to be denser and more carefully designed, so that qubits are less likely to wait for other qubits before evolving onto the next time step. Nevertheless, QASMBench includes densely packed large and medium circuits with *Ising_n-26* having a Gate Density factor of 0.8462 for large scale circuits, and *BB84_n-8* with a Gate Density of 0.675 in the medium scale category. Note, a low gate density may also imply the amount of internal operation dependency and the potential gain with a better circuit reform.

4.1.4 Retention Lifespan. Retention Lifespan, visualized in Figure 11, illustrates the highly varying maximum qubit lifespan within the benchmark circuits of QASMBench. Short circuits such as *Deutsch_n-2* are required to preserve a quantum state for a relatively short period of time with a retention lifespan of 1.3863, compared to the *Square-Root_n-18* circuit with a retention life span of

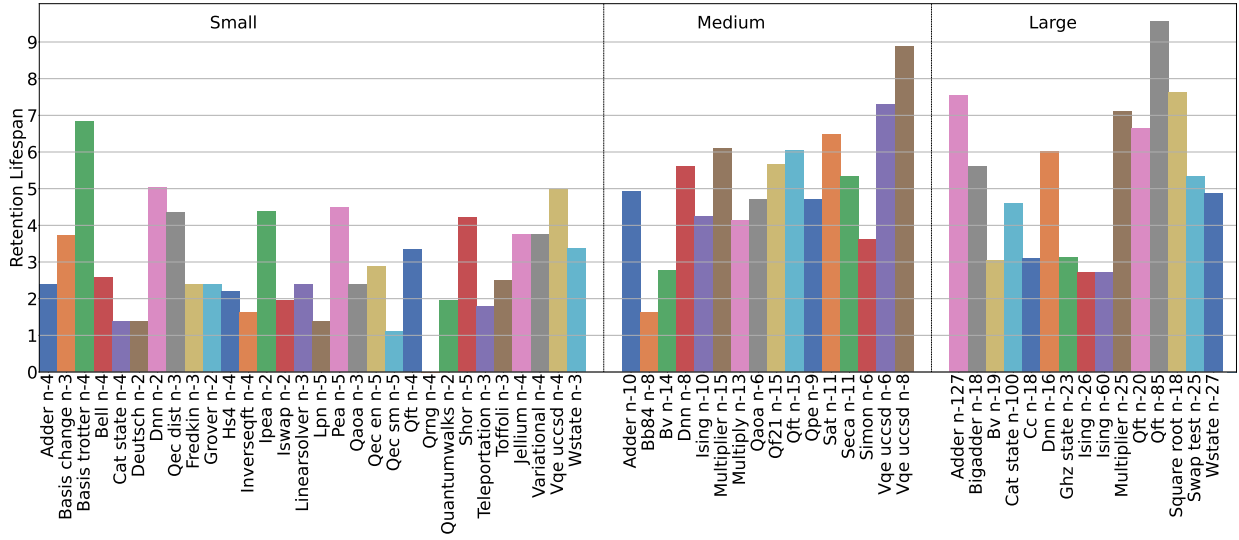


Fig. 11. Retention Lifespan of QASMBench Circuits

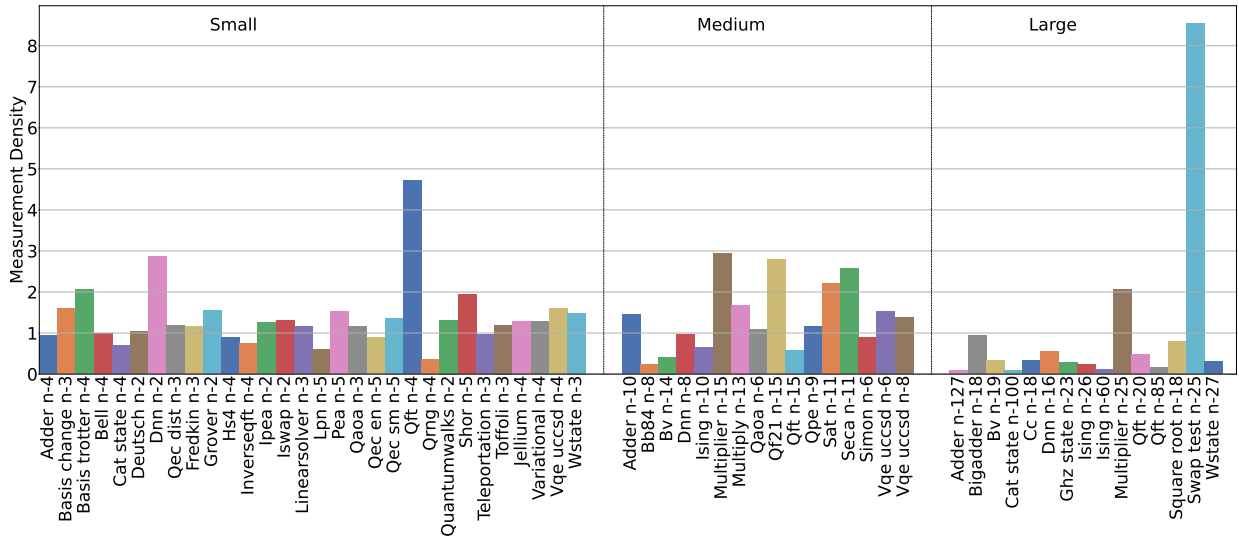


Fig. 12. Measurement Density of QASMBench Circuits

5.6168. Note, retention lifespan is related to circuit depth, but taking dependency into consideration and is specific to a certain qubit rather than the overall system.

4.1.5 Measurement Density. The measurement density for the QASMBench circuits is illustrated in Figure 12, which measures the importance of measurement operations in a circuit. As can be seen, *SWAP-test*, *multiplier*, *DNN* and *QFT* show significantly higher measurement density than the other circuits. We have already discussed Swap-Test in Section 3.5. *QFT* has a very deep circuit which explains why the only measurement at the end is of more importance. For *multiplier*, only qubits storing the product results are measured at the end. *DNN* essentially embeds the *SWAP-test* as a component for the estimation of the cost function.

4.1.6 Entanglement Variance. Figure 13 shows the entanglement variance for the QASMBench circuits. The figure demonstrates QASMBench's diverse entanglement arrangements, with densely entangled circuits such as *Dnn n-2* and *Grover n-2*, where all entanglement operations entangle

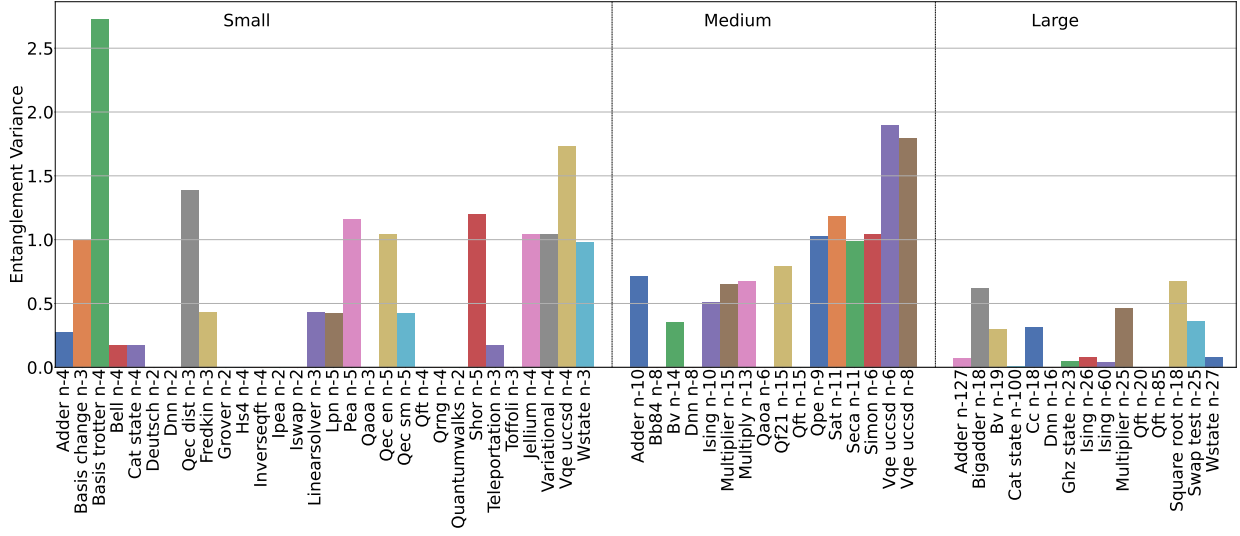


Fig. 13. Entanglement Variance of QASMBench Circuits

the only two qubits available. Therefore, the entanglement is evenly distributed across the circuit. However, compared to Basis trotter n-4, the primary bulk of entanglement is between qubits 1 and 2, whereas qubits 0 and 4 interact substantially less with other qubits. Consequently, when mapping *Basis-trotter* to a NISQ device like *ibmq_belem*, it involves much less SWAP gates if qubit-1 and 2 are mapped to Q1 and Q3 in Figure 2 than the reverse.

4.2 Evaluating QASMBench on Real NISQ Devices

We first describe the metric we used for the evaluation. Then, we evaluate applications from the small category of QASMBench on 10 IBM-Q machines. Finally, we compare among devices from three different NISQ hardware vendors.

4.2.1 Evaluation Metric. We use **fidelity** to assess the quality of executing a particular quantum algorithm in a specific quantum device. Fidelity here is defined as the distance between the actual quantum state of a NISQ device with the ideal quantum state obtained from a classical simulation. Since NISQ devices are susceptible to noise, the actual quantum state is essentially a mixed state, described by a density matrix. However, quantum observables are not directly measurable on the classical side. Therefore, we reconstruct this mixed state density matrix through state tomography.

Equation 7 shows the definition of fidelity we used in the evaluation. Its average value at runtime reflects a quantum machine's ability to induce a particular circuit.

$$F(\rho, \sigma) = \left(\text{Tr} \sqrt{\sqrt{\rho} \sigma \sqrt{\rho}} \right)^2 \quad (7)$$

where ρ is the density matrix representing the pure state $|\psi_\rho\rangle$ from a classical simulation. σ is the density matrix representing the mixed state $|\psi_\sigma\rangle$ from the noisy execution in the NISQ device. Since $|\psi_\rho\rangle$ is a pure state, Equation 7 can be further simplified as:

$$F(\rho, \sigma) = \langle \psi_\rho | \sigma | \psi_\rho \rangle \quad (8)$$

We use Equation 8 to measure the fidelity, which should be the most precise fidelity metric, to the best of our knowledge. However, density matrix tomography is extremely computationally expensive, which requires 3^n times' circuit evaluations where n is the number of qubits. Therefore, for a 10-qubit circuit, it demands 59,049 circuit evaluations (without even counting the shots per

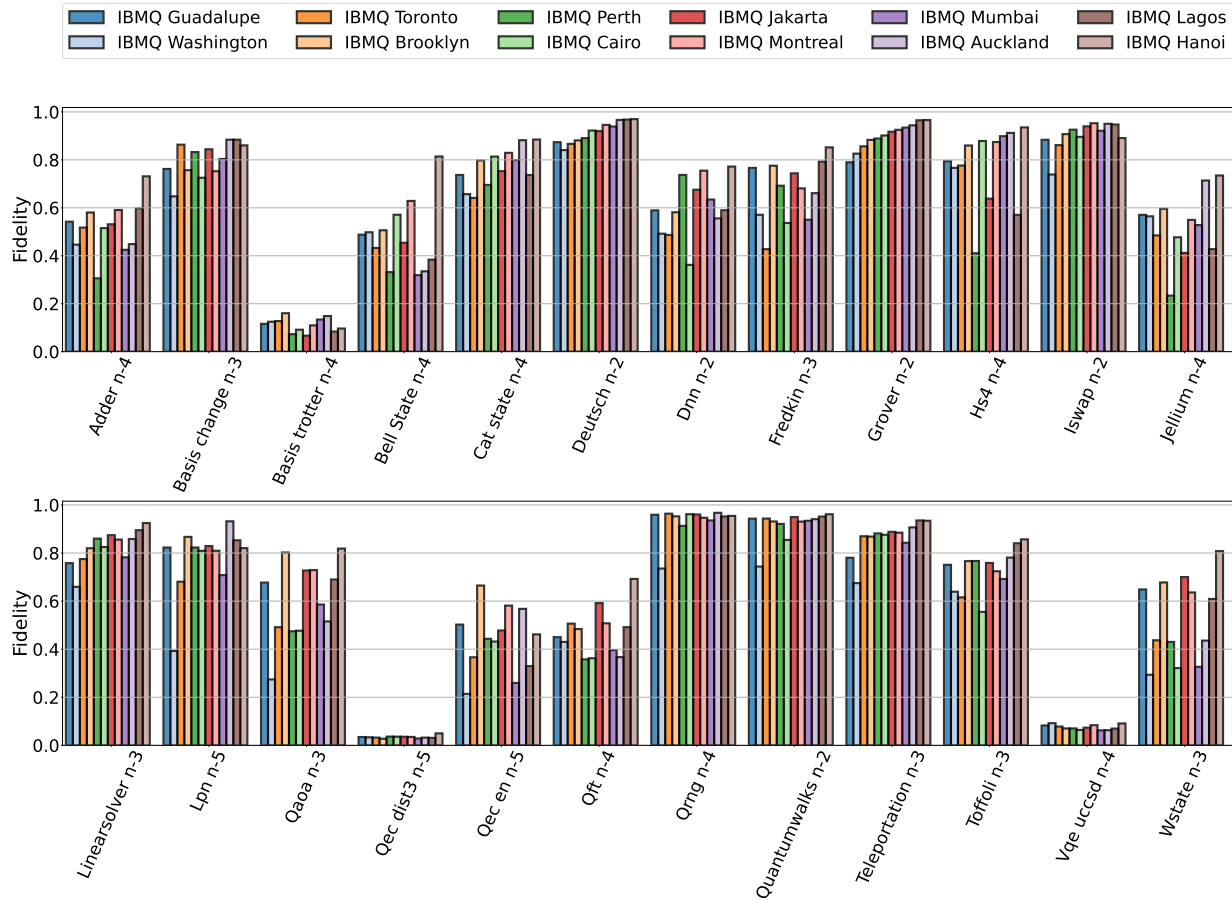


Fig. 14. Evaluations of 24 circuits from QASMBench on 12 IBM-Q machines via density matrix tomography.

evaluation). Additionally, these evaluations should be performed consecutively in a short time window and at least within the same calibration period. Otherwise, the state to be reconstructed is not consistent any more. Consequently, such an approach is only feasible for circuits with a small number of qubits. Nevertheless, active research is ongoing in this area to accelerate and make state tomography more scalable through technologies such as neural networks [104].

4.2.2 IBM-Q Evaluation. Due to the excessive cost of density matrix tomography, we evaluate 24 circuits from the small category (see Table 3) on 12 IBM-Q machines: Washington (127-qubit, QV64 which means quantum volume is 64), Guadalupe (16-qubit, QV32), Toronto (27-qubit, QV32), Brooklyn (65-qubit, QV32), Perth (7-qubit, QV32), Cairo (27-qubit, QV64), Jakarta (7-qubit, QV16), Montreal (27-qubit, QV128), Mumbai (27-qubit, QV128), Auckland (27-qubit, QV64), Lagos (7-qubit, QV32) and Hanoi (7-qubit, QV32), through PNNL’s IBM-Q Hub. We perform density matrix state tomography to calculate the fidelity using Equation 8 for each circuit, with respect to the pure state from the state-vector simulator of Qiskit. This effort comprises 25,000 circuit evaluations in total. We use 8192 shots per evaluation. The Qiskit version is 0.33.0. We use the default optimization level for the Qiskit transpiler.

Figure 14 illustrates the results. We observe that: (i) Compared among different benchmark circuits, the fidelity is generally correlated with circuit depth and width. For example, the circuit depths of `basis_trotter`, `qec_dist3`, and `vqe_uccsd` are much larger than the other circuits, with `qec_en` and `qft` stay in the middle. For `dnn n-2`, although the circuit is relatively deep (268 gates),

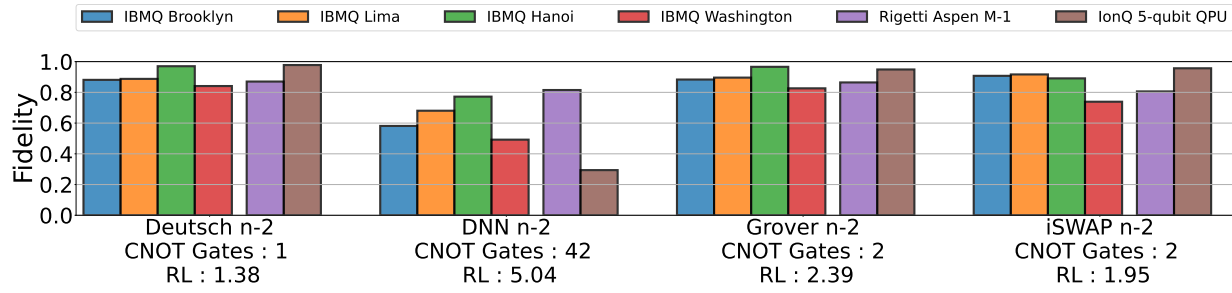


Fig. 15. Fidelity evaluation on 4 IBM-Q machines, IonQ's 5-qubit QPU and Rigetti's 80-qubit Aspen M-1 machine. RL refers to the retention lifespan of the circuit (see Equation 4).

due to less qubit, the fidelity level is better than adder $n=4$ with merely 23 gates. (ii) Compared among different machines, the fidelity appears to be weakly correlated with the qubit number of the machine and its quantum volume. Particularly, for qnn $n=2$, despite mapping to the best performing qubits of IBM-Q Washington, the fidelity level is still lower than the 7-qubit IBM-Q Hanoi with a less QV. (iii) In general, the impact from the noise, e.g., the decay noise, is significant. With merely 36 gates in qft $n=4$, the fidelity for most machines drops below 0.6 except Hanoi.

4.2.3 Evaluations of NISQ Devices. We further evaluate 4 circuits (i.e., Deutsch, DNN, Grover and iSWAP) on different NISQ devices. We use the 5-qubit IonQ trapped-ion QPU through Microsoft Azure Quantum [88], and the 80-qubit Rigetti Aspen M-1 superconducting machine through Oak Ridge Leadership Computing Facility (OLCF). We compare the fidelity with IBM-Q Brooklyn (65-qubit, QV32), Lima (5-qubit, QV8), Hanoi (27-qubit, QV64), and Washington (127-qubit, QV64). The results are shown in Figure 15. The modest evaluation is due to resource limitations.

Through this evaluation, we can observe that IonQ's trapped-ion system generally obtains higher fidelity than the other devices, for relatively shallow circuits with smaller retention lifespan. However, for a deeper circuit, the performance drops significantly compared to the other superconducting devices. This can be attributed to the lower single-gate fidelity level for the trapped ion processor (i.e., 99.35%), compared with the other superconducting devices (e.g., 99.90% for IBM-Q Washington). Despite the higher T1 time ($\geq 10^7 \mu s$) of IonQ's QPU can preserve quantum information better (with respect to $99.28 \mu s$ for IBM-Q Washington), the exponential compounding imperfect gate fidelity leads to an observable reduction in the overall performance.

5 RELATED WORK

To the best of our knowledge, QASMBench is the first benchmark suite aiming at evaluating NISQ devices using quantum applications from a broad range of domains. Existing works have already proposed to adopt randomly generated benchmarks for evaluating the error rates of different gate operations regarding a particular NISQ device [79, 80, 102]. Recently, Patel et al. [99] evaluated several IBM-Q machines using seven benchmarks and drew interesting observations regarding the error and execution time. Since the work mainly focused on extracting and validating new insights, only limited applications were included and no new metrics had been proposed.

Regarding QC metrics, Cross et al. defined the quantum volume protocol which quantified the largest random circuit of equal width and depth that a NISQ device can successfully implement [26], taking gate and measurement error into consideration. Quantum Linpack, proposed by Dong and Lin [34] recently, attempted to develop a random circuit block-encoded matrix (RACBEM) for measuring the QC capability in resolving a random system of linear equations. Benedetti et al. [8] proposed the qBAS score metric for benchmarking the shallow quantum-classical hybrid systems

using synthetic data set. These metrics, despite being comprehensive, are not quite intuitive for understanding. Besides, these metrics all involve certain randomness due to synthetically generated inputs or circuits, which do not necessarily share the properties of real quantum workloads. Given the randomness, the validation requires offline simulation in a classic computer, ultimately limiting the scalability as the simulation cost in a classical computer scales exponentially with increased number of qubits.

Regarding the general evaluation of NISQ devices, in addition to [99], LaRose [71] compared the four widely used QC programming environments: Forest/pyQuil, Qiskit, ProjectQ and QDK/Q# in terms of library support, hardware, compiler and simulator performance. McCaskey et al. [82] proposed XACC, a programming model that defines a low-level intermediate representation and compiler front-end to provide a unified abstraction for evaluating different quantum hardware and programming environments. Finally, Abhijith et al. [1] developed a great hands-on tutorial in explaining the principles of QC using 20 different quantum algorithms, using the 5-qubit IBM-Q machine as the validation platform. However, this work is mainly for education purposes using the small-scale circuits (5 qubits) rather than serving as a QC benchmark suite. SupermarQ [123] comprises a set of circuits in attempts to benchmark the current term quantum processors. Lubinski et al. evaluated NISQ processors comprehensively according to volumetric benchmarking, and provided evaluations of quantum volume on a diverse set of NISQ superconducting and trapped ion quantum processors [78].

6 CONCLUSION

In this paper, we propose a light-weighted, low-level and easy-to-use benchmark suite called QASMBench based on the OpenQASM quantum assembly language. It collects commonly seen quantum algorithms and routines from a variety of domains with distinct properties, serving the need for convenient quantum computing characterization and evaluation purposes for the computer system and architecture communities. Additionally, we propose four novel circuit metrics including gate density, retention lifespan, measurement density, and entanglement variance for assessing the execution efficiency, susceptibility to NISQ error, and potential gain from low-level optimizations. We also evaluate the fidelity of inducing QASMBench circuits on IBM-Q, IonQ and Rigetti NISQ devices through density matrix state tomography and derive new observations. QASMBench, together with the metrics and evaluations, provides a unique tool for evaluating and characterizing the properties of emerging NISQ devices, estimating the potential of optimization from quantum transpilers, and benchmarking the performance of classical quantum simulators.

ACKNOWLEDGMENTS

This material is based upon work supported by the U.S. Department of Energy, Office of Science, National Quantum Information Science Research Centers, Co-design Center for Quantum Advantage (C2QA) under contract number DE-SC0012704. This research used resources of the Oak Ridge Leadership Computing Facility, which is a DOE Office of Science User Facility supported under Contract DE-AC05-00OR22725. We also acknowledge support from Microsoft's Azure Quantum for providing credits and access to the ion-trap quantum hardware used in this paper. The Pacific Northwest National Laboratory is operated by Battelle for the U.S. Department of Energy under contract DE-AC05-76RL01830.

REFERENCES

- [1] J Abhijith, Adetokunbo Adedoyin, John Ambrosiano, Petr Anisimov, Andreas Bäertschi, William Casper, Gopinath Chennupati, Carleton Coffrin, Hristo Djidjev, David Gunter, et al. 2018. Quantum algorithm implementations for beginners. *arXiv e-prints* (2018), arXiv-1804.

Equivalence Checking of Sequential Quantum Circuits

Qisheng Wang *

Riling Li †

Mingsheng Ying ‡

Abstract

We define a formal framework for equivalence checking of sequential quantum circuits. The model we adopt is a quantum state machine, which is a natural quantum generalisation of Mealy machines. A major difficulty in checking quantum circuits (but not present in checking classical circuits) is that the state spaces of quantum circuits are continuums. This difficulty is resolved by our main theorem showing that equivalence checking of two quantum Mealy machines can be done with input sequences that are taken from some chosen basis (which are finite) and have a length quadratic in the dimensions of the state Hilbert spaces of the machines. Based on this theoretical result, we develop an (*and to the best of our knowledge, the first*) algorithm for checking equivalence of sequential quantum circuits with running time $\mathcal{O}(2^{3m+5l}(2^{3m} + 2^{3l}))$, where m and l denote the numbers of input and internal qubits, respectively. The complexity of our algorithm is comparable with that of the known algorithms for checking classical sequential circuits in the sense that both are exponential in the number of (qu)bits. Several case studies and experiments are presented.

Keywords: quantum computing, quantum circuits, sequential circuits, Mealy machines, equivalence checking.

*Qisheng Wang is with the Department of Computer Science and Technology, Tsinghua University, Beijing 100084, China (e-mail: QishengWang1994@gmail.com).

†Riling Li was with the Department of Computer Science and Technology, Tsinghua University, Beijing 100084, China. He is now with the State Key Laboratory of Computer Science, Institute of Software, Chinese Academy of Sciences, Beijing 100190, China (e-mail: lirl@ios.ac.cn).

‡Mingsheng Ying is with the State Key Laboratory of Computer Science, Institute of Software, Chinese Academy of Sciences, Beijing 100190, China, and also with the Department of Computer Science and Technology, Tsinghua University, Beijing 100084, China (e-mail: yingms@ios.ac.cn).

Contents

1	Introduction	3
2	Combinational Quantum Circuits	5
2.1	Qubits	5
2.2	Unitary Transformations	6
2.3	Quantum Measurements	6
2.4	Quantum Gates and Quantum Circuits	7
3	Sequential Quantum Circuits	8
3.1	Definition and Examples	8
3.2	Computation Process	10
3.3	Equivalence of Sequential Quantum Circuits	10
4	Quantum Mealy Machines	11
4.1	Preliminaries and Notations	11
4.2	The Model of Quantum Mealy Machines	12
4.3	Equivalence of Quantum Mealy Machines	13
5	Equivalence Checking	14
5.1	Direct Sum of Quantum Mealy Machines	14
5.2	Main Theorem	15
5.3	Algorithm	17
5.3.1	Description of the Algorithm	17
5.3.2	Correctness and Complexity	18
6	Case Studies	19
6.1	Repeat-until-Success Implementation	19
6.2	Quantum Random Walks	19
6.3	Quantum Fourier Transform	20
6.4	Quantum Half Adder	21
7	Experiments	22
8	Conclusion	24
	References	24
A	Hardware Realisation of Sequential Quantum Circuits	28
B	Completing the Proof of Theorem 5.1	30
C	Correctness Proof of Algorithm 1	32

1 Introduction

Hardware verification is emerging as an important issue in quantum computing with the recent rapid progress in quantum hardware implemented by industries like IBM, Google and Intel. A series of testing and verification techniques have already been developed for quantum circuits in the last fifteen years (see for example [1] [2] [3] [4] [5] [6] [7]). More generally, Design Automation technology has been gradually introduced into quantum computing in the last few years (see for example [8] [9] [10]).

Equivalence checking is arguably the most important formal verification technique being employed in the design flows of classical computing hardware. Equivalence checking of quantum circuits was first studied in [11] based on a quantum variant of BDDs (Binary Decision Diagrams), called QuIDD (Quantum Information Decision Diagram). The notion of miter circuit was generalised in [12] to reversible miter so that various simplification techniques of quantum circuits can be used in equivalence checking. In [13], a method for checking equivalence of multiple-valued quantum circuits was introduced by extending data structure XQDD (X-decomposition Quantum Decision Diagram) defined in [14]. In a series of papers [15] [16] [17], a scheme for checking equivalence of quantum circuits has been systematically developed using data structure QMDD (Quantum Multiple-Valued Decision Diagram) [18] [19]. Another way for checking equivalence of quantum circuits was proposed in [20] employing (a finite variant of) Feynman path integral.

As is well-known, digital logic circuits are divided into two categories: **combinational circuits** and **sequential circuits**. The output value of a combinational circuit is a function of only the current input value, while the output value of a sequential circuit depends on not only the external input value but also the stored internal information, which is designed to be fed back to the sequential circuit itself. Almost all practical digital devices contain (classical) sequential circuits.

An early application area of the model of sequential quantum circuits is the quantum control theory [21], where a quantum system is controlled by feeding the information obtained from it back to itself. In the view of sequential logic, the quantum control behaves as a sequential machine that takes the last output state as input at each time step. In particular, it was shown in [22] [23] that coherent quantum feedback control that feeds quantum information coherently back to the system can perform some tasks that cannot be done by a controller processing classical information. Subsequently, the notion of quantum feedback in quantum control theory was extended to the model of quantum feedback networks [24] [25], and a Quantum Hardware Description Language QHDL was designed in [26] for specification of photonic quantum circuits with feedback.

Recently, sequential circuit models started to play an essential role in quantum computing; for example, sequential quantum circuits are employed in the RUS (Repeat-Until-Success) scheme [27] for efficient implementation of quantum logic gates.

Example 1. $V_3 = (I + 2iZ)/\sqrt{5}$ is one of the six gates in the efficiently universal single-qubit gate set [28]. The RUS implementation of V_3 given in [27] is a sequential quantum circuit visualised in Fig. 1 (the quantum gates used there will be explained in Section 2). At each step, it is supposed to

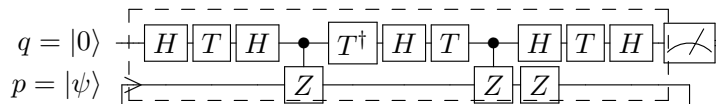


Figure 1: A sequential quantum circuit for RUS implementation of V_3 .

implement V_3 with probability $5/8$ and identity with probability $3/8$. The circuit has two variables

q and p . The outcome of the measurement on q is the output variable, indicating whether V_3 is successful implemented at the current step. Variable p is used to store internal information. In particular, this sequential circuit was physically implemented on superconducting qubits [29].

Contributions of This Paper: The existing research on equivalence checking of quantum circuits focuses on their combinational logic. In this paper, we define a formal framework for equivalence checking of sequential quantum circuits, where a natural quantum generalisation of Mealy machines is adopted to model sequential quantum circuits. Then the notion of equivalence of sequential quantum circuits can be formally defined in terms of the equivalence of quantum Mealy machines.

A major difference between quantum circuits and classical circuits is that the state spaces of quantum circuits are continuums, and in contrast the state spaces of classical circuits are finite. In the case of combinational logic, it has been observed in the previous work mentioned above that equivalence checking of quantum circuits can be done with input sequences taken from some given basis (which are finite). To extend this idea to the case of sequential quantum logic, two additional dimensions of difficulty must be overcome:

- The dependence of the output of a sequential circuit on both the external input and the internal state complicates the procedure of reducing the equivalence checking to a specific basis of the state space;
- Quantum measurements in sequential circuits are essential when checking their equivalence; whereas the measurements in combinational circuits can always be pushed to the end and thus ignored in equivalence checking.

By circumventing these difficulties, we prove that equivalence checking of two quantum Mealy machines can be done with input sequences which are taken from a given basis and have a length quadratic in the dimensions of the state Hilbert spaces of the machines. Based on it, we develop an algorithm for checking equivalence of two quantum Mealy machines (see Theorem 5.2) with time complexity $\mathcal{O}(|O|d_V d^5(d^3 + d_{in}^3))$, where $|O|$ is the number of possible measurement outcomes, d_{in} and d is the dimension of the input and internal Hilbert spaces, and d_V is the dimension of the subspace of input (mixed) states that are being checked. When checking an arbitrary possible input, we take $d_V = \mathcal{O}(d_{in}^2)$. Through this, the equivalence checking of sequential quantum circuits with m input qubits and l internal qubits can be done in $\mathcal{O}(2^{3m+5l}(2^{3m} + 2^{3l}))$ time. Our algorithm is then comparable with the known algorithms for equivalence checking of classical sequential circuits, which also has time complexity exponential in the number of bits.

To illustrate the effectiveness of our model and algorithm, we present several case studies and experiments, including testing of sequential quantum circuits in the above example and (i) quantum random walks, a tool widely used in quantum algorithm design (e.g. search and simulation); (ii) controlled quantum circuits (e.g. Toffoli gate); (iii) a circuit for testing quantum Fourier transforms; and (iv) quantum half adder. Although our examples only use several simple quantum gates, namely the Hadamard gate, CNOT gate, Toffoli gate and one-qubit rotations, the main results of this paper are not limited to these specific gates and can be applied to an arbitrarily given set of quantum gates.

Related Work: The work in [11] [18] [12] [13] [15] [20] mentioned in the above targets checking equivalence of general combinational quantum circuits. Another line of related research [30] [31] [32] but not mentioned above is concerned with equivalence checking of a special class of combinational quantum circuits used in quantum communication protocols, in particular those that can be expressed in the stabiliser formalism.

The formal model employed in this paper is quantum Mealy machines, a kind of quantum finite-state machines. There have been a large number of references for various models of quantum finite-state machines; two of the earliest papers in this area are [33] [34]; for recent surveys, see [35] [36]. The work presented in the following interesting papers are the closest to ours. Quantum finite-state machines were already used in [37] to represent quantum circuits, including sequential quantum circuits, although only several interesting examples are presented in [37], without a formal description of quantum Mealy machines; in particular, equivalence checking of sequential quantum circuits was not considered there. The models proposed in [38] and [39] are similar to ours, but they are committed to accepting strings over a finite input alphabet. However, our model allows inputs to be strings over an infinite alphabet, i.e. sequences of quantum (mixed) states (although in a finite-dimensional Hilbert space). In fact, this extension of inputs to quantum finite-state machines is helpful for checking equivalence of sequential quantum circuits.

Technically, equivalence checking of sequential quantum circuits is resolved by reducing it to that of quantum Mealy machines. Equivalence checking of (weighted) automata has a long history. The first attempt on checking equivalence of probabilistic automata was made in [40], with the observation that two probabilistic automata are equivalent if and only if they produce the same probability distribution on every input of length $\leq n_1 + n_2 - 1$, where n_1 and n_2 are the number of states in the two automata under checking, respectively. Let us call the maximal length of inputs needed in equivalence checking *the checking bound*. Then the above observation implies an equivalence checking algorithm with time complexity exponential in the number of states by directly checking every possible input of length within the checking bound. A remarkable improvement was achieved in [41] with the first polynomial-time algorithm for equivalence checking of probabilistic automata. Koshiba [42] and Liu and Qiu [38] successfully extended the method of [41] to quantum automata. They developed efficient algorithms for equivalence checking of various types of quantum automata with time complexity $O(n^8)$ with checking bound $(n_1 + n_2)^2$, where n_1, n_2 are the respective dimensions of the two quantum automata under checking, and $n = n_1 + n_2$. This checking bound was later improved in [43] to $n_1^2 + n_2^2 - 1$. Recently, the techniques for equivalence checking of quantum automata were further extended in [39] to the case of mixed quantum states, and the time complexity was improved from $O(n^8)$ to $O(n^6)$. The main idea of our method for checking equivalence of quantum Mealy machines in this paper is based on the techniques mentioned above, especially those in [38] and [39], where the machine models have a similar structure to ours.

Organisation of the Paper: For convenience of the reader, we briefly recall basics of combinational quantum circuits in Section 2. A formal description of sequential quantum circuits is given in Section 3. The notion of quantum Mealy machine is introduced in Section 4 as an abstract model of sequential quantum circuits. An algorithm for equivalence checking of quantum Mealy machines is developed in Section 5. Case studies are given in Section 6. The experiments of executing the algorithm on the examples mentioned are briefly discussed in Section 7. A brief conclusion is drawn in Section 8.

2 Combinational Quantum Circuits

In this section, we briefly review the basics of combinational quantum circuits; for more details, we refer to [44].

2.1 Qubits

The basic unit of quantum information is called a quantum bit or qubit. For each qubit q , we write $\mathcal{H}_q$ for its state Hilbert space, which is two-dimensional. Using the Dirac notation, a (pure)

state of q is represented by $|\psi\rangle = \alpha_0|0\rangle + \alpha_1|1\rangle$ with complex numbers α_0 and α_1 satisfying the normalisation condition $|\alpha_0|^2 + |\alpha_1|^2 = 1$; for example, q can be in not only the basis states $|0\rangle, |1\rangle$ but also a superposition of them like: $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$.

A sequence $\bar{q} = q_1, \dots, q_n$ of distinct qubit variables is called a quantum register. Its state Hilbert space is the tensor product $\mathcal{H}_{\bar{q}} = \bigotimes_{i=1}^n \mathcal{H}_{q_i}$, which is 2^n -dimensional. To explicitly describe a state of quantum register $\bar{q}$, let an integer $0 \leq x < 2^n$ be represented by a string $x_1 \dots x_n \in \{0, 1\}^n$ of n bits: $x = \sum_{i=1}^n x_i \cdot 2^{n-i}$. We shall not distinguish the integer x from its binary representation. Thus, each (pure) state in $\mathcal{H}_{\bar{q}}$ can be written as: $|\psi\rangle = \sum_{x=0}^{2^n-1} \alpha_x |x\rangle$, where $\{|x\rangle\}$ is called the computational basis, and complex numbers α_x satisfy the normalisation condition $\sum_{x=0}^{2^n-1} |\alpha_x|^2 = 1$. This state can also be represented by the 2^n -dimensional column vector $|\psi\rangle = (\alpha_0, \dots, \alpha_{2^n-1})^T$, where T stands for transpose. We write $\langle\psi| = (\alpha_0^*, \dots, \alpha_{2^n-1}^*)$ to denote the conjugate transpose of $|\psi\rangle$, and $\langle\psi|\phi\rangle$ to denote the inner product of $|\psi\rangle$ and $|\phi\rangle$. For example, two qubits q_1, q_2 can be in an entangled state like the EPR (Einstein-Podolsky-Rosen) pair: $|\beta\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$.

2.2 Unitary Transformations

A transformation of a quantum register $\bar{q} = q_1, \dots, q_n$ is modelled by a $2^n \times 2^n$ unitary matrix $U = (u_{ij})$ satisfying $U^\dagger U = I$, where $U^\dagger$ stands for the conjugate transpose of U and I the unit matrix. Thus, state $|\psi\rangle$ is transformed by U to a new state represented by column vector $|\psi'\rangle = U|\psi\rangle$ (matrix multiplication), i.e. $|\psi'\rangle = (\alpha'_0, \dots, \alpha'_{2^n-1})^T$ where $\alpha'_i = \sum_{j=0}^{2^n-1} u_{ij} \alpha_j$ for each $i = 0, \dots, 2^n - 1$.

Example 2. *Some unitary transformations used in quantum computing:*

- *Single-qubit transformations: Hadamard matrix $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$, $\pi/8$ gate $T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$, and Pauli matrices $X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, $Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$, $Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$. For instance, H transforms the basis states $|0\rangle, |1\rangle$ to $|+\rangle, |-\rangle$, respectively; that is, $H|0\rangle = |+\rangle$ and $H|1\rangle = |-\rangle$. Similarly, $Y|1\rangle = -i|0\rangle$, $Z|1\rangle = -|1\rangle$.*
- *The 4×4 matrix $CNOT = \begin{pmatrix} I & 0 \\ 0 & X \end{pmatrix}$ represents a unitary transformation of two qubits, called the controlled NOT. It can generate entanglement; for example, it transforms separable state $|+\rangle|0\rangle$ to the EPR pair $|\beta\rangle$.*

2.3 Quantum Measurements

According to a basic postulate of quantum mechanics, a way of reading out data from a quantum system is a quantum measurement. In quantum circuits, we mainly use the measurements in the computational basis. Let $\bar{q} = q_1, \dots, q_n$ be a quantum register. It is divided into two segments $\bar{p} = q_1, \dots, q_m$ ($m \leq n$) and $\bar{r} = q_{m+1}, \dots, q_n$. If $\bar{q}$ is in state $|\psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle$ and we perform a measurement on the first segment $\bar{p}$ in the computational basis, then output $a = (a_1, \dots, a_m) \in \{0, 1\}^m$ is obtained with probability $p(a) = \sum_{x \text{ s.t. } x_i = a_i (1 \leq i \leq m)} |\alpha_x|^2$ and after that, the state of $\bar{r}$ becomes $|\varphi_a\rangle = \frac{1}{\sqrt{p(a)}} \sum_{y \in \{0,1\}^{n-m}} \alpha_{ay} |y\rangle$, where $ay \in \{0, 1\}^n$ is the concatenation of a and y . For example, if two qubits q_1, q_2 are in the entangled state $|\beta\rangle$, and we measure q_1 in the computational basis, then we obtain outcome 0 with probability $\frac{1}{2}$ and after that, q_2 is in the basis state $|0\rangle$, and we obtain outcome 1 also with probability $\frac{1}{2}$ and after that, q_2 is in state $|1\rangle$.

2.4 Quantum Gates and Quantum Circuits

Now we are ready to formally define the notions of quantum gates and quantum circuits. Here, we adopt the algebraic language defined in [45] [46].

Definition 2.1 (Quantum gates). *For any positive integer n , if U is a $2^n \times 2^n$ unitary matrix, and $q_1, \dots, q_n$ are qubit variables, then $G \equiv U[q_1, \dots, q_n]$ is called an n -qubit gate.*

Example 3 (Hadamard gate). *Whenever the Hadamard matrix H acts on a qubit q , then we have the Hadamard gate $H[q]$. If q is currently in state $|1\rangle$, it will be in state $|-\rangle$ at the end of the Hadamard gate.*

Example 4 (Classically controlled quantum gates). *1. Let q_1, q_2 be qubit variables. Then CNOT gate $CNOT[q_1, q_2]$ is a two-qubit gate with q_1 as the control qubit and q_2 as the target qubit. It acts as follows: $CNOT[q_1, q_2]|i_1, i_2\rangle = |i_1, i_1 \oplus i_2\rangle$ for $i_1, i_2 \in \{0, 1\}$, where $\oplus$ is addition modulo 2; that is, if q_1 is set to $|1\rangle$, then q_2 is flipped, otherwise q_2 is left unchanged.*

2. Let q_1, q_2, q_3 be qubit variables. Then the Toffoli gate is defined by $Toffoli[q_1, q_2, q_3]|i_1, i_2, i_3\rangle = |i_1, i_2, i_3 \oplus i_1 i_2\rangle$ for $i_1, i_2, i_3 \in \{0, 1\}$; that is, q_3 is flipped if and only if q_1 and q_2 are both set to $|1\rangle$.

3. More generally, suppose that $c_1, \dots, c_k \in \{0, 1\}$, U is a $2^n \times 2^n$ unitary matrix, and $p_1, \dots, p_k$ and $q_1, \dots, q_n$ are qubit variables. Then we can define a controlled gate:

$$G \equiv C^{c_1} C^{c_2} \dots C^{c_k} U[p_1, p_2, \dots, p_k, q_1, q_2, \dots, q_n]$$

by

$$G|i_1, \dots, i_k, j_1, \dots, j_n\rangle = \begin{cases} |i_1, \dots, i_k\rangle U|j_1, \dots, j_n\rangle & \text{if } i_l = c_l \text{ for } 1 \leq l \leq k; \\ |i_1, \dots, i_k, j_1, \dots, j_n\rangle & \text{otherwise,} \end{cases}$$

where the first k variables $p_1, \dots, p_k$ are control qubits, and the last n variables $q_1, \dots, q_n$ are target qubits. In particular, we have: $CNOT[q_1, q_2] = C^1 X[q_1, q_2]$, $Toffoli[q_1, q_2, q_3] = C^1 C^1 X[q_1, q_2, q_3]$.

Definition 2.2 (Quantum circuits). *A quantum circuit is a sequence of quantum gates: $C \equiv G_1 \dots G_d$, where $d \geq 1$ and $G_1, \dots, G_d$ are quantum gates.*

For a quantum circuit $C = G_1 \dots G_d$, let $\bar{q} = q_1, \dots, q_n$ be all of the qubit variables occurring in C . Then for each $1 \leq i \leq d$, G_i is modelled by a unitary matrix U_i acting on a substring of $\bar{q}$. We often consider U_i as a transformation on the whole string $\bar{q}$; that is, U_i is identified with its cylindrical extension in $\mathcal{H}_{\bar{q}}$. Therefore, C is represented by unitary matrix $U = U_d \dots U_1$.

Definition 2.3 (Equivalence of Circuits). *Let C_1, C_2 be quantum circuits and $\bar{q}$ be the sequence of qubit variables occurring in C_1 and C_2 . Then C_1 and C_2 are called equivalent, written $C_1 = C_2$, if we have: $C_1|\psi\rangle = C_2|\psi\rangle$ for any $|\psi\rangle \in \mathcal{H}_{\bar{q}}$.*

Example 5. *The SWAP gate is defined by $SWAP[q_1, q_2]|i_1, i_2\rangle = |i_2, i_1\rangle$ for $i_1, i_2 \in \{0, 1\}$, which swaps the states of two qubits q_1, q_2 . It can be implemented by the following circuit consisting of three CNOT gates: $SWAP[q_1, q_2] = CNOT[q_1, q_2] CNOT[q_2, q_1] CNOT[q_1, q_2]$.*

3 Sequential Quantum Circuits

In this section, we extend the model of combinational quantum circuits defined in the last section to sequential quantum circuits.

3.1 Definition and Examples

Sequential circuits can be further divided into two subcategories: synchronous logic and asynchronous logic. As the first step, we only consider synchronous sequential quantum circuits in this paper. Recall from [47] that the output values of a sequential (classical) circuit depend both on the current inputs and on stored internal information (and thus also on the past inputs). So, a sequential circuit is a combinational circuit with memory. It is capable of storing data and also performing certain logical operations upon this data. In other words, the output values of a sequential circuit at a given time is then a function of external input values and the stored data at that time. Such a circuit model has a natural quantum generalisation:

Definition 3.1. *A (synchronous) sequential quantum circuit S consists of:*

- m input variables $q_1, \dots, q_m$, each of which is a qubit;
- m output variables $x_1, \dots, x_m$, each of which is a classical binary variable, taking value 0 or 1;
- l state variables $p_1, \dots, p_l$, each of which is also a qubit;
- a combinational quantum circuit C with $n = m + l$ qubits $q_1, \dots, q_m, p_1, \dots, p_l$;

A sequential quantum circuit is visualised in Fig. 2. Intuitively, as in a classical synchronous

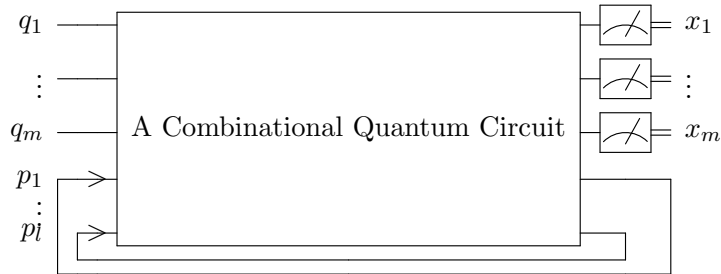


Figure 2: A synchronous sequential quantum circuit.

circuit, a clock produces a sequence of clock signals. Each signal is distributed to all of the memory elements. The output of each memory element only changes when triggered by the clock signal. The quantum hardware implementing a sequential circuit works very differently from its classical counterpart (See Appendix A for a discussion about physical implementation of the “memory” and “clock” in a sequential quantum circuit). Another difference between classical and quantum models is that in the quantum case, a measurement appears at the end of each qubit q_i , which is needed to produce a (classical) outcome x_i . To show reasonableness of the above model, let us consider two examples:

Example 6. The RUS (Repeat-Until-Success) implementation of quantum gate $V_3 = (I + 2iZ)/\sqrt{5}$ given in Example 1 can be formally described as a sequential quantum circuits with the following combinational part:

$$\begin{aligned} \mathcal{C}_{rus} = & H[q]T[q]H[q]C^1Z[q,p]T^\dagger[q]H[q]T[q] \\ & C^1Z[q,p]H[q]Z[p]T[q]H[q]. \end{aligned} \quad (1)$$

Example 7. Quantum random walks are quantum analogues of random walks, with many applications in designing quantum algorithms. Here, we consider a quantum particle moving on a 4-circle [48]:

1. The circle has 4 different positions: 0, 1, 2, 3, and thus the position space is a 4-dimensional Hilbert space $\mathcal{H}_p = \text{span}\{|0\rangle_p, |1\rangle_p, |2\rangle_p, |3\rangle_p\}$, that is, the complex vector space with $\{|0\rangle, |1\rangle, |2\rangle, |3\rangle\}$ as an orthonormal basis. In contrast to the classical case, the particle can be in a superposition of positions 0, 1, 2, 3, written $\alpha_0|0\rangle + \alpha_1|1\rangle + \alpha_2|2\rangle + \alpha_3|3\rangle$.
2. A quantum coin is used to control the walking direction. Its state space is $\mathcal{H}_c = \text{span}\{|0\rangle_c, |1\rangle_c\}$, where 0, 1 denote direction right and left, respectively. In contrast to the classical case, the coin can be in a superposition $\beta_0|0\rangle + \beta_1|1\rangle$, meaning the particle moving to left and right simultaneously.
3. In each step of the walk, the coin is flipped with a unitary operator C , and then the conditional translation of position is performed: $T|0\rangle_c|i\rangle_p = |0\rangle_c|i+1\rangle_p$ and $T|1\rangle_c|i\rangle_p = |1\rangle_c|i-1\rangle_p$ for $i = 0, 1, 2, 3$, where $i+1$ and $i-1$ are computed modulo 4. Here, we use the subscripts c, p to indicate the states of coin and position, respectively. Intuitively, if the coin is in state 0 (resp. 1), then the position is changed from i to $i+1$ (resp. $i-1$).
4. A qubit d together with a measurement is set to detect whether position 3 is reached. Whenever position 3 is reached, the detective qubit d is flipped.

As usual, we can employ two qubits p_1 and p_2 to encode the 4 positions: $|0\rangle_p = |0\rangle_{p_1}|0\rangle_{p_2}$, $|1\rangle_p = |0\rangle_{p_1}|1\rangle_{p_2}$, $|2\rangle_p = |1\rangle_{p_1}|0\rangle_{p_2}$, $|3\rangle_p = |1\rangle_{p_1}|1\rangle_{p_2}$. Then the walk can be implemented as a sequential

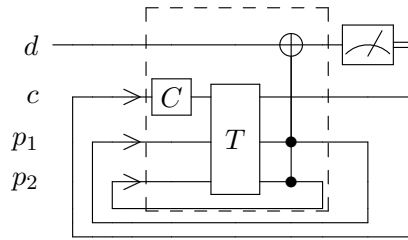


Figure 3: A sequential circuit for detecting quantum random walk.

quantum circuit shown in Fig. 3. This circuit has four variables d, c, p_1 and p_2 , where d is the only input variable, and c, p_1 and p_2 are state variables. The outcome of measurement on d is the output variable. The combinational part of the circuit is defined by

$$\mathcal{C}_C = C[c]T[c, p_1, p_2] \text{Toffoli}[p_1, p_2, d]. \quad (2)$$

3.2 Computation Process

Now we give a formal description of the computational process of a sequential quantum circuit. From the discussion in the last section, the combinational circuit C with n qubits is represented by a $2^n \times 2^n$ unitary matrix $U = (U_{uv})_{u,v=0}^{2^n-1}$. Moreover, we need the following notations: for any column vector $|\psi\rangle = (\alpha_0, \alpha_1, \dots, \alpha_{2^n-1})^T$, its conjugate transpose is row vector $\langle\psi| = (\alpha_0^*, \alpha_1^*, \dots, \alpha_{2^n-1}^*)$. Thus, the inner product of two vectors $|\psi\rangle, |\varphi\rangle$ is $\langle\psi|\varphi\rangle$ (as an ordinary matrix multiplication), and matrix entries $U_{uv} = \langle u|U|v\rangle$ if we identify integers $u, v \in \{0, 1, \dots, 2^n - 1\}$ with their binary representations $u, v \in \{0, 1\}^n$.

Let $|\varphi_1\rangle, \dots, |\varphi_i\rangle, \dots$ in $\mathcal{H}_2^{\otimes m}$ (the tensor product of m copies of the 2-dimensional Hilbert space) be an input sequence and $|\psi_0\rangle$ in $\mathcal{H}_2^{\otimes l}$ an initial state. Then the sequential circuit S behaves as follows:

Step i ($i \geq 1$): Input state $|\varphi_i\rangle = \sum_{x \in \{0,1\}^m} \alpha_{ix} |x\rangle \in \mathcal{H}_2^{\otimes m}$ to variables $q_1, \dots, q_m$. Note that at the end of step $i-1$, variables $p_1, \dots, p_l$ are in state $|\psi_{i-1}\rangle \in \mathcal{H}_2^{\otimes l}$, and this state is stored in the memory elements. Now it is output from the memory elements to the combinational circuit. Assume that $|\psi_{i-1}\rangle = \sum_{y \in \{0,1\}^l} \beta_{(i-1)y} |y\rangle$. Unitary U transforms the state of $q_1, \dots, q_m, p_1, \dots, p_l$ to:

$$U|\varphi_i\rangle|\psi_{i-1}\rangle = \sum_{a \in \{0,1\}^m, b \in \{0,1\}^l} \left(\sum_{x \in \{0,1\}^m, y \in \{0,1\}^l} \alpha_{ix} \beta_{(i-1)y} \langle ab|U|xy\rangle \right) |ab\rangle.$$

Then we measure $q_1, \dots, q_m$ in the computational basis, and the outcome $a_i = a_{i1}, \dots, a_{im} \in \{0, 1\}^m$ is output through $x_1, \dots, x_m$, respectively, with probability

$$p(a_i) = \|\langle a_i|U|\varphi_i\rangle|\psi_{i-1}\rangle\|^2 = \sum_{b \in \{0,1\}^l} \left| \sum_{x \in \{0,1\}^m, y \in \{0,1\}^l} \alpha_{ix} \beta_{(i-1)y} \langle a_i b|U|xy\rangle \right|^2. \quad (3)$$

And variable $p_1, \dots, p_l$ is in state

$$|\psi_i\rangle = \frac{\langle a_i|U|\varphi_i\rangle|\psi_{i-1}\rangle}{\sqrt{p(a_i)}} = \frac{1}{\sqrt{p(a_i)}} \sum_{b \in \{0,1\}^l} \left(\sum_{x \in \{0,1\}^m, y \in \{0,1\}^l} \alpha_{ix} \beta_{(i-1)y} \langle a_i b|U|xy\rangle \right) |b\rangle, \quad (4)$$

which is input to the memory elements and stored there.

The relationship between input $|\varphi_i\rangle$, output a_i and state variables $|\psi_i\rangle$ at each step is visualised in Fig. 4.

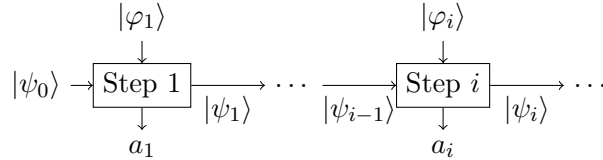


Figure 4: An illustration of the execution of a sequential quantum circuit.

3.3 Equivalence of Sequential Quantum Circuits

For an unknown sequential quantum circuit S with m input variables and l state variables, the only information that can be obtained is the measurement outcome after each step. Here, the measurement outcome is an l -bit binary string. Regarding the sequential quantum circuits as black-boxes,

the only way to distinguish them is to compare their measurement outcomes through experimental statistics. We note that the input states can be selected arbitrarily, but the measurement outcomes are probabilistic. Two sequential quantum circuits are said to be equivalent if they cannot be distinguished (with a high probability) through these experiments.

Formally, suppose a sequence of input states in $\mathcal{H}_2^{\otimes m}$ is selected to be $\varphi = |\varphi_1\rangle, \dots, |\varphi_K\rangle$. Then the sequential quantum circuit S with initial state $|\psi_0\rangle \in \mathcal{H}^{\otimes l}$ on this input sequence will produce a sequence of measurement outcomes $a = a_1, \dots, a_K$ with probability $p_S(a|\varphi, |\psi_0\rangle)$ (according to equations (3) and (4)). A natural definition of equivalence of sequential quantum circuits is given as follows.

Definition 3.2 (Equivalence of Sequential Quantum Circuits). *Suppose S_1 and S_2 are two sequential quantum circuits with m input variables and l_1 and l_2 state variables initialized to $|\psi_0^{(1)}\rangle$ and $|\psi_0^{(2)}\rangle$, respectively. Then S_1 and S_2 is said to be equivalent, if for every input sequence $\varphi = |\varphi_1\rangle, \dots, |\varphi_K\rangle$ and output sequence $a = a_1, \dots, a_K$, S_1 and S_2 on input sequence φ produce the output sequence a with the same probability, i.e. $p_{S_1}(a|\varphi, |\psi_0^{(1)}\rangle) = p_{S_2}(a|\varphi, |\psi_0^{(2)}\rangle)$.*

Remark 3.1. *Several different notions of equivalence for (combinational) quantum circuits have been introduced in the previous literature (see for example [11]). The strongest one requires that the output quantum states must be the same (see Definition 2.3 in this paper). The notion of equivalence defined above is weaker than it and only requires that the difference between the output quantum states cannot be detected by the external observation through certain quantum measurements. The applicability of this kind of equivalence is shown by the examples given in Section 6.*

4 Quantum Mealy Machines

The Mealy machine [49] is a standard model of classical sequential circuits as a kind of finite-state machine. In this section, we introduce a quantum generalisation of Mealy machines, and show how sequential quantum circuits can be modelled as quantum Mealy machines.

4.1 Preliminaries and Notations

A formal definition of a quantum Mealy machine requires more preliminaries from the quantum theory, which we briefly review here (More details can be found in the standard textbook [44]). A mixed quantum state is modelled as an ensemble $\{(p_i, |\psi_i\rangle)\}$ of pure states, meaning that the system is in state $|\psi_i\rangle$ with probability p_i , where $\sum_i p_i = 1$. It can also be described as a density matrix: $\rho = \sum_i p_i |\psi_i\rangle\langle\psi_i|$. Let $\mathcal{D}(\mathcal{H})$ denote the set of all density matrices in Hilbert space $\mathcal{H}$. After a unitary transform U is performed on a quantum system in mixed state ρ , the system will be in state $U\rho U^\dagger$. A quantum measurement is modelled by a family $M = \{M_m\}$ of matrices satisfying a normalisation condition $\sum_m M_m^\dagger M_m = I$ (the identity matrix in $\mathcal{H}$), where index m is used to denote different possible outcomes. When we perform M on a quantum system in state ρ , outcome m is obtained with probability $p(m) = \text{tr}(M_m \rho M_m^\dagger)$. Right after the measurement, if the outcome is m , the system will be in state $\rho' = \frac{1}{p(m)} M_m \rho M_m^\dagger$.

Let A and B be two quantum systems. Then the state space of the composite system AB is the tensor product $\mathcal{H}_A \otimes \mathcal{H}_B$ of A 's state space $\mathcal{H}_A$ and B 's state space $\mathcal{H}_B$. Formally, suppose $\{|i\rangle_A\}$ and $\{|j\rangle_B\}$ are orthonormal bases of $\mathcal{H}_A$ and $\mathcal{H}_B$, respectively. Then $\mathcal{H}_A \otimes \mathcal{H}_B = \text{span}\{|i\rangle_A \otimes |j\rangle_B\}$, the complex vector space with $\{|i\rangle_A \otimes |j\rangle_B\}$ as an orthonormal basis. We often use $|i\rangle_A |j\rangle_B$ or simply $|ij\rangle$ to denote $|i\rangle_A \otimes |j\rangle_B$ for short. The inner product of $|ij\rangle$ and $|vw\rangle$ is defined by $\langle ij|vw\rangle = \langle i|v\rangle \langle j|w\rangle$. The state of a subsystem can be described using a reduced density operator. Let ρ be a density

operator in $\mathcal{H}_A \otimes \mathcal{H}_B$. Then its reduced density operator for A is given as $\rho^A = \text{tr}_B(\rho)$, where tr_B is a linear map, known as partial trace over B , defined by $\text{tr}_B(|i\rangle_A \langle v| \otimes |j\rangle_B \langle w|) = \langle j|w\rangle |i\rangle_A \langle v|$. Suppose U_A and U_B are two operators on $\mathcal{H}_A$ and $\mathcal{H}_B$, respectively. Then the tensor product $U_A \otimes U_B$ is an operator in $\mathcal{H}_A \otimes \mathcal{H}_B$, defined by $(U_A \otimes U_B)(|i\rangle_A \otimes |j\rangle_B) = U_A|i\rangle_A \otimes U_B|j\rangle_B$. Moreover, the direct sum of spaces $\mathcal{H}_A$ and $\mathcal{H}_B$ is defined as $\mathcal{H}_A \oplus \mathcal{H}_B = \text{span}(\{|i\rangle_A\} \cup \{|j\rangle_B\})$ provided the two bases are disjoint. Then the direct sum of operators U_A in $\mathcal{H}_A$ and U_B in $\mathcal{H}_B$ is defined by $(U_A \oplus U_B)|i\rangle_A = U_A|i\rangle_A$ and $(U_A \oplus U_B)|j\rangle_B = U_B|j\rangle_B$.

4.2 The Model of Quantum Mealy Machines

With the preparations in the last subsection, a quantum Mealy machine can be defined as a straightforward abstraction of the model of sequential quantum circuits described in the last section.

Definition 4.1 (Quantum Mealy machines). *A quantum Mealy machine is a quintuple $\mathcal{M} = (\mathcal{H}_{in}, \mathcal{H}_s, U, M)$, where: $\mathcal{H}_{in}$ and $\mathcal{H}_s$ are both finite-dimensional Hilbert spaces, called the input space and the state space, respectively; U is a unitary operator on $\mathcal{H}_{in} \otimes \mathcal{H}_s$; and $M = \{M_a : a \in O\}$ is a quantum measurement in $\mathcal{H}_{in} \otimes \mathcal{H}_s$.*

The examples of sequential quantum circuits presented in the previous section can be properly specified using the notion of quantum Mealy machine. In our examples, we only use measurements in $\mathcal{H}_{in}$, but for generality, M in the above definition is allowed to be a measurement in $\mathcal{H}_{in} \otimes \mathcal{H}_s$. As will be seen in Section 5, this generalisation is particularly convenient in the description of equivalence checking of two quantum Mealy machines.

Example 8. *The RUS implementation circuit of quantum gate V_3 in Examples 1 and 6 can be described as a quantum Mealy machine $\mathcal{M}_{rus} = (\mathcal{H}_q, \mathcal{H}_p, U, M)$, where: input space $\mathcal{H}_q = \text{span}\{|0\rangle_q, |1\rangle_q\}$, state space $\mathcal{H}_p = \text{span}\{|0\rangle_p, |1\rangle_p\}$, unitary operator U describes the combinational quantum circuit $\mathcal{C}_{rus}$ defined by equation 1, and $M = \{M_0 = |0\rangle_q \langle 0|, M_1 = |1\rangle_q \langle 1|\}$ is the measurement in the computational basis of $\mathcal{H}_q$.*

Example 9. *The circuit implementing the quantum walk on a 4-circle in Example 7 can be described as a quantum Mealy machine $\mathcal{M}_C = (\mathcal{H}_d, \mathcal{H}_c \otimes \mathcal{H}_p, U, M)$, where: input space $\mathcal{H}_d = \text{span}\{|0\rangle_d, |1\rangle_d\}$, state space $\mathcal{H}_c \otimes \mathcal{H}_p$ with $\mathcal{H}_c = \text{span}\{|0\rangle_c, |1\rangle_c\}$ and $\mathcal{H}_p = \mathcal{H}_{p_1} \otimes \mathcal{H}_{p_2}$, where $\mathcal{H}_{p_i} = \text{span}\{|0\rangle_{p_i}, |1\rangle_{p_i}\}$ for $i = 1, 2$, unitary operator U describes the combinational quantum circuit $\mathcal{C}_C$ defined by equation 2, and $M = \{M_0 = |0\rangle_d \langle 0|, M_1 = |1\rangle_d \langle 1|\}$ is the measurement in the computational basis of $\mathcal{H}_d$.*

Now let us formally describe how a quantum Mealy machine runs. To simplify the description, for each measurement outcome $a \in O$ and input state σ , we introduce a (super-)operator: for every ρ in $\mathcal{H}_s$,

$$\mathcal{E}_{a|\sigma}(\rho) = \text{tr}_{\mathcal{H}_{in}} \left[M_a U (\sigma \otimes \rho) U^\dagger M_a^\dagger \right], \quad (5)$$

where $\text{tr}_{\mathcal{H}_{in}}$ is used to trace out the input subsystem $\mathcal{H}_{in}$. Then for any input sequence $\sigma_1, \dots, \sigma_i, \dots$ in $\mathcal{H}_{in}$ and initial (mixed) state ρ_0 in $\mathcal{H}_s$, the behaviour of machine $\mathcal{M}$ can be described as follows:

- **Step i ($i \geq 1$):** After receiving the i th input, $\mathcal{M}$ is in state $\sigma_i \otimes \rho_{i-1}$. Unitary U transforms it to $U(\sigma_i \otimes \rho_{i-1})U^\dagger$. Then we measure subsystem $\mathcal{H}_{in}$, outcome $a_i \in O$ is output with probability

$$p(a_i|\sigma_i, \rho_{i-1}) = \text{tr} \left[M_{a_i} U (\sigma_i \otimes \rho_{i-1}) U^\dagger M_{a_i}^\dagger \right], \quad (6)$$

and subsystem $\mathcal{H}_s$ is in state

$$\rho_i = \frac{1}{p(a_i|\sigma_i, \rho_{i-1})} \mathcal{E}_{a_i|\sigma_i}(\rho_{i-1}). \quad (7)$$

It is easy to see that equations (3) and (4) are special cases of (6) and (7), respectively.

The application of an input sequence is called an *experiment* on the machine. More explicitly, an input sequence $\pi = \sigma_1, \dots, \sigma_k$ and an initial state ρ_0 induce a probability distribution $p(\cdot|\pi, \rho_0)$ over output sequences O^k of length k : for every $a = a_1, \dots, a_k \in O^k$, $p(a|\pi, \rho_0) = \prod_{i=1}^k p(a_i|\sigma_i, \rho_{i-1})$, where $p(a_i|\sigma_i, \rho_{i-1})$ is defined by equation (6).

Remark 4.1. *The reader should have noticed that in the above description of running a quantum Mealy machine, the initial state in $\mathcal{H}_s$ is taken as a mixed state ρ_0 rather than a pure state $|\psi_0\rangle$. The reason behind this design decision is that even if the machine starts in a pure initial state $|\psi_0\rangle$, the subsequent states $\rho_1, \dots, \rho_i, \dots$ can still be mixed states because they are obtained by tracing out the input subsystem $\mathcal{H}_{in}$ (see equation (7)).*

Remark 4.2. *At the first glance, the notion of quantum Mealy machine defined above is similar to the one defined in [39], but they are actually very different. The one defined above is designed for modelling sequential quantum circuits, whereas the one in [39] was used for modelling combinational quantum circuits. This difference between the design purposes leads to the differences between the components of machines. First, the input to the machine in [39] is a string over a finite alphabet (that is, a sequence of classical states), while that of the machine in this paper is a sequence of quantum states in a finite-dimensional Hilbert space. In an execution, the input of the machine in [39] indicates which unitary operator is performed while the input to the machine in this paper coherently gets involved in the computation and possibly becomes entangled with the internal state of the machine. Thus, to fetch information, we must perform measurements on both input and state Hilbert spaces, rather than the only state space as in [39]. Thus we need to use partial trace (see equation (5)) to trace out the input space, while [39] does not. Furthermore, the intermediate states during the execution are mixed states in both [39] and this paper, but the reasons are different. In [39], the initial (internal) state is allowed to be a mixed state and thus the subsequent intermediate states would be mixed states too. On the contrary, the intermediate mixed states are inevitable in the machine studied in this paper. Even if the initial internal state is a pure state, the intermediate states could still be mixed states due to the possible entanglement between the input and internal states before measurement.*

4.3 Equivalence of Quantum Mealy Machines

To conclude this section, we introduce the key notion of equivalence for quantum Mealy machines.

Definition 4.2 (Equivalence of quantum Mealy machines). *Let $\mathcal{M}_i = (\mathcal{H}_{in}, \mathcal{H}_s^{(i)}, U_i, M_i)$ ($i = 1, 2$) be two quantum Mealy machines with the same input space $\mathcal{H}_{in}$ and the same outputs O (i.e. measurements $M_i = \{M_a^{(i)} : a \in O\}$ for $i = 1, 2$), let $\rho_i \in \mathcal{D}(\mathcal{H}_s^{(i)})$ ($i = 1, 2$), $V \subseteq \mathcal{D}(\mathcal{H}_{in})$, and let K be a positive integer. Then:*

1. $\mathcal{M}_1$ with initial state ρ_1 and $\mathcal{M}_2$ with initial state ρ_2 are (functionally) equivalent, written: $(\mathcal{M}_1, \rho_1) \sim (\mathcal{M}_2, \rho_2)$, if for any input sequence π and for any $a \in O^{|\pi|}$,

$$p(a|\pi, \rho_1) = p(a|\pi, \rho_2). \quad (8)$$

Note that the probability in the left-hand side of the above equation is defined in machine $\mathcal{M}_1$ with ρ_1 as its initial state, and the probability in the right-hand side is given in $\mathcal{M}_2$ with initial state ρ_2 .

2. $\mathcal{M}_1$ with ρ_1 and $\mathcal{M}_2$ with ρ_2 are V -equivalent, written: $(\mathcal{M}_1, \rho_1) \sim_V (\mathcal{M}_2, \rho_2)$, if equation (8) holds for all input sequences π in V .

3. $\mathcal{M}_1$ with ρ_1 and $\mathcal{M}_2$ with ρ_2 are K -equivalent, written: $(\mathcal{M}_1, \rho_1) \sim_K (\mathcal{M}_2, \rho_2)$, if equation [\(8\)](#) holds for all input sequences π with $|\pi| \leq K$.
4. $\mathcal{M}_1$ with ρ_1 and $\mathcal{M}_2$ with ρ_2 are (V, K) -equivalent, written: $(\mathcal{M}_1, \rho_1) \sim_{V,K} (\mathcal{M}_2, \rho_2)$, if equation [\(8\)](#) holds for all input sequences π in V with $|\pi| \leq K$.

Intuitively, $(\mathcal{M}_1, \rho_1) \sim (\mathcal{M}_2, \rho_2)$ means the probability distributions of the outputs of the two machines $\mathcal{M}_1$ and $\mathcal{M}_2$ are the same for any sequence of inputs, but V -equivalence only requires the distributions of outputs are the same for all inputs from a given range V , and K -equivalence only requires the distributions are the same for all input and output sequences of length not greater than K . Whenever $\mathcal{M}_1$ and $\mathcal{M}_2$ are the same and $(\mathcal{M}_1, \rho_1) \sim (\mathcal{M}_2, \rho_2)$ (or $(\mathcal{M}_1, \rho_1) \sim_V (\mathcal{M}_2, \rho_2)$, $(\mathcal{M}_1, \rho_1) \sim_K (\mathcal{M}_2, \rho_2)$, $(\mathcal{M}_1, \rho_1) \sim_{V,K} (\mathcal{M}_2, \rho_2)$), we simply say that ρ_1 and ρ_2 are equivalent (resp. V -equivalent, K -equivalent, (V, K) -equivalent) and write $\rho_1 \sim \rho_2$ (resp. $\rho_1 \sim_V \rho_2$, $\rho_1 \sim_K \rho_2$, $\rho_1 \sim_{V,K} \rho_2$). When two pure states $|\psi_1\rangle$ and $|\psi_2\rangle$ are equivalent, i.e. $|\psi_1\rangle\langle\psi_1| \sim |\psi_2\rangle\langle\psi_2|$, we simply write $|\psi_1\rangle \sim |\psi_2\rangle$.

To conclude this section, let us see how the equivalence of quantum Mealy machines is linked to that of sequential quantum circuits. Suppose S is a sequential quantum circuit with m input variables and l state variables. Then S can be modelled by a quantum Mealy machine $\mathcal{M}(S) = (\mathcal{H}_{in}, \mathcal{H}_s, U, M)$, where $\mathcal{H}_{in} = \mathcal{H}_2^{\otimes m}$, $\mathcal{H}_s = \mathcal{H}_2^{\otimes l}$, U is the combinational quantum circuit of S , and $M = \{M_a = |a\rangle_{in}\langle a| : a \in \{0, 1\}^m\}$ is the measurement. Then we have:

Theorem 4.1. *Suppose S_1 and S_2 are two sequential quantum circuits as Definition [3.2](#). Then S_1 and S_2 are equivalent if and only if $(\mathcal{M}(S_1), |\psi_0^{(1)}\rangle) \sim (\mathcal{M}(S_2), |\psi_0^{(2)}\rangle)$.*

Proof: It is straightforward from the definitions. ■

The above theorem provides a basis for using quantum Mealy machines in equivalence checking of sequential quantum circuits. To see this more clearly, some examples are provided in Section [4.2](#).

5 Equivalence Checking

In this section, we present the main theoretical result of this paper. In particular, we develop an algorithm for checking equivalence of two quantum Mealy machines with initial states.

5.1 Direct Sum of Quantum Mealy Machines

The basic idea of our algorithm is a reduction from equivalence checking of two quantum Mealy machines with initial states to equivalence checking of two states in their direct sum, which is defined in the following:

Definition 5.1 (Direct sum of quantum Mealy machines). *Let $\mathcal{M}_i$ ($i = 1, 2$) be the same as in Definition [4.2](#). Then the direct sum of $\mathcal{M}_1$ and $\mathcal{M}_2$ is quantum Mealy machine*

$$\mathcal{M}_1 \oplus \mathcal{M}_2 = \left(\mathcal{H}_{in}, \mathcal{H}_s^{(1)} \oplus \mathcal{H}_s^{(2)}, U_1 \oplus U_2, M \right),$$

where $\oplus$ in the right-hand side of the above equation stands for direct sum of two Hilbert spaces or two operators, and measurement $M = \{M_a : a \in O\}$ with $M_a = M_a^{(1)} \oplus M_a^{(2)}$ for every $a \in O$.

To see that $\mathcal{M}_1 \oplus \mathcal{M}_2$ is well-defined as a quantum Mealy machine, it suffices to note that for each $a \in O$, $M_a = M_a^{(1)} \oplus M_a^{(2)}$ is a matrix in $(\mathcal{H}_{in} \otimes \mathcal{H}_s^{(1)}) \oplus (\mathcal{H}_{in} \otimes \mathcal{H}_s^{(2)}) \cong \mathcal{H}_{in} \otimes (\mathcal{H}_s^{(1)} \oplus \mathcal{H}_s^{(2)})$, and normalisation condition $\sum_{a \in O} M_a^\dagger M_a = I$ (the identity matrix in $\mathcal{H}_{in} \otimes (\mathcal{H}_s^{(1)} \oplus \mathcal{H}_s^{(2)})$) is satisfied.

Remark 5.1. In [39], since the input is a sequence of classical states (that is, a string over a finite alphabet), the direct sum of two machines can be defined straightforwardly. In contrast, as discussed in Remark 4.2, the input to the machine considered in this paper is a sequence of quantum states that can be entangled with the internal states. Then a major difficulty in defining the sum of two machines is to make the input entangled with the internal states of the two machines “simultaneously”. To resolve this issue, we define the measurement operators of the sum to be on Hilbert space $\mathcal{H}_{in} \otimes (\mathcal{H}_s^{(1)} \oplus \mathcal{H}_s^{(2)})$, which keep the behaviour of each machine as if they did not interfere with each other. In comparison, the measurement operators of the sum machine defined in [39] are on $\mathcal{H}_s^{(1)} \oplus \mathcal{H}_s^{(2)}$.

5.2 Main Theorem

There are two major difficulties in designing an algorithm for equivalence checking of quantum circuits. As in checking classical circuits, it is desirable to find an upper bound of the length of the needed input sequences. On the other hand, the input state space of a classical circuit is finite; in contrast, the input state of a quantum circuit is a continuum. These difficulties are resolved by the following theorem showing that equivalence checking of quantum Mealy machines can be done using only input sequences taken from some given basis and of length quadratic in the dimensions of the state Hilbert spaces of the machines. Surprisingly, the length of checking input sequences is independent of the dimension of the input space.

Theorem 5.1. Let $\mathcal{M}_i = (\mathcal{H}_{in}, \mathcal{H}_s^{(i)}, U_i, M_i)$ ($i = 1, 2$) be two quantum Mealy machines with the same input space $\mathcal{H}_{in}$ and the same outputs O (i.e. measurements $M_i = \{M_a^{(i)} : a \in O\}$ for $i = 1, 2$), let $\rho_i \in \mathcal{D}(\mathcal{H}_s^{(i)})$ ($i = 1, 2$), and let V be a subspace of $\mathcal{D}(\mathcal{H}_{in})$. Then the following two statements are equivalent:

1. $(\mathcal{M}_1, \rho_1) \sim_V (\mathcal{M}_2, \rho_2)$.
2. $\rho_1 \sim_{B, d_1^2 + d_2^2 - 1} \rho_2$ (in $\mathcal{M}_1 \oplus \mathcal{M}_2$), where B is a basis of V , and $d_i = \dim \mathcal{H}_s^{(i)}$ for $i = 1, 2$.

Proof: Let us first outline the basic idea of the proof. We are going to prove the theorem in three steps:

- (i) $(\mathcal{M}_1, \rho_1) \sim_V (\mathcal{M}_2, \rho_2)$ iff $\rho_1 \sim_V \rho_2$ in $\mathcal{M}_1 \oplus \mathcal{M}_2$;
- (ii) $\rho_1 \sim_B \rho_2$ in $\mathcal{M}$ iff $\rho_1 \sim_{\text{span } B} \rho_2$, where $\text{span } B$ is the subspace spanned by B ;
- (iii) $\rho_1 \sim_B \rho_2$ iff $\rho_1 \sim_{B, d^2 - 1} \rho_2$, where d is the dimension of the state Hilbert space of $\mathcal{M}$.

The first two steps are intuitive and we put their tedious details in Appendix B. To prove step (iii), let ρ be a density operator and $B \subseteq \mathcal{D}(\mathcal{H}_{in})$ be a finite set with $\text{span } B = \text{span } V$. By step (ii), we have $\rho_1 \sim_V \rho_2$ iff $\rho_1 \sim_B \rho_2$. Let $D(\rho, m) = \{\mathcal{E}_{a|\pi}(\rho) : |\pi| \leq m, a \in O^{|\pi|}, \pi_i \in B\}$. We observe that $D(\rho, m) \subseteq D(\rho, m+1)$, and thus $\text{span } D(\rho, m) \subseteq \text{span } D(\rho, m+1)$ for every $m \in \mathbb{N}$.

Claim. If for some $m \in \mathbb{N}$, $\text{span } D(\rho, m) = \text{span } D(\rho, m+1)$, then $\text{span } D(\rho, m) \subseteq \text{span } D(\rho, m+\delta)$ for all $\delta \in \mathbb{N}$.

We prove this claim by induction. It is trivial for the case $\delta = 1$. Suppose it holds for $\delta > 0$. Then for every $t \leq m + \delta + 1$ and $\mathcal{E}_{a_1 \dots a_t | \sigma_1 \dots \sigma_t}(\rho) \in D(\rho, m + \delta + 1)$, we have: $\mathcal{E}_{a_1 \dots a_{t-1} | \sigma_1 \dots \sigma_{t-1}}(\rho) \in$

$\text{span } D(\rho, m + \delta) = \text{span } D(\rho, m)$ by the induction hypothesis, and

$$\begin{aligned}\mathcal{E}_{a_1 \dots a_t | \sigma_1 \dots \sigma_t}(\rho) &= \mathcal{E}_{a_t | \sigma_t}(\mathcal{E}_{a_1 \dots a_{t-1} | \sigma_1 \dots \sigma_{t-1}}(\rho)) \\ &\in \mathcal{E}_{a_t | \sigma_t}(\text{span } D(\rho, m + \delta)) \\ &= \mathcal{E}_{a_t | \sigma_t}(\text{span } D(\rho, m)) \\ &\subseteq \text{span } D(\rho, m + 1) = \text{span } D(\rho, m),\end{aligned}$$

which implies that $\text{span } D(\rho, m + \delta + 1) = \text{span } D(\rho, m)$, that is, the claim is true for $\delta + 1$.

Now we notice that the claim implies: $\text{span } D(\rho, d^2 - 1) \supseteq \text{span } D(\rho, m)$ for every $m \in \mathbb{N}$ because $\dim \text{span } D(\rho, m) \leq d^2$ and $\dim \text{span } D(\rho, 0) = 1$. Suppose ρ_1 and ρ_2 are $(B, d^2 - 1)$ -equivalent. Then $\text{tr}(\varrho) = 0$ for any $\varrho \in D(\rho, d^2 - 1)$, where $\rho = \rho_1 - \rho_2$. On the other hand, for every input sequence π and $a \in O^{|\pi|}$, we have: $\mathcal{E}_{a|\pi}(\rho) \in D(\rho, |\pi|) \subseteq \text{span } D(\rho, d^2 - 1)$. Thus, $p(a|\pi, \rho_1) - p(a|\pi, \rho_2) = \text{tr}[\mathcal{E}_{a|\pi}(\rho_1)] - \text{tr}[\mathcal{E}_{a|\pi}(\rho_2)] = \text{tr}[\mathcal{E}_{a|\pi}(\rho_1 - \rho_2)] = \text{tr}[\mathcal{E}_{a|\pi}(\rho)] = 0$, and this completes the proof of step (iii).

Finally, we can prove Theorem 5.1 by gluing the three steps. Our strategy is to show that clauses 1) and 2) of Theorem 5.1 and the following two statements are all equivalent:

- a) $\rho_1 \sim_B \rho_2$ (in $\mathcal{M}_1 \oplus \mathcal{M}_2$), where B is a basis of V ;
- b) $(\mathcal{M}_1, \rho_1) \sim_{d_1^2 + d_2^2 - 1} (\mathcal{M}_2, \rho_2)$.

a) $\implies$ 2) and 2) $\iff$ b) are trivial. By steps (i) and (ii), it holds that 1) $\iff$ a). Therefore, we only need to prove that 2) $\implies$ a). Let $\mathcal{E}_{a|\pi}^{(1)}, \mathcal{E}_{a|\pi}^{(2)}$ and $\mathcal{E}_{a|\pi}$ be the super-operator defined in $\mathcal{M}_1, \mathcal{M}_2$ and $\mathcal{M} = \mathcal{M}_1 \oplus \mathcal{M}_2$ by equation (5) and generalised by equation (9), respectively, and let $\rho = \rho_1 \oplus (-\rho_2)$. Similar to the proof of step (iii), we define $D(\rho, m) = \{\mathcal{E}_{a|\pi}(\rho) : |\pi| \leq m, a \in O^{|\pi|}, \pi_i \in B\}$. From the definition of $\mathcal{M}_1 \oplus \mathcal{M}_2$, it holds that $\mathcal{E}_{a|\pi}(\rho) = \mathcal{E}_{a|\pi}^{(1)}(\rho_1) \oplus (-\mathcal{E}_{a|\pi}^{(2)}(\rho_2))$. Thus, we conclude that $D(\rho, m) \subseteq \mathcal{D}(\mathcal{H}_s^{(1)}) \oplus \mathcal{D}(\mathcal{H}_s^{(2)})$, which implies:

$$\dim \text{span } D(\rho, m) \leq \dim \mathcal{D}(\mathcal{H}_s^{(1)}) + \dim \mathcal{D}(\mathcal{H}_s^{(2)}) \leq d_1^2 + d_2^2.$$

With the above inequality, we obtain that $\text{span } D(\rho, d_1^2 + d_2^2 - 1) \supseteq \text{span } D(\rho, m)$ for every $m \in \mathbb{N}$. Then the remaining part of the proof can be carried out in the same way as the proof of step (iii). $\blacksquare$

Remark 5.2. The reader might noticed that high-level structure of the above proof is similar to the proof of the main result in [39]. This reason is that both of the proofs are quantum generalization of the technique of [41] for probabilistic automata, which is now a standard and basic technique in automata theory. However, the difference between the machines studied in [39] and this paper as well as the difference between their direct sums (see Remarks 4.2 and 5.1) leads to some differences in the proof details. In particular, the input to the quantum automata considered in the previous literature including [39] is taken from a finite alphabet and thus can be easily enumerated. To reduce the equivalence of two machines to a property that is easy to check, an observation in the above proof is that only a basis of the input Hilbert space matters, thanks to the linearity of quantum super-operators.

Clause 2) of Theorem 5.1 indicates that the length of input sequences for checking equivalence of two quantum Mealy machines is required to be quadratic in the dimensions of their state spaces. Essentially, the quadratic length comes from the dimensions of the spaces of mixed states, which occur inevitably even if the input is a pure state, because measurements can be performed at the

middle of a computation. Intuitively, the space spanned by the mixed states that are generated by the input and output sequences of length not greater than l is of dimension not less than $\min\{l+1, d\}$, where $d = d_1^2 + d_2^2$ is the dimension of the space of all possible mixed states in the state Hilbert space (This choice of d was also used in [43]. Here we derive it naturally from the dimension of density operators). Therefore, l is required to set to be at least $d-1$ in order to make that the spanned space coincides with the space of all possible mixed states. On the other hand, it is not enough to require input sequences to be less than or equal to the dimension of the state Hilbert space, as already pointed out in [39].

In Theorem 5.1, the choice of the basis B of subspace V can be arbitrary. For the most common case that $V = \mathcal{D}(\mathcal{H}_{in})$, there is a convenient basis of the space of all density operators that consists of only stabilizer states [50]. As shown in [32], stabilizer formalism can greatly simplify the equivalence checking of some special quantum systems.

5.3 Algorithm

An algorithm for checking equivalence of two quantum Mealy machines can be directly derived from Theorem 5.1 by enumerating all possible input sequences in B within length $d_1^2 + d_2^2 - 1$, but its complexity is $d^{\mathcal{O}(d_1^2 + d_2^2)}$, where $d = \dim V$, exponential in the dimensions d_1 and d_2 of state Hilbert spaces of the machines. Here, we are able to develop a more efficient algorithm with a time complexity polynomial in d_1 and d_2 .

5.3.1 Description of the Algorithm

We assume that B is a basis of V , which is a given subspace of $\mathcal{D}(\mathcal{H}_{in})$, and O is the set of outputs. Note that the choice of B is arbitrary; that is, B can be any basis of V . In particular, when $V = \mathcal{D}(\mathcal{H}_{in})$, we can choose B to be the computational basis. In this algorithm, the queue is maintained to be monotonic in an admissible order on the set $(B \times O)^*$ that first compares the length and then the lexicographical order. That is, $u < v$ for two different $u, v \in (B \times O)^*$, if $|u| < |v|$, or $|u| = |v|$ and u has priority over v in lexicographical order. As we will see in the complexity analysis given in Section 5.3.2, the admissible order in the queue enables the algorithm to terminate in a time polynomial in d_1 and d_2 . For input sequence $\pi = \sigma_1, \dots, \sigma_k$ and output sequence $a = a_1, \dots, a_k$, we define:

$$\mathcal{E}_{a|\pi} = \mathcal{E}_{a_k|\sigma_k} \circ \dots \circ \mathcal{E}_{a_1|\sigma_1} \quad (9)$$

(composition of (super-)operators), where super-operators $\mathcal{E}_{a_i|\sigma_i}$ are defined in $\mathcal{M}_1 \oplus \mathcal{M}_2$. In case of $k = 0$ (i.e. the input sequence is empty), $\mathcal{E}_{a|\pi}$ is defined to be $\mathcal{I}$, the identity (super-)operator, which transforms any density operator to itself, i.e. $\mathcal{I}(\rho) = \rho$ for every ρ .

The algorithm is based on Lex-BFS (lexicographical breadth-first search). A queue Q is used to maintain possible candidates of (π, a) in the admissible order discussed above, and a set $\mathfrak{B}$ is used to collect a basis of $\mathcal{E}_{a|\pi}(\rho)$ for all π and a chosen from the queue Q in the execution of the algorithm, where $\rho = \rho_1 \oplus (-\rho_2)$.

1. Initially, a pair $(\pi = \epsilon, a = \epsilon)$ of empty sequences is pushed into Q , and the set $\mathfrak{B}$ is set to be empty.
2. Whenever the queue is not empty, pop the front element (π, a) of the queue, and check whether $\mathcal{E}_{a|\pi}(\rho) \in \text{span } \mathfrak{B}$ or not. If not, add $\mathcal{E}_{a|\pi}(\rho)$ into $\mathfrak{B}$, and push $(\pi\sigma, ax)$ into Q for all $\sigma \in B$ and $x \in O$.

3. When the queue is empty, check whether $\text{tr}(\varrho) = 0$ for all $\varrho \in \mathfrak{B}$ or not. If yes, then claim that $\rho_1 \sim_V \rho_2$, and $\rho_1 \not\sim_V \rho_2$ otherwise. It should be noted that $\text{tr}(\mathcal{E}_{a|\pi}(\rho)) = 0$ implies the equivalence of $\mathcal{M}_1$ and $\mathcal{M}_2$ on input sequence π and output sequence a .

The algorithm described above is formalised in Algorithm [1](#).

Algorithm 1 Checking equivalence of quantum Mealy machines.

Input: $\mathcal{M}_i, \rho_i$ ($i = 1, 2$) and B the same as in Theorem [5.1](#).

Output: Whether $(\mathcal{M}_1, \rho_1) \sim_V (\mathcal{M}_2, \rho_2)$ or not.

- 1: $\rho \leftarrow \rho_1 \oplus (-\rho_2)$.
 - 2: $\mathfrak{B} \leftarrow \emptyset$.
 - 3: Let Q be an empty queue and push (ϵ, ϵ) into Q .
 - 4: **while** Q is not empty **do**
 - 5: Pop the front element (π, a) of Q .
 - 6: **if** $\mathcal{E}_{a|\pi}(\rho) \notin \text{span } \mathfrak{B}$ **then**
 - 7: Add $\mathcal{E}_{a|\pi}(\rho)$ into $\mathfrak{B}$.
 - 8: Push $(\pi\sigma, ax)$ into Q for $\sigma \in B$ and $x \in O$.
 - 9: **end if**
 - 10: **end while**
 - 11: **return true** if $\text{tr}(\varrho) = 0$ for every $\varrho \in \mathfrak{B}$, and **false** otherwise.
-

5.3.2 Correctness and Complexity

Correctness of Algorithm [1](#) can be proved using Theorem [5.1](#). For readability, the detailed proof is put in Appendix [C](#).

The complexity of Algorithm [1](#) is analysed as follows. All mixed states are supposed to be stored as matrices with appropriate dimensions, and all quantum operations are then seen as the corresponding matrix operations. Since $\mathfrak{B}$ is maintained to be linearly independent, $|\mathfrak{B}| \leq d_1^2 + d_2^2$, where $d_i = \dim \mathcal{H}_s^{(i)}$ for $i = 1, 2$. Thus, there are at most $|B||O||\mathfrak{B}| \leq d_V(d_1^2 + d_2^2)|O|$ elements that are pushed into the queue, where $d_V = \dim V = |B|$. For each element $\mathcal{E}_{a|\pi}(\rho)$ in the queue, it requires to check whether $\mathcal{E}_{a|\pi}(\rho) \in \text{span } \mathfrak{B}$, which needs $\mathcal{O}((d_1^2 + d_2^2)^3)$ time (for example, using Gaussian elimination). Here, we also need $\mathcal{O}(d_{in}^3(d_1^3 + d_2^3))$ (by matrix multiplication) to compute $\mathcal{E}_{a|\pi}(\rho)$ based on previous calculations, where $d_{in} = \dim \mathcal{H}_{in}$. Therefore, the total time complexity is $\mathcal{O}(|O|d_V(d_1^2 + d_2^2)^4) + \mathcal{O}(|O|d_V d_{in}^3(d_1^3 + d_2^3)(d_1^2 + d_2^2)) = \mathcal{O}(|O|d_V d^5(d^3 + d_{in}^3))$, where $d = d_1 + d_2$. On the other hand, only the mixed states in $\mathfrak{B}$ are required to be stored, while other mixed states are calculated only when needed. For example, $\mathcal{E}_{\pi\sigma|ax}(\rho)$ can be calculated by $\mathcal{E}_{\pi\sigma|ax}(\rho) = \mathcal{E}_{\sigma|x}(\mathcal{E}_{\pi|a}(\rho))$. Therefore, the space complexity is $\mathcal{O}(|\mathfrak{B}|^2) = \mathcal{O}((d_1^2 + d_2^2)^2)$. Formally, we have:

Theorem 5.2. *There is an algorithm with time complexity $\mathcal{O}(|O|d_V d^5(d^3 + d_{in}^3))$ and space complexity $\mathcal{O}((d_1^2 + d_2^2)^2)$ that, given $\mathcal{M}_i, \rho_i$ ($i = 1, 2$) and V be the same as in Theorem [5.1](#), checks whether $\rho_1 \sim_V \rho_2$ or not, where $d = d_1 + d_2$, $d_i = \dim \mathcal{H}_s^{(i)}$ for $i = 1, 2$ and $d_V = \dim V$.*

A careful comparison between the complexity of our algorithm for checking equivalence of sequential quantum circuits and the complexity of the known best algorithm for checking classical circuits should be helpful for understanding the difference between the classical and quantum cases. To check equivalence of two sequential quantum circuits with m input qubit variables and l_1 and l_2 state qubit variables, respectively, we need to phrase them into the model of our quantum Mealy machine with $|O| = 2^m$, $d_V = \dim \mathcal{D}(\mathcal{H}_{in}) = 2^{2m}$, $d_i = 2^{l_i}$ for $i = 1, 2$ and $d = d_1 + d_2 = 2^l$,

where $l = \max\{l_1, l_2\}$. The overall complexity is then $\mathcal{O}(2^{3m+5l}(2^{3m} + 2^{3l}))$, as given by Theorem 5.2. On the other hand, the state space traversal based algorithms have been extensively adopted for checking equivalence of classical sequential classical circuits; see for example 47 and 51. Although several industrial improvements are proposed, for example 52 and 53, the worst case complexity remains unimproved. Table 1 gives a comparison between the complexities of checking equivalence of sequential quantum circuits and classical ones with the same numbers of input and state (qubit/Boolean) variables. From there, we see that both algorithms have time complexities exponential in the number of (qu)bits.

Table 1: Complexity comparison between quantum and classical.

	Quantum	Classical
Complexity	$\mathcal{O}(2^{3m+5l}(2^{3m} + 2^{3l}))$	$\mathcal{O}(2^{m+l})$

This table shows the complexity of checking equivalence of two sequential quantum (resp. classical) circuits with m input qubit (resp. bit) variables and l state qubit (resp. bit) variables, respectively.

6 Case Studies

To show the utility of the formal model defined in this paper and further illustrate Algorithm 1 we present several case studies in this section.

6.1 Repeat-until-Success Implementation

We first consider the repeat-until-success implementation example (Example 1 and 8). As pointed out in 27, the sequential quantum circuit in Fig. 1 will produce $V_3|\psi\rangle$ with probability $5/8$ and remain $|\psi\rangle$ with probability $3/8$, regardless of the exact state $|\psi\rangle$. Indeed, this assertion can be verified by our algorithm in the sense that given any two different quantum states $|\varphi_1\rangle$ and $|\varphi_2\rangle$, we can conclude that they have the same probability to apply a unitary operator V_3 or the identity by verifying V_{rus} -equivalence in M_{rus} , i.e. $|\varphi_1\rangle \sim_{V_{rus}} |\varphi_2\rangle$, where $V_{rus} = \text{span}\{|0\rangle_q\langle 0|\}$. We select $|\varphi_1\rangle = |0\rangle$ and $|\varphi_2\rangle = |+\rangle$, and $|\varphi_1\rangle = (|0\rangle + i|1\rangle)/\sqrt{2}$ and $|\varphi_2\rangle = (|0\rangle + 2|1\rangle)/\sqrt{5}$ as examples (see Cases qrus-1 and qrus-2 in Table 2).

6.2 Quantum Random Walks

We consider the quantum random walk example (Example 7 and 9). Our algorithm is applied to the following equivalence checking problems:

(1) **The same coin and different initial states:** Suppose the coin-flip operator C is the Hadamard gate H . In this case, the machine is denoted $\mathcal{M}_H$. We consider the two initial states $|0\rangle_c|0\rangle_p$ and $|0\rangle_c|2\rangle_p$. We want to check: whether $\mathcal{M}_H$ with $|0\rangle_c|0\rangle_p$ is equivalent to the same $\mathcal{M}_H$ with $|0\rangle_c|2\rangle_p$? In other words, whether $|0\rangle_c|0\rangle_p \sim |0\rangle_c|2\rangle_p$, i.e. $|0\rangle_c|0\rangle_p \sim_{V_{walk}} |0\rangle_c|2\rangle_p$ for $V_{walk} = \mathcal{D}(\mathcal{H}_d)$? To this end, we choose the basis $B = \{|0\rangle_d\langle 0|, |1\rangle_d\langle 1|, |+\rangle_d\langle +|, |\phi\rangle_d\langle \phi|\}$ of $\mathcal{D}(\mathcal{H}_t)$ that consists of only pure states, where $|\phi\rangle = (|0\rangle + i|1\rangle)/\sqrt{2}$. For convenience, we use $0, 1, +, \phi$ to denote the corresponding states (i.e., density matrices), and set $\rho = (|0\rangle_c\langle 0| \otimes |0\rangle_p\langle 0|) \oplus (-|0\rangle_c\langle 0| \otimes |2\rangle_p\langle 2|)$. Now let us see how the algorithm works. It first pushes (ϵ, ϵ) into an empty queue Q . Repeating the “while” loop, it produces set $\mathfrak{B} = \{\mathcal{E}_{\epsilon|\epsilon}(\rho), \mathcal{E}_{0|0}(\rho), \mathcal{E}_{1|0}(\rho), \mathcal{E}_{0|+}(\rho), \dots\}$. We see that $\mathcal{E}_{00|+00}(\rho) = \frac{1}{2}(|+\rangle_c\langle +| \otimes |1\rangle_p\langle 1|) \oplus 0 \in \mathfrak{B}$, but $\text{tr}(\mathcal{E}_{00|+00}(\rho)) = \frac{1}{2} \neq 0$. Therefore, $|0\rangle_c|0\rangle_p$ is not equivalent to $|0\rangle_c|2\rangle_p$ in $\mathcal{M}_H$ (see Case qwalk-small-1 of Table 2). In fact,

$p(000|+00, |0\rangle_c|0\rangle_p) = \frac{1}{2} \neq p(000|+00, |0\rangle_c|2\rangle_p) = 0$. This example shows an interesting difference between quantum and classical random walks. Note that two initial states $|0\rangle_c|0\rangle_p$ and $|0\rangle_c|2\rangle_p$ are symmetric in the classical case. In the quantum case, however, a superposition of states can be used in a detection. In other words, the Hadamard gate H is biased over $|0\rangle$ and $|1\rangle$, but we can use $|\phi\rangle$ as the coin state to overcome this issue [48]. Surprisingly, $|\phi\rangle_c|0\rangle_p$ is equivalent to $|\phi\rangle_c|2\rangle_p$ in $\mathcal{M}_H$, which can also be verified by our algorithm (see Case qwalk-small-2 of Table 2).

Furthermore, using our algorithm, we can discover several other interesting symmetries in $\mathcal{M}_H$: $|0\rangle_c|0\rangle_p \sim |1\rangle_c|2\rangle_p$, $|0\rangle_c|1\rangle_p \sim |1\rangle_c|1\rangle_p$ and $|0\rangle_c|2\rangle_p \sim |1\rangle_c|0\rangle_p$ (see Case qwalk-sym-1 to qwalk-sym-3 of Table 2).

Remark 6.1. *Our algorithm can be used to test the design of quantum walks against some mathematically provable properties. For example, since it can be proved that $|\phi\rangle_c|x\rangle_p \sim |\phi\rangle_c|N-x-2\rangle_p$ for every position $0 \leq x \leq N-2$ in an N -dimensional quantum walk with absorbing position $N-1$. Our algorithm can be used to check whether an implementation of quantum walk satisfies this property, and if not, the implementation is incorrect.*

(2) Two different coins: The previous example considers different initial states, but the combinational circuits there are the same. Now we replace the Hadamard gate for coin-flip by the gate $Y = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & i \\ i & 1 \end{pmatrix}$ and the corresponding machine is denoted $\mathcal{M}_Y$; that is, we set $C = Y$ in $\mathcal{M}_C$. It can be verified that $(\mathcal{M}_H, |0\rangle_c|0\rangle_p)$ is equivalent to $(\mathcal{M}_Y, |0\rangle_c|0\rangle_p)$ by our algorithm (see Case qwalk-diff of Table 2).

(3) Coins with irrational rotations: Let us further consider two coins with irrational rotations $R_{\pm} = R_x(\pm\sqrt{2})$, where $R_x(\theta) = \begin{pmatrix} \cos(\theta/2) & -i\sin(\theta/2) \\ -i\sin(\theta/2) & \cos(\theta/2) \end{pmatrix}$. In this case, the states of the machines may never repeat, as the angles of rotations are irrational. Nevertheless, it can be verified that $(\mathcal{M}_{R_+}, |0\rangle_c|0\rangle_p)$ is equivalent to $(\mathcal{M}_{R_-}, |0\rangle_c|0\rangle_p)$ by our algorithm (see Case qwalk-irr of Table 2).

6.3 Quantum Fourier Transform

A sequential circuit involving a simple quantum Fourier transform is given in Fig. 5. Suppose there are six qubits d, q, p_1, p_2, p_3, p_4 , where d is the detective qubit, and q is the control qubit: (1) If the state of q is $|0\rangle$, then a 4-qubit quantum Fourier transform is performed on p_1, p_2, p_3, p_4 . Recall that an n -qubit quantum Fourier transform is defined by $QFT_n|j\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \exp\left(\frac{2\pi i j k}{N}\right) |k\rangle$, where $N = 2^n$, and for $0 \leq j < N$, $|j\rangle$ denotes $|j_1 j_2 \dots j_n\rangle$ where j is represented by $\overline{j_1 j_2 \dots j_n}$ in binary form. (2) If the state of q is $|1\rangle$, then a $\pi/8$ gate T is performed on p_1 .

At the end of the combinational part, a CNOT gate is performed on d and p_3 with p_3 being the control qubit so that certain classical information about p_3 can be obtained by measuring d . Formally, this sequential quantum circuit has d, q as input variables and p_1, p_2, p_3, p_4 as state variables, and its combinational part is described by

$$\mathcal{C}_{qft} = C^0 QFT_4[q, p_1, p_2, p_3, p_4] C^1 T[q, p_1] CNOT[p_3, d]. \quad (10)$$

As an illustrative example, we assume the initial state to be either $|0\rangle_{p_1}|0\rangle_{p_2}|0\rangle_{p_3}|0\rangle_{p_4}$ or $|1\rangle_{p_1}|0\rangle_{p_2}|0\rangle_{p_3}|0\rangle_{p_4}$, and we want to determine which is the case. Suppose the input states are only available from a subspace V_{qft} of $\mathcal{D}(\mathcal{H}_d \otimes \mathcal{H}_q)$: $V_{qft} = \text{span}\{|0\rangle_d\langle 0| \otimes |0\rangle_q\langle 0|, |0\rangle_d\langle 0| \otimes |1\rangle_q\langle 1|\}$. We hope to find an input sequence that can distinguish the two cases if we examine the statistics of output sequences. Note that at each step there are four possible measurement outcomes, i.e. $0_d 0_q, 0_d 1_q, 1_d 0_q$

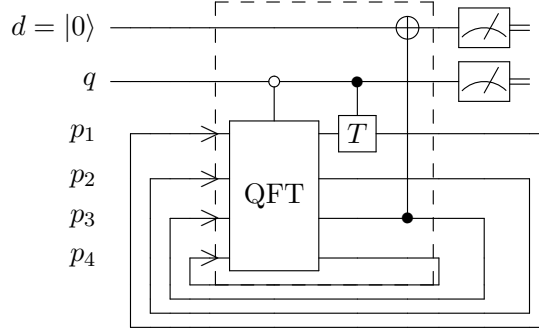


Figure 5: A sequential circuit for the quantum Fourier transform.

and $1_d 1_q$. Indeed, our algorithm can find an input sequence $\pi = |0\rangle_d |0\rangle_q, |0\rangle_d |0\rangle_q, |0\rangle_d |1\rangle_q, |0\rangle_d |0\rangle_q$ and present an output sequence $a = 0_d 0_q, 0_d 0_q, 0_d 1_q, 0_d 0_q$ such that the two possible initial states behave differently (and therefore we can distinguish them). To be precise, on input sequence π , the initial state $|0\rangle_{p_1} |0\rangle_{p_2} |0\rangle_{p_3} |0\rangle_{p_4}$ will produce the output sequence a with probability $0.338 \dots$ while $|1\rangle_{p_1} |0\rangle_{p_2} |0\rangle_{p_3} |0\rangle_{p_4}$ will do so with probability $0.161 \dots$ (See Case qft-small-1 of Table 2). The gap between the two cases is large enough to distinguish them.

Another example is to distinguish whether the initial state is $|0\rangle_{p_1} |0\rangle_{p_2} |0\rangle_{p_3} |0\rangle_{p_4}$ or $|0\rangle_{p_1} |1\rangle_{p_2} |0\rangle_{p_3} |0\rangle_{p_4}$. Again, our algorithm can find an input sequence $\pi = |0\rangle_d |0\rangle_q, |0\rangle_d |1\rangle_q, |0\rangle_d |0\rangle_q, |0\rangle_d |1\rangle_q, |0\rangle_d |0\rangle_q$ and give an output sequence $a = 0_d 0_q, 0_d 1_q, 0_d 0_q, 0_d 1_q, 0_d 0_q$ with which the two cases can be distinguished (See Case qft-small-2 of Table 2).

6.4 Quantum Half Adder

A sequential circuit involving a quantum half adder [3] is given in Fig. 6, where $V = \sqrt{X} = \frac{1}{2} \begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix}$. There are three qubits c, a and b , where b is an accumulator that computes the sum of every a modulo 2, and c contains the carry of a and b . More precisely, this sequential quantum circuit has c, a as input variables and b as the only state variable, and its combinational part is described by

$$\mathcal{C}_{qha} = C^1 V^\dagger[b, c] CNOT[a, b] C^1 V[b, c] C^1 V[a, c] CNOT[a, c]. \quad (11)$$

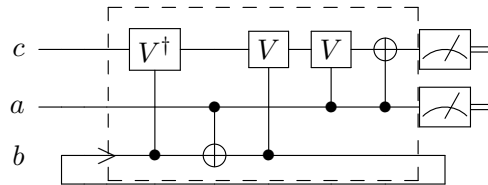


Figure 6: A sequential quantum circuit for half adders.

A classical (reversible) implementation of this sequential circuit is given in Fig. 7. The combi-

national part of sequential classical (reversible) circuit is described by

$$C_{ha} = \text{Toffoli}[a, b, c] \text{CNOT}[a, b]. \quad (12)$$

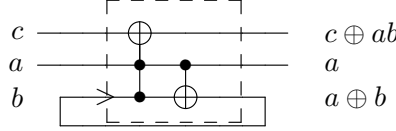


Figure 7: A sequential classical circuit for half adders.

Let us compare the sequential quantum circuit (11) and classical circuit (12) of half adder presented in Fig. 6 and Fig. 7. To this end, we should only consider the computational basis, or equivalently work on the subspace: $V_{qha} = \text{span}\{|0\rangle_c\langle 0| \otimes |0\rangle_a\langle 0|, |0\rangle_c\langle 0| \otimes |1\rangle_a\langle 1|, |1\rangle_c\langle 1| \otimes |0\rangle_a\langle 0|, |1\rangle_c\langle 1| \otimes |1\rangle_a\langle 1|\}$. In checking whether the two sequential (quantum and classical) circuits are (classically functional) equivalent, one would ask, for example, whether $(\mathcal{M}_{qha}, |0\rangle_b) \sim_{V_{qha}} (\mathcal{M}_{ha}, |0\rangle_b)$, where $\mathcal{M}_{qha}$ and $\mathcal{M}_{ha}$ denote the quantum Mealy machines corresponding to equations (11) and (12), respectively? Indeed, our algorithm returns a positive answer (See Case qhalf-adder of Table 2).

Remark 6.2. *It should be noted that in this case study of half adder, we are comparing a sequential quantum circuit with a classical (reversible) one. In such a special case, the equivalence of them (in terms of sequential quantum circuits) actually degenerates to that of classical sequential circuits. This is because in this case the input state is always in the computational basis, and the outcome probability of the sequential quantum circuits must be 0 or 1. If a sequential quantum circuit is equivalent to a classical reversible sequential circuit (with input states limited to the computational basis), then they are functionally equivalent. That is, they always behave the same on the same input sequence (in the computational basis).*

7 Experiments

Algorithm 1 is implemented in Python3 using NumPy package linked with OpenBLAS library. We test our algorithm on a workstation which has an Intel(R) Xeon(R) Platinum 8235 CPU with 16 cores. The results are collected in Table 2.

An Overview of Test Cases: Several test cases are based on repeat-until-success scheme (Example 1), quantum random walk (Example 7), testing circuits for quantum Fourier transform, quantum control circuit and quantum half adder.

It is difficult to find nontrivial equivalent test cases (that is, two equivalent sequential quantum circuits with different combinational parts). Most of our equivalent test cases are based on the symmetry of quantum random walks, and some other known facts (for example, repeat-until-success scheme). Non-equivalent test cases are trivial because random sequential quantum circuits are likely to be non-equivalent. However, two random sequential quantum circuits usually reveal their non-equivalence by a very short input sequence. It turns out that it is even difficult to meet two non-equivalent sequential quantum circuits that are equivalent in short input length (formally, they are K -equivalent for small K). Our nontrivial non-equivalent test cases are based on quantum Fourier transform.

Table 2: Experiment results of several test cases.

Test case	d_{in}	$d_{1,2}$	d_V	n	Eq	RT(s)
qwalk-small-1	2	8	4	4	No	<0.1
qwalk-small-2	2	8	4	4	Yes	<0.1
qwalk-sym-1	2	8	4	4	Yes	<0.1
qwalk-sym-2	2	8	4	4	Yes	<0.1
qwalk-sym-3	2	8	4	4	Yes	<0.1
qwalk-diff	2	8	4	4	Yes	<0.1
qwalk-irr	2	8	4	4	Yes	<0.1
qctrl-1	8	8	4	6	Yes	3.4
qctrl-2	8	8	4	6	Yes	3.4
qhalf-adder	4	2	4	3	Yes	<0.1
qft-small-1	4	16	2	6	No	0.7
qft-small-2	4	16	2	6	No	1.7
qrus-1	2	2	1	2	Yes	<0.1
qrus-2	2	2	1	2	Yes	<0.1
qwalk-large-1	2	32	4	6	Yes	40.3
qwalk-large-2	2	64	4	7	Yes	986.4
qwalk-large-3	2	128	4	8	Yes	35117.4
qft-large-1	4	32	2	7	Yes	189.6
qft-large-2	4	64	2	8	No	12.5
qft-large-3	4	128	2	9	No	50.9

d_{in} : the dimension of input Hilbert space. $d_{1,2}$: the dimension of state Hilbert space. d_V : the dimension of available input subspace. n : the number of qubits. Eq: whether $(\mathcal{M}_1, \rho_1) \sim_V (\mathcal{M}_2, \rho_2)$ or not. RT(s): the running time of the algorithm.

Efficiency Testing: Our experiment results on the test cases are summarized in Table 2. We test 6 large test cases, which are qwalk-large-1 to qwalk-large-3 and qft-large-1 to qft-large-3 in Table 2. These large test cases are similar to the corresponding small ones, but a larger number of qubits are involved in them. A big difference between the running times of qwalk-small and qwalk-large occurs due to the exponentially increasing complexity in the number of qubits. For example, the running time of qwalk-large-3 is about 10 hours even if there are only 8 qubits. The situation of qft-large is different in that the larger test cases have less running times. This is because the sequential quantum circuits in qft-large-2 and qft-large-3 are not equivalent, and our algorithm is designed (for efficiency) to halt as soon as it finds a witness that shows the non-equivalence of the circuits.

8 Conclusion

This paper defines a Mealy machine-based framework and develops an algorithm for equivalence checking of sequential quantum circuits. To the best of our knowledge, it is the first algorithm for this purpose. The complexity of this algorithm is $\mathcal{O}(2^{3m+5l}(2^{3m}+2^{3l}))$ when applied to two circuits with m input qubit variables and l_1, l_2 state qubit variables, respectively. Although the gap between it and the complexity of the best known algorithm for checking equivalence of classical sequential circuits is not very large (see Table 1), there is still some possibility to improve our algorithm. So, an important issue for future research is to find more efficient algorithms for equivalence checking of sequential quantum circuits.

It is widely believed that in the near future, only NISQ (Noisy Intermediate-Scale Quantum) devices are physically realisable and they will find real-world applications in such areas as quantum chemistry and many-body physics [54]. So, another interesting research topic is to develop algorithms for checking approximate equivalence between sequential quantum circuit and their noisy implementations. But the nature of this problem seems very different from the exact equivalence checking studied in this paper, as hinted by the results in [41] for probabilistic automata.

Model checking has been successfully applied to verify some sophisticated properties of very large classical sequential circuits; see for example [55]. In the last fifteen years, model checking techniques have been extended for verification of quantum communication protocols [56] [57] [32], quantum automata [58] and quantum Markov chains [59]. So, another interesting topic is to see how these quantum model checking techniques can be tuned for checking sequential quantum circuits.

Up to now, all of the algorithms for checking equivalence of (both combinational and sequential) quantum circuits are classical (i.e. to be run on classical computers). An even more interesting topic is to find *quantum* algorithms for equivalence checking of quantum circuits so that the existing small quantum computers can be used in designing larger and larger ones.

Acknowledgment

We thank Prof. Yuan Feng for pointing out a mistake in an early version of this paper.

References

- [1] A. Abdollahi and M. Pedram. Analysis and synthesis of quantum circuits by using quantum decision diagrams. In *Proceedings of the Conference on Design, Automation and Test in Europe*, pages 317–322, 2006.

- [51] P. Molitor and J. Mohnke. *Equivalence Checking of Digital Circuits: Fundamentals, Principles, Methods*. Kluwer Academic, 2004.
- [52] C. A. J. van Eijk. Sequential equivalence checking based on structural similarities. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 19(7):814–819, 2006.
- [53] H. Savoj, A. Mishchenko, and R. Brayton. Sequential equivalence checking for clock-gated circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 33(2):305–317, 2014.
- [54] J. Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018.
- [55] J. R. Burch, E. M. Clarke, D. E. Long, K. L. McMillan, and D. L. Dill. Symbolic model checking for sequential circuit verification. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 13(4):401–424, 1994.
- [56] S. J. Gay, R. Nagarajan, and N. Papanikolaou. QMC: a model checker for quantum systems. In *Proceedings of the 20th International Conference on Computer Aided Verification*, pages 543–547, 2008.
- [57] T. A. S. Davidson, S. J. Gay, H. Mlnarik, R. Nagarajan, and N. Papanikolaou. Model checking for communicating quantum processes. *International Journal of Unconventional Computing*, 8(1):73–98, 2012.
- [58] M. S. Ying, Y. J. Li, N. K. Yu, and Y. Feng. Model-checking linear-time properties of quantum systems. *ACM Transactions on Computational Logic*, 15(3):22:1–31, 2014.
- [59] S. G. Ying, Y. Feng, N. K. Yu, and M. S. Ying. Reachability probabilities of quantum Markov chains. In *International Conference on Concurrency Theory: CONCUR 2013 - Concurrency Theory*, pages 334–348, 2013.
- [60] M. A. Rol, F. Battistel, F. K. Malinowski, et al. Fast, high-fidelity conditional-phase gate exploiting leakage interference in weakly anharmonic superconducting qubits. *Physical Review Letters*, 132(12):120502, 2019.
- [61] M. A. Rol, L. Ciorciaro, F. K. Malinowski, et al. Time-domain characterization and correction of on-chip distortion of control pulses in a quantum processor. *Applied Physics Letters*, 116(5):054001, 2020.
- [62] Y. Chen, C. Neill, et al. Qubit architecture with high coherence and fast tunable coupling. *Physical Review Letters*, 113(22):220502, 2014.
- [63] F. Arute, K. Arya, R. Babbush, et al. Quantum supremacy using a programmable superconducting processor. *Nature*, 574:505–510, 2019.

A Hardware Realisation of Sequential Quantum Circuits

To show that the sequential quantum circuit model is physically feasible and meaningful, we briefly discuss its realization on real quantum hardware in this section. We will take superconducting quantum computing as an example, which is now one of the most advanced and promising technique paths for quantum computers.

Different from classical circuits, quantum gates in superconducting quantum computers are realized by specific pulse signals instead of concrete electronic components, so quantum data will not “flow through” the quantum gates, and conversely, the gate pulses are applied on the quantum data, which are stored in quantum registers (namely, qubits). Quantum hardware itself needs a clock to ensure correct timing for gate pulses, otherwise the computing results will be totally unpredictable. Despite these differences, sequential quantum circuits are still realizable on superconducting quantum hardware.

The realization of sequential quantum circuits requires dynamic quantum computing [29]. Dynamic quantum computing allows feedforward and feedback, which means that qubits could be measured intermediately during the computation and the measurement outcomes might in return affect the following computing. A common approach to realize dynamic quantum computing is to design a control system for quantum processors using field-programmable gate array (FPGA) [60] [61].

In our sequential quantum circuit model, it is only required that the rest qubits are still working after some qubits are measured, rather than a complete feedforward and feedback control mechanism. As stated in Section 3.1, a synchronous clock is needed in a sequential quantum circuit to synchronize the computation, which is different from the internal clock of FPGA control system (whose frequency is usually 100 MHz or higher) to guarantee correct gate pulses. A single-qubit or two-qubit unitary gate usually costs about 10 to 40 nanoseconds [62] [63]. Measurement operations usually take longer (about a few hundred nanoseconds) [29]. What we need is to design a synchronous clock with one-cycle duration longer than the whole time consumption of both the combination and measurement parts in the sequential quantum circuit (besides, the one-cycle duration of this clock should be a multiple of the one-cycle duration of the internal clock). Apparently this kind of synchronous clock is feasible and can be added to the control system. Fig. 8 shows an example of sequential quantum circuits with $n = 2l$ qubits executed on a superconducting quantum processor with $3l$ qubits.

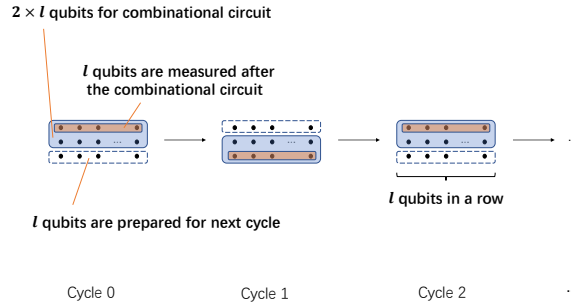


Figure 8: A sequential circuit implemented on a 2D-grid superconducting quantum processor with $3l$ qubits. Here we set $m = l$. The l qubits in middle row represent the l state variables, while the l qubits in the top row and l qubits in the bottom row alternatively represent the m ($m = l$) input variables. At Cycle 0, the first l qubits are input variables and they together with the second row are applied by a combinational circuit, and after that their measurement outcomes are passed to the l output variables. The bottom row in the dashed frame is prepared as input for Cycle 1. At Cycle 1, the qubits in the bottom row become the input variables, while the qubits in the top row are reset to prepare the input for Cycle 2. During the whole computing process, the l qubits in middle row are always active.

B Completing the Proof of Theorem 5.1

Here, we provide the details of steps (i) and (ii) outlined at the beginning of the proof of Theorem 5.1. These two steps will be carried out through a series of lemmas. We start from giving a technical lemma that gives explicit formulas for computing the transition probability and the machine's state at each step.

Lemma B.1. *Let $\mathcal{M} = \{\mathcal{H}_{in}, \mathcal{H}_s, U, M\}$ be a quantum Mealy machine. Let $\mathcal{E}_{a|\pi}$ be the super-operator defined by equation (5) and generalised by equation (9). Then for every initial state $\rho_0 \in \mathcal{D}(\mathcal{H}_s)$, input sequence π in $\mathcal{D}(\mathcal{H}_{in})$ and output sequence $a \in O^{|\pi|}$, we have*

$$p(a|\pi, \rho_0) = \text{tr} [\mathcal{E}_{a|\pi}(\rho_0)],$$

and the density operator on input sequence π and output sequence a is

$$\rho_{|\pi|} = \frac{1}{\text{tr} [\mathcal{E}_{a|\pi}(\rho_0)]} \mathcal{E}_{a|\pi}(\rho_0).$$

Proof: We prove it by induction on the length of π . It is trivial for the case that π is empty, i.e. $|\pi| = 0$. Suppose it is true for the case $|\pi| = k$, i.e.

$$p(a|\pi, \rho_0) = \text{tr} [\mathcal{E}_{a|\pi}(\rho_0)]$$

for every initial state $\rho_0 \in \mathcal{D}(\mathcal{H}_s)$, input sequence π in $\mathcal{D}(\mathcal{H}_{in})$ with $|\pi| = k$ and output sequence $a \in O^{|\pi|}$. The density operator on input sequence π and output sequence a is

$$\rho_k = \frac{1}{\text{tr} [\mathcal{E}_{a|\pi}(\rho_0)]} \mathcal{E}_{a|\pi}(\rho_0).$$

Let the next input and output be σ_{k+1} and a_{k+1} , respectively. Then the probability with output a_{k+1} on input σ_{k+1} is

$$\begin{aligned} p(a_{k+1}|\sigma_{k+1}, \rho_k) &= \text{tr} [\mathcal{E}_{a_{k+1}|\sigma_{k+1}}(\rho)] \\ &= \frac{1}{\text{tr} [\mathcal{E}_{a|\pi}(\rho_0)]} \text{tr} [\mathcal{E}_{aa_{k+1}|\pi\sigma_{k+1}}(\rho_0)]. \end{aligned}$$

By the definition, the probability on input sequence $\pi\sigma_{k+1}$ and output sequence aa_{k+1} is

$$\begin{aligned} p(aa_{k+1}|\pi\sigma_{k+1}, \rho_0) &= p(a|\pi, \rho_0)p(a_{k+1}|\sigma_{k+1}, \rho_k) \\ &= \text{tr} [\mathcal{E}_{aa_{k+1}|\pi\sigma_{k+1}}(\rho_0)], \end{aligned}$$

and the density operator is

$$\begin{aligned} \rho' &= \frac{1}{p(a_{k+1}|\sigma_{k+1}, \rho_k)} \mathcal{E}_{a_{k+1}|\sigma_{k+1}}(\rho) \\ &= \frac{1}{\text{tr} [\mathcal{E}_{aa_{k+1}|\pi\sigma_{k+1}}(\rho_0)]} \mathcal{E}_{aa_{k+1}|\pi\sigma_{k+1}}(\rho_0). \end{aligned}$$

Hence, the property also holds for the case $|\pi| = k + 1$, and the proof is completed. ■

The next lemma presents a characterisation of the dynamics of the direct sum of two machines in terms of their respective dynamics.

Lemma B.2. Let $\mathcal{M}_i = \{\mathcal{H}_{in}, \mathcal{H}_s^{(i)}, U_i, M_i\}$ ($i = 1, 2$) be two quantum Mealy machines with the same input space $\mathcal{H}_{in}$ and the same outputs O (i.e. measurements $M_i = \{M_a^{(i)} : a \in O\}$ for $i = 1, 2$). Let $\mathcal{E}_{a|\pi}^{(1)}, \mathcal{E}_{a|\pi}^{(2)}$ and $\mathcal{E}_{a|\pi}$ be the super-operator defined in $\mathcal{M}_1, \mathcal{M}_2$ and $\mathcal{M} = \mathcal{M}_1 \oplus \mathcal{M}_2$ by equation (5) and generalised by equation (9), respectively. Then

$$\mathcal{E}_{a|\pi}(\rho_1 \oplus \rho_2) = \mathcal{E}_{a|\pi}^{(1)}(\rho_1) \oplus \mathcal{E}_{a|\pi}^{(2)}(\rho_2)$$

for every $\rho_i \in \mathcal{D}(\mathcal{H}_s^{(i)})$ ($i = 1, 2$).

Proof: We prove this lemma by induction on the length of π . For the case of $|\pi| = 1$, for $a \in O$ and $\sigma \in \mathcal{D}(\mathcal{H}_{in})$, we can verify that

$$\begin{aligned} \mathcal{E}_{a|\sigma}(\rho_1 \oplus \rho_2) &= \text{tr}_{\mathcal{H}_{in}} \left[\left(M_a^{(1)} \oplus M_a^{(2)} \right) (U_1 \oplus U_2) \right. \\ &\quad \left. (\sigma \otimes (\rho_1 \oplus \rho_2)) (U_1 \oplus U_2)^\dagger \left(M_a^{(1)} \oplus M_a^{(2)} \right)^\dagger \right] \\ &= \mathcal{E}_{a|\sigma}^{(1)}(\rho_1) \oplus \mathcal{E}_{a|\sigma}^{(2)}(\rho_2). \end{aligned}$$

Now suppose it is true for the case of $|\pi| = k$, i.e.

$$\mathcal{E}_{a|\pi}(\rho_1 \oplus \rho_2) = \mathcal{E}_{a|\pi}^{(1)}(\rho_1) \oplus \mathcal{E}_{a|\pi}^{(2)}(\rho_2)$$

holds for every input sequence π with $|\pi| = k$ and output sequence a . For every $a_{k+1} \in O$ and $\sigma_{k+1} \in \mathcal{D}(\mathcal{H}_{in})$, we have

$$\begin{aligned} \mathcal{E}_{aa_{k+1}|\pi\sigma_{k+1}}(\rho_1 \oplus \rho_2) &= \mathcal{E}_{a_{k+1}|\sigma_{k+1}}(\mathcal{E}_{a|\pi}(\rho_1 \oplus \rho_2)) \\ &= \mathcal{E}_{a_{k+1}|\sigma_{k+1}}(\mathcal{E}_{a|\pi}^{(1)}(\rho_1) \oplus \mathcal{E}_{a|\pi}^{(2)}(\rho_2)) \\ &= \mathcal{E}_{aa_{k+1}|\pi\sigma_{k+1}}^{(1)}(\rho_1) \oplus \mathcal{E}_{aa_{k+1}|\pi\sigma_{k+1}}^{(2)}(\rho_2). \end{aligned}$$

Hence, the property also holds for the case $|\pi| = k + 1$, and we complete the proof. $\blacksquare$

Now we are ready to accomplish step (i). Formally, this step can be formulated as the following:

Lemma B.3. Let $\mathcal{M}_i = \{\mathcal{H}_{in}, \mathcal{H}_s^{(i)}, U_i, M_i\}$ ($i = 1, 2$) be two quantum Mealy machines with the same input space $\mathcal{H}_{in}$ and the same outputs O (i.e. measurements $M_i = \{M_a^{(i)} : a \in O\}$ for $i = 1, 2$). Let $\rho_i \in \mathcal{D}(\mathcal{H}_s^{(i)})$ ($i = 1, 2$) and $V \subseteq \mathcal{D}(\mathcal{H}_{in})$. Then $(\mathcal{M}_1, \rho_1) \sim_V (\mathcal{M}_2, \rho_2)$ iff $\rho_1 \sim_V \rho_2$ in $\mathcal{M}_1 \oplus \mathcal{M}_2$.

Proof: Let $\mathcal{M} = \mathcal{M}_1 \oplus \mathcal{M}_2$. Furthermore, let $\mathcal{E}_{a|\pi}^{(i)}$ be the super-operator defined in $\mathcal{M}_i$ for $i = 1, 2$, and $\mathcal{E}_{a|\pi}$ be that defined in $\mathcal{M}$. We also use the notation $p^{(i)}(a|\pi, \rho)$ ($i = 1, 2$) and $p(a|\pi, \rho)$ to denote the probability that $\mathcal{M}_i$ ($i = 1, 2$) and $\mathcal{M}$ with initial state ρ output a on input sequence π , respectively. Then by Lemma B.1, we have:

$$p^{(i)}(a|\pi, \rho_i) = \text{tr} \left[\mathcal{E}_{a|\pi}^{(i)}(\rho_i) \right], \quad p(a|\pi, \rho_i) = \text{tr} \left[\mathcal{E}_{a|\pi}(\rho_i) \right]$$

for $i = 1, 2$. By Lemma B.2, we have: $\mathcal{E}_{a|\pi}(\rho_i) = \mathcal{E}_{a|\pi}^{(i)}(\rho_i)$ for $i = 1, 2$. We immediately obtain that $p(a|\pi, \rho_i) = p^{(i)}(a|\pi, \rho_i)$, which implies that $(\mathcal{M}_1, \rho_1) \sim_V (\mathcal{M}_2, \rho_2)$ iff $\rho_1 \sim_V \rho_2$ in $\mathcal{M}_1 \oplus \mathcal{M}_2$ by the definition of V -equivalence. $\blacksquare$

Next, we turn to accomplish step (ii). Formally, we have:

Lemma B.4. Let $\mathcal{M} = (\mathcal{H}_{in}, \mathcal{H}_s, U, M)$ be a quantum Mealy machine, and $B \subseteq \mathcal{D}(\mathcal{H}_{in})$. Then $\rho_1 \sim_B \rho_2$ iff $\rho_1 \sim_{\text{span } B} \rho_2$ for every $\rho_1, \rho_2 \in \mathcal{D}(\mathcal{H}_s)$.

Proof: The “if” part is obvious. To prove the “only if” part, we first observe that for every input sequence π and $a \in O^{|\pi|}$, $p(a|\pi, \rho) = \text{tr} [\mathcal{E}_{a|\pi}(\rho)]$. We only need to consider the case where B is a finite set, say $B = \{\gamma_1, \gamma_2, \dots, \gamma_k\}$. Suppose that $\rho_1 \sim_B \rho_2$. Then for every input sequence $\pi = \sigma_1 \sigma_2 \dots \sigma_t$ and $a \in O^t$, where $\sigma_j \in \text{span } B$ for $1 \leq j \leq t$, we have

$$\sigma_j = \sum_{i=1}^k \alpha_{ji} \gamma_i.$$

for some coefficients α_{ji} . By Lemma B.1, we have

$$\begin{aligned} p(a|\pi, \rho_1) &= \text{tr} [\mathcal{E}_{a|\pi}(\rho_1)] \\ &= \sum_{i_1=1}^k \dots \sum_{i_t=1}^k \alpha_{1i_1} \dots \alpha_{ti_t} \text{tr} [\mathcal{E}_{a|\gamma_{i_1} \dots \gamma_{i_t}}(\rho_1)] \\ &= \sum_{i_1=1}^k \dots \sum_{i_t=1}^k \alpha_{1i_1} \dots \alpha_{ti_t} \text{tr} [\mathcal{E}_{a|\gamma_{i_1} \dots \gamma_{i_t}}(\rho_2)] \\ &= \text{tr} [\mathcal{E}_{a|\pi}(\rho_2)] = p(a|\pi, \rho_2), \end{aligned}$$

which yields $\rho_1 \sim_{\text{span } B} \rho_2$. ■

As an immediate corollary, we see that $\rho_1 \sim \rho_2$ if and only if $\rho_1 \sim_B \rho_2$ for some basis B of $\mathcal{D}(\mathcal{H}_{in})$ consisting of only pure states.

C Correctness Proof of Algorithm 1

The correctness of Algorithm 1 is essentially based on Theorem 5.1. Indeed, by Theorem 5.1 we only need to prove that $\text{span } D(\rho, d_1^2 + d_2^2 - 1) = \text{span } \mathfrak{B}$. The inclusion $\text{span } D(\rho, d_1^2 + d_2^2 - 1) \supseteq \text{span } \mathfrak{B}$ is obvious. To prove the reverse inclusion, it is sufficient to show that $\mathcal{E}_{a|\pi}(\rho) \in \text{span } \mathfrak{B}_{\leq(\pi, a)}$ for every $\mathcal{E}_{a|\pi}(\rho) \in D(\rho, d_1^2 + d_2^2 - 1)$, where

$$\mathfrak{B}_{\triangleleft(\pi, a)} = \{\mathcal{E}_{a'|\pi'}(\rho) : (\pi', a') \triangleleft (\pi, a) \text{ and } \mathcal{E}_{a'|\pi'}(\rho) \in \mathfrak{B}\}$$

and $\triangleleft$ can be $<$ or $\leq$. This can be proved by induction on the admissible order maintained in the queue. Note that $\mathcal{E}_{\epsilon|\epsilon}(\rho) = \rho \in \text{span } \mathfrak{B}$ for the basis. For every $\mathcal{E}_{ax|\pi\sigma}(\rho)$, we consider the following two cases:

Case 1. If $\mathcal{E}_{a|\pi}(\rho)$ is added into $\mathfrak{B}$ at the step when (π, a) is popped from Q in the algorithm, then $\mathcal{E}_{ax|\pi\sigma}(\rho)$ is pushed into Q in the execution of the algorithm and thus $\mathcal{E}_{ax|\pi\sigma}(\rho) \in \text{span } \mathfrak{B}_{\leq(\pi\sigma, ax)}$ is guaranteed.

Case 2. If $\mathcal{E}_{a|\pi}(\rho)$ is not added into $\mathfrak{B}$ at the step when (π, a) is popped from Q in the algorithm or even (π, a) is never in the queue, then $\mathfrak{B}_{\triangleleft(\pi, a)} = \mathfrak{B}_{\leq(\pi, a)}$. By the induction hypothesis we have: $\mathcal{E}_{a|\pi}(\rho) \in \text{span } \mathfrak{B}_{\leq(\pi, a)}$, which immediately yields

$$\begin{aligned} \mathcal{E}_{ax|\pi\sigma}(\rho) &= \mathcal{E}_{x|\sigma}(\mathcal{E}_{a|\pi}(\rho)) \\ &\in \mathcal{E}_{x|\sigma}(\text{span } \mathfrak{B}_{\leq(\pi, a)}) \\ &= \text{span } \mathcal{E}_{x|\sigma}(\mathfrak{B}_{\leq(\pi, a)}) \\ &\subseteq \text{span } \mathfrak{B}_{\leq(\pi\sigma, ax)} \\ &\subseteq \text{span } \mathfrak{B}_{\leq(\pi\sigma, ax)}. \end{aligned}$$

Hence, $\mathcal{E}_{ax|\pi\sigma}(\rho) \in \text{span } \mathfrak{B}_{\leq(\pi\sigma, ax)}$ is also guaranteed.

Therefore, it always holds that $\text{span } D(\rho, d_1^2 + d_2^2 - 1) \subseteq \text{span } \mathfrak{B}$ and correctness of the algorithm is proved.

QCircuitNet: A Large-Scale Hierarchical Dataset for Quantum Algorithm Design

Rui Yang^{1,2} Yuntian Gu³ Ziruo Wang⁴ Yitao Liang⁵ Tongyang Li^{1,2}

¹School of Computer Science, Peking University

²Center on Frontiers of Computing Studies, Peking University

³School of Intelligence Science and Technology, Peking University

⁴School of Computer Science, Beijing Institute of Technology

⁵Institute for Artificial Intelligence, Peking University

ypyangrui@pku.edu.cn guyuntian@stu.pku.edu.cn

ziruwang@bit.edu.cn yitaol@pku.edu.cn tongyangli@pku.edu.cn

ABSTRACT

Quantum computing is an emerging field recognized for the significant speedup it offers over classical computing through quantum algorithms. However, designing and implementing quantum algorithms pose challenges due to the complex nature of quantum mechanics and the necessity for precise control over quantum states. Despite the significant advancements in AI, there has been a lack of datasets specifically tailored for this purpose. In this work, we introduce QCircuitNet, the first benchmark and test dataset designed to evaluate AI’s capability in designing and implementing quantum algorithms in the form of quantum circuit codes. Unlike using AI for writing traditional codes, this task is fundamentally different and significantly more complicated due to highly flexible design space and intricate manipulation of qubits. Our key contributions include:

1. A general framework which formulates the key features of quantum algorithm design task for Large Language Models.
2. Implementation for a wide range of quantum algorithms from basic primitives to advanced applications, with easy extension to more quantum algorithms.
3. Automatic validation and verification functions, allowing for iterative evaluation and interactive reasoning without human inspection.
4. Promising potential as a training dataset through primitive fine-tuning results.

We observed several interesting experimental phenomena: fine-tuning does not always outperform few-shot learning, and LLMs tend to exhibit consistent error patterns. QCircuitNet provides a comprehensive benchmark for AI-driven quantum algorithm design, offering advantages in model evaluation and improvement, while also revealing some limitations of LLMs in this domain.

1 INTRODUCTION

Quantum computing is an emerging field in recent decades because algorithms on quantum computers may solve problems significantly faster than their classical counterparts. From the perspective of theoretical computer science, the design of quantum algorithms have been investigated in various research directions - see the survey (Dalzell et al., 2023) and the quantum algorithm zoo (Zoo, 2024). However, the design of quantum algorithms on quantum computers has been completed manually by researchers. This process is notably challenging due to highly flexible design space and extreme demands for a comprehensive understanding of mathematical tools and quantum properties.

For these reasons, quantum computing is often considered to have high professional barriers. As the discipline evolves, we aim to explore more possibilities for algorithm design and implementation in the quantum setting. This is aligned with recent advances among AI for Science, including AlphaFold (Jumper et al., 2021), AlphaGeometry (Trinh et al., 2024), etc. Recently, large language models (LLMs) have also become widely applicable among AI for science approaches (Yang et al.,

[2024; Zhang et al., 2024; Yu et al., 2024]. LLMs represent the best practice of sequential modeling methods at current stage. They have an edge over other models in possessing abundant pre-training knowledge and providing human-friendly interfaces which support human-machine collaboration. Therefore, we gear LLMs for quantum algorithm design.

As far as we know, there has not been any dataset for AI in quantum algorithm design. Existing work combining quantum computing and AI mostly targets at exploiting quantum computing for AI; there are some papers applying AI for quantum computing, but they either consider niche problems ([Nakayama et al., 2023; Schatzki et al., 2021]) or limited functions ([Tang et al., 2023; Fürutter et al., 2024]), not quantum algorithm datasets of general interest (see Section 2). However, unlike classical code generation where abundant data exist, the most challenging aspect for quantum algorithm design is the lack of sufficient data, and hence the difficulty of generalization in training AI models. Therefore, datasets for quantum algorithm design are solicited.

Descriptions of quantum algorithms in natural language could be verbose and vague. Mathematical formulas, while precise and succinct, are difficult to verify automatically. To accommodate with LLMs, we make a change of perspective by formulating quantum algorithms as programming languages. This allows for precise representation of a quantum algorithm, enables automatic verification procedure, and bridges the gap between theoretical design and circuit implementations. Furthermore, meaningful quantum algorithms which can be efficiently implemented have no more than polynomially many gates ([Poulin et al., 2011]), and thus such formulations have the theoretical benefits allowing for scalable representations.

Key Contributions. In this work, We propose QCircuitNet, the first comprehensive, structured dataset for quantum algorithm design. Technically, QCircuitNet has the following key contributions:

- It formulates the task for Large Language Models (LLMs) with a carefully designed framework encompassing the key features of quantum algorithm design, including problem description, quantum circuit codes, classical post-processing, and verification functions. It maintains the black-box nature of oracles and characterizes query complexity properly.
- It implements a wide range of quantum algorithms from basic primitives and textbook-level algorithms to advanced applications. We demonstrate the compatibility with complex algorithms through Generalized Simon’s Problem and showcase the easy extension to advanced algorithms.
- It has automatic validation and verification functions, allowing for iterative evaluation without human inspection. This further enables interactive reasoning which may improve the performance.
- It showcases the potential as a training dataset through primitive fine-tuning results. As we expand the dataset to include more algorithms and explore novel fine-tuning methods, it will hopefully contribute to interactive quantum algorithm design and implementation significantly.

2 RELATED WORK

Quantum Machine Learning. To the best of our knowledge, QCircuitNet is the first dataset tailored specifically for quantum algorithm design. Previous efforts combining quantum computing with AI primarily fall under the category of Quantum Machine Learning (QML), which aims at leveraging the unique properties of quantum systems to enhance machine learning algorithms and achieve improvements over their classical counterparts ([Schuld et al., 2015; Biamonte et al., 2017; Ciliberto et al., 2018]). Corresponding datasets often focus on encoding classical data into quantum states. For instance, MNISQ ([Placidi et al., 2023]) is a dataset of quantum circuits representing the original MNIST dataset ([LeCun et al., 1998]) generated by the AQCE algorithm ([Shirakawa et al., 2021]). Considering the intrinsic nature of quantum properties, another category of datasets focuses on collecting quantum data to demonstrate quantum advantages since classical machine learning methods can fail to characterize particular patterns of quantum data. For example, [Nakayama et al., 2023] created a VQE-generated quantum circuit dataset for classification of variational ansatzes and showed its quantum supremacy. NTangled ([Schatzki et al., 2021]) further investigated different types of entanglement and composed quantum states with various multipartite entanglement for classification. While these datasets successfully demonstrate the supremacy of quantum computing, they address relatively niche problems whose practical applications are unclear.

AI for Quantum Computing. This research direction explores the possibility of leveraging AI to facilitate the advancement of quantum computing. QDataSet (Perrier et al., 2022) collects data from simulations of one- and two-qubit systems and targets training classical machine learning algorithms for quantum control, quantum tomography, and noise mitigation. LLM4QPE (Tang et al., 2023) is a large language model style paradigm for predicting quantum system properties with pre-training and fine-tuning workflows. While the paradigm is interesting, the empirical experiments are limited to two downstream tasks: quantum phase classification and correlation prediction. Furrutter et al. (2024) studied the application of diffusion models (Sohl-Dickstein et al., 2015; Rombach et al., 2022) to quantum circuit synthesis (Saeedi & Markov, 2013; J. et al., 2022). Scalability issues must be addressed to achieve practical and meaningful unitary compilation through this methodology.

Quantum Circuit Benchmarks. The aforementioned works represent meaningful explorations at the intersection of AI and quantum computing. However, none of them considers the task which interests the quantum computing community (from the theoretical side) the most: quantum algorithm design. Our work aims to take the first step in bridging this gap. It is worth noting that several quantum algorithm circuit benchmarks already exist, such as QASMBench (Li et al., 2023), MQTBench (Quetschlich et al., 2023), and VeriQBench (Chen et al., 2022). However, these benchmarks are designed to evaluate the performance of NISQ (Noisy Intermediate-Scale Quantum) (Preskill, 2018) machines or quantum software tools, rather than for training and evaluating AI models. For instance, QASMBench includes a diverse variety of quantum circuits based on OpenQASM representation (Cross et al., 2022), covering quantum circuits with qubit sizes ranging from 2 to 127. However, it fails as a dataset for AI in that it does not capture the design patterns of each algorithm and ignores the post-processing procedure and construction of different oracles, which are crucial to quantum algorithm design. Similar limitations apply to MQTBench and VeriQBench.

3 PRELIMINARIES FOR QUANTUM COMPUTING

In this section, we will introduce necessary backgrounds for quantum computing related to this paper. Additional preliminaries can also be found in Appendix B. A more detailed introduction to quantum computing can be found in the standard textbook by Nielsen & Chuang (2000).

Quantum States. In classical computing, the basic unit is a bit. In quantum computing, the basic unit is a *qubit*. Mathematically, n ($n \in \mathbb{N}$) qubits forms an N -dimensional Hilbert space for $N = 2^n$. An n -qubit *quantum state* $|\phi\rangle$ can be written as

$$|\phi\rangle = \sum_{i=0}^{N-1} \alpha_i |i\rangle, \text{ where } \sum_{i=0}^{N-1} |\alpha_i|^2 = 1. \quad (1)$$

Here $|\cdot\rangle$ represents a column vector, also known as a ket state. The tensor product of two quantum states $|\phi_1\rangle = \sum_{i=0}^{N-1} \alpha_i |i\rangle$ and $|\phi_2\rangle = \sum_{j=0}^{M-1} \beta_j |j\rangle$ with $M = 2^m$, $m \in \mathbb{N}$ is defined as

$$|\phi_1\rangle \otimes |\phi_2\rangle = \sum_{i=0}^{N-1} \sum_{j=0}^{M-1} \alpha_i \beta_j |i, j\rangle, \quad (2)$$

where $|i, j\rangle$ is an $(n + m)$ -qubit state with first n qubits being the state $|i\rangle$ and the last m qubits being the state $|j\rangle$. When there is no ambiguity, $|\phi_1\rangle \otimes |\phi_2\rangle$ can be abbreviated as $|\phi_1\rangle |\phi_2\rangle$.

Quantum Oracles. To study a Boolean function $f: \{0, 1\}^n \rightarrow \{0, 1\}^m$, we need to gain its access. Classically, a standard setting is to being able to *query* the function, in the sense that if we input an $x \in \{0, 1\}^n$, we will get the output $f(x) \in \{0, 1\}^m$. In quantum computing, the counterpart is a quantum query, which is instantiated by a *quantum oracle*. Specifically, the function f is encoded as an oracle U_f such that for any $x \in \{0, 1\}^n$, $z \in \{0, 1\}^m$,

$$U_f |x\rangle |z\rangle = |x\rangle |z \oplus f(x)\rangle, \quad (3)$$

where $\oplus$ is the plus modulo 2. Note that a quantum query to the oracle is stronger than a classical query in the sense that the quantum query can be applied to a state in *superposition*: For an input state $\sum_i c_i |x_i\rangle |z_i\rangle$ with $\sum_i |c_i|^2 = 1$, the output state is $\sum_i c_i |x_i\rangle |z_i \oplus f(x_i)\rangle$; measuring this state gives x_i and $z_i \oplus f(x_i)$ with probability $|c_i|^2$. A classical query for x can be regarded as the special setting with $c_1 = 1$, $x_1 = x$, $z_1 = 0^m$, and $c_i = 0$ for all other i .

Quantum Gates. Similar to classical computing that can stem from logic synthesis with AND, OR, and NOT, quantum computing is also composed of basic quantum gates. For instance, the Hadamard H is the matrix $\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$, satisfying $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ and $H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$. In general, an n -qubit quantum gate is a unitary matrix from $\mathbb{C}^{2^n \times 2^n}$.

4 QCIrcuITNET DATASET

4.1 TASK SUITE

For the general purpose of quantum algorithm design, we consider three categories of tasks: oracle construction, algorithm design, and random circuit synthesis. These three tasks are crucial for devising and implementing quantum algorithms, with oracle construction serving as the premise for algorithm design, and random circuits serving as a main demonstration for quantum supremacy.

4.1.1 TASK I: ORACLE CONSTRUCTION

The construction of such an oracle U_f using quantum gates is deeply rooted in the research topic of reversible quantum logic synthesis, which remains a challenge for complex Boolean functions. In this dataset, we mainly focus on the construction of textbook-level oracles: Bernstein-Vazirani Problem (Bernstein & Vazirani, 1993), Deutsch-Jozsa Problem (Deutsch & Jozsa, 1992), Simon's Problem (Simon, 1997), and Grover's algorithm for unstructured search (Grover, 1996) (including constructions of both the oracle and the diffusion operator). There is another category of more flexible oracle construction tasks which we refer to as "Problem Encoding". For example, one can apply Grover's oracle to solving constraint problems such as SAT and triangle finding (Ambainis, 2004). Formulating problem encoding tasks for LLMs slightly differs from quantum logic synthesis, and we refer the readers to Appendix A.2 for more detailed discussion.

4.1.2 TASK II: QUANTUM ALGORITHM DESIGN

There are several challenges to address in order to formulate quantum algorithm design task precisely:

- From a theoretical perspective, the oracle is usually provided as a blackbox gate since the goal of many algorithms is to determine the property of the function $f(x)$ encoded by the oracle U_f . If the model has access to the gate implementation of the oracle, it can directly deduce the property from the circuit, failing the purpose of designing a quantum algorithm to decode the information. However, for all experimental platforms, a quantum circuit needs to be explicitly constructed to compile and run successfully, i.e., the oracle should be provided with exact gate implementation. Most tutorials and benchmarks (especially those based on Qiskit and OpenQASM) simply merge the circuit implementation of the oracle and the algorithm as a whole for demonstration purposes. When we gear LLMs for quantum algorithm design, how to separate the algorithm circuits from oracle implementation to avoid information leakage is a critical point to consider.
- A quantum algorithm constitutes not only the quantum circuit, but also its interpretation of measurement results. For example, in Simon's algorithm, the measurement results y_i are not the direct answer s to the problem, but rather satisfies the property $s \cdot y_i = 0$. Linear equations need to be solved to obtain the final answer. In this case, for a complete algorithm design, the model should also specify the post-process steps to derive the answer to the original problem.
- Quantum circuits for the same algorithm vary with different qubit number n . Although this is trivial for theoretical design, it needs to be considered when implementing concrete quantum circuits.

In this category, we cover a wide range of quantum algorithms with varying complexity, from fundamental primitives and textbook-level algorithms to *advanced applications*. For example, we implemented Generalized Simon's Problem (Ye et al., 2021), a more advanced version of the standard Simon's problem and an active area of research in recent years (Ye et al., 2021; Wu et al., 2022). The setting is formally stated as follows: given an (unknown) function $f: \mathbb{Z}_p^n \rightarrow X$ where X is a finite set and a k is a positive integer satisfying $k < n$, it is guaranteed that there exists a subgroup $S \leq \mathbb{Z}_p^n$ of rank k such that for any $x, y \in \mathbb{Z}_p^n$, $f(x) = f(y)$ iff $x - y \in S$. The goal is to find S . Intuitively, the generalized Simon's problem extends the standard Simon's problem from binary to p -ary bases and

from a single secret string to a subgroup of rank k . Beyond universal quantum algorithms, we also consider quantum teleportation and quantum key distribution, two widespread protocols in quantum information. We cover their details in Appendix B.

4.1.3 TASK III: RANDOM CIRCUIT SYNTHESIS

The third task we consider is random circuit synthesis. On the one hand, random circuit sampling is the first algorithm for showing quantum supremacy by Google in 2019 (Arute et al., 2019), and is still widely applied to demonstrate the power of quantum algorithms in recent research (Wu et al., 2021; Bluvstein et al., 2024; DeCross et al., 2024). On the other hand, this also naturally enlarges our dataset. To be more specific, we generate the dataset by randomly sampling gates from a Clifford gate set $\{H, S, CNOT\}$ and a universal set $\{H, S, T, CNOT\}$ separately. We then simulate the circuits to obtain the final quantum states. In each task, the problem description provides the vector of the final state, and the model is required to generate quantum circuits that reproduce this state using the specified gate set.

4.2 DATASET STRUCTURE

The overall structure of QCircuitNet is illustrated as follows (more details are given in Appendix A):

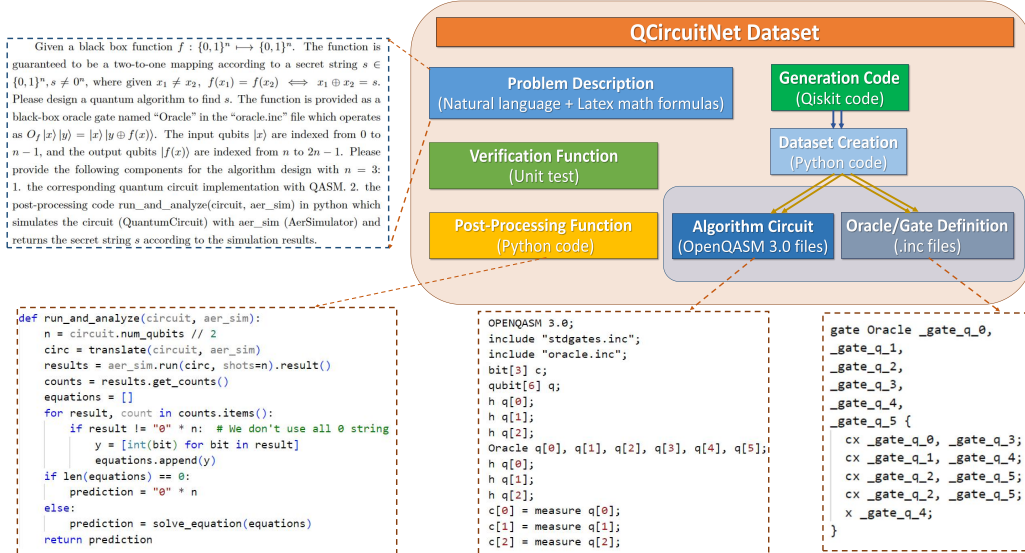


Figure 1: Structure of QCircuitNet. The components of QCircuitNet are presented in the frame on the top-right. As a showcase, this figure presents the components for Simon’s problem (Simon, 1997), including its problem description in natural language, post-processing function in python code, circuit in a .qasm file, and oracle definition in a .inc file.

Design Principles. As discussed in Section 4.1, a critical consideration in formulating the framework is the dilemma between providing the oracle as a black box for quantum algorithm design and the need for its explicit construction to execute the circuit and interpret the results, making the algorithm design complete. Additionally, model training and reference present challenges, particularly for LLMs in generating complex and precise composite gates and evaluating the results efficiently. To address these obstacles, we highlight the following construction principles, which are specially designed to adapt to these tasks:

- For algorithm design tasks, as discussed in Section 4.1.2, we provide the oracle as a black-box gate named "Oracle" with the explicit definition in a separate "oracle.inc" library, which is supported by the OpenQASM 3.0 grammar. In this way, we make sure that the model can use the oracle without

accessing its underlying function, which solves the problem of isolating oracle definition from the algorithm circuit.

- For oracle construction tasks, we ask the model to directly output the quantum circuit in QASM format. For algorithm design task, we require both a quantum circuit and a post-processing function to derive the final answer from circuit execution results. Moreover, we ask the model to explicitly set the shots needed to run the circuit itself in order to characterize the query complexity, which is critical in the theoretical analysis of algorithms.
- For available quantum gates, we provide the definition of some important composite gates not included in the standard QASM gate library in a "customgates.inc". Hierarchical definition for multi-controlled X gate contains 45060 lines for qubit number $n = 14$ in OpenQASM format, which is impossible for AI models to accurately generate at the time. Providing these as a .inc file guarantees the correctness of OpenQASM's grammar while avoiding the generation of complicated gates, which is a distraction from the original design task.
- To verify models' output automatically without human evaluation, we compose verification functions to validate the syntax of QASM / Qiskit and the functionality of the implemented circuits / codes. Since comprehensive Logic Equivalence Checking (LEC) might be inefficient for the throughput of LLM inference, we perform the verification by directly checking the correctness of output with extensive test cases.

Based on these principles, we proposed the framework of QCircuitNet. Below is a more detailed explanation for the 7 components of the dataset:

1. **Problem Description:** carefully hand-crafted prompts stating the oracle to be constructed or the target problem to be solved in natural language and latex math formulas. If the problem involves the usage of a quantum oracle or composite gates beyond the standard gate library, the interfaces of the oracle / gate will also be included (input qubits, output qubits, function mechanism).
2. **Generation Code:** one general Qiskit (Javadi-Abhari et al., 2024) code to create quantum circuits for oracles or algorithms of different settings, such as distinct secret strings or various qubit numbers. We choose Qiskit as the main experiment platform because it is a general quantum programming software widely used for the complete workflow from creating quantum circuits to transpiling, simulation, and execution on real hardware.
3. **Algorithm Circuit:** a .qasm file storing the quantum circuit for each specific setting. We choose OpenQASM 3.0 (Cross et al., 2022) as the format to store the quantum circuits, because Qiskit, as a python library, can only create quantum circuits at runtime instead of explicitly saving the circuits at gate level.¹
4. **Post-Processing Function:** this is for Algorithm Design task only, see Section 4.1.2. The function takes a complete quantum circuit as input, uses the Qiskit AerSimulator to execute the circuit, and returns the final answer to the original problem according to the simulation results. For state preparation problems such as creating a GHZ state of n qubits, this function returns the qubit indices of the generated state.
5. **Oracle / Gate Definition:** a .inc file to provide definitions of composite gates or oracles. For oracle construction tasks, this only includes the definition of composite gates required to build the oracle. For algorithm design tasks, we also provide the gate definition of the oracle in this file, which successfully delivers the oracle in a black-box way.
6. **Verification Function:** a function to evaluate whether the implemented oracle / algorithm achieves the desired purpose with grammar validation and test cases verification. If there exist grammar errors, the function returns -1 and provides detailed error message, which can be used as feedback for the LLM to improve through interactive reasoning. If the program can execute successfully, the function returns a score between $[0, 1]$ indicating the success rate on test cases.²
7. **Dataset Creation Script:** the script to create the dataset from scratch in the format suitable for benchmarking / fine-tuning LLMs. It contains the following functions: 1. generate primitive

¹ Although currently the Qiskit APIs for importing and dumping OpenQASM 3.0 files are still in experimental stage, we choose to adopt version 3.0 over 2.0 in that it supports saving parameterized circuits, which allows for extending the framework to variational quantum algorithms (Cerezo et al., 2021).

² The verification function explicitly integrates the oracle / gate definition library with output algorithm circuit since Qiskit importer for OpenQASM 3.0 does not support non-standard gate libraries currently.

QASM circuits. 2. extract gate definitions and add include instructions to create algorithm circuit, the direct output of model. 3. validate and verify the correctness of the data points in the dataset. 4. concatenate algorithm circuit with problem description as a json file for the benchmark pipeline.

This structure of QCircuitNet provides a general framework to formulate quantum algorithm design for large language models, with an easy extension to more advanced quantum algorithms.

5 EXPERIMENTS

5.1 BENCHMARKING LLMs ON QCIRCUITNET

We benchmark the quantum algorithm design capabilities of leading closed-source and open-source large language models using QCircuitNet. The workflow of our benchmark is illustrated in Figure 2. The total computation cost is approximately equivalent to two days on an A100 GPU.

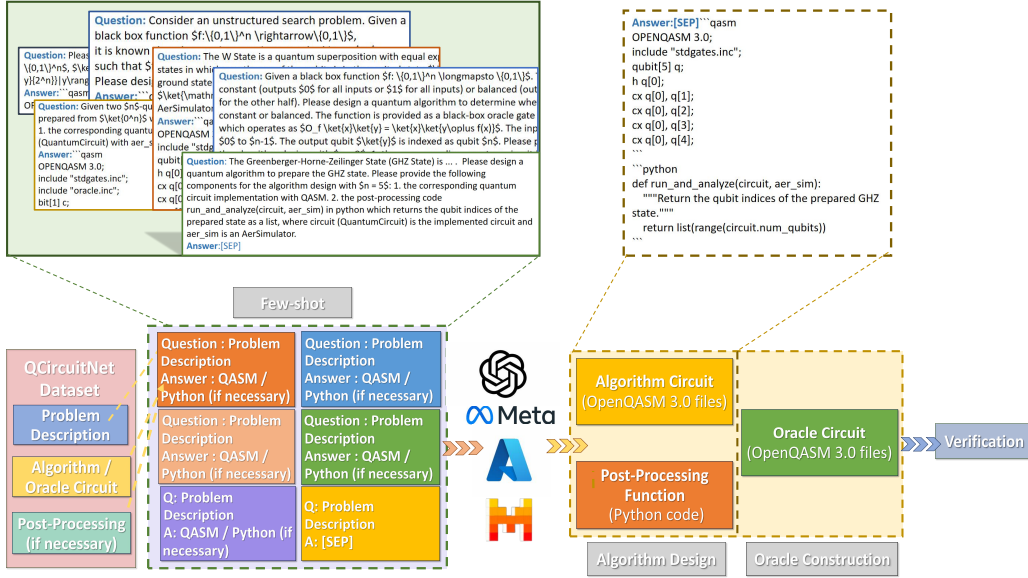


Figure 2: Flowchart of benchmarking QCircuitNet.

Models. Recently, the GPT series models have become the benchmark for generative models due to their exceptional performance. Specifically, we include two models from OpenAI, GPT-3.5-turbo (Brown et al., 2020) and GPT-4 (OpenAI et al., 2024), in our benchmark. Additionally, the LLAMA series models (Touvron et al., 2023a,b) are widely recognized as leading open-source models, and we have selected LLAMA-3-8B for our study. For a comprehensive evaluation, we also benchmark Phi-3-medium-128k (Abdin et al., 2024) and Mistral-7B-v0.3 (Jiang et al., 2023).

Prompts. We employ a few-shot learning framework, a prompting technique that has shown considerable success in generative AI (Xie et al., 2021). In this approach, we utilize either 1 or 5 examples, followed by a problem description. To ensure we do not train and test on the same quantum algorithm, we implement k-fold validation. This method involves using one problem as the test set while the remaining problems serve as the training set, rotating through each problem one at a time.

Evaluation Metrics. We use three evaluation metrics (see Appendix C.1 for more details):

1. BLEU Score: this metric measures how closely the generated code matches the reference code, with a higher BLEU score indicating greater similarity.

2. Verification function: this function checks the syntax validation and the result correctness of the code produced by the language model.
3. Byte Perplexity: this metric evaluates the model’s ability to predict the next byte in a sequence. Lower byte perplexity indicates better performance by reflecting the model’s predictive accuracy.

The results for BLEU and verification function score are shown in Figure 3, Table 1, and Table 2. We include the results of Byte Perplexity in Appendix C.2.

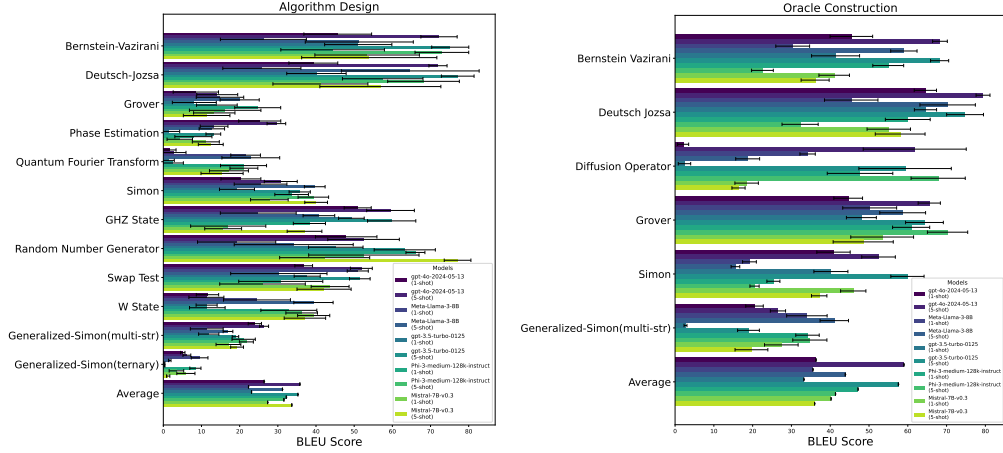


Figure 3: Benchmarking algorithm design and oracle construction in BLEU scores.

Table 1: Benchmarking algorithm design in verification function scores.

Model	Shot	Bernstein Vazirani	Deutsch Jozsa	Grover	Phase Estimation	QFT	Simon	GHZ	Random Number Generator	Swap Test	W State	Generalized Simon (multi-str)	Generalized Simon (ternary)	Avg
gpt4o	1	-0.8462 (±0.1042)	-0.5538 (±0.1986)	-0.7089 (±0.1879)	-0.8423 (±0.0000)	-1.0000 (±0.0000)	-0.6692 (±0.1447)	-0.8462 (±0.1538)	-1.0000 (±0.0000)	-1.0000 (±0.0000)	-1.0000 (±0.0000)	-0.8844 (±0.1156)	-0.6667 (±0.3333)	-0.8348
gpt4o	5	-0.3054 (±0.2086)	0.0135 (±0.2070)	-0.2071 (±0.2089)	-0.5846 (±0.3107)	-0.3692 (±0.1804)	-0.6154 (±0.1443)	-0.1538 (±0.2738)	-0.4967 (±0.2210)	-0.8700 (±0.1300)	-0.9231 (±0.0769)	-0.4889 (±0.2077)	0.0000 (±0.0000)	-0.4167
Llama3	1	-0.2308 (±0.2571)	-0.7692 (±0.1216)	-0.7143 (±0.1844)	-1.0000 (±0.1429)	-0.9231 (±0.0769)	-1.0000 (±0.0000)	-0.6154 (±0.1804)	-0.9285 (±0.0715)	-1.0000 (±0.0000)	-0.3846 (±0.1404)	-1.0000 (±0.0000)	-1.0000 (±0.0000)	-0.7972
Llama3	5	0.0769 (±0.2107)	-0.2308 (±0.1662)	-0.5393 (±0.2185)	-0.9231 (±0.0000)	-0.7692 (±0.1662)	-0.8462 (±0.1042)	-0.3846 (±0.1404)	-0.7276 (±0.1468)	-1.0000 (±0.0000)	-0.1538 (±0.1042)	-0.8889 (±0.1111)	-1.0000 (±0.0000)	-0.6155
gpt3.5	1	-0.8462 (±0.1042)	-0.7154 (±0.1503)	-0.5679 (±0.2037)	-1.0000 (±0.0000)	-1.0000 (±0.0000)	-0.6231 (±0.1680)	-0.8462 (±0.1538)	-1.0000 (±0.0000)	-1.0000 (±0.0000)	-1.0000 (±0.0000)	-0.7778 (±0.1470)	-1.0000 (±0.0000)	-0.8647
gpt3.5	5	-0.6154 (±0.1404)	-0.0571 (±0.1934)	-0.0590 (±0.1687)	-0.9154 (±0.0000)	-0.6338 (±0.1538)	-0.1646 (±0.1395)	-0.2308 (±0.2809)	-0.4513 (±0.2410)	-0.8778 (±0.1222)	-0.8462 (±0.1042)	-0.3311 (±0.1672)	0.0000 (±0.0000)	-0.4328
Phi3	1	-0.8462 (±0.1538)	-0.7750 (±0.1527)	-1.0000 (±0.0000)	-1.0000 (±0.0000)	-1.0000 (±0.0000)	-1.0000 (±0.0000)	-0.3846 (±0.2130)	-1.0000 (±0.0000)	-0.8878 (±0.1122)	-0.8462 (±0.1042)	-1.0000 (±0.0000)	-1.0000 (±0.0000)	-0.8950
Phi3	5	-0.6577 (±0.1891)	-0.3821 (±0.2342)	-0.8286 (±0.1714)	-0.6923 (±0.0000)	-1.0000 (±0.0000)	-0.6100 (±0.1425)	-0.9231 (±0.0769)	-0.3569 (±0.2402)	-0.8333 (±0.1667)	-0.8462 (±0.1042)	-0.8889 (±0.1111)	-1.0000 (±0.0000)	-0.7516
Mistral	1	-0.8462 (±0.1042)	-0.8590 (±0.1410)	-0.7107 (±0.1868)	-1.0000 (±0.0000)	-1.0000 (±0.0000)	-0.9192 (±0.0808)	-0.7692 (±0.1662)	-1.0000 (±0.0000)	-1.0000 (±0.0000)	-0.6923 (±0.1332)	-0.8889 (±0.1111)	-1.0000 (±0.0000)	-0.8905
Mistral	5	-0.6246 (±0.1664)	-0.6667 (±0.1820)	-0.4071 (±0.2106)	-1.0000 (±0.1429)	-0.9231 (±0.0769)	-0.9115 (±0.0885)	-0.6923 (±0.1332)	-0.8820 (±0.1180)	-1.0000 (±0.0000)	-0.5385 (±0.1439)	-0.8889 (±0.1111)	-0.6667 (±0.3333)	-0.7668

The results illustrate that most models achieve better scores in the five-shot setting, which indicates their ability to learn effectively from contextual examples. Notably, models perform well on tasks like Bernstein-Vazirani and Deutsch-Jozsa but struggle with more complex algorithms such as Grover, phase estimation, and quantum Fourier transform, highlighting differences in task difficulty. Furthermore, although the BLEU scores show a general trend of consistency with verification scores, some discrepancies arise, such as the swap test showing relatively high BLEU scores but incorrect algorithm generation by most models. This observation emphasizes the need for complementary evaluation metrics beyond BLEU to accurately assess model performance, which highlights the importance of our verification function. Additionally, GPT-4o and GPT-3.5 consistently excel in long-context comprehension, significantly outperforming other models across tasks, which highlights their superior in-context learning capabilities. For more detailed analysis of the experimental results, we refer to Appendix C.1.

Types of Errors Made by LLMs. In Appendix C.3, we include several case studies to illustrate and analyze various types of errors made by LLMs. For example, GPT-4o tends to use advanced OpenQASM 3.0 features unsupported by Qiskit yet and novel namespaces which might result in global

Table 2: Benchmarking oracle construction in verification function scores.

Model	Shot	Bernstein-Vazirani	Deutsch-Jozsa	Diffusion-Operator	Grover	Simon	Generalized-Simon (multi-str)	Avg
gpt4o	1	-0.3200 (± 0.0530)	-0.0100 (± 0.0438)	-0.8462 (± 0.1538)	-0.9885 (± 0.0115)	-0.4674 (± 0.0545)	-0.3750 (± 0.0870)	-0.5012
gpt4o	5	-0.1100 (± 0.0399)	0.0800 (± 0.0506)	-0.3077 (± 0.2083)	-0.9540 (± 0.0279)	-0.0870 (± 0.0295)	-0.3125 (± 0.0832)	-0.2819
Llama3	1	-0.7300 (± 0.0468)	-0.5000 (± 0.0704)	-0.3846 (± 0.1404)	-1.0000 (± 0.0000)	-0.6848 (± 0.0487)	-0.9375 (± 0.0435)	-0.7061
Llama3	5	-0.0500 (± 0.0359)	0.1700 (± 0.0551)	-0.8462 (± 0.1042)	-1.0000 (± 0.0000)	-0.6413 (± 0.0503)	-0.6875 (± 0.0832)	-0.5092
gpt3.5	1	-0.3500 (± 0.0539)	-0.0400 (± 0.0470)	-0.8462 (± 0.1538)	-1.0000 (± 0.0000)	-0.3696 (± 0.0529)	-1.0000 (± 0.0000)	-0.6010
gpt3.5	5	-0.1100 (± 0.0373)	0.0200 (± 0.0531)	-0.3077 (± 0.2083)	-0.9770 (± 0.0162)	-0.1087 (± 0.0326)	-0.4063 (± 0.0989)	-0.3149
Phi3	1	-0.6800 (± 0.0510)	-0.6100 (± 0.0584)	-0.9231 (± 0.0769)	-1.0000 (± 0.0000)	-0.7500 (± 0.0454)	-1.0000 (± 0.0000)	-0.8272
Phi3	5	-0.5400 (± 0.0521)	-0.4300 (± 0.0685)	-1.0000 (± 0.0000)	-1.0000 (± 0.0000)	-0.8370 (± 0.0387)	-0.9063 (± 0.0524)	-0.7855
Mistral	1	-0.4000 (± 0.0512)	-0.4300 (± 0.0640)	-0.9231 (± 0.0769)	-0.9540 (± 0.0279)	-0.6087 (± 0.0512)	-0.9375 (± 0.0435)	-0.7089
Mistral	5	-0.3700 (± 0.0506)	-0.1300 (± 0.0734)	-1.0000 (± 0.0000)	-0.9195 (± 0.0373)	-0.2391 (± 0.0447)	-0.9063 (± 0.0524)	-0.5942

conflicts in one-shot setting. This tendency to improvise by drawing on pre-trained knowledge rather than closely following the syntax of the example leads to avoidable "errors" and low verification scores. This issue is significantly alleviated in the 5-shot setting, highlighting GPT-4o's strong in-context learning ability.

5.2 FINE-TUNING ON QCIRCUITNET

Although QCircuitNet is targeted as a benchmark dataset at current stage, we consider fine-tuning / training from scratch based on our dataset as an interesting and important research direction. The unique nature of quantum data requires novel fine-tuning methods and model architecture designs, which could serve as a standalone topic. For a primitive demonstration, we present fine-tuning results on data from the oracle construction task here.

Following [Dettmers et al. \(2024\)](#), we quantize the model to 8-bits and then train it with LORA ([Hu et al., 2022](#)). In our experiments, we use fp16 computational datatype. We set LoRA $r = 16$, $\alpha = 32$ and add LoRA modules on all the query and value layers. We also use AdamW ([Loshchilov & Hutter, 2019](#)) and LoRA dropout of 0.05. The results are shown as follows:

Table 3: Fine-tuning oracle construction scores.

Score	Model	Setting	Bernstein-Vazirani	Deutsch-Jozsa	Grover	Simon	Clifford	Universal	Avg
BLEU	gpt4o	few-shot(5)	95.6388 (± 0.3062)	91.0564 (± 0.6650)	92.0620 (± 0.6288)	80.3390 (± 2.0900)	39.5469 (± 3.6983)	33.3673 (± 3.1007)	72.0017
	Llama3	few-shot(5)	53.5574 (± 5.2499)	69.8996 (± 5.7812)	61.3102 (± 5.4671)	26.3083 (± 2.0048)	13.0729 (± 0.9907)	13.4185 (± 1.2299)	39.5945
	Llama3	finetune	76.0480 (± 7.9255)	71.8378 (± 2.4179)	67.7892 (± 7.8900)	43.8469 (± 3.2998)	10.8978 (± 0.6169)	7.1854 (± 0.5009)	46.2675
Verification	gpt4o	few-shot(5)	0.0000 (± 0.0246)	0.4300 (± 0.0590)	0.0000 (± 0.1005)	-0.0200 (± 0.0141)	-0.0333 (± 0.0401)	-0.1023 (± 0.0443)	0.0457
	Llama3	few-shot(5)	-0.2700 (± 0.0468)	0.0900 (± 0.0668)	-0.5200 (± 0.0858)	-0.6600 (± 0.0476)	-0.7303 (± 0.0473)	-0.5056 (± 0.0549)	-0.4327
	Llama3	finetune	-0.1300 (± 0.0485)	-0.2000 (± 0.0402)	-0.3300 (± 0.0900)	-0.7400 (± 0.0441)	-0.8741 (± 0.0343)	-0.9342 (± 0.0262)	-0.5347
PPL	Llama3	few-shot(5)	1.1967 (± 0.0028)	1.1174 (± 0.0015)	1.1527 (± 0.0021)	1.1119 (± 0.0017)	1.4486 (± 0.0054)	1.4975 (± 0.0051)	1.2541
	Llama3	finetune	1.0004 (± 0.0002)	1.1090 (± 0.0014)	1.0010 (± 0.0006)	1.1072 (± 0.0011)	1.2944 (± 0.0053)	1.3299 (± 0.0055)	1.1403

We compare the performance of Llama3-8B before and after fine-tuning with case studies. Take oracle construction of Bernstein-Vazirani Problem as an example, we observed that before fine-tuning, the model would indiscriminately apply CX gates to all qubits. After fine-tuning, it began to selectively apply CX gates to qubits with '1's in the secret string. In some cases, the positions were still counted incorrectly; however, in certain instances, the model accurately identified all the positions for applying the CX gates, which is highly impressive. This improvement significantly

contributed to higher scores, suggesting that the model is starting to learn the pattern for constructing certain oracles through fine-tuning. Regarding the interesting performance decrease on Clifford and universal random circuits, we conducted additional experiments on temperature and refer to Appendix C.2 for more details.

6 CONCLUSIONS AND FUTURE WORK

In this paper, we propose QCircuitNet, the first comprehensive, structured universal quantum algorithm dataset and quantum circuit generation benchmark for AI models. This framework formulates quantum algorithm design from the programming language perspective and includes detailed descriptions and implementation of most established and important quantum algorithms / primitives, allowing for automatic verification methodologies. Benchmarking of QCircuitNet on up-to-date LLMs is systematically conducted. Fine-tuning results also showcase the potential of QCircuitNet as a training dataset. As shown by these benchmarking and fine-tuning results, QCircuitNet helps guide LLMs for reasoning and implementation. As we gradually extend the dataset, it will hopefully contribute to interactive quantum algorithm design.

We also highlight the challenges and opportunities as follows, ranging from scalability on the quantum algorithm side to data contamination on the AI side:

Scalability of the Approach. Our framework is designed to scale with increasing qubit numbers and support complex quantum algorithms as long as they are efficiently implementable with polynomial gates. The dataset generation scripts are written in a generic way which can be easily extended to arbitrarily large qubit numbers. The implementation of Generalized Simon’s Problem mentioned in Section 4.1.2 showcases the compatibility of our framework with more complex algorithms. The main bottleneck for scalability in the pipeline lies in the simulation process in verification function. For simplicity of demonstration and consideration of hardware noise, all the verification functions were run with classical simulations in our experiments. But the APIs we implemented are compatible with IBM hardware and can be easily adapted to quantum computers, which reduces the cost of classical simulation. As quantum hardware becomes widespread and the capabilities of large language models continue to advance, the dataset can be easily expanded to accommodate scaling.

Data Contamination in AI Learning. We observe a performance separation between writing general Qiskit codes and explicit gate-level circuits in QASM. Since Qiskit provides detailed tutorial with general codes for several algorithms, this may imply a *data contamination* phenomenon where LLMs rely on memorization and retrieval rather than genuine algorithm design (see Appendix C.4 for more details). Similarly, current benchmarks for AI code generation and syntax learning may also suffer from this unseen bias. Our dataset, based on QASM files created from scratch, may help circumvent this issue and serve as a stable and fair method for benchmarking AI syntax learning.

Our work leaves several **open questions** for future investigation:

- QCircuitNet is a benchmarking dataset for LLMs. It is of general interest to extend benchmarking to training, which will help LLMs better maneuver quantum algorithm design. We have implemented advanced algorithms such as the Generalized Simon’s Problem, but this in general needs implementations of more advanced algorithms to make it a more meaningful training dataset.
- Since quantum algorithms have fundamental difference from classical algorithms, novel fine-tuning methods to attempt quantum algorithm design and quantum circuit implementation, or even developments of new quantum algorithms by LLMs are solicited.
- Currently, variational quantum algorithms (Cerezo et al., 2021) can already be implemented on near-term NISQ machines (Preskill, 2018). It would be also of general interest to extend QCircuitNet to contain the design and implementation of variational quantum algorithms.

ACKNOWLEDGEMENTS

RY and TL were supported by the National Natural Science Foundation of China (Grant Numbers 92365117 and 62372006), and The Fundamental Research Funds for the Central Universities, Peking University. We would like to thank Hengli Li and Zihao Wang for their valuable discussions.

A DETAILS OF QCIRCUITNET

The QCircuitNet Dataset is available on GitHub at the following link: https://github.com/EstelYang/QCircuitNet_Dataset

QCircuitNet has the following directory structure:

```

QCircuitNet
├── Oracle Construction ..... All data for the oracle construction task
│   ├── Quantum Logic Synthesis ..... Textbook-level and advanced oracles
│   └── Problem Encoding ..... Oracles encoding application scenarios
├── Algorithm Design ..... All data for the quantum algorithm design task
│   ├── Quantum Computing ..... Universal quantum computing algorithms
│   └── Quantum Information ..... Quantum information tasks and protocols
├── Random Circuits ..... All data for the random circuit synthesis task
│   ├── Clifford ..... Random circuits with the Clifford gate set
│   └── Universal ..... Random circuits with the universal gate set

```

In each subdirectory, there is a folder for each specific algorithm. For instance, the folder structure for Simon’s algorithm is as follows:

```

Algorithm Design
├── Quantum Computing
│   └── simon ..... All data for the Simon’s Problem
│       ├── simon-dataset.py ..... Dataset creation script
│       ├── simon-generation.py ..... Qiskit generation code
│       ├── simon-post-processing.py ..... Post-processing function
│       ├── simon-utils.py ..... Utility functions for verification
│       ├── simon-verification.py ..... Verification function
│       ├── simon-description.txt ..... Problem description
│       ├── simon-verification.txt ..... Verification results of the data points
│       ├── full circuit ..... Raw data of quantum circuits
│       ├── simon-n2
│       │   └── simon-n2-s11-k11.qasm ..... Full circuit for a concrete setting
│       ├── simon-n3
│       │   ├── simon-n3-s011-k001.qasm
│       │   └── simon-n3-s011-k101.qasm
│       └── ...
│   ├── test oracle ..... Extracted oracle definitions
│   │   ├── n2
│   │   │   ├── trial1
│   │   │   │   ├── oracle.inc ..... Oracle definition as a .inc file
│   │   │   │   └── oracle-info.txt ..... Oracle information (such as key strings)
│   │   ├── n3
│   │   │   ├── trial1
│   │   │   │   ├── oracle.inc
│   │   │   │   └── oracle-info.txt
│   │   │   ├── trial2
│   │   │   │   ├── oracle.inc
│   │   │   │   └── oracle-info.txt
│   │   └── ...
│   ├── simon-n2.qasm ..... Algorithm circuit for model output
│   ├── simon-n3.qasm
│   └── ...

```

We expect to extend QCircuitNet under this general structure.

A.1 FORMAT

In this subsection, we provide concrete examples to illustrate the different components of QCircuitNet. We use the case of Simon's Problem throughout the demonstration to achieve better consistency. For further details, please check the code repository.

1. **Problem Description:** this is the carefully hand-crafted description of the task in natural language and latex math formulas. The description is provided as one template for each algorithm, and the concrete settings (such as the qubit number) are replaced when creating the data points in json. The file is named as "{algorithm_name}_description.txt".

Problem Description Template for Simon's Problem

Given a black box function $f : \{0, 1\}^n \mapsto \{0, 1\}^n$. The function is guaranteed to be a two-to-one mapping according to a secret string $s \in \{0, 1\}^n, s \neq 0^n$, where given $x_1 \neq x_2, f(x_1) = f(x_2) \iff x_1 \oplus x_2 = s$. Please design a quantum algorithm to find s . The function is provided as a black-box oracle gate named "Oracle" in the "oracle.inc" file which operates as $O_f |x\rangle |y\rangle = |x\rangle |y \oplus f(x)\rangle$. The input qubits $|x\rangle$ are indexed from 0 to $n - 1$, and the output qubits $|f(x)\rangle$ are indexed from n to $2n - 1$. Please provide the following components for the algorithm design with $n = \{\text{qubit number}\}$: 1. the corresponding quantum circuit implementation with {QASM / Qiskit}. 2. the post-processing code `run_and_analyze(circuit, aer_sim)` in python which simulates the circuit (QuantumCircuit) with `aer_sim` (AerSimulator) and returns the secret string s according to the simulation results.

2. **Generation Code:** one general Qiskit code to create quantum circuits of different settings. Note that the oracle for the problem is provided as a black-box gate "oracle" here. This code is used to generate the raw data, but can also be used as a testing benchmark for writing Qiskit codes. The file is named as "{algorithm_name}_generation.py".

```
from Qiskit import QuantumCircuit

def simon_algorithm(n, oracle):
    """Generates a Simon algorithm circuit.

    Parameters:
    - n (int): number of qubits
    - s (str): the secret string of length n

    Returns:
    - QuantumCircuit: the Simon algorithm circuit
    """
    # Create a quantum circuit on 2n qubits
    simon_circuit = QuantumCircuit(2 * n, n)

    # Initialize the first register to the |+> state
    simon_circuit.h(range(n))

    # Append the Simon's oracle
    simon_circuit.append(oracle, range(2 * n))

    # Apply a H-gate to the first register
    simon_circuit.h(range(n))

    # Measure the first register
    simon_circuit.measure(range(n), range(n))

    return simon_circuit
```

Listing 1: Qiskit generation code for Simon's algorithm.

3. **Algorithm Circuit:** the OpenQASM 3.0 format file storing the quantum circuit in gate level for each specific setting. Note that the explicit construction of "Oracle" is provided separately in "oracle.inc" file, which guarantees the usage of oracle in a black-box way. This file is named as "{algorithm_name}_n{qubit_number}.qasm".

```
OPENQASM 3.0;
include "stdgates.inc";
include "oracle.inc";
bit[3] c;
qubit[6] q;
h q[0];
h q[1];
h q[2];
Oracle q[0], q[1], q[2], q[3], q[4], q[5];
h q[0];
h q[1];
h q[2];
c[0] = measure q[0];
c[1] = measure q[1];
c[2] = measure q[2];
```

Listing 2: OpenQASM 3.0 Code for Simon's algorithm with $n = 3$.

4. **Post-Processing Function:** this function simulates the quantum circuit and derives the final answer to the problem. The file is named as "{algorithm_name}_post_processing.py".

```
from sympy import Matrix
import numpy as np
from Qiskit import transpile

def mod2(x):
    return x.as_numer_denom()[0] % 2

def solve_equation(string_list):
    """
    A^T | I
    after the row echelon reduction, we can get the basis of the
    ↪ nullspace of A in I
    since we just need the string in binary form, so we can just use
    ↪ the basis
    if row == n-1 --> only one
    if row < n-1 --> get the first one (maybe correct or wrong)
    """
    M = Matrix(string_list).T

    # Augmented : M | I
    M_I = Matrix(np.hstack([M, np.eye(M.shape[0], dtype=int)]))

    # RREF row echelon form , indices of the pivot columns
    # If x % 2 = 0, it will not be chosen as pivot (modulo 2)
    M_I_rref = M_I.rref(iszerofunc=lambda x: x % 2 == 0)

    # Modulo 2
    M_I_final = M_I_rref[0].applyfunc(mod2)

    # Non-Trivial solution
    if all(value == 0 for value in M_I_final[-1, : M.shape[1]]):
        result_s = "".join(str(c) for c in M_I_final[-1, M.shape[1] :])

    # Trivial solution
    else:
        result_s = "0" * M.shape[0]
```

```

return result_s

def run_and_analyze(circuit, aer_sim):
    n = circuit.num_qubits // 2
    circ = transpile(circuit, aer_sim)
    results = aer_sim.run(circ, shots=n).result()
    counts = results.get_counts()
    equations = []
    for result, count in counts.items():
        if result != "0" * n: # We don't use all 0 string
            y = [int(bit) for bit in result]
            equations.append(y)
    if len(equations) == 0:
        prediction = "0" * n
    else:
        prediction = solve_equation(equations)
    return prediction

```

Listing 3: Post-processing code for Simon's algorithm.

5. **Oracle / Gate Definition:** this .inc file provides the definitions of composite gates or oracles. The file is named "customgates.inc" for oracle construction tasks and "oracle.inc" for algorithm design tasks.

```

gate Oracle _gate_q_0, _gate_q_1, _gate_q_2, _gate_q_3, _gate_q_4,
    ↪ _gate_q_5 {
    cx _gate_q_0, _gate_q_3;
    cx _gate_q_1, _gate_q_4;
    cx _gate_q_2, _gate_q_5;
    cx _gate_q_2, _gate_q_5;
    x _gate_q_3;
}

```

Listing 4: One test case oracle for Simon's algorithm with $n = 3$.

For algorithm design tasks, this .inc file is accompanied with an "oracle_info.txt" file to describe the encoded information of the oracle. This helps the verification function to check the correctness of the derived answer by the model. The above test case is equipped with the following information text:

oracle_info.txt for Simon's Problem with qubit number 3 and test case 2.

Secret string: 100
Key string: 001

6. **Verification Function:** the function to evaluate the output with grammar validation and test cases verification. The file is named as "{algorithm_name}_verification.py".

```

from simon_utils import *

def check_model(qasm_string, code_string, n):
    """Check the Simon model."""
    # Verify the syntax of the QASM code with the first test case
    ↪ oracle
    t = 1
    with open(f"test_oracle/n{n}/trial{t}/oracle.inc", "r") as file:
        oracle_def = file.read()
    full_qasm = plug_in_oracle(qasm_string, oracle_def)
    circuit = verify_qasm_syntax(full_qasm)
    if circuit is None:
        return -1
    try:

```

```

exec(code_string, globals())
aer_sim = AerSimulator()
total_success = 0
total_fail = 0
t_range = min(10, 4 * (n - 2))
shots = 10
for t in range(1, 1 + t_range):
    print(f"    Running Test Case {t}")
    with open(f"test_oracle/n{n}/trial{t}/oracle.inc", "r") as
        ↪ file:
        oracle_def = file.read()
        full_qasm = plug_in_oracle(qasm_string, oracle_def)
        circuit = loads(full_qasm)
        with open(f"test_oracle/n{n}/trial{t}/oracle_info.txt", "r"
            ↪ ) as file:
            content = file.read()
            match = re.search(r"Secret string: ([01]+)", content)
            if match:
                secret_string = match.group(1)
            else:
                raise ValueError("Secret string not found in the file."
                    ↪ )

    cnt_success = 0
    cnt_fail = 0
    for shot in range(shots):
        prediction = run_and_analyze(circuit.copy(), aer_sim)
        if not isinstance(prediction, str):
            raise TypeError("Predicted secret string should be
                ↪ a string.")
        if prediction == secret_string:
            cnt_success += 1
        else:
            cnt_fail += 1
    print(f"    Success: {cnt_success}/{shots}, Fail: {
        ↪ cnt_fail}/{shots}")
    total_success += cnt_success
    total_fail += cnt_fail
print(f"Total Success: {total_success}; Total Fail: {total_fail}
    ↪ ")
return total_success / (total_fail + total_success)

except Exception as e:
    print(f"Error: {e}")
    return -1

```

Listing 5: Verification function for Simon's algorithm.

This verification function is accompanied with an "{algorithm_name}_utils.py" file to provide necessary utility functions.

```

from Qiskit.qasm3 import loads
from Qiskit_aer import AerSimulator
import re

def print_and_save(message, text):
    print(message)
    text.append(message)

def plug_in_oracle(qasm_code, oracle_def):
    """Plug-in the oracle definition into the QASM code."""
    oracle_pos = qasm_code.find('include "oracle.inc";')
    if oracle_pos == -1:

```

```

        raise ValueError("Oracle include statement not found in the
        ↪ file")
    full_qasm = (
        qasm_code[:oracle_pos]
        + oracle_def
        + qasm_code[oracle_pos + len('include "oracle.inc";') :]
    )
    return full_qasm

def verify_qasm_syntax(output):
    """Verify the syntax of the output and return the corresponding
    ↪ QuantumCircuit (if it is valid)."""
    assert isinstance(output, str)
    try:
        # Parse the OpenQASM 3.0 code
        circuit = loads(output)
        print(
            "    The OpenQASM 3.0 code is valid and has been
            ↪ successfully loaded as a QuantumCircuit."
        )
        return circuit
    except Exception as e:
        print(f"    Error: The OpenQASM 3.0 code is not valid. Details:
        ↪ {e}")
        return None

```

Listing 6: Utility functions for verification of Simon’s algorithm.

7. **Dataset Creation Script:** this script involves all the code necessary to create the data points from scratch. The file is named as "{algorithm_name}_dataset.py". The main function looks like this:

```

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument(
        "-f",
        "--func",
        choices=["qasm", "json", "gate", "check"],
        help="The function to call: generate qasm circuit, json dataset
        ↪ or extract gate definition.",
    )
    args = parser.parse_args()
    if args.func == "qasm":
        generate_circuit_qasm()
    elif args.func == "json":
        generate_dataset_json()
    elif args.func == "gate":
        extract_gate_definition()
    elif args.func == "check":
        check_dataset()

```

Listing 7: Main function of the dataset script for Simon’s algorithm.

Here the "generate_circuit_qasm()" function generates the raw data of quantum circuits in OpenQASM 3.0 format where the algorithm circuit and the oracle definition are blended, then "extract_gate_definition()" function extracts the definition of oracles and formulates the algorithm circuits into the format suitable for model output. The "check_dataset()" function is used to check the correctness of the created data points and "generate_dataset_json()" function to combine the data into json format for easy integration with the benchmarking pipeline.

A.2 DISCUSSION OF MORE TASKS

Problem Encoding. In Section [4.1.1](#), we mentioned another category of oracle construction tasks referred to as "Problem Encoding", which involves applying quantum algorithms, such as Grover’s

algorithm, to solve practical problems such as SAT and triangle finding. The crux of this process is encoding the problem constraints into Grover’s oracle, thereby making this a type of oracle construction task. Unlike quantum logic synthesis, which encodes an explicit function $f(x)$ as a unitary operator U_f , this task involves converting the constraints of a particular problem into the required oracle form. We provide implementations of several concrete problems in this directory as demonstrations and will include more applications in future work.

Quantum Information Protocols. In the "Quantum Information" section of the "Algorithm Design" task, we have also implemented three important quantum information protocols: Quantum Teleportation, Superdense Coding, and Quantum Key Distribution (BB84). A brief introduction to these protocols can be found in Appendix [B](#). We did not include the experiments for these protocols as they involve communication between two parties, which is challenging to characterize with a single OpenQASM 3.0 file. We recommend revising the post-processing function as a general classical function to schedule the communication and processing between different parties specifically for these protocols. The fundamental quantum circuits and processing codes are provided in the repository.

A.3 DATASHEET

Here we present a datasheet for the documentation of QCircuitNet.

Motivation

- *For what purpose was the dataset created?* It was created as a benchmark for the capability of designing and implementing quantum algorithms for LLMs.
- *Who created the dataset (e.g., which team, research group) and on behalf of which entity (e.g., company, institution, organization)?* The authors of this paper.
- *Who funded the creation of the dataset?* We will reveal the funding resources in the Acknowledgement section of the final version.

Composition

- *What do the instances that comprise the dataset represent (e.g., documents, photos, people, countries)?* The dataset comprises problem description, generation code, algorithm circuit, post-processing function, oracle / gate definition, verification function, and dataset creation script for various quantum algorithms.
- *How many instances are there in total (of each type, if appropriate)?* The dataset has 5 algorithms for oracle construction task and 10 algorithms for algorithm design task used for experiments. There are 3 quantum information protocols and additional problem encoding tasks not included for experiments.
- *Does the dataset contain all possible instances or is it a sample (not necessarily random) of instances from a larger set?* The dataset contains instances with restricted qubit numbers due to the current scale of real quantum hardware.
- *What data does each instance consist of?* Qiskit codes, OpenQASM 3.0 codes, python scripts, and necessary text information.
- *Are relationships between individual instances made explicit?* Yes, the way to create different instances are clearly described in Appendix [A.1](#).
- *Are there recommended data splits?* Yes, we recommend splitting the data according to different algorithms in algorithm design task.
- *Are there any errors, sources of noise, or redundancies in the dataset?* There might be some small issues due to the dumping process of Qiskit and programming mistakes (if any).
- *Is the dataset self-contained, or does it link to or otherwise rely on external resources (e.g., websites, tweets, other datasets)?* The dataset is self-contained.
- *Does the dataset contain data that might be considered confidential (e.g., data that is protected by legal privilege or by doctor-patient confidentiality, data that includes the content of individuals’ non-public communications)?* No.

- *Does the dataset contain data that, if viewed directly, might be offensive, insulting, threatening, or might otherwise cause anxiety?* No.

Collection Process

- *How was the data associated with each instance acquired?* The data is created by first composing Qiskit codes for each algorithm and then converting to OpenQASM 3.0 files using `Qiskit.qasm3.dump` function, with additional processing procedure.
- *What mechanisms or procedures were used to collect the data (e.g., hardware apparatuses or sensors, manual human curation, software programs, software APIs)?* Manual human programming and Qiskit APIs.
- *Who was involved in the data collection process (e.g., students, crowd workers, contractors), and how were they compensated (e.g., how much were crowd workers paid)?* Nobody other than the authors of the paper.
- *Over what timeframe was the data collected?* The submitted version of the dataset was created in May and June 2024.

Uses

- *Has the dataset been used for any tasks already?* It has been used in this paper to benchmark LLM’s ability for quantum algorithm design.
- *Is there a repository that links to any or all papers or systems that use the dataset?* The only paper which uses the dataset for now is this paper.

Distribution

- *Will the dataset be distributed to third parties outside of the entity (e.g., company, institution, organization) on behalf of which the dataset was created?* Yes, the dataset will be made publicly available on the Internet after the review process.
- *How will the dataset be distributed (e.g., tarball on website, API, GitHub)?* It will be distributed on the GitHub platform.
- *Will the dataset be distributed under a copyright or other intellectual property (IP) license, and/or under applicable terms of use (ToU)?* The dataset is distributed under CC BY 4.0.
- *Have any third parties imposed IP-based or other restrictions on the data associated with the instances?* No.
- *Do any export controls or other regulatory restrictions apply to the dataset or to individual instances?* No.

Maintenance

- *Who will be supporting/hosting/maintaining the dataset?* The authors of this paper.
- *How can the owner/curator/manager of the dataset be contacted (e.g., email address)?* The email for contact will be provided after the review process.
- *Is there an erratum?* Not at this time.
- *Will the dataset be updated (e.g., to correct labeling errors, add new instances, delete instances)?* Yes, it will be continually updated.
- *If others want to extend/augment/build on/contribute to the dataset, is there a mechanism for them to do so?* Yes, they can do so with the GitHub platform.

A.4 COPYRIGHT AND LICENSING TERMS

This work is distributed under a CC BY 4.0 license. The implementation of the code references open-source projects such as Qiskit, QuantumKatas, Cirq, and NWQBench. We bear responsibility in case of violation of rights.

B ADDITIONAL PRELIMINARIES FOR QUANTUM COMPUTING AND QUANTUM INFORMATION

Quantum Circuit Diagram. A quantum algorithm is composed of a series of quantum gates. By default, a quantum algorithm starts from the all-0 state $|0^n\rangle$. A quantum algorithm can be illustrated by its quantum gate diagram, drawn from left to right. The initial all-0 state is placed at the left side of the diagram. After that, whenever we apply a quantum gate, it is placed on the corresponding qubits, from left to right. At the end of the quantum gates, we need to measure and read the outputs, and these measurements are placed at the right side of the diagram. See Figure 4 for the quantum gate diagram of Simon’s algorithm (Simon, 1997).

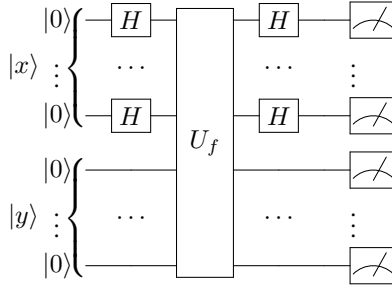


Figure 4: Quantum gate diagram of Simon’s algorithm.

Superdense Coding. Superdense coding (Bennett & Wiesner, 1992) is a quantum communication protocol that allows Alice to transmit two classical bits of information to Bob by sending only one qubit, given that they share a pair of entangled qubits. The protocol can be divided into five steps:

1. **Preparation:** Charlie prepares a maximally entangled Bell state, such as $|\beta_{00}\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$.
2. **Sharing:** Charlie sends the qubit 1 to Alice and the qubit 2 to Bob. Alice and Bob can be separated by an arbitrary distance.
3. **Encoding:** Depending on the two classical bits $zx \in \{00, 01, 10, 11\}$ that Alice wants to send, she applies the corresponding quantum gate operation to her qubit, transforming the Bell state $|\beta_{00}\rangle$ into one of the four Bell states:

$$\begin{aligned} |\beta_{00}\rangle &= \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) \text{ if } zx = 00 \\ |\beta_{01}\rangle &= \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle) \text{ if } zx = 01 \\ |\beta_{10}\rangle &= \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle) \text{ if } zx = 10 \\ |\beta_{11}\rangle &= \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle) \text{ if } zx = 11 \end{aligned}$$

Alice achieves these transformations by applying the operation $Z^z X^x$ to her qubit, where Z is the phase-flip gate, X is the bit-flip gate. Specifically:

- If $zx = 00$, Alice applies $Z^0 X^0 = I$ (identity gate).
 - If $zx = 01$, Alice applies $Z^0 X^1 = X$ (bit-flip gate).
 - If $zx = 10$, Alice applies $Z^1 X^0 = Z$ (phase-flip gate).
 - If $zx = 11$, Alice applies $Z^1 X^1 = ZX = iY$ gate.
4. **Sending:** Alice sends her qubit to Bob through a quantum channel.

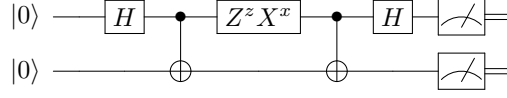


Figure 5: Quantum circuit diagram for superdense coding.

5. **Decoding:** Bob applies a CNOT gate followed by a Hadamard gate to the two qubits, transforming the entangled state into the corresponding computational basis state $|zx\rangle$. By measuring the qubits, Bob obtains the two classical bits zx sent by Alice.

Superdense coding exploits the properties of quantum entanglement to transmit two classical bits of information using only one qubit. The quantum circuit diagram for superdense coding is shown in Figure 5.

Quantum Teleportation. Quantum teleportation (Bennett et al., 1993) is a technique for transferring quantum information from a sender (Alice) to a receiver (Bob) using shared entanglement and classical communication. The protocol can be described as follows:

1. **Preparation:** Telamon prepares a maximally entangled Bell state, such as $|\beta_{00}\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$.
2. **Sharing:** Alice has qubit 1 in the state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, which she wants to teleport to Bob. Telamon shares qubit 2 with Alice and qubit 3 with Bob, creating the shared entangled state $|\beta_{00}\rangle_{23}$.
3. **Encoding:** Alice wants to teleport an unknown quantum state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ to Bob. She applies a CNOT gate to qubits 1 and 2, with qubit 1 as the control and qubit 2 as the target. Then, she applies a Hadamard gate to qubit 1. The resulting state of the three-qubit system is:

$$|\Psi\rangle = \frac{1}{2} [|\beta_{00}\rangle(\alpha|0\rangle + \beta|1\rangle) + |\beta_{01}\rangle(\alpha|1\rangle + \beta|0\rangle) \\ + |\beta_{10}\rangle(\alpha|0\rangle - \beta|1\rangle) + |\beta_{11}\rangle(\alpha|1\rangle - \beta|0\rangle)]$$

4. **Measurement:** Alice measures qubits 1 and 2 in the Bell basis and obtains one of four possible outcomes: $|\beta_{00}\rangle$, $|\beta_{01}\rangle$, $|\beta_{10}\rangle$, or $|\beta_{11}\rangle$. This measurement collapses the three-qubit state into one of the following:

$$\begin{aligned} & |\beta_{00}\rangle \otimes (\alpha|0\rangle + \beta|1\rangle) \\ & |\beta_{01}\rangle \otimes (\alpha|1\rangle + \beta|0\rangle) \\ & |\beta_{10}\rangle \otimes (\alpha|0\rangle - \beta|1\rangle) \\ & |\beta_{11}\rangle \otimes (\alpha|1\rangle - \beta|0\rangle) \end{aligned}$$

5. **Classical Communication:** Alice sends the result of her measurement (two classical bits) to Bob via a classical channel.
6. **Reconstruction:** Depending on the classical information received from Alice, Bob applies the operation $Z^z X^x$ to qubit 3, where z and x correspond to the two classical bits sent by Alice:

- If Alice measured $|\beta_{00}\rangle$, she sends $zx = 00$, and Bob applies $Z^0 X^0 = I$ (identity operation).
- If Alice measured $|\beta_{01}\rangle$, she sends $zx = 01$, and Bob applies $Z^0 X^1 = X$ (bit-flip).
- If Alice measured $|\beta_{10}\rangle$, she sends $zx = 10$, and Bob applies $Z^1 X^0 = Z$ (phase-flip).
- If Alice measured $|\beta_{11}\rangle$, she sends $zx = 11$, and Bob applies $Z^1 X^1 = ZX = iY$ (bit-flip and phase-flip).

After applying the appropriate operation, Bob's qubit 3 will be in the state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, which is the original state that Alice wanted to teleport.

The quantum circuit diagram for quantum teleportation is shown in Figure 6.

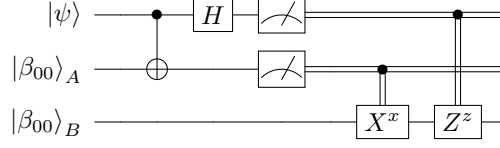


Figure 6: Quantum circuit diagram for quantum teleportation

Quantum Key Distribution. Quantum key distribution (QKD) (Bennett & Brassard, 1984) is a secure communication protocol that allows two parties, Alice and Bob, to produce a shared random secret key, which can then be used to encrypt and decrypt messages. The security of QKD is based on the fundamental principles of quantum mechanics that measuring a qubit can change its state. One of the most well-known QKD protocols is the BB84 protocol, which works as follows:

1. Alice randomly generates a bit string and chooses a random basis (X or Z) for each bit. She then encodes the bits into qubits using the chosen bases and sends them to Bob through a quantum channel.
2. Bob measures the received qubits in randomly chosen bases (X or Z) and records the results.
3. Alice and Bob communicate over a public classical channel to compare their basis choices. They keep only the bits for which their basis choices coincide and discard the rest.
4. Alice and Bob randomly select a subset of the remaining bits and compare their values. If the error rate is below a certain threshold, they conclude that no eavesdropping has occurred, and the remaining bits can be used as a secret key. If the error rate is too high, they abort the protocol, as it indicates the presence of an eavesdropper (Eve).

The security of the BB84 protocol relies on the fact that any attempt by Eve to measure the qubits during transmission will introduce detectable errors, alerting Alice and Bob to the presence of an eavesdropper.

C ADDITIONAL EXPERIMENT RESULTS

In this section, we include detailed analysis of the experiments and additional experiment results. In Section C.1, we introduce the metrics: BLEU score, verification score, and byte perplexity, and provide a detailed analysis for the experiments on BLEU and verification score. In Section C.2, we include additional experiments on perplexity score and temperatures. In Section C.3, we present concrete cases of typical patterns observed in model outputs. In Section C.4, we analyze the data contamination phenomenon revealed by our benchmark.

C.1 METRICS

BLEU Score. Bilingual Evaluation Understudy (BLEU) score is a metric used to evaluate the quality of machine-translated text compared to human-translated text. It measures how close the machine translation is to one or more reference translations. The BLEU score evaluates the quality of text generated by comparing it with one or more reference texts. It does this by calculating the n-gram precision, which means it looks at the overlap of n-grams (contiguous sequences of n words) between the generated text and the reference text. Originally the BLEU score ranges from 0 to 1, where 1 indicates a perfect match with the reference translations. Here rescaling the score makes it ranges from 0 to 100.

The BLEU score, originally designed for machine translation, can also be effectively used for evaluating algorithm generation tasks. Just as BLEU measures the similarity between machine-translated text and human reference translations, it can measure the similarity between a generated algorithm and a gold-standard algorithm. This involves comparing sequences of tokens to assess how closely the generated output matches the reference solution. In the context of algorithm generation, n-grams can represent sequences of tokens or operations in the code. BLEU score captures the precision

of these n-grams, ensuring that the generated code aligns closely with the expected sequences found in the reference implementation.

The formula for BLEU score is given by:

$$\text{BLEU} = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right).$$

where BP is the acronym for brevity penalty, w_n is the weight for the n-gram precision (typically $\frac{1}{N}$ for uniform weights), p_n is the precision for n-grams. BP is calculated as:

$$BP = \begin{cases} 1 & \text{if } c > r \\ e^{1 - \frac{r}{c}} & \text{if } c \leq r \end{cases}.$$

where c is the length of the generated text and r is the length of the reference text. Furthermore, n-gram precision p_n is calculated as:

$$p_n = \frac{\sum_{C \in \text{Candidates}} \sum_{n\text{-gram} \in C} \min(\text{Count}(n\text{-gram in candidate}), \text{Count}(n\text{-gram in references}))}{\sum_{C \in \text{Candidates}} \sum_{n\text{-gram} \in C} \text{Count}(n\text{-gram in candidate})}.$$

This formulation ensures that the BLEU score takes into account both the precision of the generated n-grams and the overall length of the translation, providing a balanced evaluation metric.

In our experiments, the BLEU scores for various quantum algorithm design tasks are illustrated in Figure 3(a). This figure not only displays the average performance of each model but also highlights the differences in performance across individual quantum algorithm tasks. The first notable observation is that the figure clearly demonstrates the varying levels of difficulty among quantum algorithms. For example, models achieve higher BLEU scores on tasks such as Bernstein-Vazirani and Deutsch-Jozsa, whereas they perform significantly worse on tasks like Grover, phase estimation, and quantum Fourier transform. This indicates that the former tasks are considerably easier than the latter ones. Another significant observation is that most models score higher in a five-shot prompt compared to a one-shot prompt, which confirms the large language models' ability to improve performance through contextual learning.

Similar patterns are observed in oracle construction tasks, as illustrated in Figure 3(b). The figure highlights that the Diffusion Operator task is notably more challenging than the Grover oracle construction task. Interestingly, we found that adding more in-context examples actually reduced the performance of the Phi-3-medium-128k-instruct and Mistral-7B-v0.3 models. This decline in performance could be attributed to the significant differences between each oracle construction task, which may be too out-of-distribution. Consequently, the additional examples might cause the models to overfit to the specific examples provided in the context, rather than generalizing well across different tasks.

Detailed Analysis of Verification Score. In addition to evaluating the BLEU score, we conducted an experiment to measure the correctness of the machine-generated algorithms, and the results are shown in Table 1. By running a verification function, we discovered that phase estimation and the swap test are significantly more challenging than other problems, leading most models to score -1 (indicating they cannot even generate the correct syntax). Notably, the BLEU score for the swap test is above average compared to other algorithms, yet almost none of the models produced a correct algorithm. This discrepancy highlights a critical limitation of using BLEU as a metric for algorithm evaluation. BLEU measures average similarity, but even a single mistake in an algorithm can render it entirely incorrect, thus failing to capture the true accuracy and functionality of the generated algorithms. Another important finding is that in a five-shot setting, GPT-4 and GPT-3.5 surpass all other models by a large margin. This demonstrates their exceptional capabilities, particularly in long-context comprehension and in-context learning. These models not only excel in understanding and generating text based on minimal examples but also maintain high performance over extended sequences, highlighting their advanced architecture and training methodologies.

The verification results of the oracle construction task, as shown in Table 2, confirm our previous conclusions. In the five-shot setting, GPT-4 and GPT-3.5 consistently outperform all other models.

Additionally, this table highlights the inconsistency between BLEU scores and verification scores. For instance, while the Diffusion Operator task achieves the lowest BLEU score, it is the Grover oracle construction that receives the lowest verification score. This discrepancy suggests that BLEU scores may not fully capture the performance of models in certain complex tasks, and it is necessary to include verification score as a comprehensive evaluation.

Byte Perplexity. Perplexity is a measure of how well a probability distribution or a probabilistic model predicts a sample. In the context of language models, it quantifies the uncertainty of the model when it comes to predicting the next element in a sequence. Byte perplexity specifically deals with sequences of bytes, which are the raw binary data units used in computer systems. For our purposes, we consider byte perplexity under UTF-8 encoding, a widely used character encoding standard that represents each character as one or more bytes.

For a given language model, let $p(x_i|x_{<i})$ be the probability of the i -th byte x_i given the preceding bytes $x_{<i}$. If we have a sequence of bytes $x = (x_1, x_2, \dots, x_N)$, the perplexity $\text{PPL}(x)$ of the model on this sequence is defined as:

$$\text{PPL}(x) = 2^{-\frac{1}{N} \sum_{i=1}^N \log_2 p(x_i|x_{<i})}.$$

A notable feature of byte perplexity is that, it does not rely on any specific tokenizer, making it versatile for comparing different models. Therefore, byte perplexity can be used to measure the performance in quantum algorithm generation tasks. In such tasks, a lower byte perplexity indicates a better-performing model, as it means the model is more confident in its predictions of the next byte in the sequence.

C.2 ADDITIONAL EXPERIMENTS ON PPL SCORE AND TEMPERATURES

The Byte Perplexity results, shown in Figure 7, provide valuable insights into the performance of our model. Evaluated in a zero-shot setting, byte perplexity trends closely mirror those observed with BLEU scores. This alignment suggests that our model’s predictive capabilities are consistent across Perplexity and BLEU evaluation metrics. Specifically, in the context of quantum algorithm design tasks, the results indicate that the Bernstein-Vazirani and Deutsch-Jozsa algorithms are relatively straightforward for the model, whereas the Simon algorithm presents greater difficulty. This differentiation highlights the varying levels of complexity inherent in these quantum algorithms.

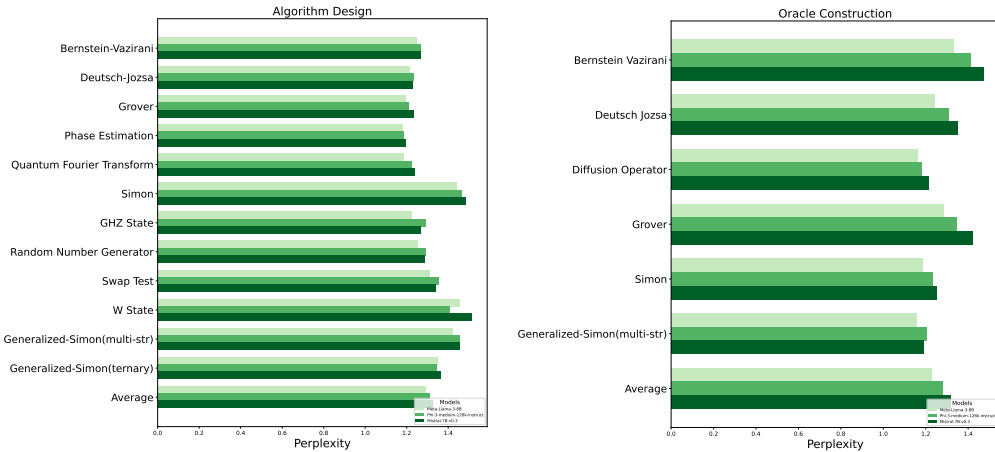


Figure 7: Benchmarking algorithm design and oracle construction in perplexity scores.

Regarding the counter-intuitive phenomenon where the performance on Clifford and universal random circuits decreases after fine-tuning, we conducted additional experiments and fine-tuned the model on 4,800 samples specifically for the Clifford task. Upon closer inspection, we observed that the model more frequently generated outputs with infinite loops and increased monotony, often producing

repetitive gate patterns and repeatedly cycling over the same qubit after fine-tuning. We further conducted experiments with different "temperature" parameters, which control the randomness of predictions. Formally, let $T > 0$ be the temperature, z_i be the raw score for token i , the probability for token i is computed as $p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$. Typically, lower temperatures make the model more conservative, while higher temperatures flatten the distribution, increasing the likelihood of generating originally less probable sequences. The results are shown in Table 4.

Table 4: Clifford Model Fine-Tuning Results Across Different Temperature Settings

Model	Setting	Temperature	BLEU	Verification
Llama3	few-shot(5)	0	13.3796(± 0.9508)	-0.6582(± 0.0360)
		0.2	12.5688(± 0.8276)	-0.6526(± 0.0372)
		1	53.0431(± 3.8422)	-0.1914(± 0.0361)
Llama	finetune	0	7.6261(± 0.3433)	-0.8895(± 0.0247)
		0.2	13.8714(± 0.6536)	-0.7873(± 0.0306)
		1	32.5241(± 2.0548)	-0.2072(± 0.0358)

One possible explanation for this counter-intuitive result lies in the challenge of encoding quantum state vectors within a language model. In the problem description, the target quantum state is represented by a complex vector with four decimal places of precision, where the dimension scales as with the number of qubits. It is a well-known fact that LLMs generally struggle with very long floating-point numbers, which might contribute to the observed performance decline.

Another potential reason could be overfitting during fine-tuning, particularly for tasks that require high output diversity. The varying degrees of intrinsic difficulty and the amount of relevant pre-training knowledge across different tasks likely played a role. Oracle constructions are relatively simple for the model to learn. For example, in the Bernstein-Vazirani algorithm, the model only needs to apply a CNOT gate at positions corresponding to '1' bits. In contrast, the random circuits in the Clifford and Universal tasks involve more general and complex quantum state transformations, making them significantly more challenging. These tasks are also less common during pre-training, which could have hindered the model's ability to generalize without overfitting. This challenge is one of the reasons we initially considered a few-shot learning approach to be suitable.

While these are plausible hypotheses, we acknowledge that further investigation is required to draw definitive conclusions. We consider this an intriguing topic that warrants additional research.

C.3 CASE STUDY

After carefully examining the model's output, we observed several interesting patterns. We present a series of case studies to illustrate these observations and provide possible explanations.

Low Score for GPT-4o in One-Shot Setting. At first glance, it is surprising that GPT-4o performs poorly on many quantum algorithms in the algorithm design task in the one-shot setting compared to Llama3-8B. Given that Llama3-8B has a relatively smaller parameter scale, the results should have been the other way around. A closer examination of the model's output reveals the potential reason: while Llama3-8B closely mimics the input examples, GPT-4o tends to improvise, resulting in outputs that are not well captured by the current syntax support. Here are several concrete examples.

This is the OpenQASM 3.0 code output for the W state with $n = 7$. In this code, GPT-4o uses the advanced "for" loop syntax newly introduced in OpenQASM 3.0 to create the circuit. Although the code fails to produce the W state, it is syntactically correct. However, the Qiskit.qasm3 import module, which converts OpenQASM 3.0 files to QuantumCircuit objects and is used in our verification function to check the correctness of the syntax of output OpenQASM codes, is still in the experimental stage and does not support many of OpenQASM 3.0's advanced features. As a result, GPT-4o's use of these features causes the code to fail syntax validation, resulting in a score of -1.

```
OPENQASM 3.0;
include "stdgates.inc";
qubit[7] q;
h q[0];
```

```
for i in[1:6] {
    cx q[i-1], q[i];
}
```

Listing 8: OpenQASM 3.0 Code output by GPT-4o for W state with $n = 7$.

Here is another example where GPT-4o decides to assign novel names to its qubit registers, leading to a conflict in the symbol table in Scope.GLOBAL. If we substitute all the registers x , y , and s with new names, the code can pass syntax validation successfully and is close to the correct solution.

```
OPENQASM 3.0;
include "stdgates.inc";
include "oracle.inc";
bit[9] s;
qubit[10] x;
qubit[11] y;
h x[0];
h x[1];
h x[2];
h x[3];
h x[4];
h x[5];
h x[6];
h x[7];
h x[8];
Oracle x[0], x[1], x[2], x[3], x[4], x[5], x[6], x[7], x[8], y;
h x[0];
h x[1];
h x[2];
h x[3];
h x[4];
h x[5];
h x[6];
h x[7];
h x[8];
measure x[0] -> s[0];
measure x[1] -> s[1];
measure x[2] -> s[2];
measure x[3] -> s[3];
measure x[4] -> s[4];
measure x[5] -> s[5];
measure x[6] -> s[6];
measure x[7] -> s[7];
measure x[8] -> s[8];
```

Listing 9: OpenQASM 3.0 Code output by GPT-4o for Bernstein-Vazirani Problem with $n = 9$.

Since Llama3-8B tends to follow the provided example more closely, it avoids such kind of errors. These factors may help explain why GPT-4o performs poorly in the one-shot setting compared to Llama3-8B.

High Score for GPT-4o in Five-Shot Setting. In contrast to its relatively low score in the one-shot setting, GPT-4o achieves the best overall performance in the five-shot setting for both oracle construction and algorithm design tasks.

Unlike in the one-shot setting, where GPT-4o frequently uses advanced OpenQASM 3.0 features, it learns from the examples in context and uses simpler syntax in the five-shot setting. For the W state task, here is an output in the five-shot setting:

```
OPENQASM 3.0;
include "stdgates.inc";
qubit[3] q;
h q[0];
cx q[0], q[1];
```

```
h q[1];
cx q[1], q[2];
h q[2];
```

Listing 10: OpenQASM 3.0 Code output by GPT-4o for W state with $n = 3$.

This output avoids the use of "for" loops and successfully passes the syntax validation test, although it still cannot generate the W state correctly.

In addition to adapting to plain syntax through in-context learning, GPT-4o achieves outstanding performance on more complicated tasks such as phase estimation. Here is the model output of GPT-4o on the phase estimation task with qubit number $n = 2$.

```
OPENQASM 3.0;
include "stdgates.inc";
include "oracle.inc";
bit[2] c;
qubit[2] q;
Psi q[0];
h q[1];
CU_0 q[0], q[1];
h q[1];
c[0] = measure q[1];
c[1] = measure q[0];
```

Listing 11: OpenQASM 3.0 Code output by GPT-4o for Phase Estimation with $n = 2$.

```
from Qiskit import transpile

def run_and_analyze(circuit, aer_sim):
    """Run the circuit and analyze the result."""
    circ = transpile(circuit, aer_sim)
    result = aer_sim.run(circ, shots=1).result()
    counts = result.get_counts()
    phase_str = list(counts.keys())[0]
    phase = int(phase_str, 2) / 2**2
    return phase
```

Listing 12: Post-processing code output by GPT-4o for Phase Estimation with $n = 2$.

This suite of OpenQASM 3.0 circuits and post-processing functions successfully outputs the phase within the required precision for the test case, resulting in an impressive verification score of 1.0. Despite the small number of qubits and differences from the reference implementation, the accuracy achieved is noteworthy.

These phenomena reflect that GPT-4o has impressive in-context learning abilities and overall better capabilities in designing and implementing quantum algorithms.

C.4 DATA CONTAMINATION

In our experiments, we observed an interesting phenomenon of performance discrepancy between implementing quantum algorithms with Qiskit codes for the general setting and with OpenQASM codes for a concrete case.

We provided the following prompt:

Prompt for Bernstein-Vazirani Problem with Qiskit

Please write the code with Qiskit implementing Bernstein-Vazirani Algorithm with qubit number $n = 5$.

The model answered with the following text and codes:

Text Answer by GPT4o

Here's a Python code using Qiskit to implement the Bernstein-Vazirani algorithm with $n = 5$ qubits. The secret string used in this example is $s = 10101$ (you can change this to any string of your choice).

```
from qiskit import QuantumCircuit, Aer, transpile, assemble
from qiskit.visualization import plot_histogram

# Secret string s for the Bernstein-Vazirani problem
s = '10101'

# Number of qubits
n = len(s)

# Create a Quantum Circuit with n+1 qubits (one ancillary qubit)
bv_circuit = QuantumCircuit(n+1, n)

# Apply Hadamard gates to all qubits except the ancillary qubit
bv_circuit.h(range(n))

# Apply X and Hadamard gate to the ancillary qubit
bv_circuit.x(n)
bv_circuit.h(n)

# Construct the Oracle
for i, bit in enumerate(s):
    if bit == '1':
        bv_circuit.cx(i, n)

# Apply Hadamard gates to the first n qubits
bv_circuit.h(range(n))
# Measure the first n qubits
bv_circuit.measure(range(n), range(n))
# Visualize the circuit
bv_circuit.draw('mpl')

# Run the circuit using Qiskit Aer simulator
simulator = Aer.get_backend('qasm_simulator')
t_qc = transpile(bv_circuit, simulator)
qobj = assemble(t_qc)
result = simulator.run(qobj).result()

# Get the result counts and plot the histogram
counts = result.get_counts()
plot_histogram(counts)
```

Listing 13: Qiskit code for Bernstein-Vazirani Problem generated by GPT4o in the zero-shot setting.

This code, although adopts an older version of Qiskit syntax, can be compiled and run successfully with the correct answer. At first glance, this indicates that GPT4o has fully grasped the mechanism of the Bernstein-Vazirani Algorithm and the syntax of Qiskit. Similar performances can be observed with OpenQASM codes:

```
// OpenQASM 2.0
OPENQASM 2.0;
include "qelib1.inc";

// Declare 5 qubits for input and 1 ancilla qubit
qreg q[6];
creg c[5]; // classical register for measuring the result

// Step 1: Apply Hadamard gates to the input qubits
h q[0];
```

```

h q[1];
h q[2];
h q[3];
h q[4];

// Apply Hadamard to the ancillary qubit and set it to |1>
x q[5];
h q[5];

// Step 2: Oracle implementation (CNOT gates based on secret string s =
// ↪ 10110)
cx q[0], q[5]; // Secret string bit 1
// No cx for q[1] because the secret string bit is 0
cx q[2], q[5]; // Secret string bit 1
cx q[3], q[5]; // Secret string bit 1
// No cx for q[4] because the secret string bit is 0

// Step 3: Apply Hadamard gates to the input qubits again
h q[0];
h q[1];
h q[2];
h q[3];
h q[4];

// Step 4: Measure the qubits to get the result
measure q[0] -> c[0];
measure q[1] -> c[1];
measure q[2] -> c[2];
measure q[3] -> c[3];
measure q[4] -> c[4];

```

Listing 14: OpenQASM code for Bernstein-Vazirani Problem generated by GPT4o.

However, if we ask the GPT4o model to implement the code for a given secret string, for example $s = 000101$, the model fails to apply the CX gate on the correct qubits.

```

OPENQASM 3.0;
include "stdgates.inc";

gate Oracle _gate_q_0, _gate_q_1, _gate_q_2, _gate_q_3, _gate_q_4,
    ↪ _gate_q_5, _gate_q_6 {
    // Apply CX gates according to the secret string "000101"
    cx _gate_q_2, _gate_q_6;
    cx _gate_q_5, _gate_q_6;
}

bit[6] c;
qubit[7] q;

// Apply Hadamard gates to the first 6 qubits
h q[0];
h q[1];
h q[2];
h q[3];
h q[4];
h q[5];

// Prepare the ancilla qubit in the |1> state and apply a Hadamard gate
x q[6];
h q[6];

// Call the oracle
Oracle q[0], q[1], q[2], q[3], q[4], q[5], q[6];

// Apply Hadamard gates to the first 6 qubits again

```

```

h q[0];
h q[1];
h q[2];
h q[3];
h q[4];
h q[5];

// Measure the first 6 qubits
c[0] = measure q[0];
c[1] = measure q[1];
c[2] = measure q[2];
c[3] = measure q[3];
c[4] = measure q[4];
c[5] = measure q[5];

```

Listing 15: OpenQASM code for Bernstein-Vazirani Problem with $s = 000101$ in few-shot setting.

We reminded the model of the mechanism with the following prompt:

Reminder Prompt

Remember that in the construction of oracles for Bernstein-Vazirani Problem, you should apply cx gates where the control qubits are where the string bit equals 1, and the target qubit is the ancillary qubit.

There was no improvement. We further asked the model about the indices of the bits in the string which equal 1. It provided the following answer:

GPT4o's answer for counting indices

In the secret string $s="000101"$, the bits that equal 1 are found at the following indices (0-based):
 Index 2
 Index 5
 These indices correspond to the positions where the string has a '1'.

This reveals the incapability for GPT models to count the bit of 1 and apply corresponding gates to construct the oracle. Considering that Qiskit has implemented tutorial codes for several representative quantum algorithms, the seemingly outstanding performance of quantum algorithm code generation might be a deceptive result caused by data contamination. Therefore, only a carefully designed quantum algorithm dataset can avoid the effects of data contamination, allowing for an effective evaluation of the model's genuine capability in quantum algorithm design and implementation. This dataset is also meaningful for testing general AI code generation and syntax learning, where no existing AI dataset could substitute us.

Clifford Circuit Optimization with Templates and Symbolic Pauli Gates

Sergey Bravyi¹, Ruslan Shaydulin², Shaohan Hu³, and Dmitri Maslov¹

¹IBM Quantum, IBM Thomas J. Watson Research Center, Yorktown Heights, NY 10598

²Mathematics and Computer Science Division, Argonne National Laboratory, Lemont, IL 60439

³JPMorgan Chase & Co., New York, NY 10017

The Clifford group is a finite subgroup of the unitary group generated by the Hadamard, the CNOT, and the Phase gates. This group plays a prominent role in quantum error correction, randomized benchmarking protocols, and the study of entanglement. Here we consider the problem of finding a short quantum circuit implementing a given Clifford group element. Our methods aim to minimize the entangling gate count assuming all-to-all qubit connectivity. First, we consider circuit optimization based on template matching and design Clifford-specific templates that leverage the ability to factor out Pauli and SWAP gates. Second, we introduce a symbolic peephole optimization method. It works by projecting the full circuit onto a small subset of qubits and optimally recompiling the projected subcircuit via dynamic programming. CNOT gates coupling the chosen subset of qubits with the remaining qubits are expressed using symbolic Pauli gates. Software implementation of these methods finds circuits that are only 0.2% away from optimal for 6 qubits and reduces the two-qubit gate count in circuits with up to 64 qubits by 64.7% on average, compared to the Aaronson–Gottesman canonical form [3].

1 Introduction

One of the central challenges in quantum computation is the problem of generating a short schedule of physically implementable quantum gates realizing a given unitary operation, otherwise known as the quantum circuit synthesis/optimization problem. In this paper, we focus on a restricted class of quantum circuits belonging to the Clifford group, which is a subgroup of the group of all unitary transformations. Clifford group elements play a crucial role in quantum error correction [25], quantum state distillation [6, 20], randomized benchmarking [21, 23], study of entanglement [5, 25], and, more recently, shadow tomography [2, 16], to name some application areas. Clifford group elements are important and frequently encountered subsets of physical-level and fault-tolerant quantum circuits; sometimes, an entire quantum algorithm can be a Clifford circuit (e.g., Bernstein–Vazirani [25] and its generalizations [9]).

A special property of the Clifford group that plays the central role in many applications is being a unitary 2-design [11, 12]. It guarantees that a random uniformly distributed element of the Clifford group has exactly the same second order moments as the Haar random unitary operator. Thus random Clifford operators can serve as a substitute for Haar random unitaries in any application that depends only on the second order moments. However, in contrast to Haar random unitaries, any Clifford operators admit an efficient implementation by a quantum circuit. For example, randomized benchmarking [21, 23] provides a scalable fidelity metric for multi-qubit operations which is insensitive to the state preparation and measurement errors. Randomized benchmarking works by measuring the decay rate of a signal generated by a sequence of random Clifford operators of varying length. The 2-design property ensures that the effective noise model obtained after averaging over the Clifford group is the depolarizing channel with a single unknown noise

parameter. As another example, classical shadows [16] provide a succinct classical description of a multi-qubit quantum state that can be efficiently measured in an experiment without performing the full state tomography. At the same time, a classical shadow determines many physically relevant properties of a state such as expected values of observables. A classical shadow of a quantum state ρ is obtained by repeatedly preparing a state $U\rho U^\dagger$ with a random Clifford operator U and measuring each qubit in the computational basis. The ability to realize a random element of the Clifford group by a short quantum circuit plays the central role in the above examples.

Clifford circuits also serve as a basis change transformation in quantum simulation algorithms. For example, simultaneous diagonalization of mutually commuting Pauli operators by a Clifford basis change can reduce the circuit depth for simulating quantum chemistry Hamiltonians [30]. Another example is tapering off qubits for quantum simulations by identifying Pauli-type symmetries of quantum chemistry Hamiltonians [8, 28]. Such symmetry operators can be mapped to single-qubit Pauli Z by applying a suitable Clifford circuit after which the respective qubits can be removed from the simulation.

Earlier studies of the synthesis of n -qubit Clifford circuits resulted in the construction of asymptotically optimal (i.e., optimal up to a constant factor) implementations in the number of gates used. Specifically, the canonical form introduced by Aaronson and Gottesman [3] accomplishes this using $\Theta(n^2/\log(n))$ gates [26]. In contrast, in this paper we focus on the practical aspects of Clifford circuit optimization—our goal is to implement a given Clifford unitary by a circuit with the smallest possible number of entangling gates. We focus on the minimization of the CNOT gate count, drawing motivation from physical layer realizations where entangling gates come at a higher cost than the single-qubit gates, and ignore the connectivity constraints. While in the worst-case scenario ignoring connectivity may lead to an $O(n)$ blowup in the CNOT gate count or depth (consider the cost of implementation of the maximal-distance $\text{CNOT}(x_1; x_n)$ gate in a linear chain with n qubits), known difference between the upper bound on the circuit depth between all-to-all and Linear Nearest Neighbor (LNN) architectures remains small. Indeed, for all-to-all architecture the best known upper bound on the two-qubit gate depth is $\frac{10}{3}n + O(\log(n))$ (obtained by combining Lemma 8 in [7] with Corollary III.2.2 in [15], and noting that CZ gate layer can be implemented in depth $(n-1)$ or n depending on whether n is even or odd), and the best-known lower bound is $\Omega\left(\frac{n}{\log(n)}\right)$ (obtained by a slight modification of the counting argument employed in [26]). In the LNN, upper bound is $9n$ [7], and lower bound is $2n+1$ [22]. The above suggests that executing a (random) Clifford circuit in restricted architectures (LNN may often be embedded in other architectures) comes with a relatively small overhead. We also note that our methods and algorithms can be straightforwardly modified to respect a restricted connectivity and target depth minimization rather than gate count minimization.

Current approaches to the synthesis of exactly optimal Clifford circuits are prohibitively expensive even for small parameters: the largest number of qubits for which optimal Clifford circuits are known is six [10]. Using these exhaustive tools leaves little hope of scaling optimal implementations beyond six qubits. Thus, efficient heuristics are desirable for practical applicability. Here we focus on the synthesis and optimization of Clifford circuits that cannot be obtained optimally, namely, circuits with $n > 6$ qubits.

Here we develop heuristic approaches for the synthesis and optimization of Clifford circuits. Our algorithms and their implementation bridge the gap between nonscalable methods for the synthesis of exactly optimal Clifford circuits and the suboptimal (albeit asymptotically optimal) synthesis methods. Our circuit synthesizer is based on the reduction of the tableau matrix representing Clifford unitary to the identity, while applying gates on both the input and output sides. Our optimization approach is based on the extension and modification of two circuit optimization techniques: template matching [24] and peephole optimization [27]. To generate an optimized circuit for a specific Clifford unitary, we first compile it using the tableau representation and then apply the optimization techniques to the compiled circuit. We note that the optimization techniques can be applied independently of the synthesizer considered in this paper.

The first optimization technique we develop is a Clifford-specific extension of the template matching method [24]. We discuss previous results on template matching in depth in [Subsection 2.1](#). We introduce a three-stage approach that leverages the observation that in Clifford circuits Pauli gates can always be “pushed” to the end of the circuit without changing the non-Pauli Clifford gates (i.e., Hadamard, controlled-

NOT, and Phase gates) and that all SWAP gates can be factored out of any quantum circuit by qubit relabeling. We thus partition the circuit into “compute,” “SWAP,” and “Pauli” stages by “pushing” Pauli and SWAP gates to the end of the circuit. Next we optimize the “compute” stage using templates. Then we optimize the “SWAP” stage by exploiting the fact that a SWAP gate can be implemented at the effective cost of one entangling gate if it can be merged with a CNOT or a CZ gate.

The second technique we develop is symbolic peephole optimization. It is inspired by the peephole optimization method first introduced in the context of reversible computations [27]. At each step, the symbolic peephole algorithm considers subcircuits spanning a small set of qubits (2 and 3 in this paper) by introducing symbolic Pauli gates (SPGs) to replace the two-qubit gates that entangle qubits in the chosen set with a qubit outside of it. The resulting Clifford+SPG subcircuit is optimized via dynamic programming using a library of optimal circuits.

We numerically evaluate the proposed methods using two sets of benchmarks. The first benchmark is based on the database of optimal Clifford circuits constructed in [10]. We consider a selection of 1,003 randomly sampled 6-qubit Clifford unitaries, conditional on the optimal CNOT gate implementation cost being higher than 4 (otherwise, it is easy to implement such a unitary optimally). The set of tools developed in this work is able to recover an optimal (in terms of the CNOT count) implementation for 97.9% of the circuits, while producing circuits no more than one CNOT away from the optimal count in the worst case. Second, to evaluate the performance on “large” circuits, we consider a toy model of Hamiltonian evolution with a graph state Hamiltonian, defined as follows. For a given graph with n nodes, the Hamiltonian evolution performs the transformation $(\text{CZ} \cdot \text{H})^t$, where CZ gates apply to graph edges, H gates apply to graph nodes (individual qubits), and t is the evolution time. At integer times, the evolution by such a Hamiltonian is described by a Clifford unitary. Implementing it as a circuit $\text{CZ} \cdot \text{H}$ repeated t times turns out to be less efficient than implementing it by using the techniques reported here. The methods we developed are evaluated on a collection of 2,264 circuits and shown to reduce the average CNOT gate count by 64.7% compared with the methods proposed by Aaronson and Gottesman in [3]. We make the full benchmark and the raw results available online [1].

The rest of the paper is organized as follows. We begin by briefly revisiting relevant concepts and defining the notations (Section 2). We next discuss previous results that our work is based on (Subsection 2.1, Subsection 2.2). Following this discussion, we describe the proposed methods (Section 3), report numerical results, and evaluate the performance (Section 4). We conclude with a short summary (Section 5).

2 Background

We assume basic familiarity with quantum computing concepts, stabilizer formalism, and Clifford circuits. Below we briefly introduce relevant concepts and notations. For detailed discussion, the reader is referred to [25] and [3].

Clifford circuits (also known as stabilizer circuits) consist of Hadamard (H), Phase (S, also known as P gate), and controlled-NOT (CNOT) gates, as well as Pauli X, Y, and Z gates. We use I to denote the identity gate/matrix. We also utilize the controlled-Z (CZ) gate, which can be constructed as a circuit with Hadamard and CNOT gates as follows,

$$\begin{array}{c} \bullet \\ | \\ \bullet \end{array} = \begin{array}{c} \bullet \\ | \\ \boxed{\text{H}} \oplus \boxed{\text{H}} \end{array}. \quad (1)$$

Clifford circuits acting on n qubits generate a finite group $\mathcal{C}_n$, known as the Clifford group. An important property of Clifford circuits is that Clifford gates H, S, and CNOT map tensor product of Pauli matrices into tensor products of Pauli matrices. This property can be employed to “push” Pauli gates through the Clifford gates H, S, and CNOT as follows:

$$HX = ZH, \quad HY = -YH, \quad HZ = XH, \quad SX = YS, \quad (2)$$

$$SY = -XS, \quad SZ = ZS, \quad (3)$$

$$\text{CNOT}_{1,2}X_1 = X_1X_2\text{CNOT}_{1,2}, \quad \text{CNOT}_{1,2}X_2 = X_2\text{CNOT}_{1,2}, \quad (4)$$

$$\text{CNOT}_{1,2}Z_2 = Z_1Z_2\text{CNOT}_{1,2}, \quad \text{CNOT}_{1,2}Z_1 = Z_1\text{CNOT}_{1,2}. \quad (5)$$

Our approach combines two building blocks: Clifford-specific extension of template matching and symbolic peephole optimization. Below we briefly review these techniques. While the developed methods reduce both single- and two-qubit gate count, in this paper we focus on the optimization of the number of two-qubit gates it takes to implement a Clifford group element. The reason for our focus is that the leading quantum information processing technologies, trapped ions [13] and superconducting circuits [17], both feature two-qubit gates that take longer time and have higher error rates compared with those of single-qubit gates.

2.1 Template Matching

A size m template [24] is a sequence of m gates that implements the identity function:

$$T = G_0G_1 \dots G_{m-1} = I.$$

The templates can be used to optimize a target circuit as follows. First, a subcircuit $G_iG_{i+1 \pmod m} \dots G_{i+p-1 \pmod m}$ of the template is matched with a subcircuit in the given circuit. If the gates in the target circuit can be moved together, this sequence of gates can be replaced with the inverse of the other $m-p$ gates in the template. The larger the length p of the matched sequence is, the more beneficial it is to perform the replacement, and for any $p > \frac{m}{2}$ the gate count is reduced. The exact criteria for the application of the template depends on the choice of the objective optimization criteria (e.g., depth, total gate count, 2-qubit gate count). More formally, for parameter p , $\frac{m}{2} \leq p \leq m$, the template T can be applied in two directions as follows,

$$\textbf{Forward: } G_iG_{i+1 \pmod m} \dots G_{i+p-1 \pmod m} \rightarrow G_{i-1 \pmod m}^\dagger G_{i-2 \pmod m}^\dagger \dots G_{i+p \pmod m}^\dagger, \quad (6)$$

$$\textbf{Backward: } G_i^\dagger G_{i-1 \pmod m}^\dagger \dots G_{i-p+1 \pmod m}^\dagger \rightarrow G_{i+1 \pmod m}^\dagger G_{i+2 \pmod m}^\dagger \dots G_{i-p \pmod m}^\dagger.$$

Any template T of size m should be independent of smaller templates; that is, an application of a smaller template should not decrease the number of gates in T or make it equal to another template. Circuit optimization using template matching is an iterative procedure where at each step we start at an index gate and attempt to match a given template by considering gates left to the index gate in the target circuit. If the matched gates can be moved together and the substitution is beneficial, the template is applied as defined above. This step is repeated by incrementing the position of the index gate by one when no match is found until the last gate is reached.

Circuit optimization with templates was originally proposed in [24]. This work has been extended with the introduction of graph-based matching techniques [18]. While the methods in these references are applicable to Clifford circuits since they are defined for universal quantum circuits, neither of them leverages the particular structure Clifford circuits have for optimization. After completion of the present work we became aware that template-based optimization techniques have been recently applied to Clifford circuits in [29].

2.2 Peephole Optimization of Quantum Circuits

Peephole optimization [27] is an iterative local optimization technique that optimizes a circuit by considering subcircuits spanning small subsets of qubits A and attempting to replace them with an optimized version drawn from a database (or synthesized on the spot in some other versions). At each step, for a given gate all subcircuits on a fixed small number of qubits (e.g., $|A|=4$ in [19]) including that gate are considered. For each subcircuit, its cost and the optimal cost (retrieved from the database of precomputed optimal circuits)

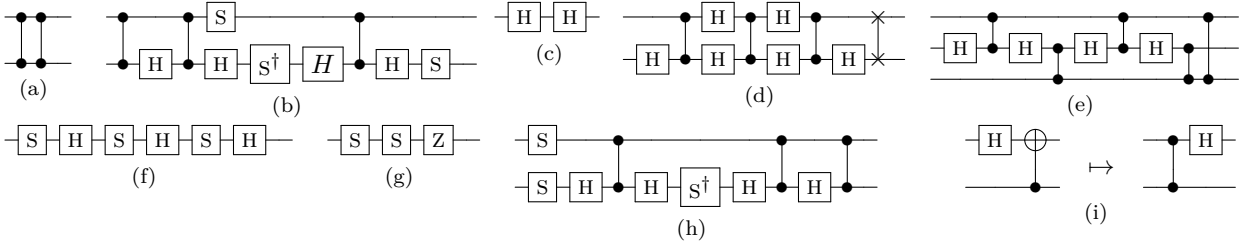


Figure 1: Templates (a)–(h) are used for template matching. The rewriting rule (i) is used for Hadamard gate pushing.

of the unitary it implements are compared. If a substitution is beneficial, the given subcircuit is replaced with its optimal implementation. The step is repeated for all gates until a convergence criterion is satisfied. Peephole optimization of reversible circuits was introduced in [27] and identified to be complementary to template matching. Since its introduction in the context of reversible computations, this approach has been applied to Clifford circuits [19].

The performance of the standard peephole optimization is limited by the need to store the entire database of optimal circuits in memory and to perform $O\left(\binom{n-2}{|A|-2}g^3\right)$ lookups, where g is the number of gates in the circuit [27]. Furthermore, since the size of the n -qubit Clifford group (inclusive of the Pauli group) equals $2^{2n+n^2} \prod_{j=1}^n (2^{2j}-1)$ and grows very quickly with n , it is unlikely that all optimal circuits can be found and stored in a suitable database for more than 6 qubits [10].

3 Algorithms

We introduce two algorithms for Clifford circuit optimization and apply them to the problem of compiling optimized Clifford circuits. The first algorithm is a Clifford-specific extension of the template matching technique, which we describe in Subsection 3.2. The second algorithm is symbolic peephole optimization, detailed in Subsection 3.3.

These optimizations can be applied in at least the following two ways. First, if the input is a Clifford unitary, we begin by synthesizing a circuit using a “greedy” compiler (described in Subsection 3.1) and then reduce the gate count by our proposed circuit optimization techniques. Second, if the input is already a Clifford circuit, we can either resynthesize it or apply the circuit optimizations directly. The gate count in the final circuit can be further decreased at the cost of increasing the runtime by a constant factor if the circuit is resynthesized k times using a randomized version of the “greedy” compiler, the k circuits are optimized individually, and the best of the k results is picked. Note that the k repetitions can be done in parallel.

3.1 “Greedy” Compiler

Suppose $U \in \mathcal{C}_n$ is a Clifford unitary to be compiled and $L \in \mathcal{C}_n$ is an operator that reproduces the action of U on a single pair of Pauli operators, x_j and z_j . In other words, $UPU^{-1} = LPL^{-1}$ for $P \in \{x_j, z_j\}$. The requisite operator L , as well as a Clifford circuit with $O(n)$ CNOTs implementing L , can be easily constructed for any given qubit j by using the standard stabilizer formalism [3]. Then the operator $L^{-1}U$ acts trivially on the j th qubit and can be considered as an element of the Clifford group $\mathcal{C}_{n-1}$. The greedy compiler applies this operation recursively such that each step reduces the number of qubits by one. A qubit j , picked at each recursion step, is chosen such that the operator L has the minimum CNOT count. In the randomized version of the algorithm, qubit j is picked randomly. The compiler runs in time $O(n^3)$ and outputs a circuit with the CNOT count at most $3n^2/4 + O(n)$. We also developed and employ a bidirectional version of the greedy

compiler that follows the same strategy as above except that each recursion step applies a transformation $U \leftarrow L^{-1}UR^{-1}$, where $L, R \in \mathcal{C}_n$ are chosen such that after the transformation U acts trivially on the j th qubit and the combined CNOT count of L and R is minimized. In [Section 4](#), we use the bidirectional version of the greedy compiler as it leads to lower CNOT costs of optimized circuits. We include a detailed description of the greedy compilers in [Appendix A](#).

3.2 Template Matching for Clifford Circuits

We extend template matching, described in [Subsection 2.1](#), by introducing a three-stage approach that takes advantage of the observation that Clifford gates map tensor products of Pauli matrices into tensor products of Pauli matrices. Below we describe the features used in the proposed three-stage approach. In [Subsection 3.4](#), we combine this approach with symbolic peephole optimization.

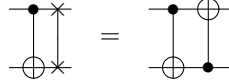
First, we partition the circuit into three stages, “compute,” “SWAP,” and “Pauli”, by pushing SWAP and Pauli gates to the end of the circuit. Paulis are “pushed” according to the rules in [Eqs. \(2, 3, 4, 5\)](#). This step results in the construction of the “compute” stage consisting of H, S, CNOT, and CZ gates only.

Second, we apply the template matching to the “compute” stage. We further simplify template matching by converting all two-qubit gates into CZ gates (at the cost of introducing two Hadamard gates when the CNOT is considered) before performing template optimization. Templates are applied as described in [Subsection 2.1](#). The list of templates is given in [Fig. 1a-1h](#).

We reduce the single-qubit gate count and increase the opportunities for template application by introducing Hadamard and Phase gate pushing. Specifically, assuming that a circuit was optimized with templates, the idea is then to “push” Hadamard and Phase gates to one side of the two-qubit gates as far as possible. “Pushing” a gate through a two-qubit gate is implemented as the application of a template where a fixed subsequence must be matched. For example, the rule in [Fig. 1i](#) can be used to push a Hadamard to the right of the CNOT gate.

Note that once the circuit is optimized in terms of the two-qubit gate count, template matching can be applied to reduce the single-qubit gate count by restricting the set of templates and how they are applied. This can be accomplished by applying templates spanning a single qubit and considering certain applications of templates with an even number of two-qubit gates.

Third, we consider SWAP gate optimization as a separate problem. SWAP optimization is performed by observing that a SWAP gate can be implemented at the effective cost of one two-qubit gate if it is aligned with a two-qubit gate (CNOT or CZ) as, for example, in the following.



In order to reduce the number of SWAPs, the SWAP stage is resynthesized with the goal of aligning as many SWAPs as possible with the two-qubit gates in the “compute” stage.

3.3 Symbolic Peephole Optimization

As outlined in [Subsection 2.2](#), various methods were proposed to create a database of optimal few-qubit Clifford circuits; some employ such databases to perform peephole optimization of larger Clifford circuits. However, these methods are limited to few-qubit subcircuits that must be completely decoupled from the remaining qubits. To address this limitation, we introduce a modified approach to Clifford circuit optimization, *symbolic* peephole optimization.

Consider a circuit $U \in \mathcal{C}_n$ and a small subset of qubits $A \subseteq [n]$. Our goal is to meaningfully define and optimize the restriction of U onto A . Let $B = [n] \setminus A$ be the complement of A . We say that a CNOT gate is *entangling* if it couples A and B . Assume without loss of generality that each entangling CNOT has its target qubit in the set A (otherwise, switch the control and the target by adding extra Hadamards). Partition

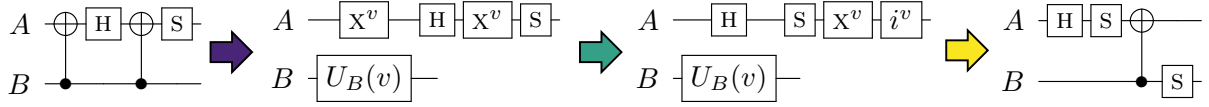


Figure 2: Example of symbolic peephole optimization. *Purple arrow*: each entangling CNOT gate is replaced with a symbolic Pauli gate (SPG) X^v , where $v \in \{0, 1\}$. Now the subcircuit acting on A is isolated from the remaining qubits. *Green arrow*: The subcircuit acting on A is optimized to reduce the number of SPGs. Here we used the commutation rules $HX^v = Z^v H$, $X^v Z^v = (-iY)^v$, and $SX^v = -X^v S$. *Yellow arrow*: The subcircuits acting on A and B are merged by replacing the SPG X^v with the CNOT. The phase factor i^v is replaced with the phase gate S acting on B .

entangling CNOTs into groups such that all CNOTs in the same group have the same control bit. Let k be the number of groups. Expanding each entangling CNOT as $|0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes X$, one can write

$$U = \sum_{v \in \{0,1\}^k} U_A(v) \otimes U_B(v),$$

where $U_A(v)$ is a Clifford circuit obtained from U by retaining all gates acting on A and replacing each entangling CNOT from the i th group with the Pauli gate X^{v_i} acting on the target qubit of the respective CNOT. Likewise, $U_B(v)$ is a (nonunitary) circuit obtained from U by retaining all gates acting on B and replacing each entangling CNOT from the i th group with the projector $|v_i\rangle\langle v_i|$ acting on the control qubit of the respective CNOT. We refer to the single-qubit gates X^{v_i} , Y^{v_i} , and Z^{v_i} as *Symbolic Pauli Gates (SPGs)*. These are similar to controlled Pauli gates except that the control qubit is replaced by a symbolic variable $v_i \in \{0, 1\}$.

A symbolic Clifford circuit $U_A(v)$ can be optimized as a regular Clifford circuit on $|A|$ qubits with the following caveats. First, $U_A(v)$ must be expressed by using the Clifford+SPG gate set. The cost of $U_A(v)$ should be defined as the number of CNOTs plus the number of SPGs. Second, the optimization must respect the temporal order of SPGs. In other words, if $i < j$, then all SPGs controlled by v_i must be applied before SPGs controlled by v_j . Third, the optimization must preserve the overall phase of $U_A(v)$ modulo phase factors $(-1)^{v_j}$ or i^{v_j} . The phase factors can be generated by single-qubit gates Z or S applied to control qubits of the entangling CNOTs. These conditions guarantee that the optimized circuit $U_A(v)$ can be lifted to a full circuit $U' \in \mathcal{C}_n$ that is functionally equivalent to U . A toy optimization example is shown in Fig. 2.

We now describe the optimization of $U_A(v)$ in more detail. Let $\mathcal{P}_A$ and $\mathcal{C}_A$ be the groups of Pauli and Clifford operators acting on A , respectively. The circuit $U_A(v)$ can be compactly specified by a k -tuple of Pauli operators $P_1, P_2, \dots, P_k \in \mathcal{P}_A$ and a Clifford operator $R \in \mathcal{C}_A$ such that $U_A(v) = P_k^{v_k} \dots P_2^{v_2} P_1^{v_1} R$ for all $v \in \{0, 1\}^k$. Indeed, any SPG can be commuted to the left since Clifford gates map Pauli operators to Pauli operators. The most general Clifford+SPG circuit that implements $U_A(v)$ can be parameterized as

$$U_A(v) = (U_k^{-1} Q_k^{v_k} U_k) \dots (U_2^{-1} Q_2^{v_2} U_2) (U_1^{-1} Q_1^{v_1} U_1) R \quad (7)$$

for some Clifford operators $U_j \in \mathcal{C}_A$ and Pauli operators $Q_j = U_j P_j U_j^{-1} \in \mathcal{P}_A$. The cost of the circuit in Eq. 7 includes the CNOT count of subcircuits $U_j U_{j-1}^{-1}$ and the SPG count of controlled Pauli operators $Q_j^{v_j}$. Note that $Q_j^{v_j}$ is a product of $|Q_j|$ single-qubit SPGs, where $|Q_j|$ is the Hamming weight of Q_j . Denoting $U_0 := R^{-1}$, one can express the cost of the circuit in Eq. (7) as

$$f = \$(U_k) + \sum_{i=1}^k \$(U_i U_{i-1}^{-1}) + \sum_{i=1}^k |U_i P_i U_i^{-1}|. \quad (8)$$

Here $\$(V)$ is the CNOT cost of a Clifford operator $V \in \mathcal{C}_A$. Our goal is to minimize the cost function f over all k -tuples $U_1, U_2, \dots, U_k \in \mathcal{C}_A$. We claim that the global minimum of f can be computed in time $O(k)$, as long as $|A| = O(1)$. The key observation is that the function f is a sum of terms that

depend on at most two consecutive variables U_i and U_{i-1} . Such functions can be minimized efficiently using the dynamic programming method; see, for example, [4]. Indeed, define intermediate cost functions $f_1, f_2, \dots, f_k : \mathcal{C}_A \rightarrow \mathbb{Z}_+$ such that f_j is obtained from f by removing the term $\$(U_k)$, retaining the first j terms in the sums over i , and taking the minimum over $U_1, U_2, \dots, U_{j-1}$. More formally,

$$f_1(U_1) = \$(U_1 R) + |U_1 P_1 U_1^{-1}| \quad (9)$$

and

$$f_j(U_j) = \min_{U_1, U_2, \dots, U_{j-1} \in \mathcal{C}_A} \sum_{i=1}^j \$(U_i U_{i-1}^{-1}) + \sum_{i=1}^j |U_i P_i U_i^{-1}|$$

for $j = 2, 3, \dots, k$. Using the induction in j , one can easily check that

$$f_j(U_j) = |U_j P_j U_j^{-1}| + \min_{U_{j-1} \in \mathcal{C}_A} f_{j-1}(U_{j-1}) + \$(U_j U_{j-1}^{-1}) \quad (10)$$

for $j = 2, 3, \dots, k$. Below we assume that a lookup table specifying the CNOT cost $\$(V)$ for all $V \in \mathcal{C}_A$ is available. Then one can compute a lookup table of f_1 by iterating over all $U_1 \in \mathcal{C}_A$ and evaluating the right-hand side of Eq. (9). Proceeding inductively, one can compute a lookup table of f_j with $j = 2, 3, \dots, k$ by iterating over all $U_j \in \mathcal{C}_A$ and evaluating the right-hand side of Eq. (10). Each step takes time roughly $|\mathcal{C}_A|^2 = O(1)$ since we assumed that $|A| = O(1)$. Finally, use the identity

$$\min_{U_1, U_2, \dots, U_k \in \mathcal{C}_A} f(U_1, U_2, \dots, U_k) = \min_{U_k \in \mathcal{C}_A} \$(U_k) + f_k(U_k) \quad (11)$$

to compute the global minimum of f . Thus, the full computation takes time $O(k)$.

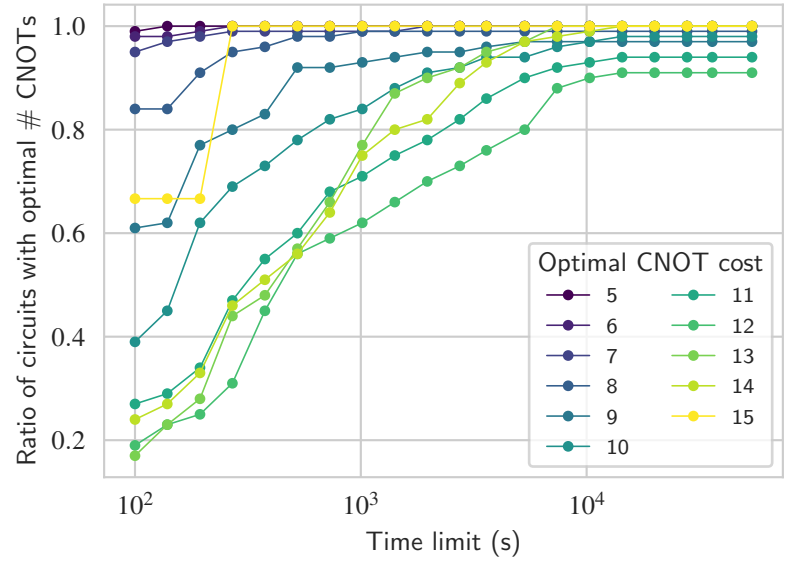
To make the above algorithm more practical, we exploited symmetries of the cost function, Eq. (8). Namely, function f is invariant under multiplying U_j on the left by any element of the local subgroup $\mathcal{C}_A^0 \subseteq \mathcal{C}_A$ generated by the single-qubit gates H_a and S_a with $a \in A$. In other words, $f(U_1, U_2, \dots, U_k)$ depends only on the right cosets of the local subgroup $\mathcal{C}_A^0 U_j$. Thus one can restrict the minimizations in Eqs. (10,11) to some fixed set of coset representatives $\mathcal{R} \subset \mathcal{C}_A$ such that each coset $\mathcal{C}_A^0 V$ has a unique representative $r(V) \in \mathcal{R}$. We chose $r(V)$ as the left-reduced form of V defined in [10, Lemma 2]. This lemma provides an algorithm for computing $r(V)$ with the runtime $O(|A|^2)$. Now each variable U_i takes only $|\mathcal{R}| = |\mathcal{C}_A|/|\mathcal{C}_A^0| = |\mathcal{C}_A|/24^{|A|}$ values. For example, $|\mathcal{R}|=20$ and $|\mathcal{R}|=6720$ for $|A|=2$ and $|A|=3$, respectively. Likewise, it suffices to compute the lookup table for the CNOT cost $\$(V)$ only for $V \in \mathcal{R}$. This computation was performed using the breadth-first search on the Clifford group $\mathcal{C}_A$.

An important open question concerns the selection of the subsets A to be considered. From numerical experiments with $|A| \in \{2, 3\}$, our most successful strategy turned out to be the random subset selection. Specifically, we generate a list of all $\binom{n}{2}$ pairs and $\binom{n}{3}$ triples of qubits. We run passes of the symbolic peephole method first on pairs of qubits and next on triples of qubits until no further improvement can be obtained. At each pass of the symbolic peephole optimization, we randomly reshuffle both lists and run optimization on all the subsets in the reshuffled order. We continue passes until either the optimal CNOT count is reached (for circuits for which the optimal CNOT count is known) or there is no improvement between two consecutive passes.

3.4 Full Algorithm

We combine the components described above in the following way. We begin by synthesizing the circuit using the “greedy” compiler described in Subsection 3.1. Then the synthesized circuit is optimized as follows. First, the circuit is partitioned into three stages. Second, template matching and SWAP gate merging is performed until a pass yields no further optimization. Third, symbolic peephole optimization is performed, as described in Subsection 3.3. Lastly, a single pass of template matching is performed to reduce the single-qubit gate count.

(a) The ratio of circuits for which the implemented methods recover optimal CNOT count. The “nonsmoothness” of the line for the CNOT count of 15 is due to only 3 circuits being considered.



(b) Mean running time of the implemented methods. Note that the actual running times are significantly lower than the time limit (black line) for most instances.

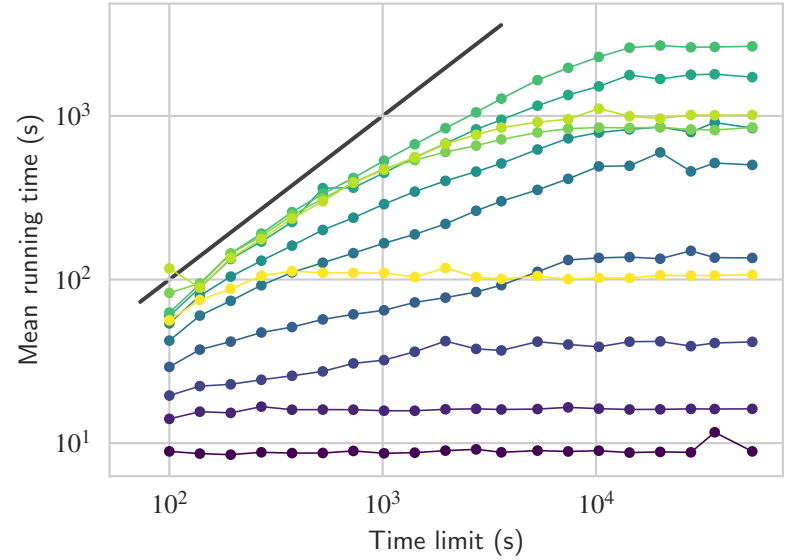


Figure 3: Quality of the solution (a) and the mean running time (b) for 6-qubit circuits with known optimal CNOT gate count. To demonstrate the trade-off between running time and the quality of the solution, we consider 20 time limits between 100 seconds and 15 hours. We observe that for all problems there exists a time limit at which the ratio of the recovered optimal circuits and the mean running time stops increasing. This value depends on the hardness of the circuits; for the hardest circuits (optimal gate count of 12) the metrics are saturated at ≈ 4 hours. With this time limit we recover the optimal CNOT count for 97.9% of the circuits and observe the difference of 0.2% between the average optimal CNOT count and the average CNOT count recovered by our software. Small deviations from monotonic growth of running time and quality with the time limit are due to the experiments being performed on a heterogeneous computing cluster and the random nature of the algorithm implementation. The mean running time being above the time limit for time limit 100s is due to letting template matching complete even after the time limit is triggered.

Type	n_q	$t_{\max}$	C_{orig}	$C_{\text{A-G}}$	C_{greedy}	C_{opt}	r (%)	$C_{\text{no SWAP}}$	C_{tket}
Path graph	5	12	26.00	12.00	11.50	7.58	36.81	4.50	4.50
Path graph	15	32	231.00	69.72	73.06	37.22	46.62	22.50	22.62
Path graph	25	52	636.00	159.27	146.15	72.73	54.33	43.50	43.88
Path graph	35	72	1241.00	270.79	224.67	109.78	59.46	66.53	67.17
Path graph	45	92	2046.00	421.72	305.65	148.40	64.81	89.96	90.98
Path graph	55	112	3051.00	589.71	384.27	188.29	68.07	113.80	115.46
Cycle graph	5	10	27.50	19.60	12.90	7.80	60.20	7.10	7.30
Cycle graph	15	30	232.50	163.17	76.00	45.90	71.87	32.67	33.60
Cycle graph	25	50	637.50	424.60	155.56	88.12	79.25	60.72	61.94
Cycle graph	35	70	1242.50	796.24	236.14	133.07	83.29	89.10	90.66
Cycle graph	45	90	2047.50	1279.60	321.24	179.00	86.01	118.01	120.00
Cycle graph	55	110	3052.50	1872.48	407.36	227.57	87.85	147.04	149.19
Square lattice	4	4	10.00	6.00	5.00	3.50	41.67	3.50	3.50
Square lattice	9	8	54.00	35.25	33.12	16.50	53.19	15.38	16.50
Square lattice	16	12	156.00	95.42	93.17	40.08	57.99	40.42	45.33
Square lattice	25	24	500.00	254.12	206.50	97.38	61.68	89.75	102.38
Square lattice	36	36	1110.00	621.36	409.89	239.50	61.46	221.39	249.31
Square lattice	49	16	714.00	858.44	553.12	320.19	62.70	298.62	343.44
Square lattice	64	252	14168.00	2189.41	1237.07	887.38	59.47	824.60	904.80
Triangular lattice	3	6	10.50	2.83	3.33	2.83	0.00	2.83	2.83
Triangular lattice	6	10	49.50	22.60	18.60	9.70	57.08	9.60	10.20
Triangular lattice	10	36	333.00	72.58	60.36	29.78	58.97	26.42	28.78
Triangular lattice	15	90	1365.00	187.18	128.17	65.44	65.04	61.40	66.96
Triangular lattice	21	24	562.50	303.08	211.67	117.33	61.29	109.58	117.79
Triangular lattice	28	300	9481.50	726.63	418.46	272.20	62.54	256.57	272.00
Triangular lattice	36	60	2562.00	928.92	556.60	356.87	61.58	340.77	369.03
Triangular lattice	45	300	16254.00	1909.62	1023.72	801.87	58.01	736.90	767.44
Triangular lattice	55	72	4927.50	2195.36	1229.04	968.68	55.88	895.53	922.10
Hexagonal lattice	6	6	21.00	15.33	14.00	8.00	47.83	7.50	7.50
Hexagonal lattice	24	24	375.00	285.88	197.25	101.83	64.38	91.67	101.58
Hexagonal lattice	54	120	4356.00	1711.33	874.77	585.89	65.76	539.67	606.44
Heavy hexagon lattice	12	12	78.00	79.50	58.92	28.42	64.26	21.58	22.67
Heavy hexagon lattice	54	120	3630.00	1747.48	899.80	609.90	65.10	561.50	625.98

Table 1: Optimization results for Hamiltonian evolution circuits. For each graph on n_q qubits, we generate and optimize $t_{\max}$ circuits corresponding to all integer numbers of steps between 1 and $t_{\max} = \min(t_p, 300)$. C_{orig} is the average CNOT gate count in the original circuits, $C_{\text{A-G}}$ is the average CNOT gate count of the circuits in Aaronson–Gottesman canonical form [3], C_{greedy} is the average CNOT gate count of the circuits produced by the bidirectional “greedy” compiler, C_{opt} is the average CNOT gate count of the optimized circuits, and $r = (C_{\text{A-G}} - C_{\text{opt}})/C_{\text{A-G}}$ is the improvement in the average CNOT gate count over the Aaronson–Gottesman canonical form. For all runs we set the time limit to 36 hours and stop both peephole optimization and template matching when the time limit is reached. We note that the “greedy” compiler by itself (without any further optimization) reduces the CNOT gate count by 48.6% compared to [3]. We additionally compare the performance of our methods with the CliffordSimp method of tket framework [29] applied to the output of our “greedy” compiler (column C_{tket}). As tket ignores the SWAP gates, we modified our implementation such that once the SWAPs are factored out in template matching phase, they are ignored. The CNOT gate counts are presented in column $C_{\text{no SWAP}}$. We observe that our optimizations result in CNOT counts that are 6.58% lower on average as compared to tket, with larger improvements (up to 17.8%) observed for harder (deeper) circuits.

4 Experimental Results

We ran two sets of computational experiments designed to test the performance of our synthesis and optimization algorithms, detailed in the next two subsections. In addition, we compared our results to [29] as well as to 8-qubit T gate free circuits from [14]. The comparison to [29] is detailed in Table 1 and the comparison to [14] reads 24.4529 (obtained using 10,000 random samples) to 50+ in [14, Figure 3].

4.1 Recovering Optimal CNOT Count for Clifford Unitaries on Six Qubits

First we compare the proposed heuristic methods with the optimal Clifford compiler for $n \leq 6$ qubits [10]. The latter uses breadth-first search on the Clifford group to construct a database specifying the optimal CNOT gate count of each Clifford operator. As shown in [10], the optimal CNOT gate count for 6-qubit Clifford operators takes values $0, 1, \dots, 15$. We generate 1,003 uniformly sampled random Clifford unitaries with the CNOT gate counts between 5 and 15. We consider only unitaries with the CNOT gate count ≥ 5 because one needs at least 5 CNOTs to entangle all 6 qubits. For the CNOT gate counts from 5 to 14, we consider 100 circuits for each cost value. For the CNOT gate count of 15, there are only 3 Clifford circuits (modulo single-qubit Cliffords on the left and on the right and modulo qubit permutations) to consider [10].

For each Clifford unitary, we start by synthesizing it using the bidirectional “greedy” compiler. The optimization is run as described in Subsection 3.4. The circuit is then resynthesized by using the randomized version of the compiler, and the resynthesized circuit is optimized. This process is repeated until the time limit is reached, and the circuit with the lowest CNOT count is chosen as the output. Note that we also stop the peephole optimization when the time limit is reached, but we allow template matching to complete. The reason is that template matching is fast as compared to peephole optimization and allowing it to complete results in the actual running time above the time limit by only 0.66% of the instances considered.

The quality of the solution obtained by the implemented methods as a function of the time limit is shown in Fig. 3a. Our algorithm converges before exhausting the time limit on most instances. Fig. 3b shows actual observed mean running time as a function of the time limit. We note that the combination of the iterative nature of symbolic peephole optimization and the randomized resynthesis allows the user to trade off the quality of the optimization and the running time as desired.

4.2 Circuits for Hamiltonian Evolution

To evaluate the performance of the proposed methods on circuits with $n > 6$ qubits, we consider a toy model of Hamiltonian time evolution. Suppose $G = (V, E)$ is a fixed graph with n vertices. We place a qubit at each vertex of G . Define a Hamiltonian evolution circuit with time t as

$$\left(\prod_{(i,j) \in E} CZ_{i,j} \prod_{j=1}^n H_j \right)^t.$$

The layers of Hadamard and CZ gates model time evolution under an external magnetic field and nearest neighbor two-qubit interactions, respectively. We consider several choices for the interaction graph. First, we take instances of the path and cycle graphs with the number of qubits $n \in \{5, 15, 25, 35, 45, 55\}$. Second, we include all three regular plane tessellations (by triangles, squares, and hexagons). We choose the numbers of vertices between 6 and 64 such that the convex hull spanned by the centers of masses of individual tiles in the gapless regular tiling is congruent to the basic tile. Third, we consider a heavy hexagon grid, obtained from hexagonal tessellation by adding a node in the middle of each edge. This set includes some of the frequently appearing qubit-to-qubit connectivities/architectures. We consider the number of layers $1 \leq t \leq t_{\max} = \min(t_p, 300)$, where t_p is the period such that the Hamiltonian evolution with the number of layers t_p produces the identity transformation. For each interaction graph G we compute the CNOT gate count of optimized circuits averaged over the number of layers $t = 1, 2, \dots, t_{\max}$. The total number of circuits considered is 2,264. We set the time limit to 36 hours and stop both peephole optimization and template

matching when the time limit is reached, only allowing the current pass of template matching to complete. Allowing the current pass of template matching to complete results in only a small ratio of problems (4.5%) to significantly ($\geq 10\%$) exceed the time limit. The results are reported in [Table 1](#). The maximum graph size in these experiments is $n=64$ because we represent n -qubit Pauli operators by a pair of 64-bit integers in our C++ implementation; this limitation can be easily removed by revising the data structure.

5 Conclusion

We reported a bidirectional synthesis approach and two circuit optimization techniques that extend known approaches to Clifford circuits by exploiting the unique properties of the Clifford group. We demonstrate the effectiveness of these methods by recovering optimal CNOT gate count for 98.9% 6-qubit circuits (over 1,003 samples) and by reducing the CNOT gate count by 64.7% on average for Hamiltonian evolution circuits with up to 64 qubits (2,264 circuits considered), compared to Aaronson–Gottesman canonical form [3]. We show evidence of the improvement in the gate count by a factor of 2 compared to other techniques, such as [14].

Appendix A

In this section, we describe greedy synthesizers that we developed to obtain initial circuits implementing the Clifford unitaries considered. There are two versions of the synthesizer: unidirectional and bidirectional.

Let us start with the unidirectional version. Suppose C is an n -qubit Clifford operator. The greedy compiler sequentially constructs Clifford circuits $L_1, L_2, \dots, L_n$ such that $C = L_1 L_2 \cdots L_n C_n$, where C_j acts trivially on the first j qubits. The circuit L_j has the CNOT cost $\$(L_j) \leq 3(n-j)/2 + O(1)$. By assumption, C_n acts trivially on all n qubits, that is, C_n is proportional to the identity. In other words, $C = \omega L_1 L_2 \cdots L_n$ for some irrelevant phase factor ω . This gives a circuit implementing C with the CNOT cost

$$\sum_{j=1}^n \$(L_j) \leq \sum_{j=1}^n (3(n-j)/2 + O(1)) = 3n^2/4 + O(n).$$

The desired circuits L_j are constructed using the following lemma.

Lemma 1. *There exists an algorithm that takes as input anti-commuting n -qubit Pauli operators O and O' and outputs a Clifford circuit $L \in \mathcal{C}_n$ such that*

$$L^{-1}OL = x_1 \quad \text{and} \quad L^{-1}O'L = z_1. \quad (12)$$

The circuit L has the CNOT cost $\$(O, O') := \$(L) \leq 3n/2 + O(1)$. The algorithm has runtime $O(n)$.

We refer to a Clifford operator L satisfying Eq. (12) as a disentangler for the pair (O, O') .

To implement a Clifford operator C as a circuit, apply the lemma to Pauli operators $O = Cx_1C^{-1}$ and $O' = Cz_1C^{-1}$. Let L be the corresponding disentangler and $C_1 = L^{-1}C$. From Eq. (12) we obtain $C_1x_1 = x_1C_1$ and $C_1z_1 = z_1C_1$. This means that C_1 commutes with all Pauli operators on the first qubit, which is possible only if C_1 acts trivially on the first qubit. We can now apply the lemma to C_1 . Ignoring the trivial action of C_1 on the first qubit, one can regard C_1 as an element of the smaller Clifford group $\mathcal{C}_{n-1}$, thereby reducing the number of qubits that need to be considered by one. After n applications of the lemma we obtain the desired disentanglers $L_1, L_2, \dots, L_n$.

Proof of Lemma 1. Let us say that Pauli operators O and O' are in the standard form if their action on any qubit j falls into one of the five cases shown below.

Case	A	B	C	D	E
O_j	X	X	X	I	I
O'_j	Z	X	I	Z	I

Recall that the single-qubit Clifford group $\mathcal{C}_1$ acts by permutations on the Pauli operators X, Y, and Z. Thus one can transform any Pauli pair (O, O') into the standard form by applying a layer of single-qubit Clifford operators. This gives rise to a partition of n qubits into five disjoint subsets A, B, C, D , and E . Note that A has odd size since otherwise O and O' would commute. Let $A(j)$ be the j -th qubit of A . Consider the following circuit.

Algorithm 1 Disentangling circuit

```

1: if  $1 \notin A$  then
2:   SWAP $_{1,A(1)}$ 
3: end if ▷ Now  $O_1 = X$  and  $O'_1 = Z$ 
4: for  $j \in C$  do ▷ Clean  $O$  from  $C$ 
5:   CNOT $_{1,j}$ 
6: end for
7: for  $j \in D$  do ▷ Clean  $O'$  from  $D$ 
8:   CNOT $_{j,1}$ 
9: end for
10:  $i \leftarrow$  first qubit of  $B$ 
11: for  $j \in B \setminus \{i\}$  do ▷ Clean  $(O, O')$  from  $B \setminus \{i\}$ 
12:   CNOT $_{i,j}$ 
13: end for
14: CNOT $_{i,1}H_i$ CNOT $_{1,i}$  ▷ Clean  $(O, O')$  from  $i$ 
15:  $k \leftarrow (|A| - 1)/2$  ▷ Due to anticommutativity,  $|A|$  is odd; group all but qubit 1 in pairs
16: for  $j = 1$  to  $k$  do ▷ Clean  $(O, O')$  from  $A \setminus \{1\}$ 
17:   CNOT $_{A(2j+1),A(2j)}$ 
18:   CNOT $_{A(2j),A(1)}$ 
19:   CNOT $_{A(1),A(2j+1)}$  ▷ Perform simultaneous mapping  $XXX \mapsto XII$  and  $ZZZ \mapsto ZII$ 
20: end for

```

Let L be the operator realized by the above circuit combined with the initial layer of single-qubit Cliffords. Direct inspection shows that L has the desired property, Eq. (12), up to sign factors. The latter can be fixed by applying Pauli X_1 or Y_1 or Z_1 as the first gate of L . Simple algebra shows that L has the CNOT gate count of at most $(3/2)|A| + |B| + |C| + |D| + O(1) \leq 3n/2 + O(1)$. $\square$

Below we use the notation $\$(O, O')$ for the CNOT gate count of the disentangling circuit constructed in Algorithm 1.

Our implementation of the greedy compiler optimizes the order in which the qubits are disentangled. Namely, suppose that at some step j of the compiler a subset of qubits S_j has been disentangled such that $|S_j| = j$ and $C = L_1 L_2 \cdots L_j C_j$, where C_j acts trivially on S_j . Let

$$p = \arg \min_{p \notin S_j} \$(C_j X_p C_j^{-1}, C_j Z_p C_j^{-1})$$

be a qubit with the smallest disentangling cost. Let L be the circuit disentangling $C_j X_p C_j^{-1}$ and $C_j Z_p C_j^{-1}$. Set $S_{j+1} = S \cup \{p\}$ and $L_{j+1} = L \cdot \text{SWAP}_{1,p}$. Then $C = L_1 L_2 \cdots L_{j+1} C_{j+1}$, where C_{j+1} acts trivially on S_{j+1} . Thus one can proceed inductively. The extra SWAP gates are isolated and incorporated back into the compiled circuit as described in Subsection 3.2.

The greedy synthesizer described above has the runtime $O(n^3)$. Indeed, consider the first step of the synthesis. Since the disentangling cost $\$(O, O')$ can be computed in time $O(n)$, picking a qubit with the

smallest disentangling cost takes time $O(n^2)$. Computing the disentangling circuit L and the product $L^{-1}C$ takes time $O(n^2)$ since L contains $O(n)$ gates and the action of a single gate can be simulated in time $O(n)$ using the stabilizer formalism [3]. Thus the full runtime of the greedy synthesizer is $O(n^3)$.

The bidirectional greedy synthesizer sequentially constructs Clifford circuits $L_1, L_2, \dots, L_n$ and $R_1, R_2, \dots, R_n$ such that

$$C = (L_1 L_2 \cdots L_j) C_j (R_j \cdots R_2 R_1),$$

where C_j acts trivially on the first j qubits. This gives a circuit implementing C with the CNOT cost at most

$$\sum_{j=1}^n \$(L_j) + \$(R_j).$$

Consider the first step, $j=1$. Let us construct the circuits $L=L_1$ and $R=R_1$ (all subsequent steps are analogous). Our goal is to minimize the combined cost $\$(L) + \(R) subject to the constraint that $C_1 = L^{-1}CR^{-1}$ acts trivially on the first qubit. Equivalently, C_1 should commute with the Pauli operators x_1 and z_1 . Define n -qubit Pauli operators

$$P := R^{-1}x_1R, \quad P' := R^{-1}z_1R, \quad O := CPC^{-1}, \quad \text{and} \quad O' := CP'C^{-1}.$$

By definition, R^{-1} is a disentangler for the pair (P, P') . Simple algebra shows that C_1 commutes with the Pauli operators x_1 and z_1 if and only if L is a disentangler for the pair (O, O') . The above shows that minimizing the combined cost $\$(L) + \(R) subject to the constraint that $C_1 = L^{-1}CR^{-1}$ acts trivially on the first qubit is equivalent to minimizing the function

$$f(P, P') = \$(P, P') + \$(CPC^{-1}, CP'C^{-1})$$

over all pairs of n -qubit anti-commuting Pauli operators P and P' . Note that $f(P, P')$ can be computed in time $O(n)$ for a given pair (P, P') . Once the optimal pair (P, P') is found, one chooses L and R^{-1} as disentanglers for the Pauli pairs (O, O') and (P, P') , respectively. Since the total number of n -qubit anti-commuting Pauli pairs grows exponentially with n , the global minimum of $f(P, P')$ cannot be computed exactly for large n . To make the problem tractable, we restricted the minimization to Pauli operators P, P' with weight at most two. The number of such pairs (P, P') is at most $O(n^3)$ since the anti-commutativity condition implies that the supports of P and P' must overlap on at least one qubit. Now the minimum of $f(P, P')$ can be computed in time $O(n^4)$, and thus the full runtime of the compiler is $O(n^5)$. Note that the unidirectional greedy compiler described earlier corresponds to $R=I$, that is, $P=x_1$ and $P'=z_1$. Thus the bidirectional compiler subsumes the unidirectional one, even with the restricted minimization domain.

Disclaimer

This paper was prepared for information purposes with contributions from the Future Lab for Applied Research and Engineering (FLARE) Group of JPMorgan Chase & Co. and its affiliates, and is not a product of the Research Department of JPMorgan Chase & Co. JPMorgan Chase & Co. makes no explicit or implied representation and warranty, and accepts no liability, for the completeness, accuracy or reliability of information, or the legal, compliance, tax or accounting effects of matters contained herein. This document is not intended as investment research or investment advice, or a recommendation, offer or solicitation for the purchase or sale of any security, financial instrument, financial product or service, or to be used in any way for evaluating the merits of participating in any transaction.

Acknowledgments

This work was supported in part by the U.S. Department of Energy (DOE), Office of Science, Office of Advanced Scientific Computing Research AIDE-QC and FAR-QC projects and by the Argonne LDRD

© 2020 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

OPTIMALITY STUDY OF EXISTING QUANTUM COMPUTING LAYOUT SYNTHESIS TOOLS

Bochen Tan
CS, UCLA
bctan@cs.ucla.edu

Jason Cong
CS, UCLA

July 14, 2020

ABSTRACT

Layout synthesis, an important step in quantum computing, processes quantum circuits to satisfy device layout constraints. In this paper, we construct QUEKO benchmarks for this problem, which have known optimal depths and gate counts. We use QUEKO to evaluate the optimality of current layout synthesis tools, including Cirq from Google, Qiskit from IBM, t|ket) from Cambridge Quantum Computing, and recent academic work. To our surprise, despite over a decade of research and development by academia and industry on compilation and synthesis for quantum circuits, we are still able to demonstrate large optimality gaps: 1.5-12x on average on a smaller device and 5-45x on average on a larger device. This suggests substantial room for improvement of the efficiency of quantum computer by better layout synthesis tools. Finally, we also prove the NP-completeness of the layout synthesis problem for quantum computing. We have made the QUEKO benchmarks open-source.

1 Introduction

Recently, a quantum processor “Sycamore” from Google was shown to have a clear advantage over classical supercomputers on a problem named sampling random quantum circuit [1]. It is widely expected that in the near future, quantum computing (QC) will outperform its classical counterpart solving more and more problems. Achieving this computational advantage, however, requires executing larger and larger quantum circuits.

A quantum circuit consists of quantum gates acting on qubits. It was shown that only gates acting on one or two qubits are required for universal quantum computing [2]. After a quantum circuit is designed, it needs to be mapped to a QC device. However, qubit connections required by two-qubit gates are often greatly constrained by QC device layouts. QC layout synthesis resolves this issue by producing an initial mapping from the qubits in the circuit to the physical qubits on the QC device, adjusting the mapping to legalize two-qubit gates by inserting some new gates, and scheduling all the gates. The resulting circuit preserves the original functionality and is executable on the QC device.

Quantum circuits in earlier experiments used to be only dozens of gates on small devices, e.g., those with 5 or fewer qubits. In those cases, layout synthesis was usually realized by exhaustive enumeration. However, the task is increasingly intractable as the circuits get deeper and wider. Nowadays, a cutting-edge QC experiment requires the execution of a circuit of 53 qubits, 1113 single-qubit gates, and 430 two-qubit gates [3]. For a general circuit of this size, the number of possible initial mappings is $53!$, and the subsequent scheduling and legalization steps have large solution space as well. Clearly, design automation is necessary. In addition, the size of the circuits that QC hardware is able to execute

has been scaling exponentially in the past few years [3]. The fast increase of hardware capacity presents an even bigger challenge to layout synthesis.

Several layout synthesis tools are available and there are also benchmarks that help us to compare them. However, it is currently unknown how far these tools are away from the optimal solutions. In this paper, we present QUEKO benchmarks (quantum mapping examples with known optimal) which are quantum circuits with known optimal depth for the given QC device. Then, we evaluate four existing QC layout synthesis tools with QUEKO, namely `tket`¹, greedy router included in `Cirq`², `DenseLayout` plus `StochasticSwap` included in `Qiskit`³, and the recent academic work by Zulehner et al. [4]. To our surprise, rather large optimality gaps are discovered even for feasible-depth circuits: 1.5-12x on average on a smaller device and 5-45x on average on a larger device.

This result is somewhat surprising as, in comparison, the result of the VLSI circuit placement optimality study conducted more than 15 years ago using the PEKO benchmarks [5] revealed that the optimality gaps of the leading academic and industrial placers were 1.43-2.12X on average for circuits with over two million placable objects, while the quantum circuits used in this study have only up to 54 qubits and about 35 thousand quantum gates, yet shown a much large optimality gap.

The optimality gaps revealed in this study have a strong implication. If we can consistently halve the circuit depth by better layout synthesis, we effectively double the decoherence time of a QC device, which is equivalent to a large advancement in experimental physics and electrical engineering. Therefore, the gaps call for more research investments into QC layout synthesis. To draw a parallel, the VLSI placement optimality study using the PEKO benchmarks [5] spurred further research investment in circuit placement, resulting in wirelength reduction equivalent to two or more generations of Moore's Law scaling, but in a more cost-efficient way [6].

The rest of this paper is organized as follows: Sec. 2 reviews relevant background of QC, Sec. 3 formulates the QC layout synthesis problem; Sec. 4 reviews related work; Sec. 5 provides the construction of QUEKO benchmarks; Sec. 6 evaluates aforementioned tools with QUEKO; Sec. 7 proves the NP-completeness of QC layout synthesis; Sec. 8 gives conclusion.

2 Background

2.1 Qubits

A qubit is in a quantum state $|\psi\rangle$ represented by a vector in two-dimensional Hilbert space with L^2 -norm equals 1

$$|\psi\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \alpha \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \beta \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \alpha|0\rangle + \beta|1\rangle, \quad (1)$$

where the two basis vectors are $|0\rangle$ and $|1\rangle$. A quantum state of multiple qubits lies in the tensor product of individual Hilbert spaces. For instance, a general two-qubit state $|\phi\rangle$ is

$$|\phi\rangle = \alpha|0\rangle|0\rangle + \beta|0\rangle|1\rangle + \gamma|1\rangle|0\rangle + \lambda|1\rangle|1\rangle = (\alpha \ \beta \ \gamma \ \lambda)^T, \quad (2)$$

where we omit the tensor product notation $\otimes$ between $|\cdot\rangle$ s for convenience. A joint state of two individual qubits $|\psi_1\rangle \otimes |\psi_0\rangle$ is

$$|\psi_1\rangle|\psi_0\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix} \otimes \begin{pmatrix} \alpha' \\ \beta' \end{pmatrix} = (\alpha\alpha' \ \alpha\beta' \ \beta\alpha' \ \beta\beta')^T. \quad (3)$$

2.2 Quantum Gates

A quantum gate transforms an input state to an output state. For example, some common single-qubit gates are X , H , and T . $\dagger$ means transpose complex conjugate.

$$X|\psi\rangle = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \begin{pmatrix} \beta \\ \alpha \end{pmatrix}, \quad H = \frac{\sqrt{2}}{2} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}, \quad T^\dagger = \begin{pmatrix} 1 & 0 \\ 0 & e^{-i\pi/4} \end{pmatrix}. \quad (4)$$

Two common two-qubit gates are CZ and CX (also named $CNOT$).

$$CZ = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}, \quad CX = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}. \quad (5)$$

¹<https://github.com/CQCL/pytket>

²<https://github.com/quantumlib/Cirq>

³<https://qiskit.org/>

NAND gates are sufficient for universal classical computing. For universal quantum computing, there are multiple complete gate sets. Table 1 lists three such sets chosen by QC frameworks Cirq², Qiskit³, and pyQuil⁴. Other gates may be input to these frameworks, but would be translated into a combination of these native gates. The exact matrix representations of these gates are not specified here because they are irrelevant to the purpose of this paper.

Table 1: Complete quantum gate set examples

Framework	Single-qubit gate	Two-qubit gate
Cirq (Google)	$Z(\lambda)$, phased X power	CZ
Qiskit (IBM)	$U_1(\lambda), U_2(\phi, \lambda), U_3(\theta, \phi, \lambda)$	CX
pyQuil (Rigetti)	$RZ(\lambda), RX(k\pi/2)$	CZ

Another important gate is CCX , or Toffoli, gate which is universal for reversible logic and thus essential for QC logic synthesis [7]. It is a quantum gate on three qubits and can be decomposed into a set of single-qubit and two-qubit gates as shown in the following subsection.

$$CCX = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \quad (6)$$

An important property between two gates is whether they commute. If the result of first applying gate 0 followed by gate 1 is the same with the result of the other way around, we say these two gates *commute*. If two gates act on totally different qubits, they always commute. For gates acting on a same qubit, it is nontrivial to judge in general. T and CZ commute because they are both diagonal matrices; but X and H do not commute, since $XH \neq HX$.

For more background knowledge, the reader can refer to a comprehensive textbook on quantum computing and quantum information [2], or a concise textbook on quantum computing [8] with more computer science flavor.

2.3 Quantum Circuit

In QC, a circuit or program is usually input as a piece of QASM code [9], e.g., Fig. 1a. The code is rather simple to read, merely specifying each gate sequentially like instructions in traditional assembly language. We thus define a *quantum circuit* to be a list of quantum gates $g_1 g_2 \dots g_M$.

It is important to note that the qubits in a quantum circuit are *logical qubits* denoted as $q_1, \dots, q_N$. We do not use “logical” to indicate error correction in this paper. Additionally, we denote *qubit count* by N , *gate count* by M , *single-qubit gate count* by M_1 , and *two-qubit gate count* by M_2 . For instance, in Fig. 1, $N = 3$, $M = 15$, $M_1 = 9$, $M_2 = 6$.

We use the notation of gate like a set of qubits. We say the cardinality $|g| = 1$ if g is a single-qubit gate and $|g| = 2$ if g is a two-qubit gate. $g \cup g'$ denotes the set of qubits involved in g or g' . $g \cap g'$ denotes the set of qubits involved in g and g' . For instance, in Fig. 1, $q_1, q_2 \in g_2$, $q_0 \in g_{13}$, $|g_4| = |g_6| = 2$, $|g_7| = |g_9| = 1$, $g_1 \cap g_2 = \{q_2\}$, and $g_8 \cup g_9 = \{q_0, q_1, q_2\}$.

A 1D circuit diagram can also represent the circuit, e.g., Fig. 1b. In such a diagram, each wire stands for a logical qubit. We draw the control qubit of CX gate as $\bullet$ and the target qubit as $\oplus$. The 1D diagram provides some implicit timing information, i.e., the gates on the same horizontal wire are executed from left to right. However, vertical alignment has no indication on timing, e.g., g_8 and g_9 can be executed simultaneously, but we separate them by some horizontal distance purely to avoid overlapping in the diagram.

2.4 Quantum Computing Device Representation

We represent the layout of a QC device with a *device graph* $G = (P, E)$ where each node p stands for a *physical qubit* and each edge e stands for a connection that enables two-qubit entangling gates, i.e., we can only perform such gates

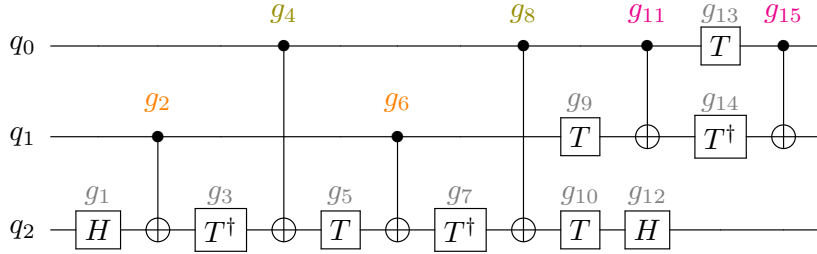
⁴<https://github.com/rigetti/pyquil>

```

OPENQASM 2.0;
include "qelib1.inc";
qreg q[3]; // Initiate 3 logical qubits
h q[2]; //  $g_1$ 
cx q[1], q[2]; //  $g_2$ 
tdg q[2]; //  $g_3$ 
cx q[0], q[2]; //  $g_4$ 
t q[2]; //  $g_5$ 
cx q[1], q[2]; //  $g_6$ 
tdg q[2]; //  $g_7$ 
cx q[0], q[2]; //  $g_8$ 
t q[1]; //  $g_9$ 
t q[2]; //  $g_{10}$ 
cx q[0], q[1]; //  $g_{11}$ 
h q[2]; //  $g_{12}$ 
t q[0]; //  $g_{13}$ 
tdg q[1]; //  $g_{14}$ 
cx q[0], q[1]; //  $g_{15}$ 

```

(a) QASM code of Toffoli circuit



(b) 1D diagram of Toffoli circuit



(c) Two-qubit gate set of Toffoli circuit

Figure 1: Toffoli circuit (Single-qubit gates are colored gray. Identical two-qubit gates applied at different times have the same color, e.g., g_2 and g_6 are orange because they are both $CX(q_1, q_2)$ but at different times.)

on two physical qubits that are connected. This graph is also named as *coupling graph* or *qubit connectivity*. Device graphs used in this paper are shown in Fig. 2.

In the coupling graphs, we assume the edges are bi-directional. We made this decision based on the following reasons. First, if the two-qubit gate in the gate library is symmetric, like CZ gate used by Google and Rigetti as shown in Table 1, then it does not make sense to have directed connections. Second, even for asymmetric gates like CX , most of the IBM quantum computers⁵ now support both directions for every edge. Third, as we shall see in Sec. 5, our construction of QUEKO benchmarks works for directed graphs as well. So, the assumption of bi-directionality does not change the conclusion of this study.

3 Problem Formulation

Layout synthesis is divided into two sub-tasks: *initial placement* that produces an initial mapping from logical qubits to physical qubits, and *gate scheduling* that decides when and where to execute each input gate and insert SWAP gates to make two-qubit gates legal on the device graph. However, a SWAP gate may increase the depth, and definitely increase the gate count and the error rate. In the whole QC compilation/mapping workflow, there are usually circuit optimization

⁵<https://quantum-computing.ibm.com/>

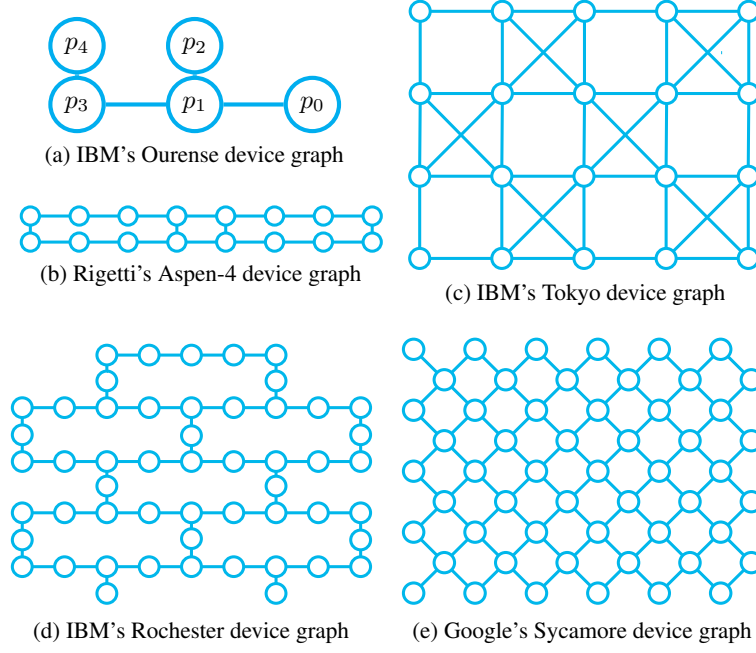


Figure 2: QC device examples

stages before or after layout synthesis. During the optimization stages, gate reduction and commutation are performed, e.g., [10]. We assume that these optimizations are already applied prior to layout synthesis, so that every input gate should be executed and the relative order of input gates should not change in the layout synthesis process.

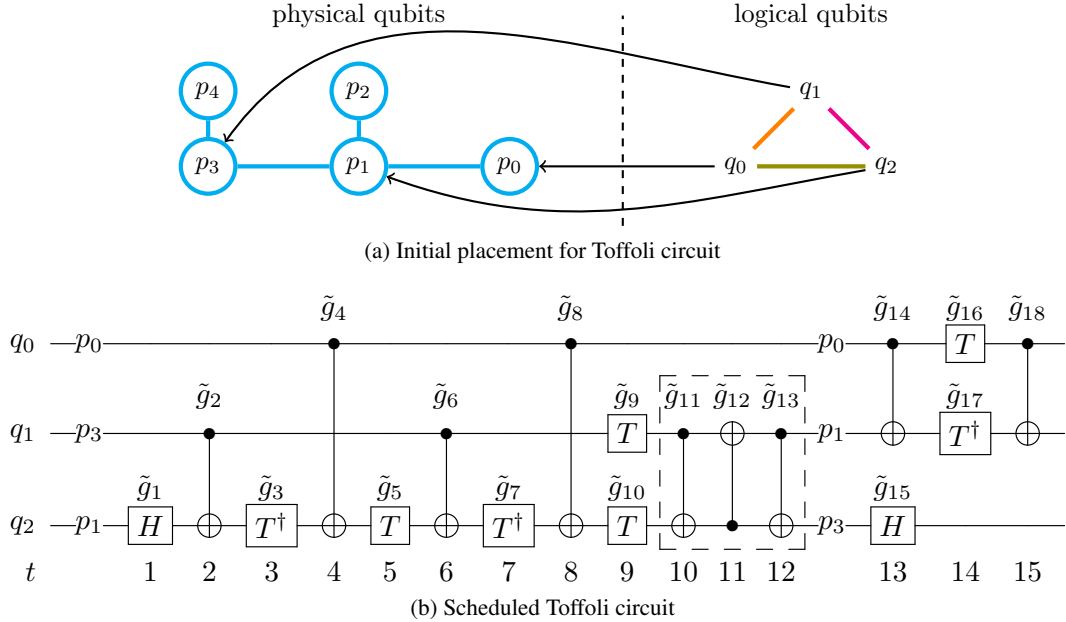


Figure 3: Quantum computing layout synthesis for Toffoli circuit on IBM's Ourense device

3.1 Initial Placement

During initial placement, we need to find a mapping from logical qubits in the quantum circuit to physical qubits on the device $\mu_0 : Q \rightarrow P$ that benefits subsequent gate scheduling. If the two-qubit gate set, consisting of all the two-qubit gates, can be embedded in the device graph during initial placement (e.g., Fig. 1c can be embedded in Fig. 2c), gate scheduling does not necessarily need to insert any gates. However, the case is not so ideal in general. For example, if we want to map the Toffoli circuit as shown in Fig. 1b onto device Ourense as shown in Fig. 2a, there must be some additional gates, since the device graph does not contain any triangles, but the two-qubit gates in Fig. 1c forms a triangle. A valid initial placement in this case is given in Fig. 3a where $\mu_0(q_0) = p_0$, $\mu_0(q_1) = p_3$, and $\mu_0(q_2) = p_1$.

3.2 Gate Scheduling

Given a quantum circuit $g_1 \dots g_M$, e.g., Fig. 1b, gate scheduling produces the *spacetime coordinates* (t_j, x_j) for each gate. The coordinates specify when and where the gates are applied. We say that a gate is scheduled to *cycle* t if its time coordinate is t . For a single-qubit gate, the space coordinate is a physical qubit, i.e., $x_j \in P$; for a two-qubit gate, it is an edge in the device graph, i.e., $x_j \in E$. SWAP gates may need to be inserted during gate scheduling to ensure that all two-qubit gates are executable. The input gate plus the inserted SWAP gates constitute the scheduled gate list $\tilde{g}_1, \dots, \tilde{g}_{\tilde{M}}$. Since only SWAP gates are inserted and all the input gates are contained in the scheduled gate list, the functionality of input circuit remains unchanged after the layout synthesis process. Additionally, gate scheduling must respect *dependencies* in the quantum circuit. If gate g acts on qubit q , then g can only be executed after all prior gates, which act on qubit q , are executed.

A valid but not necessarily optimal gate scheduling example is given in Fig. 3b. The time coordinates for all the gates are displayed at the bottom, e.g., $t_1 = 1$ and $t_{15} = 13$. The space coordinates can be inferred from the mapping, e.g. $x_1 = p_1$, $x_2 = (p_3, p_1)$, $x_{14} = (p_0, p_1)$, and $x_{15} = p_3$. There is an injective map from the original gates to the scheduled gates: $f(i) = i$ for $i = 1$ to 10 and $f(i) = i + 3$ for $i = 11$ to 15 such that $g_i = \tilde{g}_{f(i)}$ for $i = 1$ to 15. The three CX gates in the dashed box $\tilde{g}_{11}$, $\tilde{g}_{12}$, and $\tilde{g}_{13}$ constitute a SWAP gate. The adjusted mapping is shown after the them. The SWAP gate is inserted so that $\tilde{g}_{14}$ and $\tilde{g}_{15}$ are on connected qubits p_0 and p_1 , thus executable.

3.3 Formal Definition of (Depth-Optimal) Layout Synthesis Problem in Quantum Computing

Input A device graph $G = (P, E)$ and a list of quantum gates $g_1 \dots g_M$ acting on logical qubit set Q . All the input gates are in the gate set of the device, e.g., a set from Table 1. Logical qubits are less or equal than physical qubits, i.e., $|Q| \leq |P|$.

Output An initial mapping $\mu_0 : Q \rightarrow P$, and a scheduled quantum circuit consists of a new list of gates $\tilde{g}_1 \dots \tilde{g}_{\tilde{M}}$, including SWAP gates, where each gate has a spacetime coordinate (t_j, x_j) . We use tilde to denote that a gate is scheduled from here on.

Constraints

- **Feasible two-qubit gates:** all the two-qubit gates in the scheduled circuit must be on two qubits connected in the device graph. Formally, for $j = 1$ to $\tilde{M}$, if $|\tilde{g}_j| = 2$, then $x_j \in E$.
- **Executing all gates:** all input gates should be executed. Formally, there is an injective map $f : \{1, \dots, M\} \rightarrow \{1, \dots, \tilde{M}\}$ such that $g_i = \tilde{g}_{f(i)}$ for $i = 1$ to M .
- **Respecting dependencies:** for $i, i' = 1$ to M , if $i < i'$ and $g_i \cap g_{i'} \neq \emptyset$ then $t_{f(i)} < t_{f(i')}$.

Objective Minimizing circuit *depth* T , which is the maximal time coordinate of all the scheduled gates, i.e., $T \equiv \max_{j=1, \dots, \tilde{M}} t_j$.

In this paper, we use depth as the default objective but other objectives can be used as well, e.g., the number of additional gates $\tilde{M} - M$, or the fidelity of the scheduled circuit. The output and the constraints of the problem are independent of the objective. However, with other objectives like fidelity, more input information may be required.

4 Related Work

In the most general sense, the task of QC layout synthesis is generating a quantum circuit that satisfies QC device constraints and fulfills the functionality of the input circuit. Related works on this problem include [4, 11–36].

These works may have some variations on the problem in mind. [11, 13, 17, 26, 34] focus on multidimensional array device graphs (linear array for 1D, grid for 2D, and so on). [30] focuses specifically on SU(4) circuits and includes post-synthesis optimization. [36] focuses on adjusting the mapping after synthesis to improve fidelity.

As pointed out in Sec. 2.2 it is nontrivial to judge if two quantum gates commute. If two gates do not commute, changing their relative order would result in a functionally different circuit. Therefore, in general, performing commutations are seen to be part of logic synthesis for quantum computing, which happens before layout synthesis, the main topic of this paper. Thus, to better isolate layout synthesis, we stated in Sec. 3 that, as an assumption, all commutation-based optimization have been performed beforehand. This assumption is also respected in all of the evaluations afterwards and all the other related works listed above except [21, 27–29]. [21] considers a few basic commutation rules, but the solutions it compares to do not use dependencies to encode the relative orders between the gates. Thus, it is unclear whether the gain of [21] is because of commutations, or encoding with dependencies. [27–29] consider the layout synthesis of QAOA circuits. All the two-qubit gates used in QAOA circuits [37] are ZZ -phase gates that commute with each other, which means the relative order of two ZZ -phase gates can be changed even if they act on a same qubit. However, these gates rarely appear in other existing quantum circuits, so we decide to treat QAOA circuits as a special case instead of jeopardizing our efforts in isolating layout synthesis.

Existing works on quantum layout synthesis can be characterized by based on optimization metrics used. The metric can be the additional “cost”, which is usually proportional to the number of additional gates [4, 12, 24, 26, 30]; or circuit depth like this paper, since the qubits can only function well within the decoherence time [11, 25, 27–29]; or circuit fidelity, since nowadays a common practice is executing a circuit multiple times and analysing the statistics of the results [32, 34–36]; or a mix of the above [31, 33].

Detailed discussions of the complexity of QC layout synthesis can be found in Sec. 7. These discussions indicate that large scale instances cannot be solved both exactly and efficiently. From the perspective of solution techniques, the current works can be divided into two categories. The first group focuses on deriving the exact solution for moderate-sized instances with the help of solvers [15, 18, 22, 25–29, 34, 35]. [18, 26] use a PBO (pseudo Boolean optimizer) to decide a set of SWAP gates that legalizes all the two-qubit gates, but they do not explicitly schedule the gates. The same goes for [15, 22], which use a MIP (mixed integer programming) solver and a SMT (satisfiability modulo theories) solver correspondingly. [27–29] use a temporal planner to schedule specifically QAOA circuits. The closest previous works concerning this paper are [25, 34, 35]. However, [34, 35] use a SMT solver to maximize fidelity. [25] splits circuits into “levels” and inserts gates to transform the mapping between the levels. This model of quantum circuit may not yield an optimal solution. Under this imperfect “levels” model, [25] aims to derive a depth-optimal solution with integer linear programming.

The second group of related works use heuristic search techniques [4, 11, 14, 16, 17, 19–21, 23, 24, 30–33]. We only discuss the works targeting general device layouts below [4, 12, 19–21, 23, 24, 30–33]. The general approach is splitting the circuit into small sub-circuits for which the layout synthesis can be done efficiently, and then searching for the mapping transformation between these sub-circuits. A sub-circuit can be a “level” or “layer” mentioned in the last paragraph [4, 20, 23, 24, 31–33], a set of several levels [12], several levels but for a few specific qubits [30], or individual gates [19, 21]. In order to find the mapping transformation, [19] inserts SWAP gates to move the two qubits that required by the next two-qubit gate in the shortest path; [21, 24, 31] also consider distances between qubits of further two-qubit gates; [32] additionally considers fidelity in the qubit movements; [4, 30] use the sum-of-distances plus the number of SWAP gates as the cost function in A* search; [33] uses bidirectional search; [20] uses a 4-approximation algorithm; [23] exploits some existing approximate solution of token swapping; [12] recursively considers SWAP gates as cuts in the device graph.

The complexity also brings difficulty to the evaluation of these solutions. Currently, the benchmarks usually are quantum circuit libraries of some realistic functions, e.g., [10] and RevLib [38], or certain random circuits, which are thought to be the worst-case scenario, e.g., SU(4) circuits [30]. So far, researchers can only compare against each other, but do not know how far they are from the optimum. This paper aims to fill in this gap.

The QC layout synthesis problem is still quite new to compiler and design automation communities, so the name of the problem varies. It can be placement [12, 15], routing [24], compiling quantum circuits [21, 27–30, 34, 35], quantum circuit transformation [20], mapping circuits to QC architectures [4, 18, 22, 25, 26, 31, 33], conversion [13] or optimization [14] of circuits in QC architecture, realization of quantum circuits [16, 17], or qubit allocation [19, 23, 32, 36].

5 QUEKO Benchmarks

This work is inspired by PEKO [5], placement examples with known optimal. Placement is a crucial step in classical integrated circuit design, where modules are placed on a chip with objective of minimizing total wirelength. Although this problem is NP-hard, the PEKO algorithm is able to generate benchmarks with known optimal solutions.

Similarly, for a generic input quantum circuit and a generic device graph, finding the scheduled circuit with optimal depth is NP-complete, which will be proved in Sec. 7. However, it is feasible to construct some benchmarks with known optimal solution. Given a target device graph G and a target depth T , we can construct an depth-optimal circuit. Then, by re-labelling the qubits, we derive a QUEKO benchmark.

Additionally, QUEKO can be customized for a given feature: gate density vector (d_1, d_2) . The two components intuitively stand for the densities of single-qubit gates and two-qubit gates in the whole circuit. Suppose a circuit has n logical qubits, M_1 single-qubit gates, M_2 two-qubit gates, and a longest dependency chain of l , then $d_1 = M_1/(n \cdot l)$ and $d_2 = 2M_2/(n \cdot l)$. For example, in Fig. 1b, $n = 3$, $l = 11$, $M_1 = 9$, and $M_2 = 6$, so $d_1 \approx 0.27$ and $d_2 \approx 0.36$. Likewise, we can extract (d_1, d_2) from other existing circuits with known functionalities, so that the QUEKO benchmarks imitate some real-world circuits and, at the same time, have known optimal depths.

The construction of QUEKO, as shown in Algorithm 1, starts with checking the validity of input data by calculating the number of single-qubit and two-qubit gates M_1 and M_2 . If $M_1 + M_2 < T$, then there would be too few gates to generate a circuit with depth T ; if $M_1 + 2M_2 > N \cdot T$, then there would be too many gates for the given depth and device graph. We define the matching bound u of a graph G to be the minimal size of maximal matchings of G . This means we can find at least u edges in G that pair-wisely share no vertices. If $M_2 > u \cdot T$, then there could be too many two-qubit gates for the given depth and device graph. In short, if $M_1 + M_2 < T$, $M_1 + 2M_2 > N \cdot T$, or $M_2 > u \cdot T$, we return an error to reject the input data. Otherwise, we proceed to three phases: backbone construction, sprinkling, and scrambling.

5.1 Backbone Construction Phase

This phase “grows” a sequence of T gates, each depending on the previous one, constituting a dependency chain of length T . This chain serves as the “backbone” of the circuit. For example, we start from the device graph as Fig. 4a (which is just Fig. 2a rotated), and pick three executable gates $\tilde{g}_1, \tilde{g}_2$, and $\tilde{g}_3$ whose spacetime coordinates are $(1, (p_0, p_1))$, $(2, p_1)$, $(3, (p_1, p_2))$. They constitute a dependency chain of length $T = 3$, since all of them act on p_1 . This is shown in Fig. 4b, where gates at different cycles are put on different “slices” from left to right. The “backbone” is colored green. Note that if the given graph is directed, then we make sure that the direction of the two-qubit gate we choose is consistent with the direction of the corresponding edge. Thus, our construction also works for directed device graphs.

To be more rigorous, we first choose a random node or edge of G as x_1 . In every iteration afterwards, we randomly choose x_k that overlaps with x_{k-1} . Thus, $x_k \cap x_{k-1} \neq \emptyset$, which enforces $t_k > t_{k-1} = k - 1$ by dependency constraint. On the other hand, since $\tilde{g}_k$ is executable, it can at most take a single cycle, i.e., the optimal $t_k = t_{k-1} + 1 = k$. Gate sequence $\tilde{g}_1, \tilde{g}_2, \dots, \tilde{g}_T$ constitutes a dependency chain of length T . Because of this “backbone”, the final depth of the scheduled circuit cannot be lower than T . Note that we do not need to use any SWAP gates for backbone construction.

5.2 Sprinkling Phase

The backbone construction phase uses T gates in total, we then randomly “sprinkle” the rest $M_1 + M_2 - T$ gates, e.g., $\tilde{g}_4, \tilde{g}_5, \tilde{g}_6, \tilde{g}_7$ shown in Fig. 4c. We randomly select spacetime coordinates (t_i, x_i) , $(1 \leq t_i \leq T)$ that does not overlap with any existing gates with time coordinate t_i . Since all the time coordinates of gates sprinkled are less or equal to T , the backbone is not lengthened in this phase. After sprinkling, a circuit with gates $\tilde{g}_1 \dots \tilde{g}_{M_1+M_2}$ is created. Its gates are all executable; its depth is T ; its gate density vector approximates (d_1, d_2) . (There could be minor rounding errors in the ceiling function.)

It is worthy of noting that though only one longest dependency chain is explicitly generated in the backbone construction phase, the sprinkling phase may implicitly generate more. For example, $\tilde{g}_4$ depends on $\tilde{g}_7$; if we “sprinkles” a gate on p_4 at cycle 3, then another dependency chain of length 3 would exist in the output circuit. The higher the gate densities, the more likely that these implicit longest dependency chains are generated.

Algorithm 1 QUEKO construction

Input: a device graph $G = (P, E)$ with $|P| = N$ and its matching bound u , a depth target T , and a gate density vector (d_1, d_2)

Output: QUEKO benchmark $g_1 g_2 \dots g_{M_1+M_2}$, where M_1 and M_2 are the numbers of single-/two-qubit gates

```
1:  $M_1 \leftarrow \lceil d_1 \cdot N \cdot T \rceil$ ,  $M_2 \leftarrow \lceil d_2 \cdot N \cdot T/2 \rceil$ 
2: if  $M_1 + M_2 < T$  or  $M_1 + 2M_2 > N \cdot T$  or  $M_2 > u \cdot T$  then
3:   return error: input data not admissible
4: end if
5:  $m_1 \leftarrow 0$ ,  $m_2 \leftarrow 0$  // how many single-qubit gates and two-qubit gates we have used
   // Backbone construction phase
6: for  $i = 1$  to  $T$  do
7:    $j \leftarrow \text{rand}(\{1, 2\})$  // randomly decide single-qubit or two-qubit gate
8:   if  $j = 2$  and  $m_2 < M_2$  then
9:      $x_i \leftarrow \text{rand}(E)$ 
10:    while  $i > 1$  and  $x_i \cap x_{i-1} = \emptyset$  do
11:       $x_i \leftarrow \text{rand}(E)$ 
12:    end while
13:     $t_i \leftarrow i$ ,  $m_2 \leftarrow m_2 + 1$ 
14:  else
15:     $x_i \leftarrow \text{rand}(P)$ 
16:    while  $i > 1$  and  $x_i \cap x_{i-1} = \emptyset$  do
17:       $x_i \leftarrow \text{rand}(P)$ 
18:    end while
19:     $t_i \leftarrow i$ ,  $m_1 \leftarrow m_1 + 1$ 
20:  end if
21: end for
   // Sprinkling phase
22: for  $i = T + 1$  to  $M_1 + M_2$  do
23:    $j \leftarrow \text{rand}(\{1, 2\})$ 
24:   if  $j = 2$  and  $m_2 < M_2$  then
25:      $(t_i, x_i) \leftarrow \text{rand}(\{1, 2, \dots, T\} \times E)$ 
26:     while  $\exists l \in \{1, \dots, i\}$  such that  $t_i = t_l$  and  $x_i \cap x_l \neq \emptyset$  do
27:        $(t_i, x_i) \leftarrow \text{rand}(\{1, 2, \dots, T\} \times E)$ 
28:     end while
29:      $m_2 \leftarrow m_2 + 1$ 
30:   else
31:      $(t_i, x_i) \leftarrow \text{rand}(\{1, 2, \dots, T\} \times P)$ 
32:     while  $\exists l \in \{1, \dots, i\}$  such that  $t_i = t_l$  and  $x_i \cap x_l \neq \emptyset$  do
33:        $(t_i, x_i) \leftarrow \text{rand}(\{1, 2, \dots, T\} \times P)$ 
34:     end while
35:      $m_1 \leftarrow m_1 + 1$ 
36:   end if
37: end for
   // Scrambling phase
38:  $\tau \leftarrow$  a random mapping from  $P$  to  $Q$ 
39: for  $i = 1$  to  $M_1 + M_2$  do
40:   if  $x_i = (p, p') \in E$  then
41:      $g_i \leftarrow$  two-qubit gate( $\tau(x_i.p), \tau(x_i.p')$ )
42:   else
43:      $g_i \leftarrow$  single-qubit gate( $\tau(x_i)$ )
44:   end if
45: end for
   // Output
46: sort  $g_i$  according to  $t_i$ ,  $i = 1$  to  $M_1 + M_2$ 
47: return  $g_1 g_2 \dots g_{M_1+M_2}$ 
```

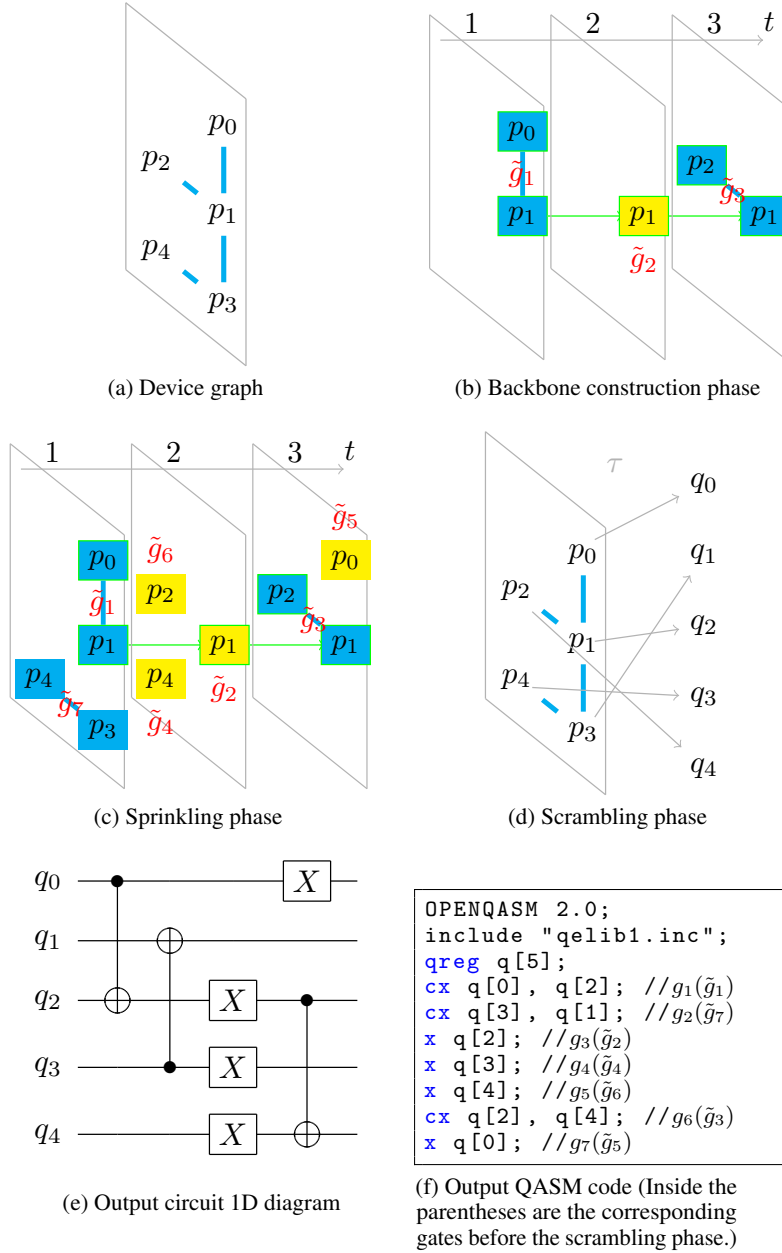


Figure 4: QUEKO construction visualization

5.3 Scrambling Phase

As shown in Fig. 4d, we generate a random mapping τ from physical qubits to logical qubits and apply τ to the space coordinates of $\tilde{g}_1\tilde{g}_2\ldots\tilde{g}_{M_1+M_2}$. For instance, $x_1 = (p_0, p_1)$, so the resulting gate g_1 is a two-qubit gate on logical qubits $\tau(p_0) = q_0$ and $\tau(p_1) = q_2$; $x_7 = (p_3, p_4)$, so g_7 is a two-qubit gate on $\tau(p_3) = q_1$ and $\tau(p_4) = q_3$; g_6 is a single-qubit gate on $\tau(p_2) = q_4$... The specific types of single-qubit gates and two-qubit gates are not important, since QUEKO is only for layout synthesis, not for circuit optimization. We use X as the single-qubit gate and CX as the two-qubit gate.

5.4 Output

Sort the gates $g_1 g_2 \dots g_{M_1+M_2}$ according to the time coordinates to transfer the timing information originally in these time coordinates to the relative order inside the output gate list. The result is a QUEKO benchmark, as shown in Fig. 4e and Fig. 4f.

As we have proven, the depth of the output circuit is at least T because of the backbone. A QC layout synthesis tool can meet the optimum by finding the initial mapping that is the inverse of the scrambling mapping τ . Therefore, QUEKO circuits have known optimal depth T .

Note that QUEKO circuits also have known optimal gate count $M_1 + M_2$. Since we assume that, in layout synthesis, all the input gates need to be executed, the result produced by the tools has at least as many gates as the QUEKO circuit. The optimal gate count is also met with the optimal initial mapping τ^{-1} , since no SWAP gates are needed in this case.

6 Experiment

6.1 Experimental Setup

To evaluate QC layout synthesis tools with QUEKO, device graphs, depths and sizes, and gate density vectors are required. We specify the choice of these parameters and the choice of tools to evaluate in this subsection. Because of the randomness in our construction, we generate ten QUEKO benchmarks for each triplet of the parameters. These benchmarks are made open-source⁶ under the BSD license. For evaluation, we fed each one of these benchmarks to four layout synthesis tools, as shown in Fig. 5. All the experiments were run on a Ubuntu 16.04 server, which has two Intel Xeon E5-2699v3 as CPUs and 128GB main memory.⁷

6.1.1 Device Graph

We used representative devices from three different QC hardware providers. Sycamore from Google⁸, Tokyo and Rochester from IBM⁹, and Aspen-4 from Rigetti¹⁰. The graphs of these devices are shown in Fig. 2. Sycamore has 54 qubits, of which 53 are active; Rochester also has 53 qubits. Both of them are state-of-the-art devices, but Sycamore has richer connectivity. Aspen-4 has 16 qubits, and Tokyo has 20 qubits. They are both highly competitive devices, but Tokyo has greater connectivity.

Also, we have only listed superconducting devices because they are by far the most advanced QC devices. This does not mean that QUEKO cannot generalize to other technologies such as quantum dot¹⁰ because our approach is valid as long as the basic quantum gates of this technology are single-qubit and two-qubit gates.

6.1.2 Depth and Size

We constructed two sets of benchmarks with different depth ranges. The corresponding size of these benchmarks can be deduced from the depth and the gate density vector, as shown in Algorithm 1. The first set has depths from 5 to 45, which is the *near-term feasible benchmarks* (B_{NTF}). In fact, one of the largest quantum circuits executed nowadays has depths 41¹¹, which is about the same with the upper bound of B_{NTF} . We intended to find out the layout synthesis performance within the current execution capacity. The sizes of B_{NTF} benchmarks range from 1136 to 34506 quantum gates. The second set of benchmarks, denoted as B_{SS} has depth from 100 to 900 which are *benchmarks for scaling study*. B_{SS} represents the performance of these tools when the decoherence time of QC device improves in the future. The sizes of B_{SS} benchmarks range from 37 to 1727 quantum gates.

6.1.3 Gate Density Vector

We picked two special gate density vectors in the experiment: (0.51, 0.4) based on the quantum circuits used in Google’s quantum supremacy experiment¹², denoted “QSE” below, and (0.27, 0.36) based on the Toffoli circuit, denoted “TFL”

⁶<https://github.com/UCLA-VAST/QUEKO-benchmark>

⁷We did not compare the runtime of these four tools, as some of them are implemented in Python and others in C. The inherent efficiency difference of programming languages renders a direct runtime comparison unfair. More fundamentally, since we use a noiseless model of quantum computers in this paper, layout synthesis would be required only once, so performance rather than runtime is the major concern. For other studies on mitigating changing noise properties of quantum devices, the runtime would be more important.

⁸<https://quantum-computing.ibm.com/>

⁹<https://www.rigetti.com/qpu>

¹⁰<https://sqc.com.au>

below. It is beneficial to study QSE, since it is the only circuit so far with which experimental QC has shown a clear advantage. We chose the TFL because existing QC logic synthesis algorithms are based largely on reversible logic synthesis, which uses TFL as a fundamental element [7]. Some B_{NTF} have TFL density and others have QSE density. All B_{SS} have QSE density. We also swept through possible gate density vectors and generated *benchmarks for impact of gate density* (B_{IGD}).

6.1.4 Layout Synthesis Tools

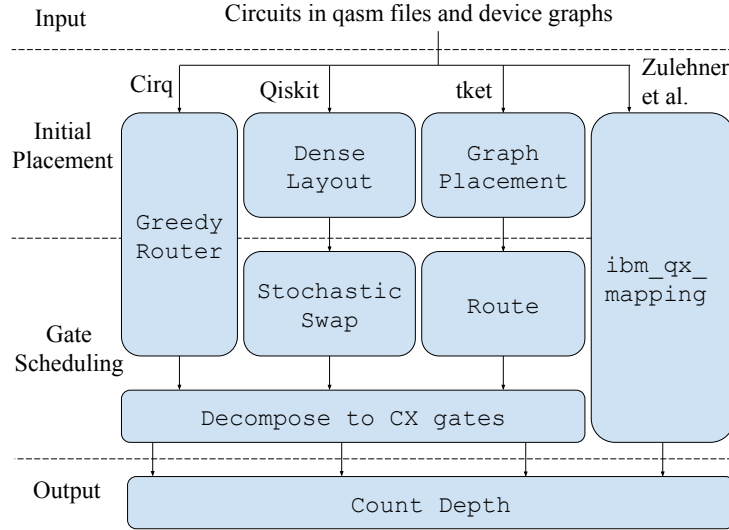


Figure 5: Workflow of the experiments

Currently, Google, IBM, and Rigetti are considered front-runners of superconducting QC. Inside their QC frameworks (Cirq, Qiskit, and pyQuil), there are tools for layout synthesis. Unfortunately, pyQuil does not provide options to breakdown the whole compilation into optimization and layout synthesis, so pyQuil was excluded from the experiments. We also included a recent academic work from Zulehner et al. [4], which is open-source [1].

We used greedy router in Cirq version 0.6.0 as one of the layout synthesis tools, as shown in Fig. 5. So far, only one router named greedy has been released, which contains an initial placement policy and a SWAP insertion policy based on heuristic search. Note that greedy router does not transform the gates into gates that are implementable on Google devices in Table 1, so the resulting circuit it produces contains the original input gates and SWAP gates it inserts. For a fair comparison of depth, in all the experiments, we decompose SWAP gates inserted by the tools to three CX gates, like in Fig. 3b.

Qiskit offers the most precise control over the so-called “transpiler”. The transpilation is divided into individual passes, and users can define their own “pass manager” to make use of various transpiling modules that are offered. For the layout synthesis problem, there are Layout modules generating initial mapping and Swap modules inserting SWAP gates to the circuit to enable two-qubit gates. Among the various combinations, we chose DenseLayout and StochasticSwap as shown in Fig. 5, which seemed to have the best overall performance. DenseLayout maps the logical qubits to an area on the device graph with dense connections. StochasticSwap perturbs the distance matrix of physical qubits and performs heuristic search for SWAP gates. The version of Qiskit in the experiments is 0.14.1.

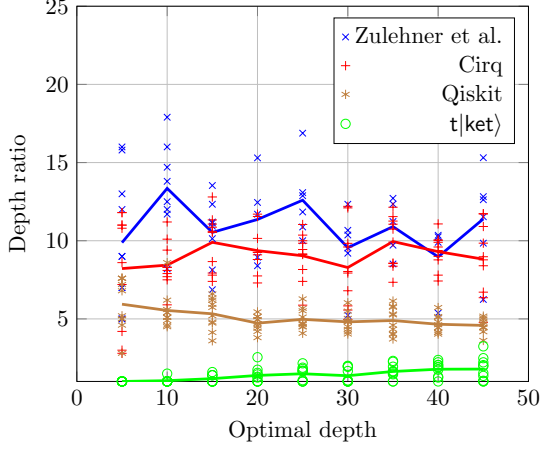
Another highly competitive router, t|ket), comes from Cambridge Quantum Computing. Graph Placement uses graph monomorphism to derive initial mapping. Route performs heuristic search for SWAP gates. We used t|ket) version 0.4.1 in the experiments.

Since all the tools evaluated use heuristics at some stages, sub-optimality is expected. Note that we only use default setup on all the modules in all the tools. Changing setup parameters in some of these modules specifically for QUEKO benchmarks may lead to better performance in the following experiments, but may lead to worse performance on other circuits.

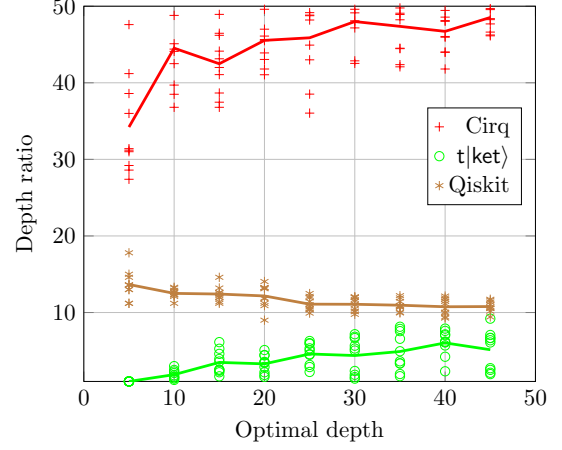
¹¹ https://github.com/iic-jku/ibm_qx_mapping

6.2 Experimental Results

6.2.1 Performance on B_{NTF}



(a) Smaller device (Aspen-4), sparser circuits (TFL)

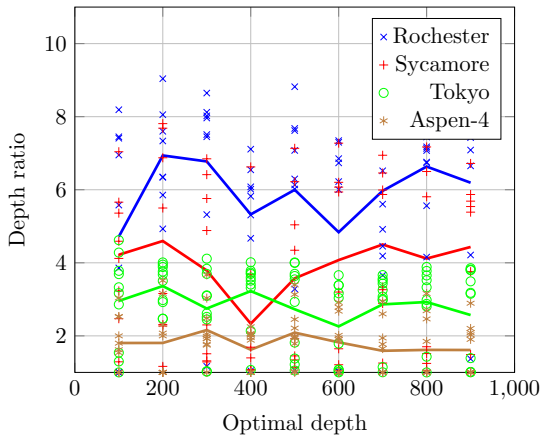


(b) Larger device (Sycamore), denser circuits (QSE)

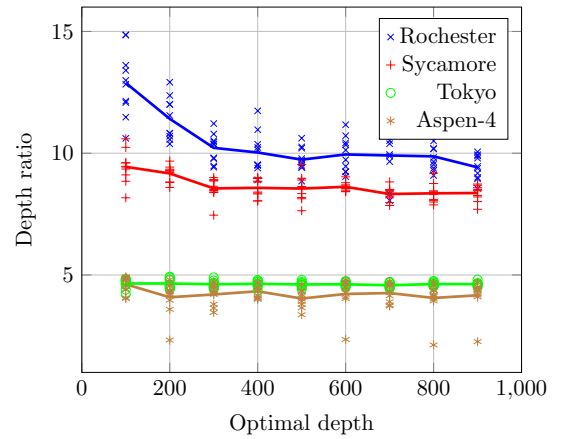
Figure 6: Performance of QC layout synthesis tools on B_{NTF} (Lines are average.)

In Fig. 6, the horizontal axis is the optimal depth and the vertical axis is the depth ratio, which is the depth of layout synthesis result divided by the optimal depth T . In the case of a smaller device (Aspen-4) and sparser circuits (TFL), the optimality gap on average is about 12x for [4], 10x for Cirq, 5x for Qiskit and 1.5x for $t|ket\rangle$. In the case of a larger device (Sycamore) and denser circuits (QSE), the optimality gap on average is about 11x to 14x for Qiskit. The optimality gaps of Cirq and $t|ket\rangle$ grow with depth correspondingly from 35x to 50x and from 1x to 5x. Zulehner et al. is not in the Fig. 6b, because for the larger device, it took so much memory that the operating system constantly killed it before finishing. This also happens sometimes for the smaller device experiments, so there are less blue data points than the other types of points in Fig. 6a.

6.2.2 Performance on B_{SS}



(a) $t|ket\rangle$ scaling behavior



(b) Qiskit scaling behavior

Figure 7: $t|ket\rangle$ and Qiskit performance on B_{SS} (Lines are average.)

We studied further scaling of $t|ket\rangle$ and Qiskit on different devices as shown in Fig. 7. The optimality gaps on average by $t|ket\rangle$ are about 5x to 7x for Rochester, 3x to 4x for Sycamore, 3x for Tokyo, and 2x for Aspen-4. Note that for a

fixed depth and a fixed device, the optimality gaps by $t|ket\rangle$ varies rather widely. $t|ket\rangle$ managed to find the optimal mappings for some QUEKO benchmarks. In general, as the depth increases, the depth ratio by Qiskit decreases at first and then converges to a value. The reason for this phenomenon may be that as the circuit deepens, the influence of initial placement gets smaller than the influence of SWAP insertion. For Qiskit, the optimality gaps on Rochester decreased from 13x to 10x on average; on Sycamore, Tokyo and Aspen-4 are about 8x, 5x, and 4x on average. It can be seen that larger devices (Rochester and Sycamore) bring about larger optimality gaps. If the number of physical qubits are close, then richer connectivity (Sycamore versus Rochester) brings about smaller optimality gaps.

6.2.3 Performance on B_{IGD}

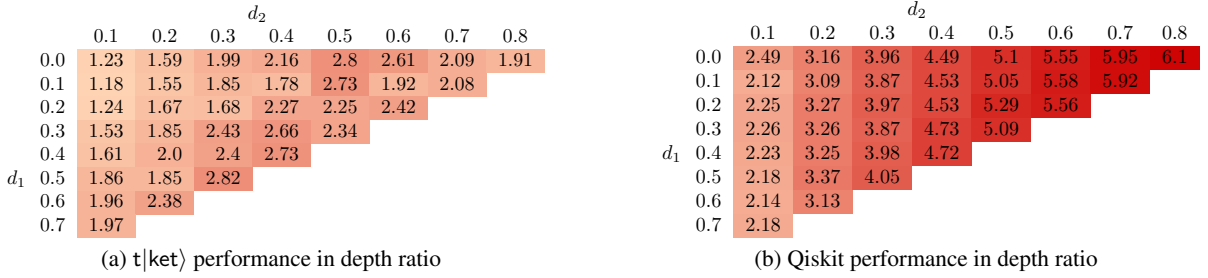


Figure 8: $t|ket\rangle$ and Qiskit performance on B_{IGD} (Data are 10-time average.)

To better understand the impact of gate density on layout synthesis performance, we fixed the device to Tokyo and the depth to 45, and swept through possible gate densities. The results are shown in Fig. 8. Fixing a column, the single-qubit gate density increases as we go down, Qiskit seems to be rather insensitive to this change, which is sensible since the single-qubit gates do not induce difficulty in layout synthesis. However, $t|ket\rangle$ is still sensitive to this change. Both tools are more sensitive to the change in the horizontal direction than in the vertical direction. Since the challenge to layout synthesis comes mainly, if not solely, from the two-qubit gates, this result is expected. The depth ratio of $t|ket\rangle$ decreases when the two-qubit gate density is very high. This is because when the circuit is dense with two-qubit gates, graph monomorphism algorithms can extract more information from the first few layers to narrow down better initial mappings.

7 Complexity

Seeing the large optimality gap, it is natural for us to investigate the computational complexity of the depth-optimal QC layout synthesis problem, which was unknown till this point. Several related results are shown, e.g., determining the minimal number of SWAP gates to insert is NP-complete. [19] proves this theorem by reduction from subgraph isomorphism problem. [12] proves the NP-completeness of depth-optimal initial placement where SWAP gates are not allowed. The NP-completeness of depth-optimal QC layout synthesis for QAOA circuits is proven in [39] by reduction from 3-SAT. In this section, we prove this for general quantum circuits by reduction from Hamiltonian cycle problem, as Theorem 1 states.

Theorem 1. *Depth-optimal QC layout synthesis is NP-complete.*

Proof. The original QC depth-optimal layout synthesis problem is not easier than its decision version: input and constraints remain the same; but output whether the depth of the scheduled circuit can be lower or equal to T . Inspired by [12], we show that the problem of determining whether a Hamiltonian cycle exists in a graph is reducible to the QC depth-decision layout synthesis problem. The former is NP-complete, so the latter is also NP-complete.

Suppose the graph for the Hamiltonian cycle problem is $G_H = (V_H, E_H)$, where $|V_H| = N$. We construct a depth-decision QC layout synthesis problem using G_H as the device graph and N as the target depth. The input circuit is N “levels” of gates. Level l contains a two-qubit gate $g_l = CX(q_l, q_{(l+1) \bmod N})$. All the other logical qubits at level l are fully occupied by single-qubit gates.

If there exists a Hamiltonian cycle in G_H , $(p_1, p_2, \dots, p_N, p_1)$, then let the initial mapping be $\mu_0 : q_i \mapsto p_i$ for $i = 1$ to N . It is easy to see that, with this mapping, all the gates in the constructed circuit can be executed.

On the other hand, if there exists a scheduled circuit with depth within N , we first claim that the mapping cannot change during the execution of the circuit. Every gate in a level depends on some gate in the last level. So every gate in level l

has a dependency chain of length l , which is the earliest cycle it can be scheduled to. This means, if any SWAP gates are inserted in gate scheduling, certain dependency chain must lengthen and the depth of the scheduled circuit is larger than N . Therefore, if a solution within N cycles exists, each gate in level l must be scheduled at exactly cycle l and there can be no SWAP gates inserted. It is also easy to see that the input CX gates cannot constitute any SWAP gates. Therefore, the mapping from logical to physical qubits remains μ_0 throughout all the cycles in the scheduled circuit. The gates $CX(q_l, q_{(l+1) \bmod N})$ being all executable means that they are mapped to edges of G_H . This means $\mu_0(q_1), \mu_0(q_2), \dots, \mu_0(q_N), \mu_0(q_1)$ is a Hamiltonian cycle in G_H .

In conclusion, we established the equivalence between a Hamiltonian cycle in G_H and the existence of QC layout synthesis solution within depth N . Thus, the Hamiltonian cycle problem is reducible to the QC layout synthesis problem. The latter is NP-complete, since the former is known to be NP-complete. $\square$

8 Conclusion and Future Work

In this paper, we formulated the problem of quantum computing layout synthesis and proved its NP-completeness. We constructed QUEKO benchmarks, each has a known optimal depth for the given device. With QUEKO, we examined four existing quantum computing layout synthesis tools, greedy router in Cirq, t|ket>, DenseLayout plus StochasticSwap in Qiskit, and Zulehner et al. [4] and showed rather surprising results. Despite over ten years of research and development efforts by both academia and industry, the current QC compilation flow is far from optimal. In fact, even combining the best performances of all tools evaluated, the optimality gaps range from 1.5x to 45x on average for circuits of feasible depth on existing devices. These gaps reveal that there is substantial room for research into QC layout synthesis, potentially equivalent to an order of improvement of the decoherence time, which would require much higher investment in quantum device technologies to achieve. We plan to use QUEKO benchmarks as a guide to better layout synthesis tools. On the other hand, our construction method opens up room for future extension. With more specific control over the amount and structures of dependency chains, we believe it is possible to generate benchmarks with known optimal that are even more representative of real applications, which sometimes requires SWAP gates in optimal solutions. In addition, we plan to extend our research to construct examples with known optimal solutions for fidelity optimization, as multiple studies have shown that fidelity is a very important metric for quantum circuits in the NISQ era.

Acknowledgment

The authors would like to thank Iris Cong, Silas Dilkes, Dmitri Maslov, and Nengkun Yu for valuable comments on the manuscript. This work is partially supported by NEC under the Center for Domain-Specific Computing Industrial Partnership Program.

References

- [1] Frank Arute, Kunal Arya, Ryan Babbush, Dave Bacon, Joseph C Bardin, Rami Barends, Rupak Biswas, Sergio Boixo, Fernando GSL Brandao, David A Buell, et al. Quantum supremacy using a programmable superconducting processor. *Nature*, 574(7779):505–510, 2019.
- [2] Michael A Nielsen and Isaac L Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, Cambridge, UK, 2010.
- [3] Jay Gambetta and Sarah Sheldon. Cramming more power into a quantum device. <https://www.ibm.com/blogs/research/2019/03/power-quantum-device/>, March 2019.
- [4] A. Zulehner, A. Paller, and R. Wille. Efficient mapping of quantum circuits to the IBM QX architectures. In *2018 Design, Automation Test in Europe Conference Exhibition (DATE)*, pages 1135–1138, March 2018.
- [5] Chin-Chih Chang, J. Cong, M. Romesis, and Min Xie. Optimality and scalability study of existing placement algorithms. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 23(4):537–549, April 2004.
- [6] ‘Huge opportunity’ in IC design optimization gained by Semiconductor Research Corporation, National Science Foundation: CAD innovation could save industry billions. <https://www.src.org/newsroom/press-release/2007/41/>, December 2007.